

ESP32 Interior - Sistema de Fermentacion de Cafe

1.0

Generado por Doxygen 1.17.0

1 Jerarquía de directorios	1
1.1 Directorios	1
2 Índice de clases	5
2.1 Lista de clases	5
3 Índice de archivos	7
3.1 Lista de archivos	7
4 Documentación de directorios	9
4.1 Referencia del directorio main/drivers	9
4.2 Referencia del directorio main/hal	10
4.3 Referencia del directorio main	10
4.4 Referencia del directorio main/mqtt	10
4.5 Referencia del directorio main/tasks	11
4.6 Referencia del directorio main/utils	11
4.7 Referencia del directorio main/wifi	11
5 Documentación de clases	13
5.1 Referencia de la estructura control_cmd_t	13
5.1.1 Descripción detallada	13
5.1.2 Documentación de datos miembro	13
5.1.2.1 bomba	13
5.1.2.2 enfriar	14
5.1.2.3 mezclar	14
5.1.2.4 servo_angle	14
5.2 Referencia de la estructura datetime_t	14
5.2.1 Descripción detallada	14
5.2.2 Documentación de datos miembro	15
5.2.2.1 day	15
5.2.2.2 hours	15
5.2.2.3 minutes	15
5.2.2.4 month	15
5.2.2.5 seconds	15
5.2.2.6 year	15
5.3 Referencia de la estructura display_data_t	16
5.3.1 Descripción detallada	16
5.3.2 Documentación de datos miembro	16
5.3.2.1 co2	16
5.3.2.2 datetime	16
5.3.2.3 mqtt_ok	16
5.3.2.4 ph	17
5.3.2.5 sd_ok	17
5.3.2.6 temperatura	17

5.3.2.7 wifi_ok	17
5.4 Referencia de la estructura sensor_data_t	17
5.4.1 Descripción detallada	18
5.4.2 Documentación de datos miembro	18
5.4.2.1 co2	18
5.4.2.2 co2_raw	18
5.4.2.3 corriente	18
5.4.2.4 datetime	18
5.4.2.5 ph	19
5.4.2.6 temperatura	19
5.4.2.7 voltaje	19
6 Documentación de archivos	21
6.1 Referencia del archivo main/drivers/ds18b20.c	21
6.1.1 Descripción detallada	22
6.1.2 Documentación de funciones	22
6.1.2.1 ds18b20_init()	22
6.1.2.2 ds18b20_read_temperature()	23
6.1.2.3 ow_low()	24
6.1.2.4 ow_read()	24
6.1.2.5 ow_read_bit()	25
6.1.2.6 ow_read_byte()	25
6.1.2.7 ow_release()	25
6.1.2.8 ow_reset()	26
6.1.2.9 ow_write_bit()	26
6.1.2.10 ow_write_byte()	27
6.1.3 Documentación de variables	27
6.1.3.1 ds_pin	27
6.1.3.2 TAG	27
6.2 ds18b20.c	27
6.3 Referencia del archivo main/drivers/ds18b20.h	29
6.3.1 Descripción detallada	30
6.3.2 Documentación de funciones	30
6.3.2.1 ds18b20_init()	30
6.3.2.2 ds18b20_read_temperature()	31
6.4 ds18b20.h	32
6.5 Referencia del archivo main/drivers/font5x7.h	32
6.5.1 Descripción detallada	33
6.5.2 Documentación de variables	33
6.5.2.1 font5x7	33
6.6 font5x7.h	34
6.7 Referencia del archivo main/drivers/mq135.c	35

6.7.1 Descripción detallada	36
6.7.2 Documentación de «define»	36
6.7.2.1 MQ135_ADC_CHANNEL	36
6.7.2.2 NUM_SAMPLES	36
6.7.2.3 TAG	36
6.7.3 Documentación de funciones	37
6.7.3.1 mq135_init()	37
6.7.3.2 mq135_read()	37
6.7.3.3 mq135_read_raw()	38
6.7.3.4 mq135_read_voltage()	38
6.8 mq135.c	39
6.9 Referencia del archivo main/drivers/mq135.h	39
6.9.1 Descripción detallada	40
6.9.2 Documentación de funciones	40
6.9.2.1 mq135_init()	40
6.9.2.2 mq135_read()	41
6.9.2.3 mq135_read_raw()	41
6.9.2.4 mq135_read_voltage()	42
6.10 mq135.h	42
6.11 Referencia del archivo main/drivers/oled_display.c	43
6.11.1 Descripción detallada	44
6.11.2 Documentación de «define»	44
6.11.2.1 OLED_ADDR	44
6.11.2.2 OLED_HEIGHT	44
6.11.2.3 OLED_PAGES	45
6.11.2.4 OLED_WIDTH	45
6.11.3 Documentación de funciones	45
6.11.3.1 oled_clear()	45
6.11.3.2 oled_draw_char()	46
6.11.3.3 oled_draw_pixel()	46
6.11.3.4 oled_draw_string()	47
6.11.3.5 oled_init()	48
6.11.3.6 oled_send_command()	49
6.11.3.7 oled_send_data()	49
6.11.3.8 oled_update()	50
6.11.4 Documentación de variables	51
6.11.4.1 framebuffer	51
6.11.4.2 mutex_i2c	51
6.12 oled_display.c	51
6.13 Referencia del archivo main/drivers/oled_display.h	54
6.13.1 Descripción detallada	54
6.13.2 Documentación de funciones	55

6.13.2.1 oled_clear()	55
6.13.2.2 oled_draw_char()	55
6.13.2.3 oled_draw_pixel()	56
6.13.2.4 oled_draw_string()	57
6.13.2.5 oled_init()	57
6.13.2.6 oled_update()	58
6.14 oled_display.h	59
6.15 Referencia del archivo main/drivers/ph_sensor.c	59
6.15.1 Descripción detallada	60
6.15.2 Documentación de «define»	61
6.15.2.1 NUM_SAMPLES	61
6.15.2.2 PH_ADC_CHANNEL	61
6.15.2.3 PH_MAX_VOLTAGE	61
6.15.2.4 PH_MIN_VOLTAGE	61
6.15.2.5 PH_OFFSET	61
6.15.2.6 PH_SLOPE	62
6.15.3 Documentación de funciones	62
6.15.3.1 ph_init()	62
6.15.3.2 ph_read()	62
6.15.3.3 ph_read_voltage()	63
6.15.4 Documentación de variables	64
6.15.4.1 TAG	64
6.16 ph_sensor.c	64
6.17 Referencia del archivo main/drivers/ph_sensor.h	65
6.17.1 Descripción detallada	66
6.17.2 Documentación de funciones	66
6.17.2.1 ph_init()	66
6.17.2.2 ph_read()	67
6.17.2.3 ph_read_voltage()	68
6.18 ph_sensor.h	68
6.19 Referencia del archivo main/drivers/rtc_ds3231.c	69
6.19.1 Descripción detallada	70
6.19.2 Documentación de «define»	70
6.19.2.1 DS3231_ADDR	70
6.19.2.2 DS3231_CONTROL_A1_INTERRUPT	70
6.19.3 Documentación de funciones	71
6.19.3.1 bcd_to_dec()	71
6.19.3.2 dec_to_bcd()	71
6.19.3.3 ds3231_clear_alarm_flag()	72
6.19.3.4 ds3231_get_datetime()	72
6.19.3.5 ds3231_init()	73
6.19.3.6 ds3231_set_alarm_seconds()	74

6.19.4 Documentación de variables	75
6.19.4.1 mutex_i2c	75
6.19.4.2 TAG	76
6.20 rtc_ds3231.c	76
6.21 Referencia del archivo main/drivers/rtc_ds3231.h	77
6.21.1 Descripción detallada	78
6.21.2 Documentación de funciones	78
6.21.2.1 ds3231_clear_alarm_flag()	78
6.21.2.2 ds3231_get_datetime()	79
6.21.2.3 ds3231_init()	80
6.21.2.4 ds3231_set_alarm_seconds()	81
6.22 rtc_ds3231.h	82
6.23 Referencia del archivo main/drivers/sd_card.c	82
6.23.1 Descripción detallada	83
6.23.2 Documentación de «define»	84
6.23.2.1 PIN_CLK	84
6.23.2.2 PIN_CS	84
6.23.2.3 PIN_MISO	84
6.23.2.4 PIN_MOSI	84
6.23.3 Documentación de funciones	85
6.23.3.1 sd_card_append_pending_mqtt()	85
6.23.3.2 sd_card_close_pending_mqtt()	85
6.23.3.3 sd_card_delete_pending_mqtt()	86
6.23.3.4 sd_card_init()	86
6.23.3.5 sd_card_is_mounted()	88
6.23.3.6 sd_card_open_pending_mqtt_read()	88
6.23.3.7 sd_card_pending_mqtt_exists()	89
6.23.3.8 sd_card_read_pending_mqtt_line()	89
6.23.3.9 sd_card_write_line()	90
6.23.4 Documentación de variables	91
6.23.4.1 mount_point	91
6.23.4.2 pending_mqtt_path	91
6.23.4.3 sd_mounted	91
6.23.4.4 spi_mutex	91
6.23.4.5 TAG	92
6.24 sd_card.c	92
6.25 Referencia del archivo main/drivers/sd_card.h	95
6.25.1 Descripción detallada	96
6.25.2 Documentación de funciones	96
6.25.2.1 sd_card_append_pending_mqtt()	96
6.25.2.2 sd_card_close_pending_mqtt()	97
6.25.2.3 sd_card_delete_pending_mqtt()	98

6.25.2.4 sd_card_init()	98
6.25.2.5 sd_card_is_mounted()	99
6.25.2.6 sd_card_open_pending_mqtt_read()	100
6.25.2.7 sd_card_pending_mqtt_exists()	100
6.25.2.8 sd_card_read_pending_mqtt_line()	101
6.25.2.9 sd_card_write_line()	101
6.26 sd_card.h	102
6.27 Referencia del archivo main/drivers/servo.c	103
6.27.1 Descripción detallada	104
6.27.2 Documentación de «define»	104
6.27.2.1 SERVO_CHANNEL	104
6.27.2.2 SERVO_FREQ	105
6.27.2.3 SERVO_MAX_DUTY	105
6.27.2.4 SERVO_MAX_PULSE_US	105
6.27.2.5 SERVO_MIN_PULSE_US	105
6.27.2.6 SERVO_MODE	105
6.27.2.7 SERVO_PERIOD_US	105
6.27.2.8 SERVO_RESOLUTION	106
6.27.2.9 SERVO_TIMER	106
6.27.3 Documentación de funciones	106
6.27.3.1 servo_init()	106
6.27.3.2 servo_set_angle()	107
6.27.4 Documentación de variables	108
6.27.4.1 servo_pin	108
6.27.4.2 TAG	109
6.28 servo.c	109
6.29 Referencia del archivo main/drivers/servo.h	110
6.29.1 Descripción detallada	110
6.29.2 Documentación de funciones	111
6.29.2.1 servo_init()	111
6.29.2.2 servo_set_angle()	112
6.30 servo.h	113
6.31 Referencia del archivo main/drivers/voltage_sensor.c	114
6.31.1 Descripción detallada	114
6.31.2 Documentación de «define»	115
6.31.2.1 ADC_OFFSET	115
6.31.2.2 DIVIDER_RATIO	115
6.31.2.3 NUM_SAMPLES	115
6.31.2.4 TAG	115
6.31.2.5 VOLTAGE_ADC_CHANNEL	115
6.31.3 Documentación de funciones	116
6.31.3.1 voltage_sensor_init()	116

6.31.3.2 voltage_sensor_read()	116
6.32 voltage_sensor.c	117
6.33 Referencia del archivo main/drivers/voltage_sensor.h	118
6.33.1 Descripción detallada	118
6.33.2 Documentación de funciones	119
6.33.2.1 voltage_sensor_init()	119
6.33.2.2 voltage_sensor_read()	119
6.34 voltage_sensor.h	120
6.35 Referencia del archivo main/hal/adc_manager.c	120
6.35.1 Descripción detallada	121
6.35.2 Documentación de funciones	122
6.35.2.1 adc_manager_init()	122
6.35.2.2 adc_manager_read_voltage()	123
6.35.3 Documentación de variables	124
6.35.3.1 adc1_cali_handle	124
6.35.3.2 adc1_handle	124
6.35.3.3 TAG	124
6.36 adc_manager.c	125
6.37 Referencia del archivo main/hal/adc_manager.h	126
6.37.1 Descripción detallada	126
6.37.2 Documentación de funciones	127
6.37.2.1 adc_manager_init()	127
6.37.2.2 adc_manager_read_voltage()	128
6.38 adc_manager.h	129
6.39 Referencia del archivo main/hal/i2c_manager.c	129
6.39.1 Descripción detallada	130
6.39.2 Documentación de «define»	131
6.39.2.1 I2C_MASTER_FREQ_HZ	131
6.39.2.2 I2C_MASTER_NUM	131
6.39.2.3 I2C_MASTER_SCL_IO	131
6.39.2.4 I2C_MASTER_SDA_IO	131
6.39.2.5 TAG	131
6.39.3 Documentación de funciones	132
6.39.3.1 i2c_manager_init()	132
6.39.3.2 i2c_manager_read()	133
6.39.3.3 i2c_manager_write()	134
6.39.3.4 i2c_manager_write_raw()	135
6.39.4 Documentación de variables	136
6.39.4.1 mutex_i2c	136
6.40 i2c_manager.c	137
6.41 Referencia del archivo main/hal/i2c_manager.h	139
6.41.1 Descripción detallada	140

6.41.2 Documentación de funciones	140
6.41.2.1 i2c_manager_init()	140
6.41.2.2 i2c_manager_read()	141
6.41.2.3 i2c_manager_write()	143
6.41.2.4 i2c_manager_write_raw()	144
6.41.3 Documentación de variables	145
6.41.3.1 mutex_i2c	145
6.42 i2c_manager.h	146
6.43 Referencia del archivo main/main.c	146
6.43.1 Descripción detallada	147
6.43.2 Documentación de «define»	148
6.43.2.1 WIFI_CONNECT_TIMEOUT_MS	148
6.43.3 Documentación de funciones	148
6.43.3.1 app_main()	148
6.43.4 Documentación de variables	151
6.43.4.1 TAG	151
6.43.4.2 task_energia_handle	151
6.43.4.3 task_sensores_handle	151
6.44 main.c	152
6.45 Referencia del archivo main/mqtt/mqtt_manager.c	154
6.45.1 Descripción detallada	156
6.45.2 Documentación de «define»	156
6.45.2.1 MQTT_BROKER_URI	156
6.45.2.2 MQTT_CONNECTED_BIT	157
6.45.2.3 MQTT_ERROR_BIT	157
6.45.2.4 MQTT_PUBLISHED_BIT	157
6.45.3 Documentación de funciones	157
6.45.3.1 mqtt_event_handler()	157
6.45.3.2 mqtt_is_connected()	158
6.45.3.3 mqtt_manager_start()	159
6.45.3.4 mqtt_manager_stop()	160
6.45.3.5 mqtt_publish()	160
6.45.3.6 mqtt_wait_connected()	161
6.45.4 Documentación de variables	162
6.45.4.1 mqtt_client	162
6.45.4.2 mqtt_connected	163
6.45.4.3 mqtt_event_group	163
6.45.4.4 TAG	163
6.46 mqtt_manager.c	163
6.47 Referencia del archivo main/mqtt/mqtt_manager.h	165
6.47.1 Descripción detallada	166
6.47.2 Documentación de funciones	166

6.47.2.1 mqtt_is_connected()	166
6.47.2.2 mqtt_manager_start()	167
6.47.2.3 mqtt_manager_stop()	167
6.47.2.4 mqtt_publish()	168
6.47.2.5 mqtt_wait_connected()	169
6.48 mqtt_manager.h	170
6.49 Referencia del archivo main/tasks/task_actuadores.c	171
6.49.1 Descripción detallada	171
6.49.2 Documentación de «define»	172
6.49.2.1 MIX_DURATION_MS	172
6.49.2.2 PIN_BOMBA	172
6.49.2.3 PIN_MOTOR	172
6.49.2.4 PIN_SERVO	173
6.49.3 Documentación de funciones	173
6.49.3.1 actuadores_init()	173
6.49.3.2 task_actuadores()	173
6.49.4 Documentación de variables	175
6.49.4.1 TAG	175
6.49.4.2 task_energia_handle	175
6.50 task_actuadores.c	175
6.51 Referencia del archivo main/tasks/task_actuadores.h	176
6.51.1 Descripción detallada	177
6.51.2 Documentación de funciones	177
6.51.2.1 task_actuadores()	177
6.52 task_actuadores.h	178
6.53 Referencia del archivo main/tasks/task_control.c	179
6.53.1 Descripción detallada	179
6.53.2 Documentación de «define»	180
6.53.2.1 CO2_THRESHOLD	180
6.53.2.2 MIX_INTERVAL_CYCLES	180
6.53.3 Documentación de funciones	181
6.53.3.1 task_control()	181
6.53.4 Documentación de variables	182
6.53.4.1 TAG	182
6.54 task_control.c	182
6.55 Referencia del archivo main/tasks/task_control.h	183
6.55.1 Descripción detallada	183
6.55.2 Documentación de funciones	184
6.55.2.1 task_control()	184
6.56 task_control.h	185
6.57 Referencia del archivo main/tasks/task_display.c	185
6.57.1 Descripción detallada	186

6.57.2 Documentación de funciones	187
6.57.2.1 task_display()	187
6.58 task_display.c	189
6.59 Referencia del archivo main/tasks/task_display.h	190
6.59.1 Descripción detallada	190
6.59.2 Documentación de funciones	191
6.59.2.1 task_display()	191
6.60 task_display.h	193
6.61 Referencia del archivo main/tasks/task_energia.c	193
6.61.1 Descripción detallada	194
6.61.2 Documentación de funciones	195
6.61.2.1 task_energia()	195
6.61.3 Documentación de variables	196
6.61.3.1 TAG	196
6.61.3.2 task_sensores_handle	196
6.62 task_energia.c	196
6.63 Referencia del archivo main/tasks/task_energia.h	197
6.63.1 Descripción detallada	197
6.63.2 Documentación de funciones	198
6.63.2.1 task_energia()	198
6.64 task_energia.h	199
6.65 Referencia del archivo main/tasks/task_logger.c	199
6.65.1 Descripción detallada	200
6.65.2 Documentación de funciones	200
6.65.2.1 task_logger()	200
6.65.3 Documentación de variables	202
6.65.3.1 TAG	202
6.66 task_logger.c	202
6.67 Referencia del archivo main/tasks/task_logger.h	203
6.67.1 Descripción detallada	203
6.67.2 Documentación de funciones	204
6.67.2.1 task_logger()	204
6.68 task_logger.h	205
6.69 Referencia del archivo main/tasks/task_mqtt.c	205
6.69.1 Descripción detallada	206
6.69.2 Documentación de «define»	207
6.69.2.1 MQTT_CONNECT_TIMEOUT_MS	207
6.69.2.2 MQTT_PENDING_PAYLOAD_SIZE	207
6.69.2.3 MQTT_PUBLISH_TIMEOUT_MS	207
6.69.2.4 MQTT_TOPIC	208
6.69.3 Documentación de funciones	208
6.69.3.1 build_payload()	208

6.69.3.2 resend_pending_mqtt_messages()	208
6.69.3.3 save_pending_payload()	209
6.69.3.4 task_mqtt()	210
6.69.4 Documentación de variables	211
6.69.4.1 task_energia_handle	211
6.70 task_mqtt.c	212
6.71 Referencia del archivo main/tasks/task_mqtt.h	214
6.71.1 Descripción detallada	214
6.71.2 Documentación de funciones	215
6.71.2.1 task_mqtt()	215
6.72 task_mqtt.h	217
6.73 Referencia del archivo main/tasks/task_sensores.c	217
6.73.1 Descripción detallada	218
6.73.2 Documentación de «define»	219
6.73.2.1 ALPHA	219
6.73.3 Documentación de funciones	219
6.73.3.1 task_sensores()	219
6.73.4 Documentación de variables	222
6.73.4.1 baseline	222
6.73.4.2 baseline_initialized	222
6.73.4.3 filtered	222
6.73.4.4 TAG	222
6.74 task_sensores.c	223
6.75 Referencia del archivo main/tasks/task_sensores.h	225
6.75.1 Descripción detallada	225
6.75.2 Documentación de funciones	226
6.75.2.1 task_sensores()	226
6.76 task_sensores.h	229
6.77 Referencia del archivo main/utls/data_types.h	229
6.77.1 Descripción detallada	230
6.78 data_types.h	230
6.79 Referencia del archivo main/utls/queues.c	231
6.79.1 Descripción detallada	232
6.79.2 Documentación de funciones	232
6.79.2.1 queues_init()	232
6.79.3 Documentación de variables	233
6.79.3.1 queue_control	233
6.79.3.2 queue_display	233
6.79.3.3 queue_logger	233
6.79.3.4 queue_mqtt	233
6.79.3.5 queue_sensores	234
6.80 queues.c	234

6.81 Referencia del archivo main/utls/queues.h	234
6.81.1 Descripción detallada	235
6.81.2 Documentación de funciones	236
6.81.2.1 queues_init()	236
6.81.3 Documentación de variables	236
6.81.3.1 queue_control	236
6.81.3.2 queue_display	237
6.81.3.3 queue_logger	237
6.81.3.4 queue_mqtt	237
6.81.3.5 queue_sensores	237
6.82 queues.h	238
6.83 Referencia del archivo main/wifi/wifi_manager.c	238
6.83.1 Descripción detallada	239
6.83.2 Documentación de «define»	240
6.83.2.1 WIFI_CONNECTED_BIT	240
6.83.2.2 WIFI_FAIL_BIT	240
6.83.2.3 WIFI_MAX_RETRIES	240
6.83.2.4 WIFI_PASS	240
6.83.2.5 WIFI_SCAN_MAX_AP	240
6.83.2.6 WIFI_SSID	240
6.83.3 Documentación de funciones	241
6.83.3.1 auth_mode_name()	241
6.83.3.2 wifi_event_handler()	241
6.83.3.3 wifi_init()	243
6.83.3.4 wifi_is_connected()	244
6.83.3.5 wifi_scan_print()	245
6.83.3.6 wifi_wait_connected()	246
6.83.4 Documentación de variables	247
6.83.4.1 retry_count	247
6.83.4.2 TAG	247
6.83.4.3 wifi_event_group	247
6.84 wifi_manager.c	248
6.85 Referencia del archivo main/wifi/wifi_manager.h	251
6.85.1 Descripción detallada	251
6.85.2 Documentación de funciones	252
6.85.2.1 wifi_init()	252
6.85.2.2 wifi_is_connected()	253
6.85.2.3 wifi_scan_print()	254
6.85.2.4 wifi_wait_connected()	255
6.86 wifi_manager.h	256

Capítulo 1

Jerarquía de directorios

1.1. Directorios

drivers	9
ds18b20.c	21
ds18b20.h	29
font5x7.h	32
mq135.c	35
mq135.h	39
oled_display.c	43
oled_display.h	54
ph_sensor.c	59
ph_sensor.h	65
rtc_ds3231.c	69
rtc_ds3231.h	77
sd_card.c	82
sd_card.h	95
servo.c	103
servo.h	110
voltage_sensor.c	114
voltage_sensor.h	118
hal	10
adc_manager.c	120
adc_manager.h	126
i2c_manager.c	129
i2c_manager.h	139
main	10
drivers	9
ds18b20.c	21
ds18b20.h	29
font5x7.h	32
mq135.c	35
mq135.h	39
oled_display.c	43
oled_display.h	54
ph_sensor.c	59
ph_sensor.h	65
rtc_ds3231.c	69
rtc_ds3231.h	77
sd_card.c	82

sd_card.h	95
servo.c	103
servo.h	110
voltage_sensor.c	114
voltage_sensor.h	118
hal	10
adc_manager.c	120
adc_manager.h	126
i2c_manager.c	129
i2c_manager.h	139
mqtt	10
mqtt_manager.c	154
mqtt_manager.h	165
tasks	11
task_actuadores.c	171
task_actuadores.h	176
task_control.c	179
task_control.h	183
task_display.c	185
task_display.h	190
task_energia.c	193
task_energia.h	197
task_logger.c	199
task_logger.h	203
task_mqtt.c	205
task_mqtt.h	214
task_sensores.c	217
task_sensores.h	225
utils	11
data_types.h	229
queues.c	231
queues.h	234
wifi	11
wifi_manager.c	238
wifi_manager.h	251
main.c	146
mqtt	10
mqtt_manager.c	154
mqtt_manager.h	165
tasks	11
task_actuadores.c	171
task_actuadores.h	176
task_control.c	179
task_control.h	183
task_display.c	185
task_display.h	190
task_energia.c	193
task_energia.h	197
task_logger.c	199
task_logger.h	203
task_mqtt.c	205
task_mqtt.h	214
task_sensores.c	217
task_sensores.h	225
utils	11
data_types.h	229
queues.c	231

queues.h	234
wifi	11
wifi_manager.c	238
wifi_manager.h	251

Capítulo 2

Índice de clases

2.1. Lista de clases

Lista de clases, estructuras, uniones e interfaces con breves descripciones:

control_cmd_t		
	Comandos generados por la lógica de control	13
datetime_t		
	Estructura para almacenar fecha y hora	14
display_data_t		
	Información destinada a la interfaz de usuario	16
sensor_data_t		
	Datos adquiridos por los sensores del sistema	17

Capítulo 3

Índice de archivos

3.1. Lista de archivos

Lista de todos los archivos con breves descripciones:

main/ main.c	
Punto de entrada del sistema de fermentación basado en ESP32	146
main/drivers/ ds18b20.c	
Implementación del driver para el sensor DS18B20 mediante protocolo OneWire	21
main/drivers/ ds18b20.h	
Driver para el sensor de temperatura DS18B20 mediante protocolo OneWire	29
main/drivers/ font5x7.h	
Definición de una fuente bitmap de 5x7 píxeles para pantallas OLED	32
main/drivers/ mq135.c	
Implementación del driver para adquisición de señales del sensor MQ135	35
main/drivers/ mq135.h	
Driver para adquisición de la señal analógica del sensor MQ135	39
main/drivers/ oled_display.c	
Implementación del controlador OLED SSD1306 mediante interfaz I2C	43
main/drivers/ oled_display.h	
Driver para pantalla OLED SSD1306 mediante interfaz I2C	54
main/drivers/ ph_sensor.c	
Implementación del driver para sensor de pH basado en adquisición analógica	59
main/drivers/ ph_sensor.h	
Driver para sensor de pH basado en adquisición analógica	65
main/drivers/ rtc_ds3231.c	
Implementación del driver para el RTC DS3231	69
main/drivers/ rtc_ds3231.h	
Driver para el reloj en tiempo real DS3231 mediante interfaz I2C	77
main/drivers/ sd_card.c	
Implementación del módulo de almacenamiento en tarjeta SD mediante SPI	82
main/drivers/ sd_card.h	
Módulo para gestión de tarjeta SD mediante interfaz SPI	95
main/drivers/ servo.c	
Implementación del control de servomotor usando PWM mediante LEDC	103
main/drivers/ servo.h	
Driver para control de servomotores mediante PWM usando el periférico LEDC del ESP32	110
main/drivers/ voltage_sensor.c	
Implementación del módulo de medición de voltaje	114
main/drivers/ voltage_sensor.h	
Driver para medición de voltaje mediante conversión ADC	118

main/hal/ adc_manager.c	
Implementación de la capa de abstracción para el ADC del ESP32	120
main/hal/ adc_manager.h	
Capa de abstracción para el convertidor analógico-digital (ADC) del ESP32	126
main/hal/ i2c_manager.c	
Implementación de la capa de abstracción para el bus I2C del ESP32	129
main/hal/ i2c_manager.h	
Capa de abstracción para el bus I2C del ESP32	139
main/mqtt/ mqtt_manager.c	
Implementación del gestor de comunicación MQTT	154
main/mqtt/ mqtt_manager.h	
Interfaz para la gestión de comunicación MQTT	165
main/tasks/ task_actuadores.c	
Implementación de la tarea de ejecución de actuadores	171
main/tasks/ task_actuadores.h	
Declaración de la tarea encargada del control de actuadores	176
main/tasks/ task_control.c	
Implementación de la lógica de control del sistema	179
main/tasks/ task_control.h	
Declaración de la tarea de control del sistema	183
main/tasks/ task_display.c	
Implementación de la tarea de visualización del sistema	185
main/tasks/ task_display.h	
Declaración de la tarea de visualización del sistema	190
main/tasks/ task_energia.c	
Implementación de la gestión energética del sistema	193
main/tasks/ task_energia.h	
Declaración de la tarea de gestión energética del sistema	197
main/tasks/ task_logger.c	
Implementación de la tarea de registro de datos	199
main/tasks/ task_logger.h	
Declaración de la tarea de registro de datos	203
main/tasks/ task_mqtt.c	
Implementación de la tarea de comunicación MQTT	205
main/tasks/ task_mqtt.h	
Declaración de la tarea de comunicación MQTT	214
main/tasks/ task_sensores.c	
Implementación de la tarea de adquisición de sensores	217
main/tasks/ task_sensores.h	
Declaración de la tarea de adquisición de sensores	225
main/utills/ data_types.h	
Definición de estructuras de datos compartidas del sistema	229
main/utills/ queues.c	
Implementación de las colas de comunicación del sistema	231
main/utills/ queues.h	
Declaración de las colas de comunicación del sistema	234
main/wifi/ wifi_manager.c	
Implementación del gestor de conectividad WiFi	238
main/wifi/ wifi_manager.h	
Interfaz para la gestión de conectividad WiFi	251

Capítulo 4

Documentación de directorios

4.1. Referencia del directorio main/drivers

Archivos

- archivo [ds18b20.c](#)
Implementación del driver para el sensor DS18B20 mediante protocolo OneWire.
- archivo [ds18b20.h](#)
Driver para el sensor de temperatura DS18B20 mediante protocolo OneWire.
- archivo [font5x7.h](#)
Definición de una fuente bitmap de 5x7 píxeles para pantallas OLED.
- archivo [mq135.c](#)
Implementación del driver para adquisición de señales del sensor MQ135.
- archivo [mq135.h](#)
Driver para adquisición de la señal analógica del sensor MQ135.
- archivo [oled_display.c](#)
Implementación del controlador OLED SSD1306 mediante interfaz I2C.
- archivo [oled_display.h](#)
Driver para pantalla OLED SSD1306 mediante interfaz I2C.
- archivo [ph_sensor.c](#)
Implementación del driver para sensor de pH basado en adquisición analógica.
- archivo [ph_sensor.h](#)
Driver para sensor de pH basado en adquisición analógica.
- archivo [rtc_ds3231.c](#)
Implementación del driver para el RTC DS3231.
- archivo [rtc_ds3231.h](#)
Driver para el reloj en tiempo real DS3231 mediante interfaz I2C.
- archivo [sd_card.c](#)
Implementación del módulo de almacenamiento en tarjeta SD mediante SPI.
- archivo [sd_card.h](#)
Módulo para gestión de tarjeta SD mediante interfaz SPI.
- archivo [servo.c](#)
Implementación del control de servomotor usando PWM mediante LEDC.
- archivo [servo.h](#)
Driver para control de servomotores mediante PWM usando el periférico LEDC del ESP32.
- archivo [voltage_sensor.c](#)
Implementación del módulo de medición de voltaje.
- archivo [voltage_sensor.h](#)
Driver para medición de voltaje mediante conversión ADC.

4.2. Referencia del directorio main/hal

Archivos

- archivo [adc_manager.c](#)
Implementación de la capa de abstracción para el ADC del ESP32.
- archivo [adc_manager.h](#)
Capa de abstracción para el convertidor analógico-digital (ADC) del ESP32.
- archivo [i2c_manager.c](#)
Implementación de la capa de abstracción para el bus I2C del ESP32.
- archivo [i2c_manager.h](#)
Capa de abstracción para el bus I2C del ESP32.

4.3. Referencia del directorio main

Directorios

- directorio [drivers](#)
- directorio [hal](#)
- directorio [mqtt](#)
- directorio [tasks](#)
- directorio [utils](#)
- directorio [wifi](#)

Archivos

- archivo [main.c](#)
Punto de entrada del sistema de fermentación basado en ESP32.

4.4. Referencia del directorio main/mqtt

Archivos

- archivo [mqtt_manager.c](#)
Implementación del gestor de comunicación MQTT.
- archivo [mqtt_manager.h](#)
Interfaz para la gestión de comunicación MQTT.

4.5. Referencia del directorio main/tasks

Archivos

- archivo [task_actuadores.c](#)
Implementación de la tarea de ejecución de actuadores.
- archivo [task_actuadores.h](#)
Declaración de la tarea encargada del control de actuadores.
- archivo [task_control.c](#)
Implementación de la lógica de control del sistema.
- archivo [task_control.h](#)
Declaración de la tarea de control del sistema.
- archivo [task_display.c](#)
Implementación de la tarea de visualización del sistema.
- archivo [task_display.h](#)
Declaración de la tarea de visualización del sistema.
- archivo [task_energia.c](#)
Implementación de la gestión energética del sistema.
- archivo [task_energia.h](#)
Declaración de la tarea de gestión energética del sistema.
- archivo [task_logger.c](#)
Implementación de la tarea de registro de datos.
- archivo [task_logger.h](#)
Declaración de la tarea de registro de datos.
- archivo [task_mqtt.c](#)
Implementación de la tarea de comunicación MQTT.
- archivo [task_mqtt.h](#)
Declaración de la tarea de comunicación MQTT.
- archivo [task_sensores.c](#)
Implementación de la tarea de adquisición de sensores.
- archivo [task_sensores.h](#)
Declaración de la tarea de adquisición de sensores.

4.6. Referencia del directorio main/utils

Archivos

- archivo [data_types.h](#)
Definición de estructuras de datos compartidas del sistema.
- archivo [queues.c](#)
Implementación de las colas de comunicación del sistema.
- archivo [queues.h](#)
Declaración de las colas de comunicación del sistema.

4.7. Referencia del directorio main/wifi

Archivos

- archivo [wifi_manager.c](#)
Implementación del gestor de conectividad WiFi.
- archivo [wifi_manager.h](#)
Interfaz para la gestión de conectividad WiFi.

Capítulo 5

Documentación de clases

5.1. Referencia de la estructura control_cmd_t

Comandos generados por la lógica de control.

```
#include <data_types.h>
```

Atributos públicos

- bool [enfriar](#)
- bool [bomba](#)
- bool [mezclar](#)
- float [servo_angle](#)

5.1.1. Descripción detallada

Comandos generados por la lógica de control.

Esta estructura es producida por la tarea de control y consumida por la tarea de actuadores.

Permite mantener desacoplada la lógica de decisión del acceso directo al hardware.

Definición en la línea [86](#) del archivo [data_types.h](#).

5.1.2. Documentación de datos miembro

5.1.2.1. bomba

```
bool control_cmd_t::bomba
```

Activar bomba de circulación.

Definición en la línea [90](#) del archivo [data_types.h](#).

5.1.2.2. enfriar

```
bool control_cmd_t::enfriar
```

Activar sistema de enfriamiento.

Definición en la línea 88 del archivo [data_types.h](#).

5.1.2.3. mezclar

```
bool control_cmd_t::mezclar
```

Activar motor de mezcla.

Definición en la línea 91 del archivo [data_types.h](#).

5.1.2.4. servo_angle

```
float control_cmd_t::servo_angle
```

Ángulo objetivo del servomotor.

Definición en la línea 93 del archivo [data_types.h](#).

La documentación de esta estructura está generada del siguiente archivo:

- [main/utls/data_types.h](#)

5.2. Referencia de la estructura `datetime_t`

Estructura para almacenar fecha y hora.

```
#include <data_types.h>
```

Atributos públicos

- `uint8_t` [seconds](#)
- `uint8_t` [minutes](#)
- `uint8_t` [hours](#)
- `uint8_t` [day](#)
- `uint8_t` [month](#)
- `uint16_t` [year](#)

5.2.1. Descripción detallada

Estructura para almacenar fecha y hora.

Utilizada por el RTC DS3231 y por los módulos de registro de datos y visualización.

Definición en la línea 40 del archivo [data_types.h](#).

5.2.2. Documentación de datos miembro

5.2.2.1. `day`

```
uint8_t datetime_t::day
```

Día del mes.

Definición en la línea 45 del archivo [data_types.h](#).

5.2.2.2. `hours`

```
uint8_t datetime_t::hours
```

Horas.

Definición en la línea 44 del archivo [data_types.h](#).

5.2.2.3. `minutes`

```
uint8_t datetime_t::minutes
```

Minutos.

Definición en la línea 43 del archivo [data_types.h](#).

5.2.2.4. `month`

```
uint8_t datetime_t::month
```

Mes.

Definición en la línea 46 del archivo [data_types.h](#).

5.2.2.5. `seconds`

```
uint8_t datetime_t::seconds
```

Segundos.

Definición en la línea 42 del archivo [data_types.h](#).

5.2.2.6. `year`

```
uint16_t datetime_t::year
```

Año.

Definición en la línea 47 del archivo [data_types.h](#).

La documentación de esta estructura está generada del siguiente archivo:

- [main/utis/data_types.h](#)

5.3. Referencia de la estructura `display_data_t`

Información destinada a la interfaz de usuario.

```
#include <data_types.h>
```

Atributos públicos

- float `temperatura`
- float `ph`
- float `co2`
- bool `wifi_ok`
- bool `mqtt_ok`
- bool `sd_ok`
- `datetime_t` `datetime`

5.3.1. Descripción detallada

Información destinada a la interfaz de usuario.

Contiene las variables mostradas en la pantalla OLED, incluyendo mediciones de sensores, estados de conexión y fecha/hora actual.

Definición en la línea 104 del archivo `data_types.h`.

5.3.2. Documentación de datos miembro

5.3.2.1. `co2`

```
float display_data_t::co2
```

Concentración de CO2 actual.

Definición en la línea 108 del archivo `data_types.h`.

5.3.2.2. `datetime`

```
datetime_t display_data_t::datetime
```

Fecha y hora mostradas.

Definición en la línea 114 del archivo `data_types.h`.

5.3.2.3. `mqtt_ok`

```
bool display_data_t::mqtt_ok
```

Estado de conexión MQTT.

Definición en la línea 111 del archivo `data_types.h`.

5.3.2.4. ph

```
float display_data_t::ph
```

Valor de pH actual.

Definición en la línea 107 del archivo [data_types.h](#).

5.3.2.5. sd_ok

```
bool display_data_t::sd_ok
```

Estado de la tarjeta SD.

Definición en la línea 112 del archivo [data_types.h](#).

5.3.2.6. temperatura

```
float display_data_t::temperatura
```

Temperatura actual.

Definición en la línea 106 del archivo [data_types.h](#).

5.3.2.7. wifi_ok

```
bool display_data_t::wifi_ok
```

Estado de conexión WiFi.

Definición en la línea 110 del archivo [data_types.h](#).

La documentación de esta estructura está generada del siguiente archivo:

- [main/utils/data_types.h](#)

5.4. Referencia de la estructura sensor_data_t

Datos adquiridos por los sensores del sistema.

```
#include <data_types.h>
```

Atributos públicos

- float [temperatura](#)
- float [ph](#)
- float [co2](#)
- float [co2_raw](#)
- float [voltaje](#)
- float [corriente](#)
- [datetime_t](#) [datetime](#)

5.4.1. Descripción detallada

Datos adquiridos por los sensores del sistema.

Esta estructura agrupa todas las variables medidas durante un ciclo de adquisición.

Es utilizada para:

- Comunicación entre tareas.
- Registro en tarjeta SD.
- Publicación MQTT.
- Visualización en pantalla.

Definición en la línea 63 del archivo [data_types.h](#).

5.4.2. Documentación de datos miembro

5.4.2.1. co2

```
float sensor_data_t::co2
```

Concentración estimada de CO2.

Definición en la línea 67 del archivo [data_types.h](#).

5.4.2.2. co2_raw

```
float sensor_data_t::co2_raw
```

Lectura ADC sin procesar del MQ-135.

Definición en la línea 68 del archivo [data_types.h](#).

5.4.2.3. corriente

```
float sensor_data_t::corriente
```

Corriente consumida.

Definición en la línea 71 del archivo [data_types.h](#).

5.4.2.4. datetime

```
datetime_t sensor_data_t::datetime
```

Fecha y hora de la medición.

Definición en la línea 73 del archivo [data_types.h](#).

5.4.2.5. `ph`

```
float sensor_data_t::ph
```

Valor de pH.

Definición en la línea 66 del archivo [data_types.h](#).

5.4.2.6. `temperatura`

```
float sensor_data_t::temperatura
```

Temperatura interna (°C).

Definición en la línea 65 del archivo [data_types.h](#).

5.4.2.7. `voltaje`

```
float sensor_data_t::voltaje
```

Voltaje del sistema.

Definición en la línea 70 del archivo [data_types.h](#).

La documentación de esta estructura está generada del siguiente archivo:

- [main/utils/data_types.h](#)

Capítulo 6

Documentación de archivos

6.1. Referencia del archivo main/drivers/ds18b20.c

Implementación del driver para el sensor DS18B20 mediante protocolo OneWire.

```
#include "drivers/ds18b20.h"
#include "driver/gpio.h"
#include "esp_rom_sys.h"
#include "esp_log.h"
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
```

Funciones

- static void [ow_low](#) ()
Fuerza la línea OneWire a nivel bajo.
- static void [ow_release](#) ()
Libera la línea OneWire.
- static int [ow_read](#) ()
Lee el estado actual del bus OneWire.
- static int [ow_reset](#) ()
Realiza el reset del bus OneWire.
- static void [ow_write_bit](#) (int bit)
Escribe un bit en el bus OneWire.
- static int [ow_read_bit](#) ()
Lee un bit desde el bus OneWire.
- static void [ow_write_byte](#) (uint8_t byte)
Escribe un byte completo en el bus OneWire.
- static uint8_t [ow_read_byte](#) ()
Lee un byte completo desde el bus OneWire.
- esp_err_t [ds18b20_init](#) (gpio_num_t pin)
Inicializa el sensor DS18B20.
- float [ds18b20_read_temperature](#) (void)
Obtiene la temperatura del sensor DS18B20.

Variables

- static const char * TAG = "DS18B20"
Etiqueta utilizada por el sistema de logging ESP-IDF.
- static gpio_num_t ds_pin
GPIO utilizado para la comunicación OneWire.

6.1.1. Descripción detallada

Implementación del driver para el sensor DS18B20 mediante protocolo OneWire.

Este módulo implementa las funciones necesarias para la comunicación con un sensor DS18B20 utilizando el protocolo OneWire por software.

Incluye:

- Inicialización del bus OneWire.
- Transmisión y recepción de bits y bytes.
- Detección de presencia del dispositivo.
- Conversión y lectura de temperatura.

El sensor se utiliza para monitorear la temperatura interna del sistema de fermentación de café.

Autor

Fernando Plazas Trujillo Isabella Ordoñez Juan Daniel Constain

Definición en el archivo [ds18b20.c](#).

6.1.2. Documentación de funciones

6.1.2.1. ds18b20_init()

```
esp_err_t ds18b20_init (
    gpio_num_t pin)
```

Inicializa el sensor DS18B20.

Parámetros

<i>pin</i>	GPIO de conexión.
------------	-------------------

Devuelve

ESP_OK si se configura correctamente.

Configura el GPIO utilizado para la comunicación OneWire y habilita la resistencia pull-up requerida por el protocolo.

Esta función debe ejecutarse una única vez durante la inicialización del sistema.

Parámetros

<i>pin</i>	GPIO al que se encuentra conectado el sensor.
------------	---

Devuelve

- ESP_OK: Inicialización exitosa.
- ESP_FAIL: Error durante la configuración del dispositivo.

Definición en la línea 198 del archivo ds18b20.c.

```
00199 {
00200     ds_pin = pin;
00201
00202     gpio_set_pull_mode(ds_pin, GPIO_PULLUP_ONLY);
00203
00204     ESP_LOGI(TAG, "DS18B20 init en GPIO%d", pin);
00205
00206     return ESP_OK;
00207 }
```

6.1.2.2. ds18b20_read_temperature()

```
float ds18b20_read_temperature (
    void )
```

Obtiene la temperatura del sensor DS18B20.

Obtiene una medición de temperatura desde el DS18B20.

Devuelve

Temperatura en °C.

-127.0 si el sensor no responde.

Obtiene la temperatura del sensor DS18B20.

Ejecuta la secuencia completa de comunicación OneWire:

- Reset del bus.
- Inicio de conversión de temperatura.
- Lectura de memoria Scratchpad.
- Conversión del dato a grados Celsius.

Devuelve

Temperatura medida en grados Celsius.

Valores devueltos

-	Error de comunicación o ausencia del sensor.
127.↔	
0	

Definición en la línea 219 del archivo [ds18b20.c](#).

```

00220 {
00221     if (!ow_reset()) {
00222         ESP_LOGE(TAG, "Sensor no detectado");
00223         return -127.0;
00224     }
00225
00226     ow_write_byte(0xCC);
00227     ow_write_byte(0x44);
00228
00229     /*
00230     * Tiempo máximo de conversión para resolución de 12 bits
00231     * según la hoja de datos del DS18B20.
00232     */
00233     vTaskDelay(pdMS_TO_TICKS(750));
00234
00235     if (!ow_reset()) {
00236         return -127.0;
00237     }
00238
00239     ow_write_byte(0xCC);
00240     ow_write_byte(0xBE);
00241
00242     uint8_t temp_lsb = ow_read_byte();
00243     uint8_t temp_msb = ow_read_byte();
00244
00245     int16_t raw = (temp_msb << 8) | temp_lsb;
00246
00247     float temp = raw / 16.0;
00248
00249     return temp;
00250 }
```

6.1.2.3. ow_low()

```
void ow_low () [static]
```

Fuerza la línea OneWire a nivel bajo.

Configura temporalmente el GPIO como salida y escribe un nivel lógico bajo.

Definición en la línea 53 del archivo [ds18b20.c](#).

```

00054 {
00055     gpio_set_direction(ds_pin, GPIO_MODE_OUTPUT);
00056     gpio_set_level(ds_pin, 0);
00057 }
```

6.1.2.4. ow_read()

```
int ow_read () [static]
```

Lee el estado actual del bus OneWire.

Devuelve

0 o 1 según el nivel lógico.

Definición en la línea 75 del archivo [ds18b20.c](#).

```

00076 {
00077     return gpio_get_level(ds_pin);
00078 }
```

6.1.2.5. ow_read_bit()

```
int ow_read_bit () [static]
```

Lee un bit desde el bus OneWire.

Devuelve

Bit leído (0 o 1).

Definición en la línea 139 del archivo [ds18b20.c](#).

```
00140 {  
00141     int bit;  
00142  
00143     ow_low();  
00144     esp_rom_delay_us(3);  
00145     ow_release();  
00146     esp_rom_delay_us(10);  
00147  
00148     bit = ow_read();  
00149  
00150     esp_rom_delay_us(53);  
00151  
00152     return bit;  
00153 }
```

6.1.2.6. ow_read_byte()

```
uint8_t ow_read_byte () [static]
```

Lee un byte completo desde el bus OneWire.

Devuelve

Byte leído.

Definición en la línea 177 del archivo [ds18b20.c](#).

```
00178 {  
00179     uint8_t byte = 0;  
00180  
00181     for (int i = 0; i < 8; i++) {  
00182         byte |= (ow_read_bit() << i);  
00183     }  
00184  
00185     return byte;  
00186 }
```

6.1.2.7. ow_release()

```
void ow_release () [static]
```

Libera la línea OneWire.

Configura el GPIO como entrada permitiendo que la resistencia pull-up mantenga el bus en nivel alto.

Definición en la línea 65 del archivo [ds18b20.c](#).

```
00066 {  
00067     gpio_set_direction(ds_pin, GPIO_MODE_INPUT);  
00068 }
```

6.1.2.8. ow_reset()

```
int ow_reset () [static]
```

Realiza el reset del bus OneWire.

Envía la señal de reset y detecta la presencia del sensor.

Devuelve

1 si el dispositivo responde, 0 en caso contrario.

Definición en la línea 91 del archivo [ds18b20.c](#).

```
00092 {  
00093     ow_low();  
00094     esp_rom_delay_us(480);  
00095  
00096     ow_release();  
00097     esp_rom_delay_us(70);  
00098  
00099     int presence = !ow_read();  
00100  
00101     esp_rom_delay_us(410);  
00102  
00103     return presence;  
00104 }
```

6.1.2.9. ow_write_bit()

```
void ow_write_bit (  
    int bit) [static]
```

Escribe un bit en el bus OneWire.

Parámetros

<i>bit</i>	Valor del bit (0 o 1).
------------	------------------------

Definición en la línea 115 del archivo [ds18b20.c](#).

```
00116 {  
00117     ow_low();  
00118  
00119     if (bit) {  
00120         esp_rom_delay_us(10);  
00121         ow_release();  
00122         esp_rom_delay_us(55);  
00123     } else {  
00124         esp_rom_delay_us(65);  
00125         ow_release();  
00126         esp_rom_delay_us(5);  
00127     }  
00128 }
```


6.1.2.10. ow_write_byte()

```
void ow_write_byte (
    uint8_t byte) [static]
```

Escribe un byte completo en el bus OneWire.

Parámetros

<i>byte</i>	Byte a transmitir.
-------------	--------------------

Definición en la línea 164 del archivo [ds18b20.c](#).

```
00165 {
00166     for (int i = 0; i < 8; i++) {
00167         ow_write_bit(byte & 0x01);
00168         byte >>= 1;
00169     }
00170 }
```

6.1.3. Documentación de variables

6.1.3.1. ds_pin

```
gpio_num_t ds_pin [static]
```

GPIO utilizado para la comunicación OneWire.

Se configura durante la inicialización del driver y es utilizado por todas las funciones de acceso al sensor.

Definición en la línea 42 del archivo [ds18b20.c](#).

6.1.3.2. TAG

```
const char* TAG = "DS18B20" [static]
```

Etiqueta utilizada por el sistema de logging ESP-IDF.

Definición en la línea 34 del archivo [ds18b20.c](#).

6.2. ds18b20.c

[Ir a la documentación de este archivo.](#)

```
00001
00002
00023 #include "drivers/ds18b20.h"
00024 #include "driver/gpio.h"
00025 #include "esp_rom_sys.h"
00026 #include "esp_log.h"
00027
00028 #include "freertos/FreeRTOS.h"
00029 #include "freertos/task.h"
00030
00034 static const char *TAG = "DS18B20";
00035
00042 static gpio_num_t ds_pin;
00043
00044 // =====
```

```

00045 // LOW LEVEL
00046 // =====
00047
00053 static void ow_low()
00054 {
00055     gpio_set_direction(ds_pin, GPIO_MODE_OUTPUT);
00056     gpio_set_level(ds_pin, 0);
00057 }
00058
00065 static void ow_release()
00066 {
00067     gpio_set_direction(ds_pin, GPIO_MODE_INPUT);
00068 }
00069
00075 static int ow_read()
00076 {
00077     return gpio_get_level(ds_pin);
00078 }
00079
00080 // =====
00081 // RESET
00082 // =====
00083
00091 static int ow_reset()
00092 {
00093     ow_low();
00094     esp_rom_delay_us(480);
00095
00096     ow_release();
00097     esp_rom_delay_us(70);
00098
00099     int presence = !ow_read();
00100
00101     esp_rom_delay_us(410);
00102
00103     return presence;
00104 }
00105
00106 // =====
00107 // WRITE BIT
00108 // =====
00109
00115 static void ow_write_bit(int bit)
00116 {
00117     ow_low();
00118
00119     if (bit) {
00120         esp_rom_delay_us(10);
00121         ow_release();
00122         esp_rom_delay_us(55);
00123     } else {
00124         esp_rom_delay_us(65);
00125         ow_release();
00126         esp_rom_delay_us(5);
00127     }
00128 }
00129
00130 // =====
00131 // READ BIT
00132 // =====
00133
00139 static int ow_read_bit()
00140 {
00141     int bit;
00142
00143     ow_low();
00144     esp_rom_delay_us(3);
00145     ow_release();
00146     esp_rom_delay_us(10);
00147
00148     bit = ow_read();
00149
00150     esp_rom_delay_us(53);
00151
00152     return bit;
00153 }
00154
00155 // =====
00156 // BYTE
00157 // =====
00158
00164 static void ow_write_byte(uint8_t byte)
00165 {
00166     for (int i = 0; i < 8; i++) {
00167         ow_write_bit(byte & 0x01);
00168         byte >>= 1;
00169     }

```

```

00170 }
00171
00177 static uint8_t ow_read_byte()
00178 {
00179     uint8_t byte = 0;
00180
00181     for (int i = 0; i < 8; i++) {
00182         byte |= (ow_read_bit() << i);
00183     }
00184
00185     return byte;
00186 }
00187
00188 // =====
00189 // INIT
00190 // =====
00191
00198 esp_err_t ds18b20_init(gpio_num_t pin)
00199 {
00200     ds_pin = pin;
00201
00202     gpio_set_pull_mode(ds_pin, GPIO_PULLUP_ONLY);
00203
00204     ESP_LOGI(TAG, "DS18B20 init en GPIO%d", pin);
00205
00206     return ESP_OK;
00207 }
00208
00209 // =====
00210 // READ TEMPERATURE
00211 // =====
00212
00219 float ds18b20_read_temperature(void)
00220 {
00221     if (!ow_reset()) {
00222         ESP_LOGE(TAG, "Sensor no detectado");
00223         return -127.0;
00224     }
00225
00226     ow_write_byte(0xCC);
00227     ow_write_byte(0x44);
00228
00229     /*
00230     * Tiempo máximo de conversión para resolución de 12 bits
00231     * según la hoja de datos del DS18B20.
00232     */
00233     vTaskDelay(pdMS_TO_TICKS(750));
00234
00235     if (!ow_reset()) {
00236         return -127.0;
00237     }
00238
00239     ow_write_byte(0xCC);
00240     ow_write_byte(0xBE);
00241
00242     uint8_t temp_lsb = ow_read_byte();
00243     uint8_t temp_msb = ow_read_byte();
00244
00245     int16_t raw = (temp_msb << 8) | temp_lsb;
00246
00247     float temp = raw / 16.0;
00248
00249     return temp;
00250 }

```

6.3. Referencia del archivo main/drivers/ds18b20.h

Driver para el sensor de temperatura DS18B20 mediante protocolo OneWire.

```

#include "esp_err.h"
#include "driver/gpio.h"

```

Funciones

- `esp_err_t ds18b20_init(gpio_num_t pin)`

Inicializa el sensor DS18B20.

- float `ds18b20_read_temperature` (void)

Obtiene una medición de temperatura desde el DS18B20.

6.3.1. Descripción detallada

Driver para el sensor de temperatura DS18B20 mediante protocolo OneWire.

Este módulo proporciona las funciones necesarias para inicializar y adquirir mediciones de temperatura desde un sensor DS18B20.

El sensor se utiliza para monitorear la temperatura interna del proceso de fermentación de café.

Funcionalidades:

- Inicialización del bus OneWire.
- Lectura de temperatura en grados Celsius.
- Detección básica de errores de comunicación.

Autor

Fernando Plazas Trujillo Isabella Ordoñez Juan Daniel Constain

Definición en el archivo `ds18b20.h`.

6.3.2. Documentación de funciones

6.3.2.1. `ds18b20_init()`

```
esp_err_t ds18b20_init (  
    gpio_num_t pin)
```

Inicializa el sensor DS18B20.

Configura el GPIO utilizado para la comunicación OneWire y habilita la resistencia pull-up requerida por el protocolo.

Esta función debe ejecutarse una única vez durante la inicialización del sistema.

Parámetros

<i>pin</i>	GPIO al que se encuentra conectado el sensor.
------------	---

Devuelve

- ESP_OK: Inicialización exitosa.
- ESP_FAIL: Error durante la configuración del dispositivo.

Parámetros

<i>pin</i>	GPIO de conexión.
------------	-------------------

Devuelve

ESP_OK si se configura correctamente.

Definición en la línea 198 del archivo ds18b20.c.

```
00199 {
00200     ds_pin = pin;
00201
00202     gpio_set_pull_mode(ds_pin, GPIO_PULLUP_ONLY);
00203
00204     ESP_LOGI(TAG, "DS18B20 init en GPIO%d", pin);
00205
00206     return ESP_OK;
00207 }
```

6.3.2.2. ds18b20_read_temperature()

```
float ds18b20_read_temperature (
    void )
```

Obtiene una medición de temperatura desde el DS18B20.

Obtiene la temperatura del sensor DS18B20.

Ejecuta la secuencia completa de comunicación OneWire:

- Reset del bus.
- Inicio de conversión de temperatura.
- Lectura de memoria Scratchpad.
- Conversión del dato a grados Celsius.

Devuelve

Temperatura medida en grados Celsius.

Valores devueltos

-	Error de comunicación o ausencia del sensor.
127.↔	
0	

Obtiene una medición de temperatura desde el DS18B20.

Devuelve

Temperatura en °C.

-127.0 si el sensor no responde.

Definición en la línea 219 del archivo `ds18b20.c`.

```
00220 {
00221     if (!ow_reset()) {
00222         ESP_LOGE(TAG, "Sensor no detectado");
00223         return -127.0;
00224     }
00225
00226     ow_write_byte(0xCC);
00227     ow_write_byte(0x44);
00228
00229     /*
00230     * Tiempo máximo de conversión para resolución de 12 bits
00231     * según la hoja de datos del DS18B20.
00232     */
00233     vTaskDelay(pdMS_TO_TICKS(750));
00234
00235     if (!ow_reset()) {
00236         return -127.0;
00237     }
00238
00239     ow_write_byte(0xCC);
00240     ow_write_byte(0xBE);
00241
00242     uint8_t temp_lsb = ow_read_byte();
00243     uint8_t temp_msb = ow_read_byte();
00244
00245     int16_t raw = (temp_msb < 8) | temp_lsb;
00246
00247     float temp = raw / 16.0;
00248
00249     return temp;
00250 }
```

6.4. ds18b20.h

[Ir a la documentación de este archivo.](#)

```
00001
00021
00022 #ifndef DS18B20_H
00023 #define DS18B20_H
00024
00025 #include "esp_err.h"
00026 #include "driver/gpio.h"
00027
00043 esp_err_t ds18b20_init(gpio_num_t pin);
00044
00057 float ds18b20_read_temperature(void);
00058
00059 #endif
```

6.5. Referencia del archivo main/drivers/font5x7.h

Definición de una fuente bitmap de 5x7 píxeles para pantallas OLED.

```
#include <stdint.h>
```

Variables

- static const uint8_t `font5x7` [][5]

Tabla de caracteres bitmap de 5x7 píxeles.

6.5.1. Descripción detallada

Definición de una fuente bitmap de 5x7 píxeles para pantallas OLED.

Este archivo contiene una tabla de caracteres ASCII representados mediante matrices de 5 columnas y 7 filas.

La fuente es utilizada por el módulo oled_display para renderizar texto sobre la pantalla SSD1306.

Características:

- Fuente monoespaciada.
- Caracteres numéricos (0-9).
- Letras mayúsculas (A-Z).
- Símbolos básicos.

Cada entrada contiene 5 bytes donde cada bit representa un píxel de la columna correspondiente.

Autor

Fernando Plazas Trujillo Isabella Ordoñez Juan Daniel Constain

Definición en el archivo [font5x7.h](#).

6.5.2. Documentación de variables

6.5.2.1. font5x7

```
const uint8_t font5x7[][5] [static]
```

Tabla de caracteres bitmap de 5x7 píxeles.

Cada índice corresponde a un código ASCII.

El contenido de cada carácter está almacenado en 5 bytes, donde cada byte representa una columna vertical de píxeles.

Ejemplo:

- font5x7['A'] devuelve la representación gráfica de la letra A.
- font5x7['0'] devuelve la representación gráfica del número 0.

Esta tabla es utilizada por el controlador OLED para generar texto sobre la pantalla SSD1306.

Definición en la línea 45 del archivo `font5x7.h`.

```
00045 {
00046
00047 [48] = {0x3E,0x51,0x49,0x45,0x3E}, // 0
00048 [49] = {0x00,0x42,0x7F,0x40,0x00}, // 1
00049 [50] = {0x42,0x61,0x51,0x49,0x46}, // 2
00050 [51] = {0x21,0x41,0x45,0x4B,0x31}, // 3
00051 [52] = {0x18,0x14,0x12,0x7F,0x10}, // 4
00052 [53] = {0x27,0x45,0x45,0x45,0x39}, // 5
00053 [54] = {0x3C,0x4A,0x49,0x49,0x30}, // 6
00054 [55] = {0x01,0x71,0x09,0x05,0x03}, // 7
00055 [56] = {0x36,0x49,0x49,0x49,0x36}, // 8
00056 [57] = {0x06,0x49,0x49,0x29,0x1E}, // 9
00057
00058 [65] = {0x7E,0x11,0x11,0x11,0x7E}, // A
00059 [66] = {0x7F,0x49,0x49,0x49,0x36}, // B
00060 [67] = {0x3E,0x41,0x41,0x41,0x22}, // C
00061 [68] = {0x7F,0x41,0x41,0x22,0x1C}, // D
00062 [69] = {0x7F,0x49,0x49,0x49,0x41}, // E
00063 [70] = {0x7F,0x09,0x09,0x09,0x01}, // F
00064 [71] = {0x3E,0x41,0x49,0x49,0x7A}, // G
00065 [72] = {0x7F,0x08,0x08,0x08,0x7F}, // H
00066 [73] = {0x00,0x41,0x7F,0x41,0x00}, // I
00067 [74] = {0x20,0x40,0x41,0x3F,0x01}, // J
00068 [75] = {0x7F,0x08,0x14,0x22,0x41}, // K
00069 [76] = {0x7F,0x40,0x40,0x40,0x40}, // L
00070 [77] = {0x7F,0x02,0x0C,0x02,0x7F}, // M
00071 [78] = {0x7F,0x04,0x08,0x10,0x7F}, // N
00072 [79] = {0x3E,0x41,0x41,0x41,0x3E}, // O
00073 [80] = {0x7F,0x09,0x09,0x09,0x06}, // P
00074 [81] = {0x3E,0x41,0x51,0x21,0x5E}, // Q
00075 [82] = {0x7F,0x09,0x19,0x29,0x46}, // R
00076 [83] = {0x46,0x49,0x49,0x49,0x31}, // S
00077 [84] = {0x01,0x01,0x7F,0x01,0x01}, // T
00078 [85] = {0x3F,0x40,0x40,0x40,0x3F}, // U
00079 [86] = {0x1F,0x20,0x40,0x20,0x1F}, // V
00080 [87] = {0x7F,0x20,0x18,0x20,0x7F}, // W
00081 [88] = {0x63,0x14,0x08,0x14,0x63}, // X
00082 [89] = {0x03,0x04,0x78,0x04,0x03}, // Y
00083 [90] = {0x61,0x51,0x49,0x45,0x43}, // Z
00084
00085 [32] = {0,0,0,0,0},
00086 [46] = {0,0x60,0x60,0,0},
00087 [58] = {0,0x36,0x36,0,0},
00088 [45] = {0x08,0x08,0x08,0x08,0x08}
00089 };
```

6.6. font5x7.h

[Ir a la documentación de este archivo.](#)

```
00001
00025 #ifndef FONT5X7_H
00026 #define FONT5X7_H
00027
00028 #include <stdint.h>
00029
00045 static const uint8_t font5x7[][5] = {
00046
00047 [48] = {0x3E,0x51,0x49,0x45,0x3E}, // 0
00048 [49] = {0x00,0x42,0x7F,0x40,0x00}, // 1
00049 [50] = {0x42,0x61,0x51,0x49,0x46}, // 2
00050 [51] = {0x21,0x41,0x45,0x4B,0x31}, // 3
00051 [52] = {0x18,0x14,0x12,0x7F,0x10}, // 4
00052 [53] = {0x27,0x45,0x45,0x45,0x39}, // 5
00053 [54] = {0x3C,0x4A,0x49,0x49,0x30}, // 6
00054 [55] = {0x01,0x71,0x09,0x05,0x03}, // 7
00055 [56] = {0x36,0x49,0x49,0x49,0x36}, // 8
00056 [57] = {0x06,0x49,0x49,0x29,0x1E}, // 9
00057
00058 [65] = {0x7E,0x11,0x11,0x11,0x7E}, // A
00059 [66] = {0x7F,0x49,0x49,0x49,0x36}, // B
00060 [67] = {0x3E,0x41,0x41,0x41,0x22}, // C
00061 [68] = {0x7F,0x41,0x41,0x22,0x1C}, // D
00062 [69] = {0x7F,0x49,0x49,0x49,0x41}, // E
00063 [70] = {0x7F,0x09,0x09,0x09,0x01}, // F
00064 [71] = {0x3E,0x41,0x49,0x49,0x7A}, // G
00065 [72] = {0x7F,0x08,0x08,0x08,0x7F}, // H
```



```

00066     [73] = {0x00, 0x41, 0x7F, 0x41, 0x00}, // I
00067     [74] = {0x20, 0x40, 0x41, 0x3F, 0x01}, // J
00068     [75] = {0x7F, 0x08, 0x14, 0x22, 0x41}, // K
00069     [76] = {0x7F, 0x40, 0x40, 0x40, 0x40}, // L
00070     [77] = {0x7F, 0x02, 0x0C, 0x02, 0x7F}, // M
00071     [78] = {0x7F, 0x04, 0x08, 0x10, 0x7F}, // N
00072     [79] = {0x3E, 0x41, 0x41, 0x41, 0x3E}, // O
00073     [80] = {0x7F, 0x09, 0x09, 0x09, 0x06}, // P
00074     [81] = {0x3E, 0x41, 0x51, 0x21, 0x5E}, // Q
00075     [82] = {0x7F, 0x09, 0x19, 0x29, 0x46}, // R
00076     [83] = {0x46, 0x49, 0x49, 0x49, 0x31}, // S
00077     [84] = {0x01, 0x01, 0x7F, 0x01, 0x01}, // T
00078     [85] = {0x3F, 0x40, 0x40, 0x40, 0x3F}, // U
00079     [86] = {0x1F, 0x20, 0x40, 0x20, 0x1F}, // V
00080     [87] = {0x7F, 0x20, 0x18, 0x20, 0x7F}, // W
00081     [88] = {0x63, 0x14, 0x08, 0x14, 0x63}, // X
00082     [89] = {0x03, 0x04, 0x78, 0x04, 0x03}, // Y
00083     [90] = {0x61, 0x51, 0x49, 0x45, 0x43}, // Z
00084
00085     [32] = {0, 0, 0, 0, 0},
00086     [46] = {0, 0x60, 0x60, 0, 0},
00087     [58] = {0, 0x36, 0x36, 0, 0},
00088     [45] = {0x08, 0x08, 0x08, 0x08, 0x08}
00089 };
00090
00091 #endif

```

6.7. Referencia del archivo main/drivers/mq135.c

Implementación del driver para adquisición de señales del sensor MQ135.

```

#include "mq135.h"
#include "hal/adc_manager.h"
#include "esp_log.h"

```

defines

- #define TAG "MQ135"
Etiqueta utilizada por el sistema de logging.
- #define MQ135_ADC_CHANNEL ADC_CHANNEL_0
Canal ADC asociado al sensor MQ135.
- #define NUM_SAMPLES 10
Cantidad de muestras utilizadas para el cálculo del promedio.

Funciones

- void mq135_init (void)
Inicializa el sensor MQ135.
- int mq135_read_raw (void)
Obtiene un valor crudo estimado del ADC.
- float mq135_read_voltage (void)
Lee el voltaje promedio del sensor.
- float mq135_read (void)
Función principal de lectura del sensor MQ135.

6.7.1. Descripción detallada

Implementación del driver para adquisición de señales del sensor MQ135.

Este módulo permite obtener mediciones analógicas provenientes del sensor MQ135 utilizando el subsistema ADC del ESP32.

Funcionalidades:

- Lectura de voltaje.
- Obtención de valores ADC estimados.
- Promediado de muestras para reducción de ruido.

La conversión a concentraciones específicas de gases no forma parte de este módulo.

Autor

Fernando Plazas Trujillo Isabella Ordoñez Juan Daniel Constain

Definición en el archivo [mq135.c](#).

6.7.2. Documentación de «define»

6.7.2.1. MQ135_ADC_CHANNEL

```
#define MQ135_ADC_CHANNEL ADC_CHANNEL_0
```

Canal ADC asociado al sensor MQ135.

Corresponde al GPIO36 del ESP32.

Definición en la línea 36 del archivo [mq135.c](#).

6.7.2.2. NUM_SAMPLES

```
#define NUM_SAMPLES 10
```

Cantidad de muestras utilizadas para el cálculo del promedio.

El promediado reduce el efecto del ruido presente en la señal analógica.

Definición en la línea 43 del archivo [mq135.c](#).

6.7.2.3. TAG

```
#define TAG "MQ135"
```

Etiqueta utilizada por el sistema de logging.

Definición en la línea 29 del archivo [mq135.c](#).

6.7.3. Documentación de funciones

6.7.3.1. mq135_init()

```
void mq135_init (
    void )
```

Inicializa el sensor MQ135.

Inicializa el módulo MQ135.

Inicializa el sensor MQ135.

Realiza la configuración necesaria para la adquisición de datos desde el canal ADC asociado al sensor.

Esta función debe ejecutarse durante la inicialización del sistema antes de realizar lecturas.

Definición en la línea [52](#) del archivo [mq135.c](#).

```
00053 {
00054     ESP_LOGI(TAG, "MQ135 inicializado");
00055 }
```

6.7.3.2. mq135_read()

```
float mq135_read (
    void )
```

Función principal de lectura del sensor MQ135.

Obtiene una lectura representativa del sensor MQ135.

Devuelve

Voltaje del sensor.

Función principal de lectura del sensor MQ135.

Esta función actúa como interfaz principal del módulo y proporciona una medición simplificada para su utilización por parte de las tareas del sistema.

Actualmente retorna el voltaje promedio de salida del sensor.

Devuelve

Valor medido por el sensor.

Definición en la línea [116](#) del archivo [mq135.c](#).

```
00117 {
00118     return mq135_read_voltage();
00119 }
```

6.7.3.3. mq135_read_raw()

```
int mq135_read_raw (
    void )
```

Obtiene un valor crudo estimado del ADC.

Obtiene una lectura cruda del sensor.

Convierte el voltaje leído a una escala de 12 bits (0–4095).

Devuelve

Valor ADC estimado.

Obtiene un valor crudo estimado del ADC.

Realiza una conversión ADC y devuelve el valor digital correspondiente sin aplicar escalamiento adicional.

Devuelve

Valor digital del ADC.

Definición en la línea 68 del archivo [mq135.c](#).

```
00069 {
00070     int raw = 0;
00071
00072     float voltage = adc_manager_read_voltage(MQ135_ADC_CHANNEL);
00073
00074     raw = (int)((voltage / 3.3) * 4095);
00075
00076     ESP_LOGI(TAG, "RAW estimado: %d", raw);
00077
00078     return raw;
00079 }
```

6.7.3.4. mq135_read_voltage()

```
float mq135_read_voltage (
    void )
```

Lee el voltaje promedio del sensor.

Obtiene el voltaje promedio de salida del sensor.

Realiza múltiples lecturas para reducir ruido.

Devuelve

Voltaje promedio en voltios.

Lee el voltaje promedio del sensor.

Se realizan múltiples conversiones ADC y posteriormente se calcula un promedio para reducir el efecto del ruido presente en la señal analógica.

Devuelve

Voltaje promedio en voltios.

Definición en la línea 92 del archivo [mq135.c](#).

```
00093 {
00094     float sum = 0;
00095
00096     for (int i = 0; i < NUM_SAMPLES; i++) {
00097         sum += adc_manager_read_voltage(MQ135_ADC_CHANNEL);
00098     }
00099
00100     float avg = sum / NUM_SAMPLES;
00101
00102     ESP_LOGI(TAG, "Voltaje: %.4f V", avg);
00103
00104     return avg;
00105 }
```

6.8. mq135.c

[Ir a la documentación de este archivo.](#)

```

00001
00021
00022 #include "mq135.h"
00023 #include "hal/adc_manager.h"
00024 #include "esp_log.h"
00025
00029 #define TAG "MQ135"
00030
00036 #define MQ135_ADC_CHANNEL ADC_CHANNEL_0
00037
00043 #define NUM_SAMPLES 10
00044
00045 // =====
00046 // INIT
00047 // =====
00048
00052 void mq135_init(void)
00053 {
00054     ESP_LOGI(TAG, "MQ135 inicializado");
00055 }
00056
00057 // =====
00058 // READ RAW (ADC)
00059 // =====
00060
00068 int mq135_read_raw(void)
00069 {
00070     int raw = 0;
00071
00072     float voltage = adc_manager_read_voltage(MQ135_ADC_CHANNEL);
00073
00074     raw = (int)((voltage / 3.3) * 4095);
00075
00076     ESP_LOGI(TAG, "RAW estimado:%d", raw);
00077
00078     return raw;
00079 }
00080
00081 // =====
00082 // READ VOLTAGE
00083 // =====
00084
00092 float mq135_read_voltage(void)
00093 {
00094     float sum = 0;
00095
00096     for (int i = 0; i < NUM_SAMPLES; i++) {
00097         sum += adc_manager_read_voltage(MQ135_ADC_CHANNEL);
00098     }
00099
00100     float avg = sum / NUM_SAMPLES;
00101
00102     ESP_LOGI(TAG, "Voltaje: %.4f V", avg);
00103
00104     return avg;
00105 }
00106
00107 // =====
00108 // READ (GENERAL)
00109 // =====
00110
00116 float mq135_read(void)
00117 {
00118     return mq135_read_voltage();
00119 }

```

6.9. Referencia del archivo main/drivers/mq135.h

Driver para adquisición de la señal analógica del sensor MQ135.

Funciones

- void `mq135_init` (void)

Inicializa el módulo MQ135.

- int `mq135_read_raw` (void)

Obtiene una lectura cruda del sensor.

- float `mq135_read_voltage` (void)

Obtiene el voltaje promedio de salida del sensor.

- float `mq135_read` (void)

Obtiene una lectura representativa del sensor MQ135.

6.9.1. Descripción detallada

Driver para adquisición de la señal analógica del sensor MQ135.

Este módulo permite adquirir la señal de salida del sensor MQ135 mediante el ADC del ESP32, obteniendo lecturas en bruto y valores convertidos a voltaje.

La conversión a concentraciones específicas de gases (ppm) no se encuentra implementada en este módulo.

Autor

Fernando Plazas Trujillo Isabella Ordoñez Juan Daniel Constain

Definición en el archivo [mq135.h](#).

6.9.2. Documentación de funciones

6.9.2.1. `mq135_init()`

```
void mq135_init (  
    void )
```

Inicializa el módulo MQ135.

Inicializa el sensor MQ135.

Realiza la configuración necesaria para la adquisición de datos desde el canal ADC asociado al sensor.

Esta función debe ejecutarse durante la inicialización del sistema antes de realizar lecturas.

Inicializa el módulo MQ135.

Definición en la línea 52 del archivo [mq135.c](#).

```
00053 {  
00054     ESP_LOGI(TAG, "MQ135 inicializado");  
00055 }
```

6.9.2.2. mq135_read()

```
float mq135_read (
    void )
```

Obtiene una lectura representativa del sensor MQ135.

Función principal de lectura del sensor MQ135.

Esta función actúa como interfaz principal del módulo y proporciona una medición simplificada para su utilización por parte de las tareas del sistema.

Actualmente retorna el voltaje promedio de salida del sensor.

Devuelve

Valor medido por el sensor.

Obtiene una lectura representativa del sensor MQ135.

Devuelve

Voltaje del sensor.

Definición en la línea 116 del archivo [mq135.c](#).

```
00117 {
00118     return mq135_read_voltage();
00119 }
```

6.9.2.3. mq135_read_raw()

```
int mq135_read_raw (
    void )
```

Obtiene una lectura cruda del sensor.

Obtiene un valor crudo estimado del ADC.

Realiza una conversión ADC y devuelve el valor digital correspondiente sin aplicar escalamiento adicional.

Devuelve

Valor digital del ADC.

Obtiene una lectura cruda del sensor.

Convierte el voltaje leído a una escala de 12 bits (0–4095).

Devuelve

Valor ADC estimado.

Definición en la línea 68 del archivo [mq135.c](#).

```
00069 {
00070     int raw = 0;
00071
00072     float voltage = adc_manager_read_voltage(MQ135_ADC_CHANNEL);
00073
00074     raw = (int)((voltage / 3.3) * 4095);
00075
00076     ESP_LOGI(TAG, "RAW estimado: %d", raw);
00077
00078     return raw;
00079 }
```

6.9.2.4. mq135_read_voltage()

```
float mq135_read_voltage (
    void )
```

Obtiene el voltaje promedio de salida del sensor.

Lee el voltaje promedio del sensor.

Se realizan múltiples conversiones ADC y posteriormente se calcula un promedio para reducir el efecto del ruido presente en la señal analógica.

Devuelve

Voltaje promedio en voltios.

Obtiene el voltaje promedio de salida del sensor.

Realiza múltiples lecturas para reducir ruido.

Devuelve

Voltaje promedio en voltios.

Definición en la línea [92](#) del archivo [mq135.c](#).

```
00093 {
00094     float sum = 0;
00095
00096     for (int i = 0; i < NUM_SAMPLES; i++) {
00097         sum += adc_manager_read_voltage(MQ135_ADC_CHANNEL);
00098     }
00099
00100     float avg = sum / NUM_SAMPLES;
00101
00102     ESP_LOGI(TAG, "Voltaje: %.4f V", avg);
00103
00104     return avg;
00105 }
```

6.10. mq135.h

[Ir a la documentación de este archivo.](#)

```
00001
00017
00018 #ifndef MQ135_H
00019 #define MQ135_H
00020
00030 void mq135_init(void);
00031
00040 int mq135_read_raw(void);
00041
00051 float mq135_read_voltage(void);
00052
00064 float mq135_read(void);
00065
00066 #endif
```


6.11. Referencia del archivo main/drivers/oled_display.c

Implementación del controlador OLED SSD1306 mediante interfaz I2C.

```
#include "drivers/oled_display.h"
#include <string.h>
#include "hal/i2c_manager.h"
#include "freertos/semphr.h"
#include "font5x7.h"
```

defines

- `#define OLED_ADDR 0x3C`
Dirección I2C del controlador SSD1306.
- `#define OLED_WIDTH 128`
Ancho de la pantalla OLED en píxeles.
- `#define OLED_HEIGHT 64`
Alto de la pantalla OLED en píxeles.
- `#define OLED_PAGES 8`
Número de páginas de memoria del SSD1306.

Funciones

- `static esp_err_t oled_send_command (uint8_t cmd)`
Envía un comando al controlador SSD1306.
- `static esp_err_t oled_send_data (const uint8_t *data, size_t len)`
Envía datos gráficos al SSD1306.
- `esp_err_t oled_init (void)`
Inicializa la pantalla OLED.
- `esp_err_t oled_clear (void)`
Borra el contenido del framebuffer.
- `esp_err_t oled_draw_pixel (uint8_t x, uint8_t y)`
Dibuja un píxel en el framebuffer.
- `esp_err_t oled_draw_char (uint8_t x, uint8_t y, char c)`
Dibuja un carácter ASCII en el framebuffer.
- `esp_err_t oled_draw_string (uint8_t x, uint8_t y, const char *str)`
Dibuja una cadena de texto en el framebuffer.
- `esp_err_t oled_update (void)`
Actualiza la pantalla OLED.

Variables

- `SemaphoreHandle_t mutex_i2c`
Mutex compartido para acceso exclusivo al bus I2C.
- `static uint8_t framebuffer [1024]`
Framebuffer local de la pantalla OLED.

6.11.1. Descripción detallada

Implementación del controlador OLED SSD1306 mediante interfaz I2C.

Este módulo implementa las funciones necesarias para controlar una pantalla OLED basada en el controlador SSD1306.

Funcionalidades:

- Inicialización del controlador.
- Gestión de framebuffer local.
- Dibujo de píxeles.
- Renderizado de caracteres.
- Renderizado de cadenas de texto.
- Actualización de pantalla mediante I2C.

El acceso al bus I2C es protegido mediante un mutex compartido para garantizar la exclusión mutua entre tareas FreeRTOS.

Autor

Fernando Plazas Trujillo Isabella Ordoñez Juan Daniel Constain

Definición en el archivo [oled_display.c](#).

6.11.2. Documentación de «define»

6.11.2.1. OLED_ADDR

```
#define OLED_ADDR 0x3C
```

Dirección I2C del controlador SSD1306.

Definición en la línea 36 del archivo [oled_display.c](#).

6.11.2.2. OLED_HEIGHT

```
#define OLED_HEIGHT 64
```

Alto de la pantalla OLED en píxeles.

Definición en la línea 46 del archivo [oled_display.c](#).

6.11.2.3. OLED_PAGES

```
#define OLED_PAGES 8
```

Número de páginas de memoria del SSD1306.

Cada página contiene 8 filas de píxeles.

Definición en la línea 53 del archivo [oled_display.c](#).

6.11.2.4. OLED_WIDTH

```
#define OLED_WIDTH 128
```

Ancho de la pantalla OLED en píxeles.

Definición en la línea 41 del archivo [oled_display.c](#).

6.11.3. Documentación de funciones

6.11.3.1. oled_clear()

```
esp_err_t oled_clear (  
    void )
```

Borra el contenido del framebuffer.

Establece todos los píxeles en estado apagado.

Es necesario llamar posteriormente a [oled_update\(\)](#) para reflejar los cambios en la pantalla física.

Devuelve

- ESP_OK si la operación fue exitosa.
- Código de error en caso contrario.

Definición en la línea 200 del archivo [oled_display.c](#).

```
00201 {  
00202     memset (  
00203         framebuffer,  
00204         0,  
00205         sizeof(framebuffer));  
00206  
00207     return oled_update();  
00208 }
```

6.11.3.2. oled_draw_char()

```
esp_err_t oled_draw_char (
    uint8_t x,
    uint8_t y,
    char c)
```

Dibuja un carácter ASCII en el framebuffer.

Utiliza la fuente bitmap definida en [font5x7.h](#) para representar el carácter solicitado.

Parámetros

<i>x</i>	Coordenada horizontal inicial.
<i>y</i>	Coordenada vertical inicial.
<i>c</i>	Carácter ASCII a representar.

Devuelve

- ESP_OK si la operación fue exitosa.
- Código de error en caso contrario.

Definición en la línea 230 del archivo [oled_display.c](#).

```
00231 {
00232     if ((uint8_t)c >= sizeof(font5x7)/5)
00233         return ESP_FAIL;
00234
00235     for (int col = 0; col < 5; col++)
00236     {
00237         uint8_t line = font5x7[(uint8_t)c][col];
00238
00239         for (int row = 0; row < 7; row++)
00240         {
00241             if (line & (1 << row))
00242             {
00243                 oled_draw_pixel(
00244                     x + col,
00245                     y + row);
00246             }
00247         }
00248     }
00249
00250     return ESP_OK;
00251 }
```

6.11.3.3. oled_draw_pixel()

```
esp_err_t oled_draw_pixel (
    uint8_t x,
    uint8_t y)
```

Dibuja un píxel en el framebuffer.

Activa el píxel correspondiente a las coordenadas especificadas.

Parámetros

<i>x</i>	Coordenada horizontal.
----------	------------------------

<i>y</i>	Coordenada vertical.
----------	----------------------

Devuelve

- ESP_OK si el píxel fue dibujado correctamente.
- ESP_ERR_INVALID_ARG si las coordenadas están fuera de rango.

Definición en la línea 214 del archivo [oled_display.c](#).

```

00215 {
00216     if (x >= OLED_WIDTH)
00217         return ESP_FAIL;
00218
00219     if (y >= OLED_HEIGHT)
00220         return ESP_FAIL;
00221
00222     framebuffer[
00223         x + (y / 8) * OLED_WIDTH
00224     ] |= (1 << (y % 8));
00225
00226     return ESP_OK;
00227 }
```

6.11.3.4. oled_draw_string()

```

esp_err_t oled_draw_string (
    uint8_t x,
    uint8_t y,
    const char * str)
```

Dibuja una cadena de texto en el framebuffer.

Renderiza secuencialmente cada carácter utilizando la fuente configurada por el controlador.

Parámetros

<i>x</i>	Coordenada horizontal inicial.
<i>y</i>	Coordenada vertical inicial.
<i>str</i>	Cadena de caracteres terminada en NULL.

Devuelve

- ESP_OK si la operación fue exitosa.
- Código de error en caso contrario.

Definición en la línea 253 del archivo [oled_display.c](#).

```

00257 {
00258     while (*str)
00259     {
00260         oled_draw_char(
00261             x,
00262             y,
00263             *str);
00264
00265         x += 6;
00266         str++;
00267     }
00268
00269     return ESP_OK;
00270 }
```

6.11.3.5. oled_init()

```
esp_err_t oled_init (
    void )
```

Inicializa la pantalla OLED.

Configura el controlador SSD1306 y prepara el framebuffer para operaciones de dibujo.

Esta función debe ejecutarse una única vez durante el arranque del sistema.

Devuelve

- ESP_OK si la inicialización fue exitosa.
- Código de error en caso contrario.

Definición en la línea 148 del archivo `oled_display.c`.

```
00149 {
00150     oled_send_command(0xAE);
00151
00152     oled_send_command(0x20);
00153     oled_send_command(0x00);
00154
00155     oled_send_command(0xB0);
00156
00157     oled_send_command(0xC8);
00158
00159     oled_send_command(0x00);
00160     oled_send_command(0x10);
00161
00162     oled_send_command(0x40);
00163
00164     oled_send_command(0x81);
00165     oled_send_command(0xFF);
00166
00167     oled_send_command(0xA1);
00168
00169     oled_send_command(0xA6);
00170
00171     oled_send_command(0xA8);
00172     oled_send_command(0x3F);
00173
00174     oled_send_command(0xA4);
00175
00176     oled_send_command(0xD3);
00177     oled_send_command(0x00);
00178
00179     oled_send_command(0xD5);
00180     oled_send_command(0xF0);
00181
00182     oled_send_command(0xD9);
00183     oled_send_command(0x22);
00184
00185     oled_send_command(0xDA);
00186     oled_send_command(0x12);
00187
00188     oled_send_command(0xDB);
00189     oled_send_command(0x20);
00190
00191     oled_send_command(0x8D);
00192     oled_send_command(0x14);
00193
00194     oled_send_command(0xAF);
00195
00196     return ESP_OK;
00197 }
```

6.11.3.6. oled_send_command()

```
esp_err_t oled_send_command (
    uint8_t cmd) [static]
```

Envía un comando al controlador SSD1306.

La transmisión se realiza mediante I2C utilizando exclusión mutua sobre el bus compartido.

Parámetros

<i>cmd</i>	Comando a transmitir.
------------	-----------------------

Devuelve

- ESP_OK si la transmisión fue exitosa.
- ESP_FAIL si no fue posible acceder al bus.

Definición en la línea [84](#) del archivo [oled_display.c](#).

```
00085 {
00086     uint8_t buffer[2];
00087
00088     buffer[0] = 0x00;
00089     buffer[1] = cmd;
00090
00091     if (xSemaphoreTake(mutex_i2c, pdMS_TO_TICKS(100)))
00092     {
00093         esp_err_t ret =
00094             i2c_manager_write_raw(
00095                 OLED_ADDR,
00096                 buffer,
00097                 sizeof(buffer));
00098
00099         xSemaphoreGive(mutex_i2c);
00100
00101         return ret;
00102     }
00103
00104     return ESP_FAIL;
00105 }
```

6.11.3.7. oled_send_data()

```
esp_err_t oled_send_data (
    const uint8_t * data,
    size_t len) [static]
```

Envía datos gráficos al SSD1306.

Copia los datos al buffer de transmisión I2C y posteriormente los envía al controlador OLED.

Parámetros

<i>data</i>	Puntero al bloque de datos.
<i>len</i>	Cantidad de bytes a transmitir.

Devuelve

- ESP_OK si la transmisión fue exitosa.
- ESP_FAIL en caso de error.

Definición en la línea 120 del archivo `oled_display.c`.

```

00121 {
00122     uint8_t buffer[129];
00123
00124     buffer[0] = 0x40;
00125
00126     memcpy(&buffer[1], data, len);
00127
00128     if (xSemaphoreTake(mutex_i2c, pdMS_TO_TICKS(100)))
00129     {
00130         esp_err_t ret =
00131             i2c_manager_write_raw(
00132                 OLED_ADDR,
00133                 buffer,
00134                 len + 1);
00135
00136         xSemaphoreGive(mutex_i2c);
00137
00138         return ret;
00139     }
00140
00141     return ESP_FAIL;
00142 }
```

6.11.3.8. oled_update()

```

esp_err_t oled_update (
    void )
```

Actualiza la pantalla OLED.

Transfiere el contenido actual del framebuffer hacia la pantalla mediante el bus I2C.

Debe ejecutarse después de realizar operaciones de dibujo para que los cambios sean visibles.

Devuelve

- ESP_OK si la actualización fue exitosa.
- Código de error en caso contrario.

Definición en la línea 272 del archivo `oled_display.c`.

```

00273 {
00274     for (int page = 0;
00275          page < OLED_PAGES;
00276          page++)
00277     {
00278         oled_send_command(
00279             0xB0 + page);
00280
00281         oled_send_command(0x00);
00282         oled_send_command(0x10);
00283
00284         oled_send_data(
00285             &framebuffer[
00286                 page * OLED_WIDTH],
00287             OLED_WIDTH);
00288     }
00289
00290     return ESP_OK;
00291 }
```


6.11.4. Documentación de variables

6.11.4.1. framebuffer

```
uint8_t framebuffer[1024] [static]
```

Framebuffer local de la pantalla OLED.

Almacena una copia completa de la memoria gráfica del SSD1306. Los cambios gráficos se realizan sobre este buffer y posteriormente se transfieren al dispositivo mediante `oled_update()`.

Definición en la línea 70 del archivo `oled_display.c`.

6.11.4.2. mutex_i2c

```
SemaphoreHandle_t mutex_i2c [extern]
```

Mutex compartido para acceso exclusivo al bus I2C.

Mutex global para acceso seguro al bus I2C.

Utilizado por todos los periféricos que comparten el mismo bus, evitando condiciones de carrera entre tareas concurrentes.

Mutex compartido para acceso exclusivo al bus I2C.

Garantiza exclusión mutua cuando múltiples tareas intentan acceder simultáneamente a dispositivos conectados al mismo bus.

Definición en la línea 66 del archivo `i2c_manager.c`.

6.12. oled_display.c

[Ir a la documentación de este archivo.](#)

```
00001
00024 #include "drivers/oled_display.h"
00025
00026 #include <string.h>
00027
00028 #include "hal/i2c_manager.h"
00029 #include "freertos/semphr.h"
00030
00031 #include "font5x7.h"
00032
00036 #define OLED_ADDR      0x3C
00037
00041 #define OLED_WIDTH 128
00042
00046 #define OLED_HEIGHT 64
00047
00053 #define OLED_PAGES 8
00054
00061 extern SemaphoreHandle_t mutex_i2c;
00062
00070 static uint8_t framebuffer[1024];
00071
00084 static esp_err_t oled_send_command(uint8_t cmd)
00085 {
00086     uint8_t buffer[2];
00087
00088     buffer[0] = 0x00;
00089     buffer[1] = cmd;
```

```

00090
00091     if (xSemaphoreTake(mutex_i2c, pdMS_TO_TICKS(100)))
00092     {
00093         esp_err_t ret =
00094             i2c_manager_write_raw(
00095                 OLED_ADDR,
00096                 buffer,
00097                 sizeof(buffer));
00098
00099         xSemaphoreGive(mutex_i2c);
00100
00101         return ret;
00102     }
00103
00104     return ESP_FAIL;
00105 }
00106
00120 static esp_err_t oled_send_data(const uint8_t *data, size_t len)
00121 {
00122     uint8_t buffer[129];
00123
00124     buffer[0] = 0x40;
00125
00126     memcpy(&buffer[1], data, len);
00127
00128     if (xSemaphoreTake(mutex_i2c, pdMS_TO_TICKS(100)))
00129     {
00130         esp_err_t ret =
00131             i2c_manager_write_raw(
00132                 OLED_ADDR,
00133                 buffer,
00134                 len + 1);
00135
00136         xSemaphoreGive(mutex_i2c);
00137
00138         return ret;
00139     }
00140
00141     return ESP_FAIL;
00142 }
00143
00144 /*
00145  * Secuencia de inicialización recomendada por el datasheet
00146  * del controlador SSD1306.
00147  */
00148 esp_err_t oled_init(void)
00149 {
00150     oled_send_command(0xAE);
00151
00152     oled_send_command(0x20);
00153     oled_send_command(0x00);
00154
00155     oled_send_command(0xB0);
00156
00157     oled_send_command(0xC8);
00158
00159     oled_send_command(0x00);
00160     oled_send_command(0x10);
00161
00162     oled_send_command(0x40);
00163
00164     oled_send_command(0x81);
00165     oled_send_command(0xFF);
00166
00167     oled_send_command(0xA1);
00168
00169     oled_send_command(0xA6);
00170
00171     oled_send_command(0xA8);
00172     oled_send_command(0x3F);
00173
00174     oled_send_command(0xA4);
00175
00176     oled_send_command(0xD3);
00177     oled_send_command(0x00);
00178
00179     oled_send_command(0xD5);
00180     oled_send_command(0xF0);
00181
00182     oled_send_command(0xD9);
00183     oled_send_command(0x22);
00184
00185     oled_send_command(0xDA);
00186     oled_send_command(0x12);
00187
00188     oled_send_command(0xDB);
00189     oled_send_command(0x20);

```

```

00190
00191     oled_send_command(0x8D);
00192     oled_send_command(0x14);
00193
00194     oled_send_command(0xAF);
00195
00196     return ESP_OK;
00197 }
00198
00199
00200 esp_err_t oled_clear(void)
00201 {
00202     memset(
00203         framebuffer,
00204         0,
00205         sizeof(framebuffer));
00206
00207     return oled_update();
00208 }
00209
00210 /*
00211  * Conversión de coordenadas XY a posición dentro
00212  * de la memoria organizada por páginas del SSD1306.
00213  */
00214 esp_err_t oled_draw_pixel(uint8_t x, uint8_t y)
00215 {
00216     if (x >= OLED_WIDTH)
00217         return ESP_FAIL;
00218
00219     if (y >= OLED_HEIGHT)
00220         return ESP_FAIL;
00221
00222     framebuffer[
00223         x + (y / 8) * OLED_WIDTH
00224     ] |= (1 << (y % 8));
00225
00226     return ESP_OK;
00227 }
00228
00229
00230 esp_err_t oled_draw_char(uint8_t x, uint8_t y, char c)
00231 {
00232     if ((uint8_t)c >= sizeof(font5x7)/5)
00233         return ESP_FAIL;
00234
00235     for (int col = 0; col < 5; col++)
00236     {
00237         uint8_t line = font5x7[(uint8_t)c][col];
00238
00239         for (int row = 0; row < 7; row++)
00240         {
00241             if (line & (1 << row))
00242             {
00243                 oled_draw_pixel(
00244                     x + col,
00245                     y + row);
00246             }
00247         }
00248     }
00249
00250     return ESP_OK;
00251 }
00252
00253 esp_err_t oled_draw_string(
00254     uint8_t x,
00255     uint8_t y,
00256     const char *str)
00257 {
00258     while (*str)
00259     {
00260         oled_draw_char(
00261             x,
00262             y,
00263             *str);
00264
00265         x += 6;
00266         str++;
00267     }
00268
00269     return ESP_OK;
00270 }
00271
00272 esp_err_t oled_update(void)
00273 {
00274     for (int page = 0;
00275          page < OLED_PAGES;
00276          page++)

```

```
00277     {
00278         oled_send_command(
00279             0xB0 + page);
00280
00281         oled_send_command(0x00);
00282         oled_send_command(0x10);
00283
00284         oled_send_data(
00285             &framebuffer[
00286                 page * OLED_WIDTH],
00287             OLED_WIDTH);
00288     }
00289
00290     return ESP_OK;
00291 }
00292
```

6.13. Referencia del archivo main/drivers/oled_display.h

Driver para pantalla OLED SSD1306 mediante interfaz I2C.

```
#include "esp_err.h"
#include <stdint.h>
```

Funciones

- esp_err_t `oled_init` (void)
Inicializa la pantalla OLED.
- esp_err_t `oled_clear` (void)
Borra el contenido del framebuffer.
- esp_err_t `oled_update` (void)
Actualiza la pantalla OLED.
- esp_err_t `oled_draw_pixel` (uint8_t x, uint8_t y)
Dibuja un píxel en el framebuffer.
- esp_err_t `oled_draw_char` (uint8_t x, uint8_t y, char c)
Dibuja un carácter ASCII en el framebuffer.
- esp_err_t `oled_draw_string` (uint8_t x, uint8_t y, const char *str)
Dibuja una cadena de texto en el framebuffer.

6.13.1. Descripción detallada

Driver para pantalla OLED SSD1306 mediante interfaz I2C.

Este módulo proporciona funciones para controlar una pantalla OLED basada en el controlador SSD1306.

Funcionalidades principales:

- Inicialización de la pantalla.
- Limpieza del framebuffer.
- Actualización de la pantalla.
- Dibujo de píxeles.
- Renderizado de caracteres.
- Renderizado de cadenas de texto.

La pantalla es utilizada para visualizar información del proceso de fermentación, incluyendo variables ambientales, estado de los actuadores y eventos del sistema.

Autor

Fernando Plazas Trujillo Isabella Ordoñez Juan Daniel Constain

Definición en el archivo [oled_display.h](#).

6.13.2. Documentación de funciones**6.13.2.1. oled_clear()**

```
esp_err_t oled_clear (  
    void )
```

Borra el contenido del framebuffer.

Establece todos los píxeles en estado apagado.

Es necesario llamar posteriormente a [oled_update\(\)](#) para reflejar los cambios en la pantalla física.

Devuelve

- ESP_OK si la operación fue exitosa.
- Código de error en caso contrario.

Definición en la línea 200 del archivo [oled_display.c](#).

```
00201 {  
00202     memset (  
00203         framebuffer,  
00204         0,  
00205         sizeof(framebuffer));  
00206  
00207     return oled_update();  
00208 }
```

6.13.2.2. oled_draw_char()

```
esp_err_t oled_draw_char (  
    uint8_t x,  
    uint8_t y,  
    char c)
```

Dibuja un carácter ASCII en el framebuffer.

Utiliza la fuente bitmap definida en [font5x7.h](#) para representar el carácter solicitado.

Parámetros

<i>x</i>	Coordenada horizontal inicial.
<i>y</i>	Coordenada vertical inicial.
<i>c</i>	Carácter ASCII a representar.

Devuelve

- ESP_OK si la operación fue exitosa.
- Código de error en caso contrario.

Definición en la línea 230 del archivo `oled_display.c`.

```
00231 {
00232     if ((uint8_t)c >= sizeof(font5x7)/5)
00233         return ESP_FAIL;
00234
00235     for (int col = 0; col < 5; col++)
00236     {
00237         uint8_t line = font5x7[(uint8_t)c][col];
00238
00239         for (int row = 0; row < 7; row++)
00240         {
00241             if (line & (1 << row))
00242             {
00243                 oled_draw_pixel(
00244                     x + col,
00245                     y + row);
00246             }
00247         }
00248     }
00249
00250     return ESP_OK;
00251 }
```

6.13.2.3. oled_draw_pixel()

```
esp_err_t oled_draw_pixel (
    uint8_t x,
    uint8_t y)
```

Dibuja un píxel en el framebuffer.

Activa el píxel correspondiente a las coordenadas especificadas.

Parámetros

<i>x</i>	Coordenada horizontal.
<i>y</i>	Coordenada vertical.

Devuelve

- ESP_OK si el píxel fue dibujado correctamente.
- ESP_ERR_INVALID_ARG si las coordenadas están fuera de rango.

Definición en la línea 214 del archivo `oled_display.c`.

```
00215 {
00216     if (x >= OLED_WIDTH)
00217         return ESP_FAIL;
00218
00219     if (y >= OLED_HEIGHT)
00220         return ESP_FAIL;
00221
00222     framebuffer[
00223         x + (y / 8) * OLED_WIDTH
00224     ] |= (1 << (y % 8));
00225
00226     return ESP_OK;
00227 }
```

6.13.2.4. oled_draw_string()

```
esp_err_t oled_draw_string (
    uint8_t x,
    uint8_t y,
    const char * str)
```

Dibuja una cadena de texto en el framebuffer.

Renderiza secuencialmente cada carácter utilizando la fuente configurada por el controlador.

Parámetros

<i>x</i>	Coordenada horizontal inicial.
<i>y</i>	Coordenada vertical inicial.
<i>str</i>	Cadena de caracteres terminada en NULL.

Devuelve

- ESP_OK si la operación fue exitosa.
- Código de error en caso contrario.

Definición en la línea [253](#) del archivo [oled_display.c](#).

```
00257 {
00258     while (*str)
00259     {
00260         oled_draw_char(
00261             x,
00262             y,
00263             *str);
00264
00265         x += 6;
00266         str++;
00267     }
00268
00269     return ESP_OK;
00270 }
```

6.13.2.5. oled_init()

```
esp_err_t oled_init (
    void )
```

Inicializa la pantalla OLED.

Configura el controlador SSD1306 y prepara el framebuffer para operaciones de dibujo.

Esta función debe ejecutarse una única vez durante el arranque del sistema.

Devuelve

- ESP_OK si la inicialización fue exitosa.
- Código de error en caso contrario.

Definición en la línea 148 del archivo `oled_display.c`.

```
00149 {  
00150     oled_send_command(0xAE);  
00151  
00152     oled_send_command(0x20);  
00153     oled_send_command(0x00);  
00154  
00155     oled_send_command(0xB0);  
00156  
00157     oled_send_command(0xC8);  
00158  
00159     oled_send_command(0x00);  
00160     oled_send_command(0x10);  
00161  
00162     oled_send_command(0x40);  
00163  
00164     oled_send_command(0x81);  
00165     oled_send_command(0xFF);  
00166  
00167     oled_send_command(0xA1);  
00168  
00169     oled_send_command(0xA6);  
00170  
00171     oled_send_command(0xA8);  
00172     oled_send_command(0x3F);  
00173  
00174     oled_send_command(0xA4);  
00175  
00176     oled_send_command(0xD3);  
00177     oled_send_command(0x00);  
00178  
00179     oled_send_command(0xD5);  
00180     oled_send_command(0xF0);  
00181  
00182     oled_send_command(0xD9);  
00183     oled_send_command(0x22);  
00184  
00185     oled_send_command(0xDA);  
00186     oled_send_command(0x12);  
00187  
00188     oled_send_command(0xDB);  
00189     oled_send_command(0x20);  
00190  
00191     oled_send_command(0x8D);  
00192     oled_send_command(0x14);  
00193  
00194     oled_send_command(0xAF);  
00195  
00196     return ESP_OK;  
00197 }
```

6.13.2.6. oled_update()

```
esp_err_t oled_update (  
    void )
```

Actualiza la pantalla OLED.

Transfiere el contenido actual del framebuffer hacia la pantalla mediante el bus I2C.

Debe ejecutarse después de realizar operaciones de dibujo para que los cambios sean visibles.

Devuelve

- ESP_OK si la actualización fue exitosa.
- Código de error en caso contrario.

Definición en la línea 272 del archivo `oled_display.c`.

```
00273 {
00274     for (int page = 0;
00275          page < OLED_PAGES;
00276          page++)
00277     {
00278         oled_send_command(
00279             0xB0 + page);
00280
00281         oled_send_command(0x00);
00282         oled_send_command(0x10);
00283
00284         oled_send_data(
00285             &framebuffer[
00286                 page * OLED_WIDTH],
00287             OLED_WIDTH);
00288     }
00289
00290     return ESP_OK;
00291 }
```

6.14. oled_display.h

[Ir a la documentación de este archivo.](#)

```
00001
00025 #ifndef OLED_DISPLAY_H
00026 #define OLED_DISPLAY_H
00027
00028 #include "esp_err.h"
00029 #include <stdint.h>
00030
00044 esp_err_t oled_init(void);
00045
00058 esp_err_t oled_clear(void);
00059
00073 esp_err_t oled_update(void);
00074
00088 esp_err_t oled_draw_pixel(uint8_t x, uint8_t y);
00089
00104 esp_err_t oled_draw_char(uint8_t x, uint8_t y, char c);
00105
00120 esp_err_t oled_draw_string(uint8_t x, uint8_t y, const char *str);
00121
00122 #endif
```

6.15. Referencia del archivo main/drivers/ph_sensor.c

Implementación del driver para sensor de pH basado en adquisición analógica.

```
#include "ph_sensor.h"
#include "hal/adc_manager.h"
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "esp_log.h"
```

defines

- `#define PH_ADC_CHANNEL ADC_CHANNEL_3`
Canal ADC asociado al sensor de pH.
- `#define NUM_SAMPLES 20`
Cantidad de muestras utilizadas para el cálculo del promedio.
- `#define PH_SLOPE (-5.7f)`
Pendiente de la ecuación de calibración.
- `#define PH_OFFSET (21.34f)`
Offset de la ecuación de calibración.
- `#define PH_MIN_VOLTAGE (0.1f)`
Voltaje mínimo válido para el sensor.
- `#define PH_MAX_VOLTAGE (3.2f)`
Voltaje máximo válido para el sensor.

Funciones

- `void ph_init (void)`
Inicializa el módulo de pH.
- `float ph_read_voltage (void)`
Obtiene el voltaje promedio del sensor de pH.
- `float ph_read (void)`
Calcula el valor de pH a partir del voltaje medido.

Variables

- `static const char * TAG = "PH"`
Etiqueta utilizada por el sistema de logging.

6.15.1. Descripción detallada

Implementación del driver para sensor de pH basado en adquisición analógica.

Este módulo obtiene mediciones analógicas desde un sensor de pH, calcula un voltaje promedio mediante múltiples muestras y convierte dicho voltaje a unidades de pH utilizando una ecuación de calibración lineal.

Funcionalidades:

- Lectura de voltaje.
- Filtrado mediante promediado.
- Conversión voltaje-pH.
- Validación básica de rango de operación.

Autor

Fernando Plazas Trujillo Isabella Ordoñez Juan Daniel Constain

Definición en el archivo `ph_sensor.c`.

6.15.2. Documentación de «define»

6.15.2.1. NUM_SAMPLES

```
#define NUM_SAMPLES 20
```

Cantidad de muestras utilizadas para el cálculo del promedio.

El promediado reduce el ruido presente en la señal analógica.

Definición en la línea 39 del archivo [ph_sensor.c](#).

6.15.2.2. PH_ADC_CHANNEL

```
#define PH_ADC_CHANNEL ADC_CHANNEL_3
```

Canal ADC asociado al sensor de pH.

Corresponde al GPIO39 del ESP32.

Definición en la línea 32 del archivo [ph_sensor.c](#).

6.15.2.3. PH_MAX_VOLTAGE

```
#define PH_MAX_VOLTAGE (3.2f)
```

Voltaje máximo válido para el sensor.

Definición en la línea 64 del archivo [ph_sensor.c](#).

6.15.2.4. PH_MIN_VOLTAGE

```
#define PH_MIN_VOLTAGE (0.1f)
```

Voltaje mínimo válido para el sensor.

Definición en la línea 59 del archivo [ph_sensor.c](#).

6.15.2.5. PH_OFFSET

```
#define PH_OFFSET (21.34f)
```

Offset de la ecuación de calibración.

Corresponde al término independiente obtenido durante el proceso de calibración experimental.

Definición en la línea 54 del archivo [ph_sensor.c](#).

6.15.2.6. PH_SLOPE

```
#define PH_SLOPE (-5.7f)
```

Pendiente de la ecuación de calibración.

Relaciona el voltaje medido con el valor de pH.

Definición en la línea 46 del archivo [ph_sensor.c](#).

6.15.3. Documentación de funciones

6.15.3.1. ph_init()

```
void ph_init (  
    void )
```

Inicializa el módulo de pH.

Inicializa el módulo del sensor de pH.

Actualmente no requiere configuración específica, ya que la adquisición ADC es gestionada por `adc_manager`.

Inicializa el módulo de pH.

Realiza la configuración necesaria para habilitar la adquisición de señales analógicas asociadas al sensor.

Debe ejecutarse una única vez durante la inicialización del sistema.

Definición en la línea 81 del archivo [ph_sensor.c](#).

```
00082 {  
00083 }
```

6.15.3.2. ph_read()

```
float ph_read (  
    void )
```

Calcula el valor de pH a partir del voltaje medido.

Obtiene una medición de pH.

Utiliza una ecuación de calibración lineal:

$$\text{pH} = (\text{PH_SLOPE} * V) + \text{PH_OFFSET}$$

donde:

- V corresponde al voltaje medido.
- PH_SLOPE es la pendiente de calibración.
- PH_OFFSET es el offset de calibración.

Devuelve

Valor estimado de pH.

Valores devueltos

-	Lectura inválida o fuera de rango.
1.↔	
0	

Calcula el valor de pH a partir del voltaje medido.

Convierte el voltaje medido por el sensor a una estimación de pH utilizando la ecuación de calibración definida para el sistema.

Devuelve

Valor estimado de pH.

Valores devueltos

-	Lectura inválida o fuera de rango.
1.↔	
0	

Definición en la línea 134 del archivo [ph_sensor.c](#).

```

00135 {
00136     float voltage = ph_read_voltage();
00137
00138     if (voltage > PH_MAX_VOLTAGE ||
00139         voltage < PH_MIN_VOLTAGE)
00140     {
00141         ESP_LOGW(TAG, "Sensor pH fuera de rango");
00142         return -1.0;
00143     }
00144
00145     /*
00146      * Conversión lineal obtenida durante el proceso
00147      * de calibración del sensor.
00148      */
00149     return PH_SLOPE * voltage + PH_OFFSET;
00150 }
```

6.15.3.3. ph_read_voltage()

```

float ph_read_voltage (
    void )
```

Obtiene el voltaje promedio del sensor de pH.

Obtiene el voltaje de salida del sensor.

Realiza múltiples lecturas ADC y calcula un promedio para reducir el efecto del ruido presente en la señal.

Devuelve

Voltaje promedio en voltios.

Obtiene el voltaje promedio del sensor de pH.

Realiza múltiples conversiones ADC y calcula un valor promedio para reducir el efecto del ruido presente en la señal analógica.

Devuelve

Voltaje promedio en voltios.

Definición en la línea 97 del archivo [ph_sensor.c](#).

```
00098 {
00099     float sum = 0;
00100
00101     for (int i = 0; i < NUM_SAMPLES; i++) {
00102         sum += adc_manager_read_voltage(PH_ADC_CHANNEL);
00103
00104         /*
00105          * Se introduce un pequeño retardo entre muestras
00106          * para evitar lecturas consecutivas excesivamente
00107          * correlacionadas y mejorar el promedio obtenido.
00108          */
00109         vTaskDelay(pdMS_TO_TICKS(10));
00110     }
00111
00112     return sum / NUM_SAMPLES;
00113 }
```

6.15.4. Documentación de variables

6.15.4.1. TAG

```
const char* TAG = "PH" [static]
```

Etiqueta utilizada por el sistema de logging.

Definición en la línea 69 del archivo [ph_sensor.c](#).

6.16. ph_sensor.c

[Ir a la documentación de este archivo.](#)

```
00001
00020
00021 #include "ph_sensor.h"
00022 #include "hal/adc_manager.h"
00023 #include "freertos/FreeRTOS.h"
00024 #include "freertos/task.h"
00025 #include "esp_log.h"
00026
00032 #define PH_ADC_CHANNEL ADC_CHANNEL_3
00033
00039 #define NUM_SAMPLES 20
00040
00046 #define PH_SLOPE (-5.7f)
00047
00054 #define PH_OFFSET (21.34f)
00055
00059 #define PH_MIN_VOLTAGE (0.1f)
00060
00064 #define PH_MAX_VOLTAGE (3.2f)
00065
```

```

00069 static const char *TAG = "PH";
00070
00071 // =====
00072 // INIT
00073 // =====
00074
00081 void ph_init(void)
00082 {
00083 }
00084
00085 // =====
00086 // READ VOLTAGE
00087 // =====
00088
00097 float ph_read_voltage(void)
00098 {
00099     float sum = 0;
00100
00101     for (int i = 0; i < NUM_SAMPLES; i++) {
00102         sum += adc_manager_read_voltage(PH_ADC_CHANNEL);
00103
00104         /*
00105          * Se introduce un pequeño retardo entre muestras
00106          * para evitar lecturas consecutivas excesivamente
00107          * correlacionadas y mejorar el promedio obtenido.
00108          */
00109         vTaskDelay(pdMS_TO_TICKS(10));
00110     }
00111
00112     return sum / NUM_SAMPLES;
00113 }
00114
00115 // =====
00116 // READ PH
00117 // =====
00118
00134 float ph_read(void)
00135 {
00136     float voltage = ph_read_voltage();
00137
00138     if (voltage > PH_MAX_VOLTAGE ||
00139         voltage < PH_MIN_VOLTAGE)
00140     {
00141         ESP_LOGW(TAG, "Sensor pH fuera de rango");
00142         return -1.0;
00143     }
00144
00145     /*
00146     * Conversión lineal obtenida durante el proceso
00147     * de calibración del sensor.
00148     */
00149     return PH_SLOPE * voltage + PH_OFFSET;
00150 }

```

6.17. Referencia del archivo main/drivers/ph_sensor.h

Driver para sensor de pH basado en adquisición analógica.

Funciones

- void `ph_init` (void)
Inicializa el módulo del sensor de pH.
- float `ph_read_voltage` (void)
Obtiene el voltaje de salida del sensor.
- float `ph_read` (void)
Obtiene una medición de pH.

6.17.1. Descripción detallada

Driver para sensor de pH basado en adquisición analógica.

Este módulo permite obtener mediciones provenientes de un sensor de pH conectado al ADC del ESP32.

Funcionalidades:

- Lectura de voltaje.
- Conversión de voltaje a unidades de pH.
- Validación básica de rangos.

El sensor es utilizado para monitorear las condiciones químicas del proceso de fermentación de café.

Autor

Fernando Plazas Trujillo Isabella Ordoñez Juan Daniel Constain

Definición en el archivo [ph_sensor.h](#).

6.17.2. Documentación de funciones

6.17.2.1. `ph_init()`

```
void ph_init (
    void )
```

Inicializa el módulo del sensor de pH.

Inicializa el módulo de pH.

Realiza la configuración necesaria para habilitar la adquisición de señales analógicas asociadas al sensor.

Debe ejecutarse una única vez durante la inicialización del sistema.

Inicializa el módulo del sensor de pH.

Actualmente no requiere configuración específica, ya que la adquisición ADC es gestionada por `adc_manager`.

Definición en la línea 81 del archivo [ph_sensor.c](#).

```
00082 {
00083 }
```


6.17.2.2. ph_read()

```
float ph_read (
    void )
```

Obtiene una medición de pH.

Calcula el valor de pH a partir del voltaje medido.

Convierte el voltaje medido por el sensor a una estimación de pH utilizando la ecuación de calibración definida para el sistema.

Devuelve

Valor estimado de pH.

Valores devueltos

-	Lectura inválida o fuera de rango.
1.↔	
0	

Obtiene una medición de pH.

Utiliza una ecuación de calibración lineal:

$$\text{pH} = (\text{PH_SLOPE} * V) + \text{PH_OFFSET}$$

donde:

- V corresponde al voltaje medido.
- PH_SLOPE es la pendiente de calibración.
- PH_OFFSET es el offset de calibración.

Devuelve

Valor estimado de pH.

Valores devueltos

-	Lectura inválida o fuera de rango.
1.↔	
0	

Definición en la línea 134 del archivo [ph_sensor.c](#).

```
00135 {
00136     float voltage = ph_read_voltage();
00137
00138     if (voltage > PH_MAX_VOLTAGE ||
00139         voltage < PH_MIN_VOLTAGE)
00140     {
00141         ESP_LOGW(TAG, "Sensor pH fuera de rango");
00142         return -1.0;
00143     }
00144
00145     /*
00146     * Conversión lineal obtenida durante el proceso
00147     * de calibración del sensor.
00148     */
00149     return PH_SLOPE * voltage + PH_OFFSET;
00150 }
```

6.17.2.3. ph_read_voltage()

```
float ph_read_voltage (
    void )
```

Obtiene el voltaje de salida del sensor.

Obtiene el voltaje promedio del sensor de pH.

Realiza múltiples conversiones ADC y calcula un valor promedio para reducir el efecto del ruido presente en la señal analógica.

Devuelve

Voltaje promedio en voltios.

Obtiene el voltaje de salida del sensor.

Realiza múltiples lecturas ADC y calcula un promedio para reducir el efecto del ruido presente en la señal.

Devuelve

Voltaje promedio en voltios.

Definición en la línea 97 del archivo [ph_sensor.c](#).

```
00098 {
00099     float sum = 0;
00100
00101     for (int i = 0; i < NUM_SAMPLES; i++) {
00102         sum += adc_manager_read_voltage(PH_ADC_CHANNEL);
00103
00104         /*
00105          * Se introduce un pequeño retardo entre muestras
00106          * para evitar lecturas consecutivas excesivamente
00107          * correlacionadas y mejorar el promedio obtenido.
00108          */
00109         vTaskDelay(pdMS_TO_TICKS(10));
00110     }
00111
00112     return sum / NUM_SAMPLES;
00113 }
```

6.18. ph_sensor.h

[Ir a la documentación de este archivo.](#)

```
00001
00021 #ifndef PH_SENSOR_H
00022 #define PH_SENSOR_H
00023
00033 void ph_init(void);
00034
00044 float ph_read_voltage(void);
00045
00056 float ph_read(void);
00057
00058 #endif
```

6.19. Referencia del archivo main/drivers/rtc_ds3231.c

Implementación del driver para el RTC DS3231.

```
#include "rtc_ds3231.h"
#include "hal/i2c_manager.h"
#include "freertos/FreeRTOS.h"
#include "freertos/semphr.h"
#include "esp_log.h"
```

defines

- #define [DS3231_ADDR](#) 0x68
Dirección I2C del RTC DS3231.
- #define [DS3231_CONTROL_A1_INTERRUPT](#) 0x05
Valor de configuración del registro de control.

Funciones

- esp_err_t [ds3231_init](#) (void)
Inicializa el módulo RTC DS3231.
- static uint8_t [bcd_to_dec](#) (uint8_t val)
Convierte un valor codificado en BCD a decimal.
- static uint8_t [dec_to_bcd](#) (uint8_t val)
Convierte un valor decimal a formato BCD.
- esp_err_t [ds3231_get_datetime](#) (datetime_t *dt)
Lee la fecha y hora actual desde el DS3231.
- esp_err_t [ds3231_clear_alarm_flag](#) (void)
Limpia el flag de alarma del DS3231.
- esp_err_t [ds3231_set_alarm_seconds](#) (uint8_t seconds)
Configura una alarma basada en coincidencia de segundos.

Variables

- SemaphoreHandle_t [mutex_i2c](#)
Mutex compartido para acceso exclusivo al bus I2C.
- static const char * [TAG](#) = "DS3231"
Etiqueta utilizada por el sistema de logging.

6.19.1. Descripción detallada

Implementación del driver para el RTC DS3231.

Maneja la comunicación I2C para lectura de fecha/hora, configuración de alarmas y control del dispositivo.

Funcionalidades:

- Lectura de fecha y hora.
- Configuración de alarmas.
- Limpieza de flags de alarma.
- Conversión entre formato BCD y decimal.

El RTC DS3231 actúa como referencia temporal del sistema de fermentación y permite la generación de eventos periódicos mediante interrupciones configuradas por alarma.

Autor

Fernando Plazas Trujillo Isabella Ordoñez Juan Daniel Constain

Definición en el archivo [rtc_ds3231.c](#).

6.19.2. Documentación de «define»

6.19.2.1. DS3231_ADDR

```
#define DS3231_ADDR 0x68
```

Dirección I2C del RTC DS3231.

Definición en la línea [35](#) del archivo [rtc_ds3231.c](#).

6.19.2.2. DS3231_CONTROL_A1_INTERRUPT

```
#define DS3231_CONTROL_A1_INTERRUPT 0x05
```

Valor de configuración del registro de control.

INTCN = 1 -> Modo interrupción. A1IE = 1 -> Habilita interrupción de Alarm 1.

Definición en la línea [43](#) del archivo [rtc_ds3231.c](#).

6.19.3. Documentación de funciones

6.19.3.1. bcd_to_dec()

```
uint8_t bcd_to_dec (  
    uint8_t val) [static]
```

Convierte un valor codificado en BCD a decimal.

El DS3231 almacena los valores de fecha y hora en formato Binary Coded Decimal (BCD), por lo que es necesario convertirlos antes de utilizarlos en la aplicación.

Parámetros

<i>val</i>	Valor en formato BCD.
------------	-----------------------

Devuelve

Valor equivalente en decimal.

Definición en la línea 92 del archivo [rtc_ds3231.c](#).

```
00093 {  
00094     return (val >> 4) * 10 + (val & 0x0F);  
00095 }
```

6.19.3.2. dec_to_bcd()

```
uint8_t dec_to_bcd (  
    uint8_t val) [static]
```

Convierte un valor decimal a formato BCD.

Utilizado para escribir configuraciones de tiempo y alarmas en los registros internos del DS3231.

Parámetros

<i>val</i>	Valor decimal.
------------	----------------

Devuelve

Valor equivalente en formato BCD.

Definición en la línea 107 del archivo [rtc_ds3231.c](#).

```
00108 {  
00109     return ((val / 10) << 4) | (val % 10);  
00110 }
```

6.19.3.3. ds3231_clear_alarm_flag()

```
esp_err_t ds3231_clear_alarm_flag (
    void )
```

Limpia el flag de alarma del DS3231.

Limpia el indicador de alarma del DS3231.

Borra el bit A1F del registro de estado para permitir futuras interrupciones de alarma.

Devuelve

- ESP_OK: Operación exitosa.
- ESP_ERR_TIMEOUT: No fue posible obtener el mutex I2C.

Limpia el flag de alarma del DS3231.

Borra el flag de alarma almacenado en el registro de estado del dispositivo, permitiendo la generación de nuevas interrupciones.

Devuelve

- ESP_OK: Operación realizada correctamente.
- Código de error en caso de fallo.

Definición en la línea 169 del archivo [rtc_ds3231.c](#).

```
00170 {
00171     uint8_t status;
00172
00173     if (xSemaphoreTake(mutex_i2c, pdMS_TO_TICKS(100)) != pdTRUE) {
00174         return ESP_ERR_TIMEOUT;
00175     }
00176
00177     i2c_manager_read(DS3231_ADDR, 0x0F, &status, 1);
00178
00179     status &= ~0x01;
00180
00181     i2c_manager_write(DS3231_ADDR, 0x0F, &status, 1);
00182
00183     xSemaphoreGive(mutex_i2c);
00184
00185     return ESP_OK;
00186 }
```

6.19.3.4. ds3231_get_datetime()

```
esp_err_t ds3231_get_datetime (
    datetime_t * dt)
```

Lee la fecha y hora actual desde el DS3231.

Obtiene la fecha y hora actual del RTC.

Lee los registros internos del dispositivo y convierte los valores almacenados en formato BCD a formato decimal.

Parámetros

<i>dt</i>	Puntero a la estructura donde se almacenará la fecha y hora obtenida.
-----------	---

Devuelve

- ESP_OK: Lectura exitosa.
- ESP_ERR_TIMEOUT: No fue posible obtener el mutex I2C.

Lee la fecha y hora actual desde el DS3231.

Lee los registros internos del DS3231 y convierte los datos almacenados en formato BCD a formato decimal.

Parámetros

<i>dt</i>	Puntero a la estructura donde se almacenará la fecha y hora obtenida.
-----------	---

Devuelve

- ESP_OK: Lectura exitosa.
- Código de error en caso de fallo de comunicación.

Definición en la línea 129 del archivo [rtc_ds3231.c](#).

```

00130 {
00131     uint8_t data[7];
00132
00133     if (xSemaphoreTake(mutex_i2c, pdMS_TO_TICKS(100)) != pdTRUE) {
00134         return ESP_ERR_TIMEOUT;
00135     }
00136
00137     i2c_manager_read(DS3231_ADDR, 0x00, data, 7);
00138
00139     xSemaphoreGive(mutex_i2c);
00140
00141     /*
00142     * Conversión de registros DS3231 almacenados
00143     * en formato BCD hacia formato decimal.
00144     */
00145     dt->seconds = bcd_to_dec(data[0]);
00146     dt->minutes = bcd_to_dec(data[1]);
00147     dt->hours   = bcd_to_dec(data[2]);
00148     dt->day     = bcd_to_dec(data[4]);
00149     dt->month   = bcd_to_dec(data[5] & 0x1F);
00150     dt->year    = 2000 + bcd_to_dec(data[6]);
00151
00152     return ESP_OK;
00153 }
```

6.19.3.5. ds3231_init()

```

esp_err_t ds3231_init (
    void )
```

Inicializa el módulo RTC DS3231.

Actualmente no requiere configuración específica, ya que el acceso al dispositivo se realiza bajo demanda mediante transacciones I2C.

Devuelve

ESP_OK.

Verifica la comunicación con el dispositivo y prepara el acceso a los registros internos mediante I2C.

Esta función debe ejecutarse durante la fase de inicialización del sistema.

Devuelve

- ESP_OK: Inicialización exitosa.
- Código de error en caso de fallo de comunicación.

Definición en la línea 71 del archivo `rtc_ds3231.c`.

```
00072 {
00073     ESP_LOGI(TAG, "DS3231 inicializado");
00074     return ESP_OK;
00075 }
```

6.19.3.6. ds3231_set_alarm_seconds()

```
esp_err_t ds3231_set_alarm_seconds (
    uint8_t seconds)
```

Configura una alarma basada en coincidencia de segundos.

Configura una alarma basada en segundos.

Programa la Alarm 1 del DS3231 para generar una interrupción cuando el campo de segundos coincida con el valor indicado.

Parámetros

<i>seconds</i>	Segundo objetivo de activación (0-59).
----------------	--

Devuelve

- ESP_OK: Configuración exitosa.
- ESP_ERR_TIMEOUT: No fue posible obtener el mutex I2C.

Configura una alarma basada en coincidencia de segundos.

Programa la alarma 1 del DS3231 para activarse cuando el valor de segundos coincida con el parámetro indicado.

Esta funcionalidad puede utilizarse para generar interrupciones periódicas que despierten al sistema desde modos de bajo consumo.

Parámetros

<i>seconds</i>	Segundo de activación de la alarma. Rango válido: 0 a 59.
----------------	---

Devuelve

- ESP_OK: Configuración exitosa.
- Código de error en caso contrario.

Definición en la línea 204 del archivo [rtc_ds3231.c](#).

```

00205 {
00206     uint8_t data[4];
00207
00208     /*
00209      * Configuración de Alarm 1:
00210      * - Coincidencia únicamente sobre segundos.
00211      * - Minutos, horas y día ignorados mediante bits AlMx.
00212      */
00213     data[0] = dec_to_bcd(seconds);
00214     data[1] = 0x80;
00215     data[2] = 0x80;
00216     data[3] = 0x80;
00217
00218     if (xSemaphoreTake(mutex_i2c, pdMS_TO_TICKS(100)) != pdTRUE) {
00219         return ESP_ERR_TIMEOUT;
00220     }
00221
00222     i2c_manager_write(DS3231_ADDR, 0x07, data, 4);
00223
00224     /*
00225      * INTCN = 1 -> modo interrupción
00226      * ALIE = 1 -> habilita Alarm 1
00227      */
00228     uint8_t control = DS3231_CONTROL_A1_INTERRUPT;
00229
00230     i2c_manager_write(DS3231_ADDR, 0x0E, &control, 1);
00231
00232     xSemaphoreGive(mutex_i2c);
00233
00234     ESP_LOGI(TAG, "Alarma configurada en segundo: %d", seconds);
00235
00236     return ESP_OK;
00237 }

```

6.19.4. Documentación de variables

6.19.4.1. mutex_i2c

```
SemaphoreHandle_t mutex_i2c [extern]
```

Mutex compartido para acceso exclusivo al bus I2C.

Mutex global para acceso seguro al bus I2C.

Garantiza acceso seguro al RTC cuando múltiples tareas utilizan periféricos conectados al mismo bus.

Mutex compartido para acceso exclusivo al bus I2C.

Garantiza exclusión mutua cuando múltiples tareas intentan acceder simultáneamente a dispositivos conectados al mismo bus.

Mutex global para acceso seguro al bus I2C.

Utilizado por todos los periféricos que comparten el mismo bus, evitando condiciones de carrera entre tareas concurrentes.

Definición en la línea 66 del archivo [i2c_manager.c](#).

6.19.4.2. TAG

```
const char* TAG = "DS3231" [static]
```

Etiqueta utilizada por el sistema de logging.

Definición en la línea 56 del archivo `rtc_ds3231.c`.

6.20. rtc_ds3231.c

[Ir a la documentación de este archivo.](#)

```
00001
00023
00024 #include "rtc_ds3231.h"
00025 #include "hal/i2c_manager.h"
00026
00027 #include "freertos/FreeRTOS.h"
00028 #include "freertos/semphr.h"
00029
00030 #include "esp_log.h"
00031
00035 #define DS3231_ADDR 0x68
00036
00043 #define DS3231_CONTROL_A1_INTERRUPT 0x05
00044
00051 extern SemaphoreHandle_t mutex_i2c;
00052
00056 static const char *TAG = "DS3231";
00057
00058 // =====
00059 // INIT
00060 // =====
00061
00071 esp_err_t ds3231_init(void)
00072 {
00073     ESP_LOGI(TAG, "DS3231 inicializado");
00074     return ESP_OK;
00075 }
00076
00077 // =====
00078 // BCD <--> DEC
00079 // =====
00080
00092 static uint8_t bcd_to_dec(uint8_t val)
00093 {
00094     return (val >> 4) * 10 + (val & 0x0F);
00095 }
00096
00107 static uint8_t dec_to_bcd(uint8_t val)
00108 {
00109     return ((val / 10) << 4) | (val % 10);
00110 }
00111
00112 // =====
00113 // GET DATETIME
00114 // =====
00115
00129 esp_err_t ds3231_get_datetime(datetime_t *dt)
00130 {
00131     uint8_t data[7];
00132
00133     if (xSemaphoreTake(mutex_i2c, pdMS_TO_TICKS(100)) != pdTRUE) {
00134         return ESP_ERR_TIMEOUT;
00135     }
00136
00137     i2c_manager_read(DS3231_ADDR, 0x00, data, 7);
00138
00139     xSemaphoreGive(mutex_i2c);
00140
00141     /*
00142      * Conversión de registros DS3231 almacenados
00143      * en formato BCD hacia formato decimal.
00144      */
00145     dt->seconds = bcd_to_dec(data[0]);
00146     dt->minutes = bcd_to_dec(data[1]);
00147     dt->hours = bcd_to_dec(data[2]);
00148     dt->day = bcd_to_dec(data[4]);
```

```

00149     dt->month    = bcd_to_dec(data[5] & 0x1F);
00150     dt->year      = 2000 + bcd_to_dec(data[6]);
00151
00152     return ESP_OK;
00153 }
00154
00155 // =====
00156 // CLEAR FLAG
00157 // =====
00158
00169 esp_err_t ds3231_clear_alarm_flag(void)
00170 {
00171     uint8_t status;
00172
00173     if (xSemaphoreTake(mutex_i2c, pdMS_TO_TICKS(100)) != pdTRUE) {
00174         return ESP_ERR_TIMEOUT;
00175     }
00176
00177     i2c_manager_read(DS3231_ADDR, 0x0F, &status, 1);
00178
00179     status &= ~0x01;
00180
00181     i2c_manager_write(DS3231_ADDR, 0x0F, &status, 1);
00182
00183     xSemaphoreGive(mutex_i2c);
00184
00185     return ESP_OK;
00186 }
00187
00188 // =====
00189 // SET ALARM
00190 // =====
00191
00204 esp_err_t ds3231_set_alarm_seconds(uint8_t seconds)
00205 {
00206     uint8_t data[4];
00207
00208     /*
00209     * Configuración de Alarm 1:
00210     * - Coincidencia únicamente sobre segundos.
00211     * - Minutos, horas y día ignorados mediante bits AlMx.
00212     */
00213     data[0] = dec_to_bcd(seconds);
00214     data[1] = 0x80;
00215     data[2] = 0x80;
00216     data[3] = 0x80;
00217
00218     if (xSemaphoreTake(mutex_i2c, pdMS_TO_TICKS(100)) != pdTRUE) {
00219         return ESP_ERR_TIMEOUT;
00220     }
00221
00222     i2c_manager_write(DS3231_ADDR, 0x07, data, 4);
00223
00224     /*
00225     * INTCN = 1 -> modo interrupción
00226     * AlIE = 1 -> habilita Alarm 1
00227     */
00228     uint8_t control = DS3231_CONTROL_A1_INTERRUPT;
00229
00230     i2c_manager_write(DS3231_ADDR, 0x0E, &control, 1);
00231
00232     xSemaphoreGive(mutex_i2c);
00233
00234     ESP_LOGI(TAG, "Alarma configurada en segundo: %d", seconds);
00235
00236     return ESP_OK;
00237 }

```

6.21. Referencia del archivo main/drivers/rtc_ds3231.h

Driver para el reloj en tiempo real DS3231 mediante interfaz I2C.

```

#include "esp_err.h"
#include "utils/data_types.h"

```

Funciones

- `esp_err_t ds3231_init` (void)
Inicializa el módulo RTC DS3231.
- `esp_err_t ds3231_clear_alarm_flag` (void)
Limpia el indicador de alarma del DS3231.
- `esp_err_t ds3231_get_datetime` (datetime_t *dt)
Obtiene la fecha y hora actual del RTC.
- `esp_err_t ds3231_set_alarm_seconds` (uint8_t seconds)
Configura una alarma basada en segundos.

6.21.1. Descripción detallada

Driver para el reloj en tiempo real DS3231 mediante interfaz I2C.

Este módulo proporciona las funciones necesarias para la comunicación con el reloj de tiempo real DS3231.

Funcionalidades:

- Inicialización del dispositivo.
- Lectura de fecha y hora.
- Configuración de alarmas.
- Gestión de interrupciones mediante alarmas.
- Limpieza de flags de alarma.

El DS3231 es utilizado como referencia temporal del sistema de fermentación, permitiendo el registro de eventos con timestamp y la activación periódica de tareas mediante interrupciones.

Autor

Fernando Plazas Trujillo Isabella Ordoñez Juan Daniel Constain

Definición en el archivo [rtc_ds3231.h](#).

6.21.2. Documentación de funciones

6.21.2.1. ds3231_clear_alarm_flag()

```
esp_err_t ds3231_clear_alarm_flag (  
    void )
```

Limpia el indicador de alarma del DS3231.

Limpia el flag de alarma del DS3231.

Borra el flag de alarma almacenado en el registro de estado del dispositivo, permitiendo la generación de nuevas interrupciones.

Devuelve

- ESP_OK: Operación realizada correctamente.
- Código de error en caso de fallo.

Limpia el indicador de alarma del DS3231.

Borra el bit A1F del registro de estado para permitir futuras interrupciones de alarma.

Devuelve

- ESP_OK: Operación exitosa.
- ESP_ERR_TIMEOUT: No fue posible obtener el mutex I2C.

Definición en la línea 169 del archivo `rtc_ds3231.c`.

```
00170 {
00171     uint8_t status;
00172
00173     if (xSemaphoreTake(mutex_i2c, pdMS_TO_TICKS(100)) != pdTRUE) {
00174         return ESP_ERR_TIMEOUT;
00175     }
00176
00177     i2c_manager_read(DS3231_ADDR, 0x0F, &status, 1);
00178
00179     status &= ~0x01;
00180
00181     i2c_manager_write(DS3231_ADDR, 0x0F, &status, 1);
00182
00183     xSemaphoreGive(mutex_i2c);
00184
00185     return ESP_OK;
00186 }
```

6.21.2.2. ds3231_get_datetime()

```
esp_err_t ds3231_get_datetime (
    datetime_t * dt)
```

Obtiene la fecha y hora actual del RTC.

Lee la fecha y hora actual desde el DS3231.

Lee los registros internos del DS3231 y convierte los datos almacenados en formato BCD a formato decimal.

Parámetros

<i>dt</i>	Puntero a la estructura donde se almacenará la fecha y hora obtenida.
-----------	---

Devuelve

- ESP_OK: Lectura exitosa.
- Código de error en caso de fallo de comunicación.

Obtiene la fecha y hora actual del RTC.

Lee los registros internos del dispositivo y convierte los valores almacenados en formato BCD a formato decimal.

Parámetros

<i>dt</i>	Puntero a la estructura donde se almacenará la fecha y hora obtenida.
-----------	---

Devuelve

- ESP_OK: Lectura exitosa.
- ESP_ERR_TIMEOUT: No fue posible obtener el mutex I2C.

Definición en la línea 129 del archivo `rtc_ds3231.c`.

```

00130 {
00131     uint8_t data[7];
00132
00133     if (xSemaphoreTake(mutex_i2c, pdMS_TO_TICKS(100)) != pdTRUE) {
00134         return ESP_ERR_TIMEOUT;
00135     }
00136
00137     i2c_manager_read(DS3231_ADDR, 0x00, data, 7);
00138
00139     xSemaphoreGive(mutex_i2c);
00140
00141     /*
00142      * Conversión de registros DS3231 almacenados
00143      * en formato BCD hacia formato decimal.
00144      */
00145     dt->seconds = bcd_to_dec(data[0]);
00146     dt->minutes = bcd_to_dec(data[1]);
00147     dt->hours    = bcd_to_dec(data[2]);
00148     dt->day      = bcd_to_dec(data[4]);
00149     dt->month    = bcd_to_dec(data[5] & 0x1F);
00150     dt->year     = 2000 + bcd_to_dec(data[6]);
00151
00152     return ESP_OK;
00153 }
```

6.21.2.3. ds3231_init()

```

esp_err_t ds3231_init (
    void )
```

Inicializa el módulo RTC DS3231.

Verifica la comunicación con el dispositivo y prepara el acceso a los registros internos mediante I2C.

Esta función debe ejecutarse durante la fase de inicialización del sistema.

Devuelve

- ESP_OK: Inicialización exitosa.
- Código de error en caso de fallo de comunicación.

Actualmente no requiere configuración específica, ya que el acceso al dispositivo se realiza bajo demanda mediante transacciones I2C.

Devuelve

ESP_OK.

Definición en la línea 71 del archivo `rtc_ds3231.c`.

```

00072 {
00073     ESP_LOGI(TAG, "DS3231 inicializado");
00074     return ESP_OK;
00075 }
```

6.21.2.4. ds3231_set_alarm_seconds()

```
esp_err_t ds3231_set_alarm_seconds (
    uint8_t seconds)
```

Configura una alarma basada en segundos.

Configura una alarma basada en coincidencia de segundos.

Programa la alarma 1 del DS3231 para activarse cuando el valor de segundos coincida con el parámetro indicado.

Esta funcionalidad puede utilizarse para generar interrupciones periódicas que despierten al sistema desde modos de bajo consumo.

Parámetros

<i>seconds</i>	Segundo de activación de la alarma. Rango válido: 0 a 59.
----------------	---

Devuelve

- ESP_OK: Configuración exitosa.
- Código de error en caso contrario.

Configura una alarma basada en segundos.

Programa la Alarm 1 del DS3231 para generar una interrupción cuando el campo de segundos coincida con el valor indicado.

Parámetros

<i>seconds</i>	Segundo objetivo de activación (0-59).
----------------	--

Devuelve

- ESP_OK: Configuración exitosa.
- ESP_ERR_TIMEOUT: No fue posible obtener el mutex I2C.

Definición en la línea [204](#) del archivo [rtc_ds3231.c](#).

```
00205 {
00206     uint8_t data[4];
00207
00208     /*
00209     * Configuración de Alarm 1:
00210     * - Coincidencia únicamente sobre segundos.
00211     * - Minutos, horas y día ignorados mediante bits AlMx.
00212     */
00213     data[0] = dec_to_bcd(seconds);
00214     data[1] = 0x80;
00215     data[2] = 0x80;
00216     data[3] = 0x80;
00217
00218     if (xSemaphoreTake(mutex_i2c, pdMS_TO_TICKS(100)) != pdTRUE) {
00219         return ESP_ERR_TIMEOUT;
00220     }
00221
00222     i2c_manager_write(DS3231_ADDR, 0x07, data, 4);
00223
00224     /*
00225     * INTCN = 1 -> modo interrupción
00226     * AlIE = 1 -> habilita Alarm 1
00227     */
00228     uint8_t control = DS3231_CONTROL_A1_INTERRUPT;
00229
00230     i2c_manager_write(DS3231_ADDR, 0x0E, &control, 1);
00231
00232     xSemaphoreGive(mutex_i2c);
00233
00234     ESP_LOGI(TAG, "Alarma configurada en segundo: %d", seconds);
00235
00236     return ESP_OK;
00237 }
```

6.22. rtc_ds3231.h

[Ir a la documentación de este archivo.](#)

```
00001
00024
00025 #ifndef RTC_DS3231_H
00026 #define RTC_DS3231_H
00027
00028 #include "esp_err.h"
00029 #include "utils/data_types.h"
00030
00044 esp_err_t ds3231_init(void);
00045
00057 esp_err_t ds3231_clear_alarm_flag(void);
00058
00072 esp_err_t ds3231_get_datetime(datetime_t *dt);
00073
00091 esp_err_t ds3231_set_alarm_seconds(uint8_t seconds);
00092
00093 #endif
```

6.23. Referencia del archivo main/drivers/sd_card.c

Implementación del módulo de almacenamiento en tarjeta SD mediante SPI.

```
#include <stdio.h>
#include <stdbool.h>
#include <errno.h>
#include <string.h>
#include <sys/stat.h>
#include "esp_log.h"
#include "esp_vfs_fat.h"
#include "sdmmc_cmd.h"
#include "driver/spi_common.h"
#include "driver/sdspi_host.h"
#include "freertos/FreeRTOS.h"
#include "freertos/semphr.h"
#include "sd_card.h"
```

defines

- #define **PIN_MISO** 19
GPIO utilizado como línea MISO.
- #define **PIN_MOSI** 23
GPIO utilizado como línea MOSI.
- #define **PIN_CLK** 18
GPIO utilizado como reloj SPI.
- #define **PIN_CS** 5
GPIO utilizado como Chip Select.

Funciones

- `esp_err_t sd_card_init` (void)
Inicializa y monta la tarjeta SD.
- `esp_err_t sd_card_write_line` (const char *line)
Escribe una línea en el archivo log.csv de la SD.
- `esp_err_t sd_card_append_pending_mqtt` (const char *payload)
Guarda un payload MQTT pendiente de publicación.
- `bool sd_card_pending_mqtt_exists` (void)
Verifica la existencia de mensajes MQTT pendientes.
- `FILE * sd_card_open_pending_mqtt_read` (void)
Abre el archivo de mensajes MQTT pendientes.
- `char * sd_card_read_pending_mqtt_line` (FILE *file, char *buffer, size_t size)
Lee una línea del archivo de mensajes MQTT pendientes.
- `void sd_card_close_pending_mqtt` (FILE *file)
Cierra el archivo de mensajes MQTT pendientes.
- `esp_err_t sd_card_delete_pending_mqtt` (void)
Elimina el archivo de mensajes MQTT pendientes.
- `bool sd_card_is_mounted` (void)
Verifica si la tarjeta SD se encuentra montada.

Variables

- `static const char * TAG` = "SD"
Etiqueta utilizada por el sistema de logging.
- `static const char mount_point []` = "/sdcard"
Punto de montaje del sistema de archivos FAT.
- `static const char pending_mqtt_path []` = "/sdcard/mqtt_pending.txt"
Archivo utilizado para almacenar mensajes MQTT pendientes.
- `static SemaphoreHandle_t spi_mutex` = NULL
Mutex para acceso exclusivo al bus SPI.
- `static bool sd_mounted` = false
Indica si la tarjeta SD fue montada correctamente.

6.23.1. Descripción detallada

Implementación del módulo de almacenamiento en tarjeta SD mediante SPI.

Este módulo proporciona funciones para:

- Inicialización y montaje de la tarjeta SD.
- Escritura de registros históricos en formato CSV.
- Almacenamiento temporal de mensajes MQTT pendientes.
- Recuperación de mensajes pendientes.
- Gestión segura de acceso concurrente mediante mutex.

El módulo permite mantener la persistencia de datos del sistema de fermentación incluso ante fallos de conectividad MQTT.

El acceso al bus SPI se encuentra protegido mediante exclusión mutua para evitar condiciones de carrera entre tareas FreeRTOS.

Autor

Fernando Plazas Trujillo Isabella Ordoñez Juan Daniel Constain

Definición en el archivo `sd_card.c`.

6.23.2. Documentación de «define»

6.23.2.1. PIN_CLK

```
#define PIN_CLK 18
```

GPIO utilizado como reloj SPI.

Definición en la línea [63](#) del archivo [sd_card.c](#).

6.23.2.2. PIN_CS

```
#define PIN_CS 5
```

GPIO utilizado como Chip Select.

Definición en la línea [68](#) del archivo [sd_card.c](#).

6.23.2.3. PIN_MISO

```
#define PIN_MISO 19
```

GPIO utilizado como línea MISO.

Definición en la línea [53](#) del archivo [sd_card.c](#).

6.23.2.4. PIN_MOSI

```
#define PIN_MOSI 23
```

GPIO utilizado como línea MOSI.

Definición en la línea [58](#) del archivo [sd_card.c](#).

6.23.3. Documentación de funciones

6.23.3.1. sd_card_append_pending_mqtt()

```
esp_err_t sd_card_append_pending_mqtt (
    const char * payload)
```

Guarda un payload MQTT pendiente de publicación.

Cuando no existe conectividad con el broker MQTT, el mensaje se almacena temporalmente en la tarjeta SD para su posterior retransmisión.

Parámetros

<i>payload</i>	Mensaje MQTT a almacenar.
----------------	---------------------------

Devuelve

- ESP_OK: Operación exitosa.
- Código de error en caso contrario.

Definición en la línea 232 del archivo [sd_card.c](#).

```
00233 {
00234     if (payload == NULL) {
00235         return ESP_FAIL;
00236     }
00237
00238     if (sd_card_init() != ESP_OK) {
00239         return ESP_FAIL;
00240     }
00241
00242     if (spi_mutex != NULL) {
00243         xSemaphoreTake(spi_mutex, portMAX_DELAY);
00244     }
00245
00246     FILE *f = fopen(pending_mqtt_path, "a");
00247
00248     if (f == NULL) {
00249         ESP_LOGE(TAG, "Error abriendo pendientes MQTT (%s), errno=%d", pending_mqtt_path, errno);
00250
00251         if (spi_mutex != NULL) {
00252             xSemaphoreGive(spi_mutex);
00253         }
00254
00255         return ESP_FAIL;
00256     }
00257
00258     fprintf(f, "%s\n", payload);
00259     fflush(f);
00260     fclose(f);
00261
00262     if (spi_mutex != NULL) {
00263         xSemaphoreGive(spi_mutex);
00264     }
00265
00266     return ESP_OK;
00267 }
```

6.23.3.2. sd_card_close_pending_mqtt()

```
void sd_card_close_pending_mqtt (
    FILE * file)
```

Cierra el archivo de mensajes MQTT pendientes.

Libera los recursos asociados al archivo abierto.

Parámetros

<i>file</i>	Puntero al archivo.
-------------	---------------------

Definición en la línea 312 del archivo [sd_card.c](#).

```
00313 {
00314     if (file != NULL) {
00315         fclose(file);
00316     }
00317
00318     if (spi_mutex != NULL) {
00319         xSemaphoreGive(spi_mutex);
00320     }
00321 }
```

6.23.3.3. sd_card_delete_pending_mqtt()

```
esp_err_t sd_card_delete_pending_mqtt (
    void )
```

Elimina el archivo de mensajes MQTT pendientes.

Se utiliza una vez que todos los mensajes almacenados han sido retransmitidos correctamente al broker MQTT.

Devuelve

- ESP_OK: Archivo eliminado correctamente.
- Código de error en caso contrario.

Definición en la línea 323 del archivo [sd_card.c](#).

```
00324 {
00325     if (sd_card_init() != ESP_OK) {
00326         return ESP_FAIL;
00327     }
00328
00329     if (spi_mutex != NULL) {
00330         xSemaphoreTake(spi_mutex, portMAX_DELAY);
00331     }
00332
00333     int result = remove(pending_mqtt_path);
00334
00335     if (spi_mutex != NULL) {
00336         xSemaphoreGive(spi_mutex);
00337     }
00338
00339     if (result == 0 || errno == ENOENT) {
00340         return ESP_OK;
00341     }
00342
00343     ESP_LOGE(TAG, "Error eliminando pendientes MQTT (%s), errno=%d", pending_mqtt_path, errno);
00344     return ESP_FAIL;
00345 }
```

6.23.3.4. sd_card_init()

```
esp_err_t sd_card_init (
    void )
```

Inicializa y monta la tarjeta SD.

Inicializa la tarjeta SD.

Configura el bus SPI, el dispositivo SD y el sistema de archivos.

Devuelve

ESP_OK si la tarjeta se monta correctamente.

Inicializa y monta la tarjeta SD.

Configura el bus SPI, monta el sistema de archivos FAT y verifica la disponibilidad del dispositivo de almacenamiento.

Esta función debe ejecutarse durante la inicialización del sistema antes de realizar operaciones de lectura o escritura.

Devuelve

- ESP_OK: Tarjeta montada correctamente.
- Código de error en caso de fallo.

Definición en la línea 112 del archivo `sd_card.c`.

```

00113 {
00114     esp_err_t ret;
00115
00116     if (sd_mounted) {
00117         return ESP_OK;
00118     }
00119
00120     if (spi_mutex == NULL) {
00121         spi_mutex = xSemaphoreCreateMutex();
00122         if (spi_mutex == NULL) {
00123             ESP_LOGE(TAG, "Error creando mutex SPI");
00124             return ESP_FAIL;
00125         }
00126     }
00127
00128     sdmmc_host_t host = SDSPI_HOST_DEFAULT();
00129
00130     spi_bus_config_t bus_cfg = {
00131         .mosi_io_num = PIN_MOSI,
00132         .miso_io_num = PIN_MISO,
00133         .sclk_io_num = PIN_CLK,
00134         .quadwp_io_num = -1,
00135         .quadhd_io_num = -1,
00136         .max_transfer_sz = 4000,
00137     };
00138
00139     /*
00140     * Configuración del bus SPI utilizado para la comunicación
00141     * con la tarjeta SD.
00142     */
00143     ret = spi_bus_initialize(host.slot, &bus_cfg, SDSPI_DEFAULT_DMA);
00144     if (ret != ESP_OK && ret != ESP_ERR_INVALID_STATE) {
00145         ESP_LOGE(TAG, "Error inicializando SPI");
00146         return ret;
00147     }
00148
00149     sdspi_device_config_t slot_config = SDSPI_DEVICE_CONFIG_DEFAULT();
00150     slot_config.gpio_cs = PIN_CS;
00151     slot_config.host_id = host.slot;
00152
00153     /*
00154     * Montaje del sistema de archivos FAT sobre la tarjeta SD.
00155     */
00156     esp_vfs_fat_sdmmc_mount_config_t mount_config = {
00157         .format_if_mount_failed = false,
00158         .max_files = 5,
00159         .allocation_unit_size = 16 * 1024
00160     };
00161
00162     sdmmc_card_t *card;
00163
00164     ret = esp_vfs_fat_sdspi_mount(mount_point, &host, &slot_config, &mount_config, &card);
00165
00166     if (ret != ESP_OK) {
00167         ESP_LOGE(TAG, "Error montando SD (%s)", esp_err_to_name(ret));
00168         return ret;
00169     }
00170
00171     ESP_LOGI(TAG, "SD montada correctamente");
00172
00173     sd_mounted = true;
00174
00175     return ESP_OK;
00176 }

```

6.23.3.5. sd_card_is_mounted()

```
bool sd_card_is_mounted (
    void )
```

Verifica si la tarjeta SD se encuentra montada.

Permite a otras tareas validar la disponibilidad del sistema de almacenamiento antes de realizar operaciones de lectura o escritura.

Devuelve

- true: Tarjeta montada correctamente.
- false: Tarjeta no disponible.

Definición en la línea 347 del archivo [sd_card.c](#).

```
00348 {
00349     return sd_mounted;
00350 }
```

6.23.3.6. sd_card_open_pending_mqtt_read()

```
FILE * sd_card_open_pending_mqtt_read (
    void )
```

Abre el archivo de mensajes MQTT pendientes.

El archivo es abierto en modo lectura para permitir la recuperación de mensajes almacenados previamente.

Devuelve

Puntero al archivo abierto o NULL si ocurre un error.

Definición en la línea 280 del archivo [sd_card.c](#).

```
00281 {
00282     if (sd_card_init() != ESP_OK) {
00283         return NULL;
00284     }
00285
00286     if (spi_mutex != NULL) {
00287         xSemaphoreTake(spi_mutex, portMAX_DELAY);
00288     }
00289
00290     FILE *f = fopen(pending_mqtt_path, "r");
00291
00292     if (f == NULL) {
00293         ESP_LOGE(TAG, "Error abriendo pendientes MQTT (%s), errno=%d", pending_mqtt_path, errno);
00294
00295         if (spi_mutex != NULL) {
00296             xSemaphoreGive(spi_mutex);
00297         }
00298     }
00299
00300     return f;
00301 }
```

6.23.3.7. sd_card_pending_mqtt_exists()

```
bool sd_card_pending_mqtt_exists (  
    void )
```

Verifica la existencia de mensajes MQTT pendientes.

Comprueba si existe el archivo de almacenamiento temporal utilizado para guardar mensajes no publicados.

Devuelve

- true: Existen mensajes pendientes.
- false: No existen mensajes pendientes.

Definición en la línea 269 del archivo [sd_card.c](#).

```
00270 {  
00271     struct stat file_stat;  
00272  
00273     if (sd_card_init() != ESP_OK) {  
00274         return false;  
00275     }  
00276  
00277     return stat(pending_mqtt_path, &file_stat) == 0;  
00278 }
```

6.23.3.8. sd_card_read_pending_mqtt_line()

```
char * sd_card_read_pending_mqtt_line (  
    FILE * file,  
    char * buffer,  
    size_t size)
```

Lee una línea del archivo de mensajes MQTT pendientes.

Obtiene el siguiente registro almacenado dentro del archivo.

Parámetros

<i>file</i>	Puntero al archivo abierto.
<i>buffer</i>	Buffer donde se almacenará la línea leída.
<i>size</i>	Tamaño del buffer.

Devuelve

- Puntero al buffer si la lectura fue exitosa.
- NULL si se alcanza el final del archivo o ocurre un error.

Definición en la línea 303 del archivo [sd_card.c](#).

```
00304 {  
00305     if (file == NULL || buffer == NULL || size == 0) {  
00306         return NULL;  
00307     }  
00308  
00309     return fgets(buffer, size, file);  
00310 }
```

6.23.3.9. sd_card_write_line()

```
esp_err_t sd_card_write_line (
    const char * line)
```

Escribe una línea en el archivo log.csv de la SD.

Escribe una línea en el archivo de registro.

La operación es protegida con mutex para evitar accesos concurrentes.

Parámetros

<i>line</i>	Texto a escribir.
-------------	-------------------

Devuelve

ESP_OK si la escritura fue exitosa.

Escribe una línea en el archivo log.csv de la SD.

Añade una nueva línea al archivo CSV utilizado para almacenar las variables del proceso de fermentación.

El acceso al archivo se encuentra protegido para evitar conflictos entre tareas concurrentes.

Parámetros

<i>line</i>	Cadena de texto a almacenar.
-------------	------------------------------

Devuelve

- ESP_OK: Escritura exitosa.
- Código de error en caso contrario.

Definición en la línea 190 del archivo [sd_card.c](#).

```
00191 {
00192     if (line == NULL) {
00193         return ESP_FAIL;
00194     }
00195
00196     if (spi_mutex != NULL) {
00197         xSemaphoreTake(spi_mutex, portMAX_DELAY);
00198     }
00199
00200     /*
00201     * Apertura del archivo en modo append para preservar
00202     * los registros históricos previamente almacenados.
00203     */
00204     FILE *f = fopen("/sdcard/log.csv", "a");
00205
00206     if (f == NULL) {
00207         ESP_LOGE(TAG, "Error abriendo archivo");
00208
00209         if (spi_mutex != NULL) {
00210             xSemaphoreGive(spi_mutex);
00211         }
00212
00213         return ESP_FAIL;
00214     }
00215
00216     /*
00217     * Cada registro corresponde a una muestra del sistema
00218     * de fermentación almacenada en formato CSV.
```



```
00219     */
00220     fprintf(f, "%s\n", line);
00221
00222     fflush(f);
00223     fclose(f);
00224
00225     if (spi_mutex != NULL) {
00226         xSemaphoreGive(spi_mutex);
00227     }
00228
00229     return ESP_OK;
00230 }
```

6.23.4. Documentación de variables

6.23.4.1. mount_point

```
const char mount_point[] = "/sdcard" [static]
```

Punto de montaje del sistema de archivos FAT.

Definición en la línea 77 del archivo [sd_card.c](#).

6.23.4.2. pending_mqtt_path

```
const char pending_mqtt_path[] = "/sdcard/mqtt_pending.txt" [static]
```

Archivo utilizado para almacenar mensajes MQTT pendientes.

Definición en la línea 82 del archivo [sd_card.c](#).

6.23.4.3. sd_mounted

```
bool sd_mounted = false [static]
```

Indica si la tarjeta SD fue montada correctamente.

Definición en la línea 99 del archivo [sd_card.c](#).

6.23.4.4. spi_mutex

```
SemaphoreHandle_t spi_mutex = NULL [static]
```

Mutex para acceso exclusivo al bus SPI.

Evita accesos concurrentes desde múltiples tareas durante operaciones de lectura o escritura.

Definición en la línea 94 del archivo [sd_card.c](#).

6.23.4.5. TAG

```
const char* TAG = "SD" [static]
```

Etiqueta utilizada por el sistema de logging.

Definición en la línea 44 del archivo `sd_card.c`.

6.24. sd_card.c

[Ir a la documentación de este archivo.](#)

```
00001
00023
00024 #include <stdio.h>
00025 #include <stdbool.h>
00026 #include <errno.h>
00027 #include <string.h>
00028 #include <sys/stat.h>
00029
00030 #include "esp_log.h"
00031 #include "esp_vfs_fat.h"
00032 #include "sdmmc_cmd.h"
00033 #include "driver/spi_common.h"
00034 #include "driver/sdspi_host.h"
00035
00036 #include "freertos/FreeRTOS.h"
00037 #include "freertos/semphr.h"
00038
00039 #include "sd_card.h"
00040
00044 static const char *TAG = "SD";
00045
00046 // =====
00047 // PINES SPI
00048 // =====
00049
00053 #define PIN_MISO 19
00054
00058 #define PIN_MOSI 23
00059
00063 #define PIN_CLK 18
00064
00068 #define PIN_CS 5
00069
00070 // =====
00071 // MONTAJE
00072 // =====
00073
00077 static const char mount_point[] = "/sdcard";
00078
00082 static const char pending_mqtt_path[] = "/sdcard/mqtt_pending.txt";
00083
00084 // =====
00085 // MUTEX SPI
00086 // =====
00087
00094 static SemaphoreHandle_t spi_mutex = NULL;
00095
00099 static bool sd_mounted = false;
00100
00101 // =====
00102 // INIT SD
00103 // =====
00104
00112 esp_err_t sd_card_init(void)
00113 {
00114     esp_err_t ret;
00115
00116     if (sd_mounted) {
00117         return ESP_OK;
00118     }
00119
00120     if (spi_mutex == NULL) {
00121         spi_mutex = xSemaphoreCreateMutex();
00122         if (spi_mutex == NULL) {
00123             ESP_LOGE(TAG, "Error creando mutex SPI");
00124             return ESP_FAIL;
00125         }
00126     }
00127 }
```

```

00125     }
00126 }
00127
00128 sdmmc_host_t host = SDSPI_HOST_DEFAULT();
00129
00130 spi_bus_config_t bus_cfg = {
00131     .mosi_io_num = PIN_MOSI,
00132     .miso_io_num = PIN_MISO,
00133     .sclk_io_num = PIN_CLK,
00134     .quadwp_io_num = -1,
00135     .quadhd_io_num = -1,
00136     .max_transfer_sz = 4000,
00137 };
00138
00139 /*
00140  * Configuración del bus SPI utilizado para la comunicación
00141  * con la tarjeta SD.
00142  */
00143 ret = spi_bus_initialize(host.slot, &bus_cfg, SDSPI_DEFAULT_DMA);
00144 if (ret != ESP_OK && ret != ESP_ERR_INVALID_STATE) {
00145     ESP_LOGE(TAG, "Error inicializando SPI");
00146     return ret;
00147 }
00148
00149 sdspi_device_config_t slot_config = SDSPI_DEVICE_CONFIG_DEFAULT();
00150 slot_config.gpio_cs = PIN_CS;
00151 slot_config.host_id = host.slot;
00152
00153 /*
00154  * Montaje del sistema de archivos FAT sobre la tarjeta SD.
00155  */
00156 esp_vfs_fat_sdmmc_mount_config_t mount_config = {
00157     .format_if_mount_failed = false,
00158     .max_files = 5,
00159     .allocation_unit_size = 16 * 1024
00160 };
00161
00162 sdmmc_card_t *card;
00163
00164 ret = esp_vfs_fat_sdspi_mount(mount_point, &host, &slot_config, &mount_config, &card);
00165
00166 if (ret != ESP_OK) {
00167     ESP_LOGE(TAG, "Error montando SD (%s)", esp_err_to_name(ret));
00168     return ret;
00169 }
00170
00171 ESP_LOGI(TAG, "SD montada correctamente");
00172
00173 sd_mounted = true;
00174
00175 return ESP_OK;
00176 }
00177
00178 // =====
00179 // WRITE
00180 // =====
00181
00190 esp_err_t sd_card_write_line(const char *line)
00191 {
00192     if (line == NULL) {
00193         return ESP_FAIL;
00194     }
00195
00196     if (spi_mutex != NULL) {
00197         xSemaphoreTake(spi_mutex, portMAX_DELAY);
00198     }
00199
00200     /*
00201     * Apertura del archivo en modo append para preservar
00202     * los registros históricos previamente almacenados.
00203     */
00204     FILE *f = fopen("/sdcard/log.csv", "a");
00205
00206     if (f == NULL) {
00207         ESP_LOGE(TAG, "Error abriendo archivo");
00208
00209         if (spi_mutex != NULL) {
00210             xSemaphoreGive(spi_mutex);
00211         }
00212
00213         return ESP_FAIL;
00214     }
00215
00216     /*
00217     * Cada registro corresponde a una muestra del sistema
00218     * de fermentación almacenada en formato CSV.
00219     */

```

```

00220     fprintf(f, "%s\n", line);
00221
00222     fflush(f);
00223     fclose(f);
00224
00225     if (spi_mutex != NULL) {
00226         xSemaphoreGive(spi_mutex);
00227     }
00228
00229     return ESP_OK;
00230 }
00231
00232 esp_err_t sd_card_append_pending_mqtt(const char *payload)
00233 {
00234     if (payload == NULL) {
00235         return ESP_FAIL;
00236     }
00237
00238     if (sd_card_init() != ESP_OK) {
00239         return ESP_FAIL;
00240     }
00241
00242     if (spi_mutex != NULL) {
00243         xSemaphoreTake(spi_mutex, portMAX_DELAY);
00244     }
00245
00246     FILE *f = fopen(pending_mqtt_path, "a");
00247
00248     if (f == NULL) {
00249         ESP_LOGE(TAG, "Error abriendo pendientes MQTT (%s), errno=%d", pending_mqtt_path, errno);
00250
00251         if (spi_mutex != NULL) {
00252             xSemaphoreGive(spi_mutex);
00253         }
00254
00255         return ESP_FAIL;
00256     }
00257
00258     fprintf(f, "%s\n", payload);
00259     fflush(f);
00260     fclose(f);
00261
00262     if (spi_mutex != NULL) {
00263         xSemaphoreGive(spi_mutex);
00264     }
00265
00266     return ESP_OK;
00267 }
00268
00269 bool sd_card_pending_mqtt_exists(void)
00270 {
00271     struct stat file_stat;
00272
00273     if (sd_card_init() != ESP_OK) {
00274         return false;
00275     }
00276
00277     return stat(pending_mqtt_path, &file_stat) == 0;
00278 }
00279
00280 FILE *sd_card_open_pending_mqtt_read(void)
00281 {
00282     if (sd_card_init() != ESP_OK) {
00283         return NULL;
00284     }
00285
00286     if (spi_mutex != NULL) {
00287         xSemaphoreTake(spi_mutex, portMAX_DELAY);
00288     }
00289
00290     FILE *f = fopen(pending_mqtt_path, "r");
00291
00292     if (f == NULL) {
00293         ESP_LOGE(TAG, "Error abriendo pendientes MQTT (%s), errno=%d", pending_mqtt_path, errno);
00294
00295         if (spi_mutex != NULL) {
00296             xSemaphoreGive(spi_mutex);
00297         }
00298     }
00299
00300     return f;
00301 }
00302
00303 char *sd_card_read_pending_mqtt_line(FILE *file, char *buffer, size_t size)
00304 {
00305     if (file == NULL || buffer == NULL || size == 0) {
00306         return NULL;

```

```

00307     }
00308
00309     return fgets(buffer, size, file);
00310 }
00311
00312 void sd_card_close_pending_mqtt(FILE *file)
00313 {
00314     if (file != NULL) {
00315         fclose(file);
00316     }
00317
00318     if (spi_mutex != NULL) {
00319         xSemaphoreGive(spi_mutex);
00320     }
00321 }
00322
00323 esp_err_t sd_card_delete_pending_mqtt(void)
00324 {
00325     if (sd_card_init() != ESP_OK) {
00326         return ESP_FAIL;
00327     }
00328
00329     if (spi_mutex != NULL) {
00330         xSemaphoreTake(spi_mutex, portMAX_DELAY);
00331     }
00332
00333     int result = remove(pending_mqtt_path);
00334
00335     if (spi_mutex != NULL) {
00336         xSemaphoreGive(spi_mutex);
00337     }
00338
00339     if (result == 0 || errno == ENOENT) {
00340         return ESP_OK;
00341     }
00342
00343     ESP_LOGE(TAG, "Error eliminando pendientes MQTT (%s), errno=%d", pending_mqtt_path, errno);
00344     return ESP_FAIL;
00345 }
00346
00347 bool sd_card_is_mounted(void)
00348 {
00349     return sd_mounted;
00350 }

```

6.25. Referencia del archivo main/drivers/sd_card.h

Módulo para gestión de tarjeta SD mediante interfaz SPI.

```

#include <stdbool.h>
#include <stddef.h>
#include <stdio.h>
#include "esp_err.h"

```

Funciones

- `esp_err_t sd_card_init` (void)
Inicializa la tarjeta SD.
- `esp_err_t sd_card_write_line` (const char *line)
Escribe una línea en el archivo de registro.
- `esp_err_t sd_card_append_pending_mqtt` (const char *payload)
Guarda un payload MQTT pendiente de publicación.
- `bool sd_card_pending_mqtt_exists` (void)
Verifica la existencia de mensajes MQTT pendientes.
- `FILE * sd_card_open_pending_mqtt_read` (void)
Abre el archivo de mensajes MQTT pendientes.

- `char * sd_card_read_pending_mqtt_line` (FILE *file, char *buffer, size_t size)
Lee una línea del archivo de mensajes MQTT pendientes.
- `void sd_card_close_pending_mqtt` (FILE *file)
Cierra el archivo de mensajes MQTT pendientes.
- `esp_err_t sd_card_delete_pending_mqtt` (void)
Elimina el archivo de mensajes MQTT pendientes.
- `bool sd_card_is_mounted` (void)
Verifica si la tarjeta SD se encuentra montada.

6.25.1. Descripción detallada

Módulo para gestión de tarjeta SD mediante interfaz SPI.

Este módulo proporciona funciones para inicializar la tarjeta SD, almacenar registros históricos del sistema y gestionar mensajes MQTT pendientes cuando no existe conectividad de red.

Funcionalidades:

- Montaje del sistema de archivos FAT.
- Escritura de registros CSV.
- Almacenamiento temporal de mensajes MQTT.
- Recuperación de mensajes pendientes.
- Verificación del estado de montaje.

El acceso a la tarjeta SD es protegido mediante mecanismos de sincronización para evitar conflictos de acceso concurrente al bus SPI.

Autor

Fernando Plazas Trujillo Isabella Ordoñez Juan Daniel Constain

Definición en el archivo [sd_card.h](#).

6.25.2. Documentación de funciones

6.25.2.1. `sd_card_append_pending_mqtt()`

```
esp_err_t sd_card_append_pending_mqtt (  
    const char * payload)
```

Guarda un payload MQTT pendiente de publicación.

Cuando no existe conectividad con el broker MQTT, el mensaje se almacena temporalmente en la tarjeta SD para su posterior retransmisión.

Parámetros

<i>payload</i>	Mensaje MQTT a almacenar.
----------------	---------------------------

Devuelve

- ESP_OK: Operación exitosa.
- Código de error en caso contrario.

Definición en la línea 232 del archivo [sd_card.c](#).

```

00233 {
00234     if (payload == NULL) {
00235         return ESP_FAIL;
00236     }
00237
00238     if (sd_card_init() != ESP_OK) {
00239         return ESP_FAIL;
00240     }
00241
00242     if (spi_mutex != NULL) {
00243         xSemaphoreTake(spi_mutex, portMAX_DELAY);
00244     }
00245
00246     FILE *f = fopen(pending_mqtt_path, "a");
00247
00248     if (f == NULL) {
00249         ESP_LOGE(TAG, "Error abriendo pendientes MQTT (%s), errno=%d", pending_mqtt_path, errno);
00250
00251         if (spi_mutex != NULL) {
00252             xSemaphoreGive(spi_mutex);
00253         }
00254
00255         return ESP_FAIL;
00256     }
00257
00258     fprintf(f, "%s\n", payload);
00259     fflush(f);
00260     fclose(f);
00261
00262     if (spi_mutex != NULL) {
00263         xSemaphoreGive(spi_mutex);
00264     }
00265
00266     return ESP_OK;
00267 }

```

6.25.2.2. sd_card_close_pending_mqtt()

```

void sd_card_close_pending_mqtt (
    FILE * file)

```

Cierra el archivo de mensajes MQTT pendientes.

Libera los recursos asociados al archivo abierto.

Parámetros

<i>file</i>	Puntero al archivo.
-------------	---------------------

Definición en la línea 312 del archivo [sd_card.c](#).

```

00313 {
00314     if (file != NULL) {
00315         fclose(file);
00316     }
00317
00318     if (spi_mutex != NULL) {
00319         xSemaphoreGive(spi_mutex);
00320     }
00321 }

```

6.25.2.3. sd_card_delete_pending_mqtt()

```
esp_err_t sd_card_delete_pending_mqtt (
    void )
```

Elimina el archivo de mensajes MQTT pendientes.

Se utiliza una vez que todos los mensajes almacenados han sido retransmitidos correctamente al broker MQTT.

Devuelve

- ESP_OK: Archivo eliminado correctamente.
- Código de error en caso contrario.

Definición en la línea 323 del archivo `sd_card.c`.

```
00324 {
00325     if (sd_card_init() != ESP_OK) {
00326         return ESP_FAIL;
00327     }
00328
00329     if (spi_mutex != NULL) {
00330         xSemaphoreTake(spi_mutex, portMAX_DELAY);
00331     }
00332
00333     int result = remove(pending_mqtt_path);
00334
00335     if (spi_mutex != NULL) {
00336         xSemaphoreGive(spi_mutex);
00337     }
00338
00339     if (result == 0 || errno == ENOENT) {
00340         return ESP_OK;
00341     }
00342
00343     ESP_LOGE(TAG, "Error eliminando pendientes MQTT (%s), errno=%d", pending_mqtt_path, errno);
00344     return ESP_FAIL;
00345 }
```

6.25.2.4. sd_card_init()

```
esp_err_t sd_card_init (
    void )
```

Inicializa la tarjeta SD.

Inicializa y monta la tarjeta SD.

Configura el bus SPI, monta el sistema de archivos FAT y verifica la disponibilidad del dispositivo de almacenamiento.

Esta función debe ejecutarse durante la inicialización del sistema antes de realizar operaciones de lectura o escritura.

Devuelve

- ESP_OK: Tarjeta montada correctamente.
- Código de error en caso de fallo.

Inicializa la tarjeta SD.

Configura el bus SPI, el dispositivo SD y el sistema de archivos.

Devuelve

ESP_OK si la tarjeta se monta correctamente.

Definición en la línea 112 del archivo [sd_card.c](#).

```

00113 {
00114     esp_err_t ret;
00115
00116     if (sd_mounted) {
00117         return ESP_OK;
00118     }
00119
00120     if (spi_mutex == NULL) {
00121         spi_mutex = xSemaphoreCreateMutex();
00122         if (spi_mutex == NULL) {
00123             ESP_LOGE(TAG, "Error creando mutex SPI");
00124             return ESP_FAIL;
00125         }
00126     }
00127
00128     sdmmc_host_t host = SDSPI_HOST_DEFAULT();
00129
00130     spi_bus_config_t bus_cfg = {
00131         .mosi_io_num = PIN_MOSI,
00132         .miso_io_num = PIN_MISO,
00133         .sclk_io_num = PIN_CLK,
00134         .quadwp_io_num = -1,
00135         .quadhd_io_num = -1,
00136         .max_transfer_sz = 4000,
00137     };
00138
00139     /*
00140     * Configuración del bus SPI utilizado para la comunicación
00141     * con la tarjeta SD.
00142     */
00143     ret = spi_bus_initialize(host.slot, &bus_cfg, SDSPI_DEFAULT_DMA);
00144     if (ret != ESP_OK && ret != ESP_ERR_INVALID_STATE) {
00145         ESP_LOGE(TAG, "Error inicializando SPI");
00146         return ret;
00147     }
00148
00149     sdspi_device_config_t slot_config = SDSPI_DEVICE_CONFIG_DEFAULT();
00150     slot_config.gpio_cs = PIN_CS;
00151     slot_config.host_id = host.slot;
00152
00153     /*
00154     * Montaje del sistema de archivos FAT sobre la tarjeta SD.
00155     */
00156     esp_vfs_fat_sdmmc_mount_config_t mount_config = {
00157         .format_if_mount_failed = false,
00158         .max_files = 5,
00159         .allocation_unit_size = 16 * 1024
00160     };
00161
00162     sdmmc_card_t *card;
00163
00164     ret = esp_vfs_fat_sdspi_mount(mount_point, &host, &slot_config, &mount_config, &card);
00165
00166     if (ret != ESP_OK) {
00167         ESP_LOGE(TAG, "Error montando SD (%s)", esp_err_to_name(ret));
00168         return ret;
00169     }
00170
00171     ESP_LOGI(TAG, "SD montada correctamente");
00172
00173     sd_mounted = true;
00174
00175     return ESP_OK;
00176 }

```

6.25.2.5. sd_card_is_mounted()

```

bool sd_card_is_mounted (
    void )

```

Verifica si la tarjeta SD se encuentra montada.

Permite a otras tareas validar la disponibilidad del sistema de almacenamiento antes de realizar operaciones de lectura o escritura.

Devuelve

- true: Tarjeta montada correctamente.
- false: Tarjeta no disponible.

Definición en la línea 347 del archivo [sd_card.c](#).

```
00348 {
00349     return sd_mounted;
00350 }
```

6.25.2.6. sd_card_open_pending_mqtt_read()

```
FILE * sd_card_open_pending_mqtt_read (
    void )
```

Abre el archivo de mensajes MQTT pendientes.

El archivo es abierto en modo lectura para permitir la recuperación de mensajes almacenados previamente.

Devuelve

Puntero al archivo abierto o NULL si ocurre un error.

Definición en la línea 280 del archivo [sd_card.c](#).

```
00281 {
00282     if (sd_card_init() != ESP_OK) {
00283         return NULL;
00284     }
00285
00286     if (spi_mutex != NULL) {
00287         xSemaphoreTake(spi_mutex, portMAX_DELAY);
00288     }
00289
00290     FILE *f = fopen(pending_mqtt_path, "r");
00291
00292     if (f == NULL) {
00293         ESP_LOGE(TAG, "Error abriendo pendientes MQTT (%s), errno=%d", pending_mqtt_path, errno);
00294
00295         if (spi_mutex != NULL) {
00296             xSemaphoreGive(spi_mutex);
00297         }
00298     }
00299
00300     return f;
00301 }
```

6.25.2.7. sd_card_pending_mqtt_exists()

```
bool sd_card_pending_mqtt_exists (
    void )
```

Verifica la existencia de mensajes MQTT pendientes.

Comprueba si existe el archivo de almacenamiento temporal utilizado para guardar mensajes no publicados.

Devuelve

- true: Existen mensajes pendientes.
- false: No existen mensajes pendientes.

Definición en la línea 269 del archivo [sd_card.c](#).

```
00270 {
00271     struct stat file_stat;
00272
00273     if (sd_card_init() != ESP_OK) {
00274         return false;
00275     }
00276
00277     return stat(pending_mqtt_path, &file_stat) == 0;
00278 }
```

6.25.2.8. sd_card_read_pending_mqtt_line()

```
char * sd_card_read_pending_mqtt_line (  
    FILE * file,  
    char * buffer,  
    size_t size)
```

Lee una línea del archivo de mensajes MQTT pendientes.

Obtiene el siguiente registro almacenado dentro del archivo.

Parámetros

<i>file</i>	Puntero al archivo abierto.
<i>buffer</i>	Buffer donde se almacenará la línea leída.
<i>size</i>	Tamaño del buffer.

Devuelve

- Puntero al buffer si la lectura fue exitosa.
- NULL si se alcanza el final del archivo o ocurre un error.

Definición en la línea 303 del archivo [sd_card.c](#).

```
00304 {  
00305     if (file == NULL || buffer == NULL || size == 0) {  
00306         return NULL;  
00307     }  
00308  
00309     return fgets(buffer, size, file);  
00310 }
```

6.25.2.9. sd_card_write_line()

```
esp_err_t sd_card_write_line (  
    const char * line)
```

Escribe una línea en el archivo de registro.

Escribe una línea en el archivo log.csv de la SD.

Añade una nueva línea al archivo CSV utilizado para almacenar las variables del proceso de fermentación.

El acceso al archivo se encuentra protegido para evitar conflictos entre tareas concurrentes.

Parámetros

<i>line</i>	Cadena de texto a almacenar.
-------------	------------------------------

Devuelve

- ESP_OK: Escritura exitosa.
- Código de error en caso contrario.

Escribe una línea en el archivo de registro.

La operación es protegida con mutex para evitar accesos concurrentes.

Parámetros

<i>line</i>	Texto a escribir.
-------------	-------------------

Devuelve

ESP_OK si la escritura fue exitosa.

Definición en la línea 190 del archivo `sd_card.c`.

```

00191 {
00192     if (line == NULL) {
00193         return ESP_FAIL;
00194     }
00195
00196     if (spi_mutex != NULL) {
00197         xSemaphoreTake(spi_mutex, portMAX_DELAY);
00198     }
00199
00200     /*
00201     * Apertura del archivo en modo append para preservar
00202     * los registros históricos previamente almacenados.
00203     */
00204     FILE *f = fopen("/sdcard/log.csv", "a");
00205
00206     if (f == NULL) {
00207         ESP_LOGE(TAG, "Error abriendo archivo");
00208
00209         if (spi_mutex != NULL) {
00210             xSemaphoreGive(spi_mutex);
00211         }
00212
00213         return ESP_FAIL;
00214     }
00215
00216     /*
00217     * Cada registro corresponde a una muestra del sistema
00218     * de fermentación almacenada en formato CSV.
00219     */
00220     fprintf(f, "%s\n", line);
00221
00222     fflush(f);
00223     fclose(f);
00224
00225     if (spi_mutex != NULL) {
00226         xSemaphoreGive(spi_mutex);
00227     }
00228
00229     return ESP_OK;
00230 }

```

6.26. sd_card.h

[Ir a la documentación de este archivo.](#)

```

00001
00025
00026 #ifndef SD_CARD_H
00027 #define SD_CARD_H
00028
00029 #include <stdbool.h>
00030 #include <stddef.h>

```

```

00031 #include <stdio.h>
00032
00033 #include "esp_err.h"
00034
00049 esp_err_t sd_card_init(void);
00050
00066 esp_err_t sd_card_write_line(const char *line);
00067
00081 esp_err_t sd_card_append_pending_mqtt(const char *payload);
00082
00093 bool sd_card_pending_mqtt_exists(void);
00094
00103 FILE *sd_card_open_pending_mqtt_read(void);
00104
00118 char *sd_card_read_pending_mqtt_line(
00119     FILE *file,
00120     char *buffer,
00121     size_t size);
00122
00130 void sd_card_close_pending_mqtt(FILE *file);
00131
00142 esp_err_t sd_card_delete_pending_mqtt(void);
00143
00155 bool sd_card_is_mounted(void);
00156
00157 #endif

```

6.27. Referencia del archivo main/drivers/servo.c

Implementación del control de servomotor usando PWM mediante LEDC.

```

#include "drivers/servo.h"
#include "driver/ledc.h"
#include "esp_log.h"

```

defines

- #define **SERVO_FREQ** 50
Frecuencia PWM utilizada por el servomotor.
- #define **SERVO_TIMER** LEDC_TIMER_0
Temporizador LEDC utilizado.
- #define **SERVO_MODE** LEDC_HIGH_SPEED_MODE
Modo de operación LEDC.
- #define **SERVO_CHANNEL** LEDC_CHANNEL_0
Canal PWM utilizado.
- #define **SERVO_RESOLUTION** LEDC_TIMER_16_BIT
Resolución PWM configurada.
- #define **SERVO_MAX_DUTY** 65535
Ciclo de trabajo máximo para resolución de 16 bits.
- #define **SERVO_PERIOD_US** 20000.0f
Periodo PWM en microsegundos.
- #define **SERVO_MIN_PULSE_US** 1000.0f
Ancho de pulso correspondiente a 0°.
- #define **SERVO_MAX_PULSE_US** 2000.0f
Ancho de pulso correspondiente a 180°.

Funciones

- esp_err_t [servo_init](#) (gpio_num_t pin)
Inicializa el servomotor.
- esp_err_t [servo_set_angle](#) (float angle)
Posiciona el servomotor en el ángulo especificado.

Variables

- static const char * [TAG](#) = "SERVO"
Etiqueta utilizada por el sistema de logging.
- static gpio_num_t [servo_pin](#)
GPIO utilizado para el servomotor.

6.27.1. Descripción detallada

Implementación del control de servomotor usando PWM mediante LEDC.

Este módulo genera señales PWM compatibles con servomotores estándar utilizando el periférico LEDC del ESP32.

Características:

- Frecuencia de operación de 50 Hz.
- Resolución PWM de 16 bits.
- Conversión automática de ángulo a ciclo de trabajo.

El servomotor puede utilizarse para accionar mecanismos físicos dentro del sistema de fermentación, como sistemas de mezcla o posicionamiento.

Autor

Fernando Plazas Trujillo Isabella Ordoñez Juan Daniel Constain

Definición en el archivo [servo.c](#).

6.27.2. Documentación de «define»

6.27.2.1. SERVO_CHANNEL

```
#define SERVO_CHANNEL LEDC_CHANNEL_0
```

Canal PWM utilizado.

Definición en la línea [53](#) del archivo [servo.c](#).

6.27.2.2. SERVO_FREQ

```
#define SERVO_FREQ 50
```

Frecuencia PWM utilizada por el servomotor.

Los servomotores convencionales utilizan una frecuencia de aproximadamente 50 Hz (periodo de 20 ms).

Definición en la línea 38 del archivo [servo.c](#).

6.27.2.3. SERVO_MAX_DUTY

```
#define SERVO_MAX_DUTY 65535
```

Ciclo de trabajo máximo para resolución de 16 bits.

Definición en la línea 63 del archivo [servo.c](#).

6.27.2.4. SERVO_MAX_PULSE_US

```
#define SERVO_MAX_PULSE_US 2000.0f
```

Ancho de pulso correspondiente a 180°.

Definición en la línea 80 del archivo [servo.c](#).

6.27.2.5. SERVO_MIN_PULSE_US

```
#define SERVO_MIN_PULSE_US 1000.0f
```

Ancho de pulso correspondiente a 0°.

Definición en la línea 75 del archivo [servo.c](#).

6.27.2.6. SERVO_MODE

```
#define SERVO_MODE LEDC_HIGH_SPEED_MODE
```

Modo de operación LEDC.

Definición en la línea 48 del archivo [servo.c](#).

6.27.2.7. SERVO_PERIOD_US

```
#define SERVO_PERIOD_US 20000.0f
```

Periodo PWM en microsegundos.

50 Hz = 20 ms = 20000 us

Definición en la línea 70 del archivo [servo.c](#).

6.27.2.8. SERVO_RESOLUTION

```
#define SERVO_RESOLUTION LEDC_TIMER_16_BIT
```

Resolución PWM configurada.

Definición en la línea 58 del archivo [servo.c](#).

6.27.2.9. SERVO_TIMER

```
#define SERVO_TIMER LEDC_TIMER_0
```

Temporizador LEDC utilizado.

Definición en la línea 43 del archivo [servo.c](#).

6.27.3. Documentación de funciones

6.27.3.1. servo_init()

```
esp_err_t servo_init (  
    gpio_num_t pin)
```

Inicializa el servomotor.

Configura el temporizador LEDC y el canal PWM asociado al GPIO indicado.

Parámetros

<i>pin</i>	GPIO conectado al servomotor.
------------	-------------------------------

Devuelve

- ESP_OK: Inicialización exitosa.
- Código de error en caso contrario.

Configura el temporizador y canal PWM del periférico LEDC necesarios para generar la señal de control del servo.

Esta función debe ejecutarse una única vez antes de utilizar cualquier otra función del módulo.

Parámetros

<i>pin</i>	GPIO al que se encuentra conectado el servomotor.
------------	---

Devuelve

- ESP_OK: Inicialización exitosa.
- Código de error en caso contrario.

Definición en la línea 103 del archivo `servo.c`.

```

00104 {
00105     servo_pin = pin;
00106
00107     ledc_timer_config_t timer = {
00108         .speed_mode = SERVO_MODE,
00109         .timer_num = SERVO_TIMER,
00110         .duty_resolution = SERVO_RESOLUTION,
00111         .freq_hz = SERVO_FREQ,
00112         .clk_cfg = LEDC_AUTO_CLK
00113     };
00114
00115     ledc_timer_config(&timer);
00116
00117     ledc_channel_config_t channel = {
00118         .gpio_num = servo_pin,
00119         .speed_mode = SERVO_MODE,
00120         .channel = SERVO_CHANNEL,
00121         .intr_type = LEDC_INTR_DISABLE,
00122         .timer_sel = SERVO_TIMER,
00123         .duty = 0,
00124         .hpoint = 0
00125     };
00126
00127     ledc_channel_config(&channel);
00128
00129     ESP_LOGI(TAG, "Servo inicializado en GPIO%d", pin);
00130
00131     return ESP_OK;
00132 }

```

6.27.3.2. servo_set_angle()

```

esp_err_t servo_set_angle (
    float angle)

```

Posiciona el servomotor en el ángulo especificado.

Posiciona el servomotor en un ángulo específico.

Convierte el ángulo solicitado en un ancho de pulso PWM:

- 0°-> 1.0 ms
- 90°-> 1.5 ms
- 180°-> 2.0 ms

Posteriormente el ancho de pulso se transforma al valor de duty cycle requerido por el periférico LEDC.

Parámetros

<i>angle</i>	Ángulo deseado en grados.
--------------	---------------------------

Devuelve

- ESP_OK: Operación realizada correctamente.

Posiciona el servomotor en el ángulo especificado.

Convierte el ángulo solicitado a un ciclo de trabajo PWM compatible con servomotores estándar de 50 Hz.

El rango válido de operación es de 0° a 180°.

Parámetros

<i>angle</i>	Ángulo deseado en grados.
--------------	---------------------------

Devuelve

- ESP_OK: Operación realizada correctamente.
- ESP_ERR_INVALID_ARG: Ángulo fuera de rango.

Definición en la línea 155 del archivo [servo.c](#).

```

00156 {
00157     if (angle < 0.0f) {
00158         angle = 0.0f;
00159     }
00160
00161     if (angle > 180.0f) {
00162         angle = 180.0f;
00163     }
00164
00165     /*
00166      * Conversión lineal de ángulo a ancho de pulso.
00167      */
00168     float pulse_us =
00169         SERVO_MIN_PULSE_US +
00170         ((angle / 180.0f) *
00171          (SERVO_MAX_PULSE_US - SERVO_MIN_PULSE_US));
00172
00173     /*
00174      * Conversión de ancho de pulso a duty cycle.
00175      */
00176     uint32_t duty =
00177         (uint32_t)((pulse_us / SERVO_PERIOD_US) *
00178          SERVO_MAX_DUTY);
00179
00180     ledc_set_duty(
00181         SERVO_MODE,
00182         SERVO_CHANNEL,
00183         duty);
00184
00185     ledc_update_duty(
00186         SERVO_MODE,
00187         SERVO_CHANNEL);
00188
00189     ESP_LOGI(
00190         TAG,
00191         "Servo angle: %.2f deg | pulse: %.2f us | duty: %lu",
00192         angle,
00193         pulse_us,
00194         (unsigned long)duty);
00195
00196     return ESP_OK;
00197 }

```

6.27.4. Documentación de variables**6.27.4.1. servo_pin**

```
gpio_num_t servo_pin [static]
```

GPIO utilizado para el servomotor.

Definición en la línea 85 del archivo [servo.c](#).

6.27.4.2. TAG

```
const char* TAG = "SERVO" [static]
```

Etiqueta utilizada por el sistema de logging.

Definición en la línea 30 del archivo [servo.c](#).

6.28. servo.c

[Ir a la documentación de este archivo.](#)

```
00001
00022
00023 #include "drivers/servo.h"
00024 #include "driver/ledc.h"
00025 #include "esp_log.h"
00026
00030 static const char *TAG = "SERVO";
00031
00038 #define SERVO_FREQ 50
00039
00043 #define SERVO_TIMER LEDC_TIMER_0
00044
00048 #define SERVO_MODE LEDC_HIGH_SPEED_MODE
00049
00053 #define SERVO_CHANNEL LEDC_CHANNEL_0
00054
00058 #define SERVO_RESOLUTION LEDC_TIMER_16_BIT
00059
00063 #define SERVO_MAX_DUTY 65535
00064
00070 #define SERVO_PERIOD_US 20000.0f
00071
00075 #define SERVO_MIN_PULSE_US 1000.0f
00076
00080 #define SERVO_MAX_PULSE_US 2000.0f
00081
00085 static gpio_num_t servo_pin;
00086
00087 // =====
00088 // INIT
00089 // =====
00090
00103 esp_err_t servo_init(gpio_num_t pin)
00104 {
00105     servo_pin = pin;
00106
00107     ledc_timer_config_t timer = {
00108         .speed_mode = SERVO_MODE,
00109         .timer_num = SERVO_TIMER,
00110         .duty_resolution = SERVO_RESOLUTION,
00111         .freq_hz = SERVO_FREQ,
00112         .clk_cfg = LEDC_AUTO_CLK
00113     };
00114
00115     ledc_timer_config(&timer);
00116
00117     ledc_channel_config_t channel = {
00118         .gpio_num = servo_pin,
00119         .speed_mode = SERVO_MODE,
00120         .channel = SERVO_CHANNEL,
00121         .intr_type = LEDC_INTR_DISABLE,
00122         .timer_sel = SERVO_TIMER,
00123         .duty = 0,
00124         .hpoint = 0
00125     };
00126
00127     ledc_channel_config(&channel);
00128
00129     ESP_LOGI(TAG, "Servo inicializado en GPIO%d", pin);
00130
00131     return ESP_OK;
00132 }
00133
00134 // =====
00135 // SET ANGLE
00136 // =====
```

```

00137
00155 esp_err_t servo_set_angle(float angle)
00156 {
00157     if (angle < 0.0f) {
00158         angle = 0.0f;
00159     }
00160
00161     if (angle > 180.0f) {
00162         angle = 180.0f;
00163     }
00164
00165     /*
00166      * Conversión lineal de ángulo a ancho de pulso.
00167      */
00168     float pulse_us =
00169         SERVO_MIN_PULSE_US +
00170         ((angle / 180.0f) *
00171          (SERVO_MAX_PULSE_US - SERVO_MIN_PULSE_US));
00172
00173     /*
00174      * Conversión de ancho de pulso a duty cycle.
00175      */
00176     uint32_t duty =
00177         (uint32_t)((pulse_us / SERVO_PERIOD_US) *
00178          SERVO_MAX_DUTY);
00179
00180     ledc_set_duty(
00181         SERVO_MODE,
00182         SERVO_CHANNEL,
00183         duty);
00184
00185     ledc_update_duty(
00186         SERVO_MODE,
00187         SERVO_CHANNEL);
00188
00189     ESP_LOGI(
00190         TAG,
00191         "Servo angle: %.2f deg | pulse: %.2f us | duty: %lu",
00192         angle,
00193         pulse_us,
00194         (unsigned long)duty);
00195
00196     return ESP_OK;
00197 }

```

6.29. Referencia del archivo main/drivers/servo.h

Driver para control de servomotores mediante PWM usando el periférico LEDC del ESP32.

```

#include "esp_err.h"
#include "driver/gpio.h"

```

Funciones

- esp_err_t `servo_init` (gpio_num_t pin)
Inicializa el servomotor.
- esp_err_t `servo_set_angle` (float angle)
Posiciona el servomotor en un ángulo específico.

6.29.1. Descripción detallada

Driver para control de servomotores mediante PWM usando el periférico LEDC del ESP32.

Este módulo proporciona las funciones necesarias para controlar un servomotor estándar mediante señales PWM generadas por el periférico LEDC del ESP32.

Funcionalidades:

- Inicialización del canal PWM.
- Configuración del temporizador LEDC.
- Posicionamiento angular del servomotor.

El módulo es utilizado para accionar mecanismos físicos dentro del sistema de fermentación, como mezcla o posicionamiento de elementos mecánicos.

Autor

Fernando Plazas Trujillo Isabella Ordoñez Juan Daniel Constain

Definición en el archivo [servo.h](#).

6.29.2. Documentación de funciones

6.29.2.1. servo_init()

```
esp_err_t servo_init (  
    gpio_num_t pin)
```

Inicializa el servomotor.

Configura el temporizador y canal PWM del periférico LEDC necesarios para generar la señal de control del servo.

Esta función debe ejecutarse una única vez antes de utilizar cualquier otra función del módulo.

Parámetros

<i>pin</i>	GPIO al que se encuentra conectado el servomotor.
------------	---

Devuelve

- ESP_OK: Inicialización exitosa.
- Código de error en caso contrario.

Configura el temporizador LEDC y el canal PWM asociado al GPIO indicado.

Parámetros

<i>pin</i>	GPIO conectado al servomotor.
------------	-------------------------------

Devuelve

- ESP_OK: Inicialización exitosa.
- Código de error en caso contrario.

Definición en la línea 103 del archivo [servo.c](#).

```

00104 {
00105     servo_pin = pin;
00106
00107     ledc_timer_config_t timer = {
00108         .speed_mode = SERVO_MODE,
00109         .timer_num = SERVO_TIMER,
00110         .duty_resolution = SERVO_RESOLUTION,
00111         .freq_hz = SERVO_FREQ,
00112         .clk_cfg = LEDC_AUTO_CLK
00113     };
00114
00115     ledc_timer_config(&timer);
00116
00117     ledc_channel_config_t channel = {
00118         .gpio_num = servo_pin,
00119         .speed_mode = SERVO_MODE,
00120         .channel = SERVO_CHANNEL,
00121         .intr_type = LEDC_INTR_DISABLE,
00122         .timer_sel = SERVO_TIMER,
00123         .duty = 0,
00124         .hpoint = 0
00125     };
00126
00127     ledc_channel_config(&channel);
00128
00129     ESP_LOGI(TAG, "Servo inicializado en GPIO%d", pin);
00130
00131     return ESP_OK;
00132 }
```

6.29.2.2. servo_set_angle()

```

esp_err_t servo_set_angle (
    float angle)
```

Posiciona el servomotor en un ángulo específico.

Posiciona el servomotor en el ángulo especificado.

Convierte el ángulo solicitado a un ciclo de trabajo PWM compatible con servomotores estándar de 50 Hz.

El rango válido de operación es de 0° a 180°.

Parámetros

<i>angle</i>	Ángulo deseado en grados.
--------------	---------------------------

Devuelve

- ESP_OK: Operación realizada correctamente.
- ESP_ERR_INVALID_ARG: Ángulo fuera de rango.

Posiciona el servomotor en un ángulo específico.

Convierte el ángulo solicitado en un ancho de pulso PWM:

- 0°-> 1.0 ms
- 90°-> 1.5 ms
- 180°-> 2.0 ms

Posteriormente el ancho de pulso se transforma al valor de duty cycle requerido por el periférico LEDC.

Parámetros

<i>angle</i>	Ángulo deseado en grados.
--------------	---------------------------

Devuelve

- ESP_OK: Operación realizada correctamente.

Definición en la línea 155 del archivo `servo.c`.

```

00156 {
00157     if (angle < 0.0f) {
00158         angle = 0.0f;
00159     }
00160
00161     if (angle > 180.0f) {
00162         angle = 180.0f;
00163     }
00164
00165     /*
00166      * Conversión lineal de ángulo a ancho de pulso.
00167      */
00168     float pulse_us =
00169         SERVO_MIN_PULSE_US +
00170         ((angle / 180.0f) *
00171          (SERVO_MAX_PULSE_US - SERVO_MIN_PULSE_US));
00172
00173     /*
00174      * Conversión de ancho de pulso a duty cycle.
00175      */
00176     uint32_t duty =
00177         (uint32_t)((pulse_us / SERVO_PERIOD_US) *
00178          SERVO_MAX_DUTY);
00179
00180     ledc_set_duty(
00181         SERVO_MODE,
00182         SERVO_CHANNEL,
00183         duty);
00184
00185     ledc_update_duty(
00186         SERVO_MODE,
00187         SERVO_CHANNEL);
00188
00189     ESP_LOGI(
00190         TAG,
00191         "Servo angle: %.2f deg | pulse: %.2f us | duty: %lu",
00192         angle,
00193         pulse_us,
00194         (unsigned long)duty);
00195
00196     return ESP_OK;
00197 }

```

6.30. servo.h

[Ir a la documentación de este archivo.](#)

```

00001
00023
00024 #ifndef SERVO_H
00025 #define SERVO_H
00026
00027 #include "esp_err.h"
00028 #include "driver/gpio.h"
00029
00045 esp_err_t servo_init(gpio_num_t pin);
00046
00061 esp_err_t servo_set_angle(float angle);
00062
00063 #endif

```

6.31. Referencia del archivo main/drivers/voltage_sensor.c

Implementación del módulo de medición de voltaje.

```
#include "voltage_sensor.h"
#include "hal/adc_manager.h"
#include "esp_log.h"
```

defines

- #define TAG "VOLTAGE"
Etiqueta utilizada por el sistema de logging.
- #define VOLTAGE_ADC_CHANNEL ADC_CHANNEL_4
Canal ADC asociado al sensor de voltaje.
- #define DIVIDER_RATIO 5.0f
Relación del divisor resistivo.
- #define NUM_SAMPLES 10
Cantidad de muestras utilizadas para el promedio.
- #define ADC_OFFSET 0.48f
Corrección de offset experimental del ADC.

Funciones

- void voltage_sensor_init (void)
Inicializa el módulo de medición de voltaje.
- float voltage_sensor_read (void)
Obtiene el voltaje real del sistema.

6.31.1. Descripción detallada

Implementación del módulo de medición de voltaje.

Este módulo obtiene mediciones de voltaje mediante el ADC del ESP32, aplica compensaciones de calibración y calcula el voltaje real utilizando la relación del divisor resistivo.

Funcionalidades:

- Lectura ADC mediante la capa HAL.
- Promediado de muestras.
- Compensación de offset.
- Conversión a voltaje real.

El módulo es utilizado para monitorear el estado de la batería y apoyar las decisiones de gestión energética del sistema.

Autor

Fernando Plazas Trujillo Isabella Ordoñez Juan Daniel Constain

Definición en el archivo [voltage_sensor.c](#).

6.31.2. Documentación de «define»

6.31.2.1. ADC_OFFSET

```
#define ADC_OFFSET 0.48f
```

Corrección de offset experimental del ADC.

Este valor compensa errores sistemáticos observados durante la calibración del sistema.

Definición en la línea 61 del archivo [voltage_sensor.c](#).

6.31.2.2. DIVIDER_RATIO

```
#define DIVIDER_RATIO 5.0f
```

Relación del divisor resistivo.

Voltaje real = Voltaje ADC × DIVIDER_RATIO

Definición en la línea 45 del archivo [voltage_sensor.c](#).

6.31.2.3. NUM_SAMPLES

```
#define NUM_SAMPLES 10
```

Cantidad de muestras utilizadas para el promedio.

El promediado reduce el efecto del ruido presente en la señal analógica.

Definición en la línea 53 del archivo [voltage_sensor.c](#).

6.31.2.4. TAG

```
#define TAG "VOLTAGE"
```

Etiqueta utilizada por el sistema de logging.

Definición en la línea 31 del archivo [voltage_sensor.c](#).

6.31.2.5. VOLTAGE_ADC_CHANNEL

```
#define VOLTAGE_ADC_CHANNEL ADC_CHANNEL_4
```

Canal ADC asociado al sensor de voltaje.

Corresponde al GPIO32 del ESP32.

Definición en la línea 38 del archivo [voltage_sensor.c](#).

6.31.3. Documentación de funciones

6.31.3.1. voltage_sensor_init()

```
void voltage_sensor_init (
    void )
```

Inicializa el módulo de medición de voltaje.

Actualmente no requiere configuración específica, ya que la adquisición ADC es gestionada por `adc_manager`.

Prepara los recursos necesarios para realizar conversiones ADC a través de la capa HAL.

Esta función debe ejecutarse durante la fase de inicialización del sistema.

Definición en la línea 73 del archivo `voltage_sensor.c`.

```
00074 {
00075     ESP_LOGI(TAG, "Voltage sensor inicializado");
00076 }
```

6.31.3.2. voltage_sensor_read()

```
float voltage_sensor_read (
    void )
```

Obtiene el voltaje real del sistema.

Obtiene el voltaje real medido.

El procedimiento de medición consiste en:

- Adquirir múltiples muestras ADC.
- Calcular un promedio.
- Aplicar corrección de offset.
- Compensar el divisor resistivo.

Devuelve

Voltaje real medido en voltios.

Obtiene el voltaje real del sistema.

Realiza una lectura ADC, aplica las correcciones necesarias y compensa el efecto del divisor resistivo utilizado en la etapa de acondicionamiento.

El valor retornado corresponde al voltaje real presente en la fuente o batería monitoreada.

Devuelve

Voltaje medido en voltios.

Definición en la línea 93 del archivo `voltage_sensor.c`.

```

00094 {
00095     float sum = 0.0f;
00096
00097     /*
00098      * Promediado simple para reducir ruido.
00099      */
00100     for (int i = 0; i < NUM_SAMPLES; i++) {
00101         sum += adc_manager_read_voltage(VOLTAGE_ADC_CHANNEL);
00102     }
00103
00104     float v_adc = sum / NUM_SAMPLES;
00105
00106     /*
00107      * Corrección de offset obtenida durante
00108      * la calibración experimental.
00109      */
00110     v_adc -= ADC_OFFSET;
00111
00112     if (v_adc < 0.0f) {
00113         v_adc = 0.0f;
00114     }
00115
00116     /*
00117      * Conversión del voltaje medido en el ADC
00118      * al voltaje real mediante el divisor resistivo.
00119      */
00120     float v_real = v_adc * DIVIDER_RATIO;
00121
00122     ESP_LOGI(TAG, "Vadc corregido: %.3f V", v_adc);
00123     ESP_LOGI(TAG, "Voltaje final: %.2f V", v_real);
00124
00125     return v_real;
00126 }

```

6.32. voltage_sensor.c

[Ir a la documentación de este archivo.](#)

```

00001
00023
00024 #include "voltage_sensor.h"
00025 #include "hal/adc_manager.h"
00026 #include "esp_log.h"
00027
00031 #define TAG "VOLTAGE"
00032
00038 #define VOLTAGE_ADC_CHANNEL ADC_CHANNEL_4
00039
00045 #define DIVIDER_RATIO 5.0f
00046
00053 #define NUM_SAMPLES 10
00054
00061 #define ADC_OFFSET 0.48f
00062
00063 // =====
00064 // INIT
00065 // =====
00066
00073 void voltage_sensor_init(void)
00074 {
00075     ESP_LOGI(TAG, "Voltage sensor inicializado");
00076 }
00077
00078 // =====
00079 // READ VOLTAGE
00080 // =====
00081
00093 float voltage_sensor_read(void)
00094 {
00095     float sum = 0.0f;
00096
00097     /*
00098      * Promediado simple para reducir ruido.
00099      */

```

```

00100     for (int i = 0; i < NUM_SAMPLES; i++) {
00101         sum += adc_manager_read_voltage(VOLTAGE_ADC_CHANNEL);
00102     }
00103
00104     float v_adc = sum / NUM_SAMPLES;
00105
00106     /*
00107      * Corrección de offset obtenida durante
00108      * la calibración experimental.
00109      */
00110     v_adc -= ADC_OFFSET;
00111
00112     if (v_adc < 0.0f) {
00113         v_adc = 0.0f;
00114     }
00115
00116     /*
00117      * Conversión del voltaje medido en el ADC
00118      * al voltaje real mediante el divisor resistivo.
00119      */
00120     float v_real = v_adc * DIVIDER_RATIO;
00121
00122     ESP_LOGI(TAG, "Vadc corregido: %.3f V", v_adc);
00123     ESP_LOGI(TAG, "Voltaje final: %.2f V", v_real);
00124
00125     return v_real;
00126 }

```

6.33. Referencia del archivo main/drivers/voltage_sensor.h

Driver para medición de voltaje mediante conversión ADC.

Funciones

- void [voltage_sensor_init](#) (void)
Inicializa el módulo de medición de voltaje.
- float [voltage_sensor_read](#) (void)
Obtiene el voltaje real medido.

6.33.1. Descripción detallada

Driver para medición de voltaje mediante conversión ADC.

Este módulo permite medir niveles de voltaje utilizando el convertidor analógico-digital (ADC) del ESP32.

Debido a las limitaciones de entrada del ADC, la señal es acondicionada mediante un divisor resistivo externo, permitiendo medir tensiones superiores al rango soportado directamente por el microcontrolador.

Funcionalidades:

- Inicialización del módulo.
- Lectura de voltaje compensada.
- Conversión a voltaje real del sistema.

El módulo es utilizado para monitorear el nivel de batería y apoyar las estrategias de gestión energética del sistema de fermentación.

Autor

Fernando Plazas Trujillo Isabella Ordoñez Juan Daniel Constain

Definición en el archivo [voltage_sensor.h](#).

6.33.2. Documentación de funciones

6.33.2.1. voltage_sensor_init()

```
void voltage_sensor_init (
    void )
```

Inicializa el módulo de medición de voltaje.

Prepara los recursos necesarios para realizar conversiones ADC a través de la capa HAL.

Esta función debe ejecutarse durante la fase de inicialización del sistema.

Actualmente no requiere configuración específica, ya que la adquisición ADC es gestionada por adc_manager.

Definición en la línea 73 del archivo [voltage_sensor.c](#).

```
00074 {
00075     ESP_LOGI(TAG, "Voltage sensor inicializado");
00076 }
```

6.33.2.2. voltage_sensor_read()

```
float voltage_sensor_read (
    void )
```

Obtiene el voltaje real medido.

Obtiene el voltaje real del sistema.

Realiza una lectura ADC, aplica las correcciones necesarias y compensa el efecto del divisor resistivo utilizado en la etapa de acondicionamiento.

El valor retornado corresponde al voltaje real presente en la fuente o batería monitoreada.

Devuelve

Voltaje medido en voltios.

Obtiene el voltaje real medido.

El procedimiento de medición consiste en:

- Adquirir múltiples muestras ADC.
- Calcular un promedio.
- Aplicar corrección de offset.
- Compensar el divisor resistivo.

Devuelve

Voltaje real medido en voltios.

Definición en la línea 93 del archivo `voltage_sensor.c`.

```
00094 {
00095     float sum = 0.0f;
00096
00097     /*
00098      * Promediado simple para reducir ruido.
00099      */
00100     for (int i = 0; i < NUM_SAMPLES; i++) {
00101         sum += adc_manager_read_voltage(VOLTAGE_ADC_CHANNEL);
00102     }
00103
00104     float v_adc = sum / NUM_SAMPLES;
00105
00106     /*
00107      * Corrección de offset obtenida durante
00108      * la calibración experimental.
00109      */
00110     v_adc -= ADC_OFFSET;
00111
00112     if (v_adc < 0.0f) {
00113         v_adc = 0.0f;
00114     }
00115
00116     /*
00117      * Conversión del voltaje medido en el ADC
00118      * al voltaje real mediante el divisor resistivo.
00119      */
00120     float v_real = v_adc * DIVIDER_RATIO;
00121
00122     ESP_LOGI(TAG, "Vadc corregido: %.3f V", v_adc);
00123     ESP_LOGI(TAG, "Voltaje final: %.2f V", v_real);
00124
00125     return v_real;
00126 }
```

6.34. `voltage_sensor.h`

[Ir a la documentación de este archivo.](#)

```
00001
00027
00028 #ifndef VOLTAGE_SENSOR_H
00029 #define VOLTAGE_SENSOR_H
00030
00040 void voltage_sensor_init(void);
00041
00054 float voltage_sensor_read(void);
00055
00056 #endif
```

6.35. Referencia del archivo `main/hal/adc_manager.c`

Implementación de la capa de abstracción para el ADC del ESP32.

```
#include "adc_manager.h"
#include "esp_adc/adc_oneshot.h"
#include "esp_adc/adc_cali.h"
#include "esp_adc/adc_cali_scheme.h"
#include "esp_log.h"
```

Funciones

- void [adc_manager_init](#) (void)
Inicializa el ADC y el sistema de calibración.
- float [adc_manager_read_voltage](#) (adc_channel_t channel)
Obtiene el voltaje asociado a un canal ADC.

Variables

- static const char * [TAG](#) = "ADC_MANAGER"
Etiqueta utilizada por el sistema de logging.
- static adc_oneShot_unit_handle_t [adc1_handle](#)
Handle de la unidad ADC1 en modo One-Shot.
- static adc_cali_handle_t [adc1_cali_handle](#)
Handle de calibración ADC.

6.35.1. Descripción detallada

Implementación de la capa de abstracción para el ADC del ESP32.

Este módulo implementa el acceso al convertidor analógico-digital utilizando el modo One-Shot proporcionado por ESP-IDF.

Funcionalidades:

- Inicialización del ADC1.
- Configuración de canales analógicos.
- Calibración mediante Line Fitting.
- Conversión de valores ADC a voltaje.

Los sensores analógicos del sistema utilizan este módulo como interfaz única de acceso al ADC.

Sensores asociados:

- MQ135
- Sensor de pH
- Sensor de voltaje

Autor

Fernando Plazas Trujillo Isabella Ordoñez Juan Daniel Constain

Definición en el archivo [adc_manager.c](#).

6.35.2. Documentación de funciones

6.35.2.1. `adc_manager_init()`

```
void adc_manager_init (
    void )
```

Inicializa el ADC y el sistema de calibración.

Inicializa el subsistema ADC.

Configura:

- ADC Unit 1.
- Resolución por defecto.
- Atenuación de 12 dB.
- Esquema de calibración Line Fitting.

Además registra todos los canales utilizados por los sensores analógicos del sistema.

Inicializa el ADC y el sistema de calibración.

Configura la unidad ADC en modo One-Shot y prepara los mecanismos de calibración utilizados para mejorar la precisión de las conversiones.

Esta función debe ejecutarse una única vez durante la inicialización del sistema. Configuración de la unidad ADC1.

Configuración común para todos los canales.

Atenuación de 12 dB para ampliar el rango de medida.

Configuración de calibración ADC.

El esquema Line Fitting mejora la precisión de las conversiones compensando errores internos.

Definición en la línea [73](#) del archivo `adc_manager.c`.

```
00074 {
00078     adc_oneshot_unit_init_cfg_t init_config = {
00079         .unit_id = ADC_UNIT_1,
00080     };
00081
00082     adc_oneshot_new_unit(
00083         &init_config,
00084         &adc1_handle);
00085
00091     adc_oneshot_chan_cfg_t config = {
00092         .bitwidth = ADC_BITWIDTH_DEFAULT,
00093         .atten = ADC_ATTEN_DB_12,
00094     };
00095
00096     /*
00097     * ADC_CHANNEL_0 -> MQ135
00098     * ADC_CHANNEL_3 -> Sensor pH
00099     * ADC_CHANNEL_4 -> Sensor voltaje
00100     */
00101     adc_oneshot_config_channel(
00102         adc1_handle,
00103         ADC_CHANNEL_0,
00104         &config);
00105
00106     adc_oneshot_config_channel(
00107         adc1_handle,
00108         ADC_CHANNEL_3,
00109         &config);
```



```

00110
00111     adc_oneshot_config_channel(
00112         adc1_handle,
00113         ADC_CHANNEL_4,
00114         &config);
00115
00122     adc_cali_line_fitting_config_t cali_config = {
00123         .unit_id = ADC_UNIT_1,
00124         .atten = ADC_ATTEN_DB_12,
00125         .bitwidth = ADC_BITWIDTH_DEFAULT,
00126     };
00127
00128     adc_cali_create_scheme_line_fitting(
00129         &cali_config,
00130         &adc1_cali_handle);
00131
00132     ESP_LOGI(TAG, "ADC1 inicializado y calibrado");
00133 }

```

6.35.2.2. adc_manager_read_voltage()

```

float adc_manager_read_voltage (
    adc_channel_t channel)

```

Obtiene el voltaje asociado a un canal ADC.

El procedimiento consiste en:

- Leer el valor ADC crudo.
- Aplicar calibración.
- Convertir milivoltios a voltios.

Parámetros

<i>channel</i>	Canal ADC a convertir.
----------------	------------------------

Devuelve

Voltaje medido en voltios.

El procedimiento incluye:

- Conversión analógica-digital.
- Calibración de la lectura.
- Conversión de milivoltios a voltios.

Parámetros

<i>channel</i>	Canal ADC a convertir.
----------------	------------------------

Devuelve

Voltaje medido en voltios.

Definición en la línea 151 del archivo [adc_manager.c](#).

```
00152 {  
00153     int raw;  
00154     int voltage_mv;  
00155  
00156     /*  
00157      * Lectura ADC sin procesar.  
00158      */  
00159     adc_oneshot_read(  
00160         adc1_handle,  
00161         channel,  
00162         &raw);  
00163  
00164     /*  
00165      * Conversión de lectura cruda a milivoltios  
00166      * mediante calibración.  
00167      */  
00168     adc_cali_raw_to_voltage(  
00169         adc1_cali_handle,  
00170         raw,  
00171         &voltage_mv);  
00172  
00173     /*  
00174      * Conversión de mV a V.  
00175      */  
00176     return voltage_mv / 1000.0f;  
00177 }
```

6.35.3. Documentación de variables

6.35.3.1. adc1_cali_handle

```
adc_cali_handle_t adc1_cali_handle [static]
```

Handle de calibración ADC.

Utilizado para convertir lecturas crudas a voltajes compensados mediante el esquema de calibración.

Definición en la línea 55 del archivo [adc_manager.c](#).

6.35.3.2. adc1_handle

```
adc_oneshot_unit_handle_t adc1_handle [static]
```

Handle de la unidad ADC1 en modo One-Shot.

Definición en la línea 47 del archivo [adc_manager.c](#).

6.35.3.3. TAG

```
const char* TAG = "ADC_MANAGER" [static]
```

Etiqueta utilizada por el sistema de logging.

Definición en la línea 38 del archivo [adc_manager.c](#).

6.36. adc_manager.c

[Ir a la documentación de este archivo.](#)

```

00001
00027
00028 #include "adc_manager.h"
00029
00030 #include "esp_adc/adc_oneshot.h"
00031 #include "esp_adc/adc_cali.h"
00032 #include "esp_adc/adc_cali_scheme.h"
00033 #include "esp_log.h"
00034
00038 static const char *TAG = "ADC_MANAGER";
00039
00040 // =====
00041 // HANDLES
00042 // =====
00043
00047 static adc_oneshot_unit_handle_t adc1_handle;
00048
00055 static adc_cali_handle_t adc1_cali_handle;
00056
00057 // =====
00058 // INIT
00059 // =====
00060
00073 void adc_manager_init(void)
00074 {
00078     adc_oneshot_unit_init_cfg_t init_config = {
00079         .unit_id = ADC_UNIT_1,
00080     };
00081
00082     adc_oneshot_new_unit(
00083         &init_config,
00084         &adc1_handle);
00085
00091     adc_oneshot_chan_cfg_t config = {
00092         .bitwidth = ADC_BITWIDTH_DEFAULT,
00093         .atten = ADC_ATTEN_DB_12,
00094     };
00095
00096     /*
00097     * ADC_CHANNEL_0 -> MQ135
00098     * ADC_CHANNEL_3 -> Sensor pH
00099     * ADC_CHANNEL_4 -> Sensor voltaje
00100     */
00101     adc_oneshot_config_channel(
00102         adc1_handle,
00103         ADC_CHANNEL_0,
00104         &config);
00105
00106     adc_oneshot_config_channel(
00107         adc1_handle,
00108         ADC_CHANNEL_3,
00109         &config);
00110
00111     adc_oneshot_config_channel(
00112         adc1_handle,
00113         ADC_CHANNEL_4,
00114         &config);
00115
00122     adc_cali_line_fitting_config_t cali_config = {
00123         .unit_id = ADC_UNIT_1,
00124         .atten = ADC_ATTEN_DB_12,
00125         .bitwidth = ADC_BITWIDTH_DEFAULT,
00126     };
00127
00128     adc_cali_create_scheme_line_fitting(
00129         &cali_config,
00130         &adc1_cali_handle);
00131
00132     ESP_LOGI(TAG, "ADC1 inicializado y calibrado");
00133 }
00134
00135 // =====
00136 // READ
00137 // =====
00138
00151 float adc_manager_read_voltage(adc_channel_t channel)
00152 {
00153     int raw;
00154     int voltage_mv;
00155
00156     /*
00157     * Lectura ADC sin procesar.

```

```

00158      */
00159      adc_oneshot_read(
00160          adc1_handle,
00161          channel,
00162          &raw);
00163
00164      /*
00165       * Conversión de lectura cruda a milivoltios
00166       * mediante calibración.
00167       */
00168      adc_cali_raw_to_voltage(
00169          adc1_cali_handle,
00170          raw,
00171          &voltage_mv);
00172
00173      /*
00174       * Conversión de mV a V.
00175       */
00176      return voltage_mv / 1000.0f;
00177 }

```

6.37. Referencia del archivo main/hal/adc_manager.h

Capa de abstracción para el convertidor analógico-digital (ADC) del ESP32.

```
#include "esp_adc/adc_oneshot.h"
```

Funciones

- void [adc_manager_init](#) (void)
Inicializa el subsistema ADC.
- float [adc_manager_read_voltage](#) (adc_channel_t channel)
Obtiene el voltaje asociado a un canal ADC.

6.37.1. Descripción detallada

Capa de abstracción para el convertidor analógico-digital (ADC) del ESP32.

Este módulo implementa una interfaz de acceso al ADC utilizando el modo One-Shot proporcionado por ESP-IDF.

Funcionalidades:

- Inicialización del periférico ADC.
- Configuración de canales analógicos.
- Calibración de lecturas.
- Conversión de valores ADC a voltaje.

Esta capa permite desacoplar los drivers de sensores del hardware específico del ESP32, facilitando la reutilización y mantenimiento del software.

Los módulos:

- mq135
- ph_sensor
- voltage_sensor

utilizan este HAL para adquirir señales analógicas.

Autor

Fernando Plazas Trujillo Isabella Ordoñez Juan Daniel Constain

Definición en el archivo [adc_manager.h](#).

6.37.2. Documentación de funciones

6.37.2.1. adc_manager_init()

```
void adc_manager_init (
    void )
```

Inicializa el subsistema ADC.

Inicializa el ADC y el sistema de calibración.

Configura la unidad ADC en modo One-Shot y prepara los mecanismos de calibración utilizados para mejorar la precisión de las conversiones.

Esta función debe ejecutarse una única vez durante la inicialización del sistema.

Inicializa el subsistema ADC.

Configura:

- ADC Unit 1.
- Resolución por defecto.
- Atenuación de 12 dB.
- Esquema de calibración Line Fitting.

Además registra todos los canales utilizados por los sensores analógicos del sistema. Configuración de la unidad ADC1.

Configuración común para todos los canales.

Atenuación de 12 dB para ampliar el rango de medida.

Configuración de calibración ADC.

El esquema Line Fitting mejora la precisión de las conversiones compensando errores internos.

Definición en la línea 73 del archivo [adc_manager.c](#).

```
00074 {
00078     adc_oneshot_unit_init_cfg_t init_config = {
00079         .unit_id = ADC_UNIT_1,
00080     };
00081
00082     adc_oneshot_new_unit (
00083         &init_config,
00084         &adc1_handle);
00085
00091     adc_oneshot_chan_cfg_t config = {
00092         .bitwidth = ADC_BITWIDTH_DEFAULT,
00093         .atten = ADC_ATTEN_DB_12,
00094     };
00095
00096     /*
00097     * ADC_CHANNEL_0 -> MQ135
00098     * ADC_CHANNEL_3 -> Sensor pH
00099     * ADC_CHANNEL_4 -> Sensor voltaje
00100     */
00101     adc_oneshot_config_channel (
00102         adc1_handle,
00103         ADC_CHANNEL_0,
00104         &config);
00105
00106     adc_oneshot_config_channel (
00107         adc1_handle,
```

```

00108         ADC_CHANNEL_3,
00109         &config);
00110
00111     adc_oneshot_config_channel(
00112         adc1_handle,
00113         ADC_CHANNEL_4,
00114         &config);
00115
00122     adc_cali_line_fitting_config_t cali_config = {
00123         .unit_id = ADC_UNIT_1,
00124         .atten = ADC_ATTEN_DB_12,
00125         .bitwidth = ADC_BITWIDTH_DEFAULT,
00126     };
00127
00128     adc_cali_create_scheme_line_fitting(
00129         &cali_config,
00130         &adc1_cali_handle);
00131
00132     ESP_LOGI(TAG, "ADC1 inicializado y calibrado");
00133 }

```

6.37.2.2. `adc_manager_read_voltage()`

```

float adc_manager_read_voltage (
    adc_channel_t channel)

```

Obtiene el voltaje asociado a un canal ADC.

El procedimiento incluye:

- Conversión analógica-digital.
- Calibración de la lectura.
- Conversión de milivoltios a voltios.

Parámetros

<i>channel</i>	Canal ADC a convertir.
----------------	------------------------

Devuelve

Voltaje medido en voltios.

El procedimiento consiste en:

- Leer el valor ADC crudo.
- Aplicar calibración.
- Convertir milivoltios a voltios.

Parámetros

<i>channel</i>	Canal ADC a convertir.
----------------	------------------------

Devuelve

Voltaje medido en voltios.

Definición en la línea 151 del archivo `adc_manager.c`.

```
00152 {
00153     int raw;
00154     int voltage_mv;
00155
00156     /*
00157      * Lectura ADC sin procesar.
00158      */
00159     adc_oneshot_read(
00160         adc1_handle,
00161         channel,
00162         &raw);
00163
00164     /*
00165      * Conversión de lectura cruda a milivoltios
00166      * mediante calibración.
00167      */
00168     adc_cali_raw_to_voltage(
00169         adc1_cali_handle,
00170         raw,
00171         &voltage_mv);
00172
00173     /*
00174      * Conversión de mV a V.
00175      */
00176     return voltage_mv / 1000.0f;
00177 }
```

6.38. adc_manager.h

[Ir a la documentación de este archivo.](#)

```
00001
00030
00031 #ifndef ADC_MANAGER_H
00032 #define ADC_MANAGER_H
00033
00034 #include "esp_adc/adc_oneshot.h"
00035
00046 void adc_manager_init(void);
00047
00060 float adc_manager_read_voltage(adc_channel_t channel);
00061
00062 #endif
```

6.39. Referencia del archivo main/hal/i2c_manager.c

Implementación de la capa de abstracción para el bus I2C del ESP32.

```
#include "i2c_manager.h"
#include "driver/i2c.h"
#include "esp_log.h"
```

defines

- `#define TAG "I2C"`
Etiqueta utilizada por el sistema de logging.
- `#define I2C_MASTER_SCL_IO 22`
GPIO utilizado como línea SCL.

- `#define I2C_MASTER_SDA_IO 21`
GPIO utilizado como línea SDA.
- `#define I2C_MASTER_NUM I2C_NUM_0`
Puerto I2C utilizado por el sistema.
- `#define I2C_MASTER_FREQ_HZ 100000`
Frecuencia de operación del bus I2C.

Funciones

- `void i2c_manager_init (void)`
Inicializa el bus I2C en modo maestro.
- `esp_err_t i2c_manager_write (uint8_t addr, uint8_t reg, uint8_t *data, size_t len)`
Escribe datos en un registro de un dispositivo I2C.
- `esp_err_t i2c_manager_read (uint8_t addr, uint8_t reg, uint8_t *data, size_t len)`
Lee datos desde un registro de un dispositivo I2C.
- `esp_err_t i2c_manager_write_raw (uint8_t addr, const uint8_t *data, size_t len)`
Envía un bloque de datos sin especificar registro.

Variables

- `SemaphoreHandle_t mutex_i2c = NULL`
Mutex global para acceso seguro al bus I2C.

6.39.1. Descripción detallada

Implementación de la capa de abstracción para el bus I2C del ESP32.

Este módulo configura el ESP32 como maestro I2C y proporciona funciones genéricas para comunicación con dispositivos conectados al bus.

Funcionalidades:

- Inicialización del bus I2C.
- Escritura de registros.
- Lectura de registros.
- Escritura de bloques de datos.
- Protección mediante mutex para acceso concurrente.

Los módulos que utilizan esta capa incluyen:

- `rtc_ds3231`
- `oled_display`

Autor

Fernando Plazas Trujillo Isabella Ordoñez Juan Daniel Constain

Definición en el archivo `i2c_manager.c`.

6.39.2. Documentación de «define»

6.39.2.1. I2C_MASTER_FREQ_HZ

```
#define I2C_MASTER_FREQ_HZ 100000
```

Frecuencia de operación del bus I2C.

Configurada a 100 kHz para garantizar compatibilidad con todos los dispositivos conectados.

Definición en la línea [57](#) del archivo [i2c_manager.c](#).

6.39.2.2. I2C_MASTER_NUM

```
#define I2C_MASTER_NUM I2C_NUM_0
```

Puerto I2C utilizado por el sistema.

Definición en la línea [49](#) del archivo [i2c_manager.c](#).

6.39.2.3. I2C_MASTER_SCL_IO

```
#define I2C_MASTER_SCL_IO 22
```

GPIO utilizado como línea SCL.

Definición en la línea [39](#) del archivo [i2c_manager.c](#).

6.39.2.4. I2C_MASTER_SDA_IO

```
#define I2C_MASTER_SDA_IO 21
```

GPIO utilizado como línea SDA.

Definición en la línea [44](#) del archivo [i2c_manager.c](#).

6.39.2.5. TAG

```
#define TAG "I2C"
```

Etiqueta utilizada por el sistema de logging.

Definición en la línea [34](#) del archivo [i2c_manager.c](#).

6.39.3. Documentación de funciones

6.39.3.1. i2c_manager_init()

```
void i2c_manager_init (
    void )
```

Inicializa el bus I2C en modo maestro.

Configura:

- Pines SDA y SCL.
- Pull-ups internas.
- Frecuencia de operación.
- Driver I2C de ESP-IDF.

Además crea el mutex utilizado para sincronizar el acceso concurrente al bus.

Configura los pines SDA y SCL, la frecuencia del bus y crea los recursos necesarios para la comunicación.

Esta función debe ejecutarse una única vez durante la inicialización del sistema.

Definición en la línea [84](#) del archivo `i2c_manager.c`.

```
00085 {
00086     i2c_config_t conf = {
00087         .mode = I2C_MODE_MASTER,
00088         .sda_io_num = I2C_MASTER_SDA_IO,
00089         .scl_io_num = I2C_MASTER_SCL_IO,
00090         .sda_pullup_en = GPIO_PULLUP_ENABLE,
00091         .scl_pullup_en = GPIO_PULLUP_ENABLE,
00092         .master.clk_speed = I2C_MASTER_FREQ_HZ
00093     };
00094
00095     /*
00096      * Configuración del periférico I2C.
00097      */
00098     i2c_param_config(
00099         I2C_MASTER_NUM,
00100         &conf);
00101
00102     /*
00103      * Instalación del driver I2C.
00104      */
00105     i2c_driver_install(
00106         I2C_MASTER_NUM,
00107         conf.mode,
00108         0,
00109         0,
00110         0);
00111
00112     /*
00113      * Creación del mutex global para protección
00114      * de acceso al bus compartido.
00115      */
00116     mutex_i2c = xSemaphoreCreateMutex();
00117
00118     ESP_LOGI(TAG, "I2C inicializado");
00119 }
```

6.39.3.2. i2c_manager_read()

```
esp_err_t i2c_manager_read (  
    uint8_t addr,  
    uint8_t reg,  
    uint8_t * data,  
    size_t len)
```

Lee datos desde un registro de un dispositivo I2C.

La secuencia implementada es:

START -> Dirección dispositivo + Write -> Registro origen -> RESTART -> Dirección dispositivo + Read -> Datos
-> STOP

Parámetros

<i>addr</i>	Dirección I2C del dispositivo.
<i>reg</i>	Registro origen.
<i>data</i>	Buffer de recepción.
<i>len</i>	Número de bytes a leer.

Devuelve

Código de error ESP-IDF.

Realiza una transacción de lectura a partir del registro especificado.

Parámetros

<i>addr</i>	Dirección I2C del dispositivo.
<i>reg</i>	Registro interno a leer.
<i>data</i>	Buffer donde se almacenarán los datos.
<i>len</i>	Número de bytes a recibir.

Devuelve

- ESP_OK: Operación exitosa.
- Código de error en caso contrario.

Definición en la línea [207](#) del archivo [i2c_manager.c](#).

```
00212 {  
00213     i2c_cmd_handle_t cmd =  
00214         i2c_cmd_link_create();  
00215  
00216     i2c_master_start(cmd);  
00217  
00218     i2c_master_write_byte(  
00219         cmd,  
00220         (addr << 1) | I2C_MASTER_WRITE,  
00221         true);  
00222  
00223     i2c_master_write_byte(  
00224         cmd,  
00225         reg,  
00226         true);
```

```

00227
00228     i2c_master_start(cmd);
00229
00230     i2c_master_write_byte(
00231         cmd,
00232         (addr << 1) | I2C_MASTER_READ,
00233         true);
00234
00235     if (len > 1) {
00236         i2c_master_read(
00237             cmd,
00238             data,
00239             len - 1,
00240             I2C_MASTER_ACK);
00241     }
00242
00243     i2c_master_read_byte(
00244         cmd,
00245         data + len - 1,
00246         I2C_MASTER_NACK);
00247
00248     i2c_master_stop(cmd);
00249
00250     esp_err_t ret =
00251         i2c_master_cmd_begin(
00252             I2C_MASTER_NUM,
00253             cmd,
00254             pdMS_TO_TICKS(100));
00255
00256     i2c_cmd_link_delete(cmd);
00257
00258     return ret;
00259 }

```

6.39.3.3. i2c_manager_write()

```

esp_err_t i2c_manager_write (
    uint8_t addr,
    uint8_t reg,
    uint8_t * data,
    size_t len)

```

Escribe datos en un registro de un dispositivo I2C.

La secuencia implementada es:

START -> Dirección dispositivo + Write -> Registro destino -> Datos -> STOP

Parámetros

<i>addr</i>	Dirección I2C del dispositivo.
<i>reg</i>	Registro de destino.
<i>data</i>	Datos a transmitir.
<i>len</i>	Número de bytes a enviar.

Devuelve

Código de error ESP-IDF.

Realiza una transacción de escritura compuesta por:

- Dirección del dispositivo.
- Dirección del registro.

- Datos asociados.

Parámetros

<i>addr</i>	Dirección I2C del dispositivo.
<i>reg</i>	Registro interno de destino.
<i>data</i>	Buffer con los datos a transmitir.
<i>len</i>	Número de bytes a escribir.

Devuelve

- ESP_OK: Operación exitosa.
- Código de error en caso contrario.

Definición en la línea 143 del archivo `i2c_manager.c`.

```

00148 {
00149     i2c_cmd_handle_t cmd =
00150         i2c_cmd_link_create();
00151
00152     i2c_master_start(cmd);
00153
00154     i2c_master_write_byte(
00155         cmd,
00156         (addr << 1) | I2C_MASTER_WRITE,
00157         true);
00158
00159     i2c_master_write_byte(
00160         cmd,
00161         reg,
00162         true);
00163
00164     i2c_master_write(
00165         cmd,
00166         data,
00167         len,
00168         true);
00169
00170     i2c_master_stop(cmd);
00171
00172     esp_err_t ret =
00173         i2c_master_cmd_begin(
00174             I2C_MASTER_NUM,
00175             cmd,
00176             pdMS_TO_TICKS(100));
00177
00178     i2c_cmd_link_delete(cmd);
00179
00180     return ret;
00181 }
```

6.39.3.4. i2c_manager_write_raw()

```

esp_err_t i2c_manager_write_raw (
    uint8_t addr,
    const uint8_t * data,
    size_t len)
```

Envía un bloque de datos sin especificar registro.

Esta función es utilizada por dispositivos que requieren transmisiones directas de comandos o datos, como el controlador OLED SSD1306.

Parámetros

<i>addr</i>	Dirección I2C del dispositivo.
<i>data</i>	Datos a transmitir.
<i>len</i>	Cantidad de bytes.

Devuelve

Código de error ESP-IDF.

Esta función es utilizada por dispositivos que requieren transmisiones directas de comandos o datos, como la pantalla OLED SSD1306.

Parámetros

<i>addr</i>	Dirección I2C del dispositivo.
<i>data</i>	Datos a transmitir.
<i>len</i>	Número de bytes a enviar.

Devuelve

- ESP_OK: Operación exitosa.
- Código de error en caso contrario.

Definición en la línea 278 del archivo [i2c_manager.c](#).

```

00282 {
00283     i2c_cmd_handle_t cmd =
00284         i2c_cmd_link_create();
00285
00286     i2c_master_start(cmd);
00287
00288     i2c_master_write_byte(
00289         cmd,
00290         (addr << 1) | I2C_MASTER_WRITE,
00291         true);
00292
00293     i2c_master_write(
00294         cmd,
00295         (uint8_t *)data,
00296         len,
00297         true);
00298
00299     i2c_master_stop(cmd);
00300
00301     esp_err_t ret =
00302         i2c_master_cmd_begin(
00303             I2C_MASTER_NUM,
00304             cmd,
00305             pdMS_TO_TICKS(100));
00306
00307     i2c_cmd_link_delete(cmd);
00308
00309     return ret;
00310 }
```

6.39.4. Documentación de variables**6.39.4.1. mutex_i2c**

```
SemaphoreHandle_t mutex_i2c = NULL
```

Mutex global para acceso seguro al bus I2C.

Mutex compartido para acceso exclusivo al bus I2C.

Garantiza exclusión mutua cuando múltiples tareas intentan acceder simultáneamente a dispositivos conectados al mismo bus.

Mutex global para acceso seguro al bus I2C.

Utilizado por todos los periféricos que comparten el mismo bus, evitando condiciones de carrera entre tareas concurrentes.

Definición en la línea 66 del archivo `i2c_manager.c`.

6.40. i2c_manager.c

[Ir a la documentación de este archivo.](#)

```

00001
00025
00026 #include "i2c_manager.h"
00027
00028 #include "driver/i2c.h"
00029 #include "esp_log.h"
00030
00034 #define TAG "I2C"
00035
00039 #define I2C_MASTER_SCL_IO 22
00040
00044 #define I2C_MASTER_SDA_IO 21
00045
00049 #define I2C_MASTER_NUM I2C_NUM_0
00050
00057 #define I2C_MASTER_FREQ_HZ 100000
00058
00066 SemaphoreHandle_t mutex_i2c = NULL;
00067
00068 // =====
00069 // INIT
00070 // =====
00071
00084 void i2c_manager_init(void)
00085 {
00086     i2c_config_t conf = {
00087         .mode = I2C_MODE_MASTER,
00088         .sda_io_num = I2C_MASTER_SDA_IO,
00089         .scl_io_num = I2C_MASTER_SCL_IO,
00090         .sda_pullup_en = GPIO_PULLUP_ENABLE,
00091         .scl_pullup_en = GPIO_PULLUP_ENABLE,
00092         .master.clk_speed = I2C_MASTER_FREQ_HZ
00093     };
00094
00095     /*
00096      * Configuración del periférico I2C.
00097      */
00098     i2c_param_config(
00099         I2C_MASTER_NUM,
00100         &conf);
00101
00102     /*
00103      * Instalación del driver I2C.
00104      */
00105     i2c_driver_install(
00106         I2C_MASTER_NUM,
00107         conf.mode,
00108         0,
00109         0,
00110         0);
00111
00112     /*
00113      * Creación del mutex global para protección
00114      * de acceso al bus compartido.
00115      */
00116     mutex_i2c = xSemaphoreCreateMutex();
00117
00118     ESP_LOGI(TAG, "I2C inicializado");

```

```
00119 }
00120
00121 // =====
00122 // WRITE
00123 // =====
00124
00143 esp_err_t i2c_manager_write(
00144     uint8_t addr,
00145     uint8_t reg,
00146     uint8_t *data,
00147     size_t len)
00148 {
00149     i2c_cmd_handle_t cmd =
00150         i2c_cmd_link_create();
00151
00152     i2c_master_start(cmd);
00153
00154     i2c_master_write_byte(
00155         cmd,
00156         (addr << 1) | I2C_MASTER_WRITE,
00157         true);
00158
00159     i2c_master_write_byte(
00160         cmd,
00161         reg,
00162         true);
00163
00164     i2c_master_write(
00165         cmd,
00166         data,
00167         len,
00168         true);
00169
00170     i2c_master_stop(cmd);
00171
00172     esp_err_t ret =
00173         i2c_master_cmd_begin(
00174             I2C_MASTER_NUM,
00175             cmd,
00176             pdMS_TO_TICKS(100));
00177
00178     i2c_cmd_link_delete(cmd);
00179
00180     return ret;
00181 }
00182
00183 // =====
00184 // READ
00185 // =====
00186
00207 esp_err_t i2c_manager_read(
00208     uint8_t addr,
00209     uint8_t reg,
00210     uint8_t *data,
00211     size_t len)
00212 {
00213     i2c_cmd_handle_t cmd =
00214         i2c_cmd_link_create();
00215
00216     i2c_master_start(cmd);
00217
00218     i2c_master_write_byte(
00219         cmd,
00220         (addr << 1) | I2C_MASTER_WRITE,
00221         true);
00222
00223     i2c_master_write_byte(
00224         cmd,
00225         reg,
00226         true);
00227
00228     i2c_master_start(cmd);
00229
00230     i2c_master_write_byte(
00231         cmd,
00232         (addr << 1) | I2C_MASTER_READ,
00233         true);
00234
00235     if (len > 1) {
00236         i2c_master_read(
00237             cmd,
00238             data,
00239             len - 1,
00240             I2C_MASTER_ACK);
00241     }
00242
00243     i2c_master_read_byte(
```



```

00244         cmd,
00245         data + len - 1,
00246         I2C_MASTER_NACK);
00247
00248     i2c_master_stop(cmd);
00249
00250     esp_err_t ret =
00251         i2c_master_cmd_begin(
00252             I2C_MASTER_NUM,
00253             cmd,
00254             pdMS_TO_TICKS(100));
00255
00256     i2c_cmd_link_delete(cmd);
00257
00258     return ret;
00259 }
00260
00261 // =====
00262 // WRITE RAW
00263 // =====
00264
00278 esp_err_t i2c_manager_write_raw(
00279     uint8_t addr,
00280     const uint8_t *data,
00281     size_t len)
00282 {
00283     i2c_cmd_handle_t cmd =
00284         i2c_cmd_link_create();
00285
00286     i2c_master_start(cmd);
00287
00288     i2c_master_write_byte(
00289         cmd,
00290         (addr << 1) | I2C_MASTER_WRITE,
00291         true);
00292
00293     i2c_master_write(
00294         cmd,
00295         (uint8_t *)data,
00296         len,
00297         true);
00298
00299     i2c_master_stop(cmd);
00300
00301     esp_err_t ret =
00302         i2c_master_cmd_begin(
00303             I2C_MASTER_NUM,
00304             cmd,
00305             pdMS_TO_TICKS(100));
00306
00307     i2c_cmd_link_delete(cmd);
00308
00309     return ret;
00310 }

```

6.41. Referencia del archivo main/hal/i2c_manager.h

Capa de abstracción para el bus I2C del ESP32.

```

#include "freertos/FreeRTOS.h"
#include "freertos/semphr.h"
#include "esp_err.h"

```

Funciones

- void [i2c_manager_init](#) (void)
Inicializa el bus I2C en modo maestro.
- esp_err_t [i2c_manager_write](#) (uint8_t addr, uint8_t reg, uint8_t *data, size_t len)
Escribe datos en un registro de un dispositivo I2C.
- esp_err_t [i2c_manager_read](#) (uint8_t addr, uint8_t reg, uint8_t *data, size_t len)
Lee datos desde un registro de un dispositivo I2C.
- esp_err_t [i2c_manager_write_raw](#) (uint8_t addr, const uint8_t *data, size_t len)
Envía un bloque de datos sin especificar registro.

Variables

- SemaphoreHandle_t [mutex_i2c](#)
Mutex global para acceso seguro al bus I2C.

6.41.1. Descripción detallada

Capa de abstracción para el bus I2C del ESP32.

Este módulo proporciona una interfaz común para la comunicación con dispositivos conectados al bus I2C.

Funcionalidades:

- Inicialización del bus I2C en modo maestro.
- Lectura de registros.
- Escritura de registros.
- Escritura de bloques de datos.
- Protección mediante mutex para acceso concurrente.

Los siguientes módulos utilizan esta capa:

- [rtc_ds3231](#)
- [oled_display](#)

Esta abstracción permite desacoplar los drivers de los detalles específicos de ESP-IDF y facilita el mantenimiento del sistema.

Autor

Fernando Plazas Trujillo Isabella Ordoñez Juan Daniel Constain

Definición en el archivo [i2c_manager.h](#).

6.41.2. Documentación de funciones

6.41.2.1. [i2c_manager_init\(\)](#)

```
void i2c_manager_init (  
    void )
```

Inicializa el bus I2C en modo maestro.

Configura los pines SDA y SCL, la frecuencia del bus y crea los recursos necesarios para la comunicación.

Esta función debe ejecutarse una única vez durante la inicialización del sistema.

Configura:

- Pines SDA y SCL.
- Pull-ups internas.
- Frecuencia de operación.
- Driver I2C de ESP-IDF.

Además crea el mutex utilizado para sincronizar el acceso concurrente al bus.

Definición en la línea 84 del archivo `i2c_manager.c`.

```
00085 {
00086     i2c_config_t conf = {
00087         .mode = I2C_MODE_MASTER,
00088         .sda_io_num = I2C_MASTER_SDA_IO,
00089         .scl_io_num = I2C_MASTER_SCL_IO,
00090         .sda_pullup_en = GPIO_PULLUP_ENABLE,
00091         .scl_pullup_en = GPIO_PULLUP_ENABLE,
00092         .master.clk_speed = I2C_MASTER_FREQ_HZ
00093     };
00094
00095     /*
00096      * Configuración del periférico I2C.
00097      */
00098     i2c_param_config(
00099         I2C_MASTER_NUM,
00100         &conf);
00101
00102     /*
00103      * Instalación del driver I2C.
00104      */
00105     i2c_driver_install(
00106         I2C_MASTER_NUM,
00107         conf.mode,
00108         0,
00109         0,
00110         0);
00111
00112     /*
00113      * Creación del mutex global para protección
00114      * de acceso al bus compartido.
00115      */
00116     mutex_i2c = xSemaphoreCreateMutex();
00117
00118     ESP_LOGI(TAG, "I2C inicializado");
00119 }
```

6.41.2.2. `i2c_manager_read()`

```
esp_err_t i2c_manager_read (
    uint8_t addr,
    uint8_t reg,
    uint8_t * data,
    size_t len)
```

Lee datos desde un registro de un dispositivo I2C.

Realiza una transacción de lectura a partir del registro especificado.

Parámetros

<i>addr</i>	Dirección I2C del dispositivo.
<i>reg</i>	Registro interno a leer.
<i>data</i>	Buffer donde se almacenarán los datos.
<i>len</i>	Número de bytes a recibir.

Devuelve

- ESP_OK: Operación exitosa.
- Código de error en caso contrario.

La secuencia implementada es:

START -> Dirección dispositivo + Write -> Registro origen -> RESTART -> Dirección dispositivo + Read -> Datos
-> STOP

Parámetros

<i>addr</i>	Dirección I2C del dispositivo.
<i>reg</i>	Registro origen.
<i>data</i>	Buffer de recepción.
<i>len</i>	Número de bytes a leer.

Devuelve

Código de error ESP-IDF.

Definición en la línea [207](#) del archivo [i2c_manager.c](#).

```

00212 {
00213     i2c_cmd_handle_t cmd =
00214         i2c_cmd_link_create();
00215
00216     i2c_master_start(cmd);
00217
00218     i2c_master_write_byte(
00219         cmd,
00220         (addr << 1) | I2C_MASTER_WRITE,
00221         true);
00222
00223     i2c_master_write_byte(
00224         cmd,
00225         reg,
00226         true);
00227
00228     i2c_master_start(cmd);
00229
00230     i2c_master_write_byte(
00231         cmd,
00232         (addr << 1) | I2C_MASTER_READ,
00233         true);
00234
00235     if (len > 1) {
00236         i2c_master_read(
00237             cmd,
00238             data,
00239             len - 1,
00240             I2C_MASTER_ACK);
00241     }
00242
00243     i2c_master_read_byte(
00244         cmd,
00245         data + len - 1,
00246         I2C_MASTER_NACK);
00247
00248     i2c_master_stop(cmd);
00249
00250     esp_err_t ret =
00251         i2c_master_cmd_begin(
00252             I2C_MASTER_NUM,
00253             cmd,
00254             pdMS_TO_TICKS(100));
00255
00256     i2c_cmd_link_delete(cmd);
00257
00258     return ret;
00259 }

```

6.41.2.3. i2c_manager_write()

```
esp_err_t i2c_manager_write (  
    uint8_t addr,  
    uint8_t reg,  
    uint8_t * data,  
    size_t len)
```

Escribe datos en un registro de un dispositivo I2C.

Realiza una transacción de escritura compuesta por:

- Dirección del dispositivo.
- Dirección del registro.
- Datos asociados.

Parámetros

<i>addr</i>	Dirección I2C del dispositivo.
<i>reg</i>	Registro interno de destino.
<i>data</i>	Buffer con los datos a transmitir.
<i>len</i>	Número de bytes a escribir.

Devuelve

- ESP_OK: Operación exitosa.
- Código de error en caso contrario.

La secuencia implementada es:

START -> Dirección dispositivo + Write -> Registro destino -> Datos -> STOP

Parámetros

<i>addr</i>	Dirección I2C del dispositivo.
<i>reg</i>	Registro de destino.
<i>data</i>	Datos a transmitir.
<i>len</i>	Número de bytes a enviar.

Devuelve

Código de error ESP-IDF.

Definición en la línea 143 del archivo `i2c_manager.c`.

```
00148 {
00149     i2c_cmd_handle_t cmd =
00150         i2c_cmd_link_create();
00151
00152     i2c_master_start(cmd);
00153
00154     i2c_master_write_byte(
00155         cmd,
00156         (addr << 1) | I2C_MASTER_WRITE,
00157         true);
00158
00159     i2c_master_write_byte(
00160         cmd,
00161         reg,
00162         true);
00163
00164     i2c_master_write(
00165         cmd,
00166         data,
00167         len,
00168         true);
00169
00170     i2c_master_stop(cmd);
00171
00172     esp_err_t ret =
00173         i2c_master_cmd_begin(
00174             I2C_MASTER_NUM,
00175             cmd,
00176             pdMS_TO_TICKS(100));
00177
00178     i2c_cmd_link_delete(cmd);
00179
00180     return ret;
00181 }
```

6.41.2.4. i2c_manager_write_raw()

```
esp_err_t i2c_manager_write_raw (
    uint8_t addr,
    const uint8_t * data,
    size_t len)
```

Envía un bloque de datos sin especificar registro.

Esta función es utilizada por dispositivos que requieren transmisiones directas de comandos o datos, como la pantalla OLED SSD1306.

Parámetros

<i>addr</i>	Dirección I2C del dispositivo.
<i>data</i>	Datos a transmitir.
<i>len</i>	Número de bytes a enviar.

Devuelve

- ESP_OK: Operación exitosa.
- Código de error en caso contrario.

Esta función es utilizada por dispositivos que requieren transmisiones directas de comandos o datos, como el controlador OLED SSD1306.

Parámetros

<i>addr</i>	Dirección I2C del dispositivo.
<i>data</i>	Datos a transmitir.
<i>len</i>	Cantidad de bytes.

Devuelve

Código de error ESP-IDF.

Definición en la línea 278 del archivo [i2c_manager.c](#).

```

00282 {
00283     i2c_cmd_handle_t cmd =
00284         i2c_cmd_link_create();
00285
00286     i2c_master_start(cmd);
00287
00288     i2c_master_write_byte(
00289         cmd,
00290         (addr << 1) | I2C_MASTER_WRITE,
00291         true);
00292
00293     i2c_master_write(
00294         cmd,
00295         (uint8_t *)data,
00296         len,
00297         true);
00298
00299     i2c_master_stop(cmd);
00300
00301     esp_err_t ret =
00302         i2c_master_cmd_begin(
00303             I2C_MASTER_NUM,
00304             cmd,
00305             pdMS_TO_TICKS(100));
00306
00307     i2c_cmd_link_delete(cmd);
00308
00309     return ret;
00310 }
```

6.41.3. Documentación de variables

6.41.3.1. mutex_i2c

```
SemaphoreHandle_t mutex_i2c [extern]
```

Mutex global para acceso seguro al bus I2C.

Garantiza exclusión mutua cuando múltiples tareas acceden simultáneamente a dispositivos conectados al mismo bus.

Mutex compartido para acceso exclusivo al bus I2C.

Garantiza exclusión mutua cuando múltiples tareas intentan acceder simultáneamente a dispositivos conectados al mismo bus.

Mutex global para acceso seguro al bus I2C.

Utilizado por todos los periféricos que comparten el mismo bus, evitando condiciones de carrera entre tareas concurrentes.

Definición en la línea 66 del archivo [i2c_manager.c](#).

6.42. i2c_manager.h

[Ir a la documentación de este archivo.](#)

```

00001
00027
00028 #ifndef I2C_MANAGER_H
00029 #define I2C_MANAGER_H
00030
00031 #include "freertos/FreeRTOS.h"
00032 #include "freertos/semphr.h"
00033
00034 #include "esp_err.h"
00035
00043 extern SemaphoreHandle_t mutex_i2c;
00044
00054 void i2c_manager_init(void);
00055
00073 esp_err_t i2c_manager_write(
00074     uint8_t addr,
00075     uint8_t reg,
00076     uint8_t *data,
00077     size_t len);
00078
00094 esp_err_t i2c_manager_read(
00095     uint8_t addr,
00096     uint8_t reg,
00097     uint8_t *data,
00098     size_t len);
00099
00115 esp_err_t i2c_manager_write_raw(
00116     uint8_t addr,
00117     const uint8_t *data,
00118     size_t len);
00119
00120 #endif

```

6.43. Referencia del archivo main/main.c

Punto de entrada del sistema de fermentación basado en ESP32.

```

#include <stdio.h>
#include <stdbool.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "esp_log.h"
#include "esp_sleep.h"
#include "hal/i2c_manager.h"
#include "hal/adc_manager.h"
#include "drivers/rtc_ds3231.h"
#include "tasks/task_sensores.h"
#include "tasks/task_control.h"
#include "tasks/task_actuadores.h"
#include "tasks/task_logger.h"
#include "tasks/task_energia.h"
#include "tasks/task_mqtt.h"
#include "tasks/task_display.h"
#include "wifi/wifi_manager.h"
#include "queues.h"
#include "drivers/oled_display.h"

```

defines

- #define [WIFI_CONNECT_TIMEOUT_MS](#) 10000
Tiempo máximo de espera para conexión WiFi.

Funciones

- void **app_main** (void)
Función principal del sistema.

Variables

- TaskHandle_t **task_sensores_handle** = NULL
Handle de la tarea de sensores.
- TaskHandle_t **task_energia_handle** = NULL
Handle de la tarea de energía.
- static const char * **TAG** = "MAIN"
Etiqueta utilizada para mensajes de depuración.

6.43.1. Descripción detallada

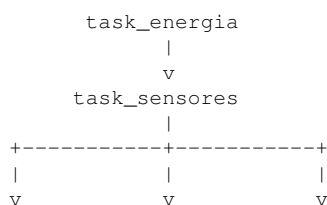
Punto de entrada del sistema de fermentación basado en ESP32.

Este módulo realiza la inicialización completa del sistema, configura los periféricos necesarios, crea las tareas FreeRTOS y pone en marcha el ciclo operativo de adquisición, control, almacenamiento y comunicación.

Funciones principales:

- Inicialización de hardware.
- Inicialización de red WiFi.
- Configuración del RTC DS3231.
- Configuración de wakeup para Deep Sleep.
- Creación de colas FreeRTOS.
- Creación de tareas del sistema.
- Inicio del ciclo operativo.

Arquitectura general:



```
task_control task_logger task_mqtt | v v task_actuadores Broker MQTT | v Actuadores
```

```

      |
      v
task_display

```

Mecanismos FreeRTOS utilizados:

- Tareas.
- Colas.
- Notificaciones.

Gestión energética:

- Deep Sleep.
- Wakeup mediante RTC DS3231.

Autor

Fernando Plazas Trujillo Isabella Ordoñez Juan Daniel Constain

Definición en el archivo [main.c](#).

6.43.2. Documentación de «define»

6.43.2.1. WIFI_CONNECT_TIMEOUT_MS

```
#define WIFI_CONNECT_TIMEOUT_MS 10000
```

Tiempo máximo de espera para conexión WiFi.

Definición en la línea 119 del archivo [main.c](#).

6.43.3. Documentación de funciones

6.43.3.1. app_main()

```
void app_main (  
    void )
```

Función principal del sistema.

Realiza toda la secuencia de inicialización:

1. Conexión WiFi.
2. Inicialización de hardware.
3. Configuración del RTC.
4. Configuración de Deep Sleep.
5. Creación de colas.
6. Creación de tareas.
7. Inicio del ciclo operativo.

Variable persistente utilizada para alternar entre los instantes de despertar.

Definición en la línea 134 del archivo `main.c`.

```

00135 {
00136     ESP_LOGI(TAG, "Iniciando sistema...");
00137
00138     // =====
00139     // INICIALIZACIÓN WIFI
00140     // =====
00141
00142     wifi_init();
00143
00144     bool wifi_connected =
00145         wifi_wait_connected(
00146             WIFI_CONNECT_TIMEOUT_MS);
00147
00148     if (wifi_connected)
00149     {
00150         ESP_LOGI(TAG, "WiFi listo");
00151     }
00152     else
00153     {
00154         ESP_LOGW(TAG,
00155             "WiFi no conectado, se continua sin red");
00156
00157         wifi_scan_print();
00158     }
00159
00160     // =====
00161     // INICIALIZACIÓN DE HARDWARE
00162     // =====
00163
00164     i2c_manager_init();
00165
00166     adc_manager_init();
00167
00168     // =====
00169     // CAUSA DE DESPERTAR
00170     // =====
00171
00172     esp_sleep_wakeup_cause_t cause =
00173         esp_sleep_get_wakeup_cause();
00174
00175     ESP_LOGI(
00176         TAG,
00177         "Wakeup cause: %d",
00178         cause);
00179
00180     // =====
00181     // CONFIGURACIÓN DE WAKEUP
00182     // =====
00183
00184     esp_sleep_enable_ext0_wakeup(
00185         GPIO_NUM_33,
00186         0);
00187
00188     // =====
00189     // INICIALIZACIÓN RTC
00190     // =====
00191
00192     ds3231_init();
00193
00194     ds3231_clear_alarm_flag();
00195
00196     // =====
00197     // CONFIGURACIÓN DE ALARMA RTC
00198     // =====
00199
00200     static RTC_DATA_ATTR int toggle = 0;
00201
00202     if (toggle == 0)
00203     {
00204         ds3231_set_alarm_seconds(30);
00205         toggle = 1;
00206     }
00207     else
00208     {
00209         ds3231_set_alarm_seconds(0);
00210         toggle = 0;
00211     }
00212
00213     // =====
00214     // CREACIÓN DE COLAS
00215     // =====
00216
00217     queues_init();

```

```
00222
00223 // =====
00224 // CREACIÓN DE TAREAS
00225 // =====
00226
00227 BaseType_t res;
00228
00229 res = xTaskCreate(
00230     task_sensores,
00231     "task_sensores",
00232     4096,
00233     NULL,
00234     5,
00235     &task_sensores_handle);
00236
00237 if (res != pdPASS)
00238 {
00239     ESP_LOGE(TAG,
00240         "Error creando task_sensores");
00241 }
00242
00243 res = xTaskCreate(
00244     task_control,
00245     "task_control",
00246     4096,
00247     NULL,
00248     5,
00249     NULL);
00250
00251 if (res != pdPASS)
00252 {
00253     ESP_LOGE(TAG,
00254         "Error creando task_control");
00255 }
00256
00257 res = xTaskCreate(
00258     task_actuadores,
00259     "task_actuadores",
00260     4096,
00261     NULL,
00262     5,
00263     NULL);
00264
00265 if (res != pdPASS)
00266 {
00267     ESP_LOGE(TAG,
00268         "Error creando task_actuadores");
00269 }
00270
00271 res = xTaskCreate(
00272     task_logger,
00273     "task_logger",
00274     4096,
00275     NULL,
00276     5,
00277     NULL);
00278
00279 if (res != pdPASS)
00280 {
00281     ESP_LOGE(TAG,
00282         "Error creando task_logger");
00283 }
00284
00285 res = xTaskCreate(
00286     task_mqtt,
00287     "task_mqtt",
00288     6144,
00289     NULL,
00290     5,
00291     NULL);
00292
00293 if (res != pdPASS)
00294 {
00295     ESP_LOGE(TAG,
00296         "Error creando task_mqtt");
00297 }
00298
00299 res = xTaskCreate(
00300     task_display,
00301     "task_display",
00302     4096,
00303     NULL,
00304     4,
00305     NULL);
00306
00307 if (res != pdPASS)
00308 {
```

```

00309     ESP_LOGE(TAG,
00310               "Error creando task_display");
00311   }
00312
00313   res = xTaskCreate(
00314     task_energia,
00315     "task_energia",
00316     4096,
00317     NULL,
00318     5,
00319     &task_energia_handle);
00320
00321   if (res != pdPASS)
00322   {
00323     ESP_LOGE(TAG,
00324             "Error creando task_energia");
00325   }
00326
00327   ESP_LOGI(TAG, "Sistema listo");
00328
00329   // =====
00330   // INICIO DEL CICLO OPERATIVO
00331   // =====
00332
00333   xTaskNotifyGive(
00334     task_energia_handle);
00335 }

```

6.43.4. Documentación de variables

6.43.4.1. TAG

```
const char* TAG = "MAIN" [static]
```

Etiqueta utilizada para mensajes de depuración.

Definición en la línea 114 del archivo [main.c](#).

6.43.4.2. task_energia_handle

```
TaskHandle_t task_energia_handle = NULL
```

Handle de la tarea de energía.

Utilizado para coordinar el ciclo general del sistema.

Utilizado para enviar notificaciones cuando finaliza un ciclo de mezcla.

Definición en la línea 109 del archivo [main.c](#).

6.43.4.3. task_sensores_handle

```
TaskHandle_t task_sensores_handle = NULL
```

Handle de la tarea de sensores.

Utilizado por task_energia para iniciar nuevos ciclos de adquisición.

Utilizado para iniciar un nuevo ciclo de adquisición.

Definición en la línea 101 del archivo [main.c](#).

6.44. main.c

[Ir a la documentación de este archivo.](#)

```

00001
00055
00056 #include <stdio.h>
00057 #include <stdbool.h>
00058
00059 #include "freertos/FreeRTOS.h"
00060 #include "freertos/task.h"
00061
00062 #include "esp_log.h"
00063 #include "esp_sleep.h"
00064
00065 // HAL
00066 #include "hal/i2c_manager.h"
00067 #include "hal/adc_manager.h"
00068
00069 // Drivers
00070 #include "drivers/rtc_ds3231.h"
00071
00072 // Tasks
00073 #include "tasks/task_sensores.h"
00074 #include "tasks/task_control.h"
00075 #include "tasks/task_actuadores.h"
00076 #include "tasks/task_logger.h"
00077 #include "tasks/task_energia.h"
00078 #include "tasks/task_mqtt.h"
00079 #include "tasks/task_display.h"
00080
00081 // Network
00082 #include "wifi/wifi_manager.h"
00083
00084 // Queues
00085 #include "queues.h"
00086
00087 // Drivers
00088 #include "drivers/rtc_ds3231.h"
00089 #include "drivers/oled_display.h"
00090
00091 // =====
00092 // HANDLES DE TAREAS
00093 // =====
00094
00101 TaskHandle_t task_sensores_handle = NULL;
00102
00109 TaskHandle_t task_energia_handle = NULL;
00110
00114 static const char *TAG = "MAIN";
00115
00119 #define WIFI_CONNECT_TIMEOUT_MS 10000
00120
00134 void app_main(void)
00135 {
00136     ESP_LOGI(TAG, "Iniciando sistema...");
00137
00138     // =====
00139     // INICIALIZACIÓN WIFI
00140     // =====
00141
00142     wifi_init();
00143
00144     bool wifi_connected =
00145         wifi_wait_connected(
00146             WIFI_CONNECT_TIMEOUT_MS);
00147
00148     if (wifi_connected)
00149     {
00150         ESP_LOGI(TAG, "WiFi listo");
00151     }
00152     else
00153     {
00154         ESP_LOGW(TAG,
00155             "WiFi no conectado, se continua sin red");
00156
00157         wifi_scan_print();
00158     }
00159
00160     // =====
00161     // INICIALIZACIÓN DE HARDWARE
00162     // =====
00163
00164     i2c_manager_init();
00165
00166     adc_manager_init();

```

```

00167
00168 // =====
00169 // CAUSA DE DESPERTAR
00170 // =====
00171
00172 esp_sleep_wakeup_cause_t cause =
00173     esp_sleep_get_wakeup_cause();
00174
00175 ESP_LOGI(
00176     TAG,
00177     "Wakeup cause: %d",
00178     cause);
00179
00180 // =====
00181 // CONFIGURACIÓN DE WAKEUP
00182 // =====
00183
00184 esp_sleep_enable_ext0_wakeup(
00185     GPIO_NUM_33,
00186     0);
00187
00188 // =====
00189 // INICIALIZACIÓN RTC
00190 // =====
00191
00192 ds3231_init();
00193
00194 ds3231_clear_alarm_flag();
00195
00196 // =====
00197 // CONFIGURACIÓN DE ALARMA RTC
00198 // =====
00199
00200 static RTC_DATA_ATTR int toggle = 0;
00201
00202 if (toggle == 0)
00203 {
00204     ds3231_set_alarm_seconds(30);
00205     toggle = 1;
00206 }
00207 else
00208 {
00209     ds3231_set_alarm_seconds(0);
00210     toggle = 0;
00211 }
00212
00213 // =====
00214 // CREACIÓN DE COLAS
00215 // =====
00216
00217 queues_init();
00218
00219 // =====
00220 // CREACIÓN DE TAREAS
00221 // =====
00222
00223 BaseType_t res;
00224
00225 res = xTaskCreate(
00226     task_sensores,
00227     "task_sensores",
00228     4096,
00229     NULL,
00230     5,
00231     &task_sensores_handle);
00232
00233 if (res != pdPASS)
00234 {
00235     ESP_LOGE(TAG,
00236         "Error creando task_sensores");
00237 }
00238
00239 res = xTaskCreate(
00240     task_control,
00241     "task_control",
00242     4096,
00243     NULL,
00244     5,
00245     NULL);
00246
00247 if (res != pdPASS)
00248 {
00249     ESP_LOGE(TAG,
00250         "Error creando task_control");
00251 }
00252
00253 res = xTaskCreate(

```

```

00258     task_actuadores,
00259     "task_actuadores",
00260     4096,
00261     NULL,
00262     5,
00263     NULL);
00264
00265     if (res != pdPASS)
00266     {
00267         ESP_LOGE(TAG,
00268                  "Error creando task_actuadores");
00269     }
00270
00271     res = xTaskCreate(
00272         task_logger,
00273         "task_logger",
00274         4096,
00275         NULL,
00276         5,
00277         NULL);
00278
00279     if (res != pdPASS)
00280     {
00281         ESP_LOGE(TAG,
00282                  "Error creando task_logger");
00283     }
00284
00285     res = xTaskCreate(
00286         task_mqtt,
00287         "task_mqtt",
00288         6144,
00289         NULL,
00290         5,
00291         NULL);
00292
00293     if (res != pdPASS)
00294     {
00295         ESP_LOGE(TAG,
00296                  "Error creando task_mqtt");
00297     }
00298
00299     res = xTaskCreate(
00300         task_display,
00301         "task_display",
00302         4096,
00303         NULL,
00304         4,
00305         NULL);
00306
00307     if (res != pdPASS)
00308     {
00309         ESP_LOGE(TAG,
00310                  "Error creando task_display");
00311     }
00312
00313     res = xTaskCreate(
00314         task_energia,
00315         "task_energia",
00316         4096,
00317         NULL,
00318         5,
00319         &task_energia_handle);
00320
00321     if (res != pdPASS)
00322     {
00323         ESP_LOGE(TAG,
00324                  "Error creando task_energia");
00325     }
00326
00327     ESP_LOGI(TAG, "Sistema listo");
00328
00329     // =====
00330     // INICIO DEL CICLO OPERATIVO
00331     // =====
00332
00333     xTaskNotifyGive(
00334         task_energia_handle);
00335 }

```

6.45. Referencia del archivo main/mqtt/mqtt_manager.c

Implementación del gestor de comunicación MQTT.


```
#include "mqtt_manager.h"
#include "freertos/FreeRTOS.h"
#include "freertos/event_groups.h"
#include "esp_event.h"
#include "esp_log.h"
#include "mqtt_client.h"
```

defines

- #define `MQTT_BROKER_URI` "mqtt://192.168.0.110:1883"
- #define `MQTT_CONNECTED_BIT` BIT0
Bit que indica conexión exitosa al broker MQTT.
- #define `MQTT_PUBLISHED_BIT` BIT1
Bit que indica publicación confirmada.
- #define `MQTT_ERROR_BIT` BIT2
Bit que indica error durante la operación MQTT.

Funciones

- static void `mqtt_event_handler` (void *handler_args, esp_event_base_t base, int32_t event_id, void *event_data)
Manejador de eventos MQTT.
- void `mqtt_manager_start` (void)
Inicializa y arranca el cliente MQTT.
- bool `mqtt_wait_connected` (uint32_t timeout_ms)
Espera la conexión con el broker MQTT.
- bool `mqtt_publish` (const char *topic, const char *payload, int qos, uint32_t timeout_ms)
Publica un mensaje en un tópico MQTT.
- void `mqtt_manager_stop` (void)
Detiene el servicio MQTT.
- bool `mqtt_is_connected` (void)
Consulta el estado de conexión MQTT.

Variables

- static const char * `TAG` = "mqtt_manager"
Etiqueta utilizada para mensajes de depuración.
- static esp_mqtt_client_handle_t `mqtt_client` = NULL
Manejador del cliente MQTT.
- static EventGroupHandle_t `mqtt_event_group` = NULL
Grupo de eventos utilizado para sincronización MQTT.
- static bool `mqtt_connected` = false
Indica si el cliente MQTT se encuentra conectado.

6.45.1. Descripción detallada

Implementación del gestor de comunicación MQTT.

Este módulo implementa la conexión y comunicación con un broker MQTT utilizando la biblioteca ESP-MQTT del ESP-IDF.

Funcionalidades principales:

- Inicialización del cliente MQTT.
- Gestión de eventos de conexión y desconexión.
- Publicación de mensajes en tópicos MQTT.
- Sincronización mediante Event Groups de FreeRTOS.
- Control del estado de conexión.

El módulo abstrae la complejidad del protocolo MQTT y proporciona una interfaz sencilla para las tareas de aplicación.

Mecanismos FreeRTOS utilizados:

- Event Groups para sincronización de eventos MQTT.

Eventos gestionados:

- Conexión al broker.
- Desconexión del broker.
- Confirmación de publicación.
- Errores de comunicación.

Autor

Fernando Plazas Trujillo Isabella Ordoñez Juan Daniel Constain

Definición en el archivo [mqtt_manager.c](#).

6.45.2. Documentación de «define»

6.45.2.1. MQTT_BROKER_URI

```
#define MQTT_BROKER_URI "mqtt://192.168.0.110:1883"
```

Definición en la línea [41](#) del archivo [mqtt_manager.c](#).

6.45.2.2. MQTT_CONNECTED_BIT

```
#define MQTT_CONNECTED_BIT BIT0
```

Bit que indica conexión exitosa al broker MQTT.

Definición en la línea 66 del archivo [mqtt_manager.c](#).

6.45.2.3. MQTT_ERROR_BIT

```
#define MQTT_ERROR_BIT BIT2
```

Bit que indica error durante la operación MQTT.

Definición en la línea 76 del archivo [mqtt_manager.c](#).

6.45.2.4. MQTT_PUBLISHED_BIT

```
#define MQTT_PUBLISHED_BIT BIT1
```

Bit que indica publicación confirmada.

Definición en la línea 71 del archivo [mqtt_manager.c](#).

6.45.3. Documentación de funciones

6.45.3.1. mqtt_event_handler()

```
void mqtt_event_handler (  
    void * handler_args,  
    esp_event_base_t base,  
    int32_t event_id,  
    void * event_data) [static]
```

Manejador de eventos MQTT.

Esta función es invocada automáticamente por la biblioteca ESP-MQTT cuando ocurre un evento relacionado con el cliente.

Gestiona:

- Conexión al broker.
- Desconexión.
- Confirmación de publicación.
- Errores de comunicación.

Además actualiza los Event Groups utilizados por las tareas de aplicación para sincronizar operaciones MQTT.

Parámetros

<i>handler_args</i>	Argumentos del manejador.
<i>base</i>	Base del evento.
<i>event_id</i>	Identificador del evento MQTT.
<i>event_data</i>	Información asociada al evento.

Definición en la línea 98 del archivo `mqtt_manager.c`.

```

00102 {
00103     esp_mqtt_event_handle_t event = event_data;
00104
00105     switch ((esp_mqtt_event_id_t) event_id)
00106     {
00107         case MQTT_EVENT_CONNECTED:
00108             ESP_LOGI(TAG, "MQTT conectado");
00109
00110             mqtt_connected = true;
00111
00112             xEventGroupSetBits(mqtt_event_group, MQTT_CONNECTED_BIT);
00113             xEventGroupClearBits(mqtt_event_group, MQTT_ERROR_BIT);
00114             break;
00115
00116         case MQTT_EVENT_DISCONNECTED:
00117             ESP_LOGW(TAG, "MQTT desconectado");
00118
00119             mqtt_connected = false;
00120
00121             xEventGroupClearBits(mqtt_event_group, MQTT_CONNECTED_BIT);
00122             break;
00123
00124         case MQTT_EVENT_PUBLISHED:
00125             ESP_LOGI(TAG, "MQTT publicado, msg_id=%d", event->msg_id);
00126             xEventGroupSetBits(mqtt_event_group, MQTT_PUBLISHED_BIT);
00127             break;
00128
00129         case MQTT_EVENT_ERROR:
00130             ESP_LOGE(TAG, "Error MQTT");
00131             xEventGroupSetBits(mqtt_event_group, MQTT_ERROR_BIT);
00132             break;
00133
00134         default:
00135             break;
00136     }
00137 }
```

6.45.3.2. mqtt_is_connected()

```

bool mqtt_is_connected (
    void )
```

Consulta el estado de conexión MQTT.

Consulta el estado actual de la conexión MQTT.

Devuelve

true si existe conexión activa con el broker.

false en caso contrario.

Consulta el estado de conexión MQTT.

Devuelve

true si el cliente está conectado al broker.

false si no existe conexión activa.

Definición en la línea 297 del archivo `mqtt_manager.c`.

```
00298 {  
00299     return mqtt_connected;  
00300 }
```

6.45.3.3. mqtt_manager_start()

```
void mqtt_manager_start (  
    void )
```

Inicializa y arranca el cliente MQTT.

Inicializa y pone en marcha el cliente MQTT.

Crea el grupo de eventos utilizado para sincronización, configura el cliente MQTT, registra el manejador de eventos y establece la conexión con el broker configurado.

Inicializa y arranca el cliente MQTT.

Configura el cliente MQTT, registra los eventos asociados y establece la conexión con el broker configurado.

Definición en la línea 145 del archivo `mqtt_manager.c`.

```
00146 {  
00147     if (mqtt_event_group == NULL)  
00148     {  
00149         mqtt_event_group = xEventGroupCreate();  
00150     }  
00151  
00152     if (mqtt_event_group == NULL)  
00153     {  
00154         ESP_LOGE(TAG, "No se pudo crear grupo de eventos MQTT");  
00155         return;  
00156     }  
00157  
00158     xEventGroupClearBits(mqtt_event_group,  
00159                         MQTT_CONNECTED_BIT | MQTT_PUBLISHED_BIT | MQTT_ERROR_BIT);  
00160  
00161     if (mqtt_client != NULL)  
00162     {  
00163         return;  
00164     }  
00165  
00166     esp_mqtt_client_config_t mqtt_cfg = {  
00167         .broker.address.uri = MQTT_BROKER_URI,  
00168     };  
00169  
00170     mqtt_client = esp_mqtt_client_init(&mqtt_cfg);  
00171     if (mqtt_client == NULL)  
00172     {  
00173         ESP_LOGE(TAG, "No se pudo crear cliente MQTT");  
00174         return;  
00175     }  
00176  
00177     esp_mqtt_client_register_event(  
00178         mqtt_client,  
00179         ESP_EVENT_ANY_ID,  
00180         mqtt_event_handler,  
00181         NULL);  
00182  
00183     ESP_LOGI(TAG, "Conectando a broker MQTT: %s", MQTT_BROKER_URI);  
00184     esp_mqtt_client_start(mqtt_client);  
00185 }
```

6.45.3.4. mqtt_manager_stop()

```
void mqtt_manager_stop (
    void )
```

Detiene el servicio MQTT.

Detiene el cliente MQTT.

Finaliza la conexión con el broker, destruye el cliente MQTT y limpia los bits de sincronización utilizados por el sistema.

Detiene el servicio MQTT.

Finaliza la conexión con el broker y libera los recursos utilizados por el módulo.

Definición en la línea 276 del archivo `mqtt_manager.c`.

```
00277 {
00278     if (mqtt_client != NULL)
00279     {
00280         esp_mqtt_client_stop(mqtt_client);
00281         esp_mqtt_client_destroy(mqtt_client);
00282         mqtt_client = NULL;
00283     }
00284
00285     if (mqtt_event_group != NULL)
00286     {
00287         xEventGroupClearBits(mqtt_event_group,
00288                             MQTT_CONNECTED_BIT | MQTT_PUBLISHED_BIT | MQTT_ERROR_BIT);
00289     }
00290 }
```

6.45.3.5. mqtt_publish()

```
bool mqtt_publish (
    const char * topic,
    const char * payload,
    int qos,
    uint32_t timeout_ms)
```

Publica un mensaje en un tópico MQTT.

Envía un mensaje al broker MQTT utilizando el nivel de QoS especificado.

Para QoS mayores a cero, la función espera la confirmación de publicación mediante Event Groups.

Parámetros

<i>topic</i>	Tópico MQTT de destino.
<i>payload</i>	Mensaje a publicar.
<i>qos</i>	Nivel de calidad de servicio.
<i>timeout_ms</i>	Tiempo máximo de espera para la confirmación.

Devuelve

true si la publicación fue exitosa.
false si ocurrió un error o timeout.

Envía una cadena de texto a un tópico específico utilizando el nivel de calidad de servicio (QoS) indicado.

Parámetros

<i>topic</i>	Tópico MQTT de destino.
<i>payload</i>	Mensaje a publicar.
<i>qos</i>	Nivel de QoS utilizado en la publicación.
<i>timeout_ms</i>	Tiempo máximo de espera para completar la operación.

Devuelve

true si la publicación fue exitosa.
false si ocurrió un error o timeout.

Definición en la línea 232 del archivo `mqtt_manager.c`.

```

00233 {
00234     if (mqtt_client == NULL || mqtt_event_group == NULL)
00235     {
00236         return false;
00237     }
00238
00239     xEventGroupClearBits(mqtt_event_group, MQTT_PUBLISHED_BIT | MQTT_ERROR_BIT);
00240
00241     int msg_id = esp_mqtt_client_publish(
00242         mqtt_client,
00243         topic,
00244         payload,
00245         0,
00246         qos,
00247         0);
00248
00249     if (msg_id < 0)
00250     {
00251         ESP_LOGE(TAG, "No se pudo publicar en MQTT");
00252         return false;
00253     }
00254
00255     if (qos == 0)
00256     {
00257         return true;
00258     }
00259
00260     EventBits_t bits = xEventGroupWaitBits(
00261         mqtt_event_group,
00262         MQTT_PUBLISHED_BIT | MQTT_ERROR_BIT,
00263         pdFALSE,
00264         pdFALSE,
00265         pdMS_TO_TICKS(timeout_ms));
00266
00267     return (bits & MQTT_PUBLISHED_BIT) != 0;
00268 }

```

6.45.3.6. mqtt_wait_connected()

```

bool mqtt_wait_connected (
    uint32_t timeout_ms)

```

Espera la conexión con el broker MQTT.

Espera hasta que se establezca la conexión MQTT.

La función bloquea la ejecución hasta que se reciba el evento de conexión o se alcance el tiempo máximo especificado.

Parámetros

<i>timeout_ms</i>	Tiempo máximo de espera en milisegundos.
-------------------	--

Devuelve

true si se estableció la conexión.
false si ocurrió timeout o error.

Espera la conexión con el broker MQTT.

Bloquea la ejecución hasta que el cliente MQTT se conecte al broker o hasta que expire el tiempo de espera.

Parámetros

<i>timeout_ms</i>	Tiempo máximo de espera en milisegundos.
-------------------	--

Devuelve

true si la conexión se estableció correctamente.
false si ocurrió un timeout o error de conexión.

Definición en la línea 198 del archivo [mqtt_manager.c](#).

```
00199 {
00200     if (mqtt_event_group == NULL)
00201     {
00202         return false;
00203     }
00204
00205     EventBits_t bits = xEventGroupWaitBits(
00206         mqtt_event_group,
00207         MQTT_CONNECTED_BIT | MQTT_ERROR_BIT,
00208         pdFALSE,
00209         pdFALSE,
00210         pdMS_TO_TICKS(timeout_ms));
00211
00212     return (bits & MQTT_CONNECTED_BIT) != 0;
00213 }
```

6.45.4. Documentación de variables

6.45.4.1. mqtt_client

```
esp_mqtt_client_handle_t mqtt_client = NULL [static]
```

Manejador del cliente MQTT.

Definición en la línea 51 del archivo [mqtt_manager.c](#).

6.45.4.2. mqtt_connected

```
bool mqtt_connected = false [static]
```

Indica si el cliente MQTT se encuentra conectado.

Definición en la línea 61 del archivo [mqtt_manager.c](#).

6.45.4.3. mqtt_event_group

```
EventGroupHandle_t mqtt_event_group = NULL [static]
```

Grupo de eventos utilizado para sincronización MQTT.

Definición en la línea 56 del archivo [mqtt_manager.c](#).

6.45.4.4. TAG

```
const char* TAG = "mqtt_manager" [static]
```

Etiqueta utilizada para mensajes de depuración.

Definición en la línea 46 del archivo [mqtt_manager.c](#).

6.46. mqtt_manager.c

[Ir a la documentación de este archivo.](#)

```
00001
00032 #include "mqtt_manager.h"
00033
00034 #include "freertos/FreeRTOS.h"
00035 #include "freertos/event_groups.h"
00036
00037 #include "esp_event.h"
00038 #include "esp_log.h"
00039 #include "mqtt_client.h"
00040
00041 #define MQTT_BROKER_URI "mqtt://192.168.0.110:1883"
00042
00046 static const char *TAG = "mqtt_manager";
00047
00051 static esp_mqtt_client_handle_t mqtt_client = NULL;
00052
00056 static EventGroupHandle_t mqtt_event_group = NULL;
00057
00061 static bool mqtt_connected = false;
00062
00066 #define MQTT_CONNECTED_BIT BIT0
00067
00071 #define MQTT_PUBLISHED_BIT BIT1
00072
00076 #define MQTT_ERROR_BIT BIT2
00077
00098 static void mqtt_event_handler(void *handler_args,
00099                               esp_event_base_t base,
00100                               int32_t event_id,
00101                               void *event_data)
00102 {
00103     esp_mqtt_event_handle_t event = event_data;
00104
00105     switch ((esp_mqtt_event_id_t) event_id)
00106     {
00107         case MQTT_EVENT_CONNECTED:
00108             ESP_LOGI(TAG, "MQTT conectado");
```

```

00109
00110         mqtt_connected = true;
00111
00112         xEventGroupSetBits(mqtt_event_group, MQTT_CONNECTED_BIT);
00113         xEventGroupClearBits(mqtt_event_group, MQTT_ERROR_BIT);
00114         break;
00115
00116     case MQTT_EVENT_DISCONNECTED:
00117         ESP_LOGW(TAG, "MQTT desconectado");
00118
00119         mqtt_connected = false;
00120
00121         xEventGroupClearBits(mqtt_event_group, MQTT_CONNECTED_BIT);
00122         break;
00123
00124     case MQTT_EVENT_PUBLISHED:
00125         ESP_LOGI(TAG, "MQTT publicado, msg_id=%d", event->msg_id);
00126         xEventGroupSetBits(mqtt_event_group, MQTT_PUBLISHED_BIT);
00127         break;
00128
00129     case MQTT_EVENT_ERROR:
00130         ESP_LOGE(TAG, "Error MQTT");
00131         xEventGroupSetBits(mqtt_event_group, MQTT_ERROR_BIT);
00132         break;
00133
00134     default:
00135         break;
00136 }
00137 }
00145 void mqtt_manager_start(void)
00146 {
00147     if (mqtt_event_group == NULL)
00148     {
00149         mqtt_event_group = xEventGroupCreate();
00150     }
00151
00152     if (mqtt_event_group == NULL)
00153     {
00154         ESP_LOGE(TAG, "No se pudo crear grupo de eventos MQTT");
00155         return;
00156     }
00157
00158     xEventGroupClearBits(mqtt_event_group,
00159                         MQTT_CONNECTED_BIT | MQTT_PUBLISHED_BIT | MQTT_ERROR_BIT);
00160
00161     if (mqtt_client != NULL)
00162     {
00163         return;
00164     }
00165
00166     esp_mqtt_client_config_t mqtt_cfg = {
00167         .broker.address.uri = MQTT_BROKER_URI,
00168     };
00169
00170     mqtt_client = esp_mqtt_client_init(&mqtt_cfg);
00171     if (mqtt_client == NULL)
00172     {
00173         ESP_LOGE(TAG, "No se pudo crear cliente MQTT");
00174         return;
00175     }
00176
00177     esp_mqtt_client_register_event(
00178         mqtt_client,
00179         ESP_EVENT_ANY_ID,
00180         mqtt_event_handler,
00181         NULL);
00182
00183     ESP_LOGI(TAG, "Conectando a broker MQTT: %s", MQTT_BROKER_URI);
00184     esp_mqtt_client_start(mqtt_client);
00185 }
00186
00198 bool mqtt_wait_connected(uint32_t timeout_ms)
00199 {
00200     if (mqtt_event_group == NULL)
00201     {
00202         return false;
00203     }
00204
00205     EventBits_t bits = xEventGroupWaitBits(
00206         mqtt_event_group,
00207         MQTT_CONNECTED_BIT | MQTT_ERROR_BIT,
00208         pdFALSE,
00209         pdFALSE,
00210         pdMS_TO_TICKS(timeout_ms));
00211
00212     return (bits & MQTT_CONNECTED_BIT) != 0;
00213 }

```

```

00214
00232 bool mqtt_publish(const char *topic, const char *payload, int qos, uint32_t timeout_ms)
00233 {
00234     if (mqtt_client == NULL || mqtt_event_group == NULL)
00235     {
00236         return false;
00237     }
00238     xEventGroupClearBits(mqtt_event_group, MQTT_PUBLISHED_BIT | MQTT_ERROR_BIT);
00239
00240     int msg_id = esp_mqtt_client_publish(
00241         mqtt_client,
00242         topic,
00243         payload,
00244         0,
00245         qos,
00246         0);
00247
00248     if (msg_id < 0)
00249     {
00250         ESP_LOGE(TAG, "No se pudo publicar en MQTT");
00251         return false;
00252     }
00253
00254     if (qos == 0)
00255     {
00256         return true;
00257     }
00258
00259     EventBits_t bits = xEventGroupWaitBits(
00260         mqtt_event_group,
00261         MQTT_PUBLISHED_BIT | MQTT_ERROR_BIT,
00262         pdFALSE,
00263         pdFALSE,
00264         pdMS_TO_TICKS(timeout_ms));
00265
00266     return (bits & MQTT_PUBLISHED_BIT) != 0;
00267 }
00268
00269 void mqtt_manager_stop(void)
00270 {
00271     if (mqtt_client != NULL)
00272     {
00273         esp_mqtt_client_stop(mqtt_client);
00274         esp_mqtt_client_destroy(mqtt_client);
00275         mqtt_client = NULL;
00276     }
00277
00278     if (mqtt_event_group != NULL)
00279     {
00280         xEventGroupClearBits(mqtt_event_group,
00281                             MQTT_CONNECTED_BIT | MQTT_PUBLISHED_BIT | MQTT_ERROR_BIT);
00282     }
00283 }
00284
00285 bool mqtt_is_connected(void)
00286 {
00287     return mqtt_connected;
00288 }
00289
00290
00291
00292
00293
00294
00295
00296
00297
00298
00299
00300

```

6.47. Referencia del archivo main/mqtt/mqtt_manager.h

Interfaz para la gestión de comunicación MQTT.

```

#include <stdbool.h>
#include <stdint.h>

```

Funciones

- void `mqtt_manager_start` (void)
Inicializa y pone en marcha el cliente MQTT.
- bool `mqtt_wait_connected` (uint32_t timeout_ms)
Espera hasta que se establezca la conexión MQTT.

- bool `mqtt_publish` (const char *topic, const char *payload, int qos, uint32_t timeout_ms)
Publica un mensaje en un tópico MQTT.
- void `mqtt_manager_stop` (void)
Detiene el cliente MQTT.
- bool `mqtt_is_connected` (void)
Consulta el estado actual de la conexión MQTT.

6.47.1. Descripción detallada

Interfaz para la gestión de comunicación MQTT.

Este módulo encapsula la configuración y operación del cliente MQTT utilizado por el sistema de monitoreo ambiental basado en ESP32.

Permite:

- Inicializar la conexión MQTT.
- Verificar el estado de conexión con el broker.
- Publicar mensajes en tópicos específicos.
- Detener el servicio MQTT.
- Consultar el estado actual de conectividad.

El módulo abstrae los detalles de la biblioteca MQTT utilizada, proporcionando una interfaz sencilla para las tareas de aplicación.

Autor

Fernando Plazas Trujillo Isabella Ordoñez Juan Daniel Constain

Definición en el archivo [mqtt_manager.h](#).

6.47.2. Documentación de funciones

6.47.2.1. `mqtt_is_connected()`

```
bool mqtt_is_connected (  
    void )
```

Consulta el estado actual de la conexión MQTT.

Consulta el estado de conexión MQTT.

Devuelve

true si el cliente está conectado al broker.
false si no existe conexión activa.

Consulta el estado actual de la conexión MQTT.

Devuelve

true si existe conexión activa con el broker.
false en caso contrario.

Definición en la línea [297](#) del archivo [mqtt_manager.c](#).

```
00298 {  
00299     return mqtt_connected;  
00300 }
```

6.47.2.2. mqtt_manager_start()

```
void mqtt_manager_start (
    void )
```

Inicializa y pone en marcha el cliente MQTT.

Inicializa y arranca el cliente MQTT.

Configura el cliente MQTT, registra los eventos asociados y establece la conexión con el broker configurado.

Inicializa y pone en marcha el cliente MQTT.

Crea el grupo de eventos utilizado para sincronización, configura el cliente MQTT, registra el manejador de eventos y establece la conexión con el broker configurado.

Definición en la línea 145 del archivo [mqtt_manager.c](#).

```
00146 {
00147     if (mqtt_event_group == NULL)
00148     {
00149         mqtt_event_group = xEventGroupCreate();
00150     }
00151
00152     if (mqtt_event_group == NULL)
00153     {
00154         ESP_LOGE(TAG, "No se pudo crear grupo de eventos MQTT");
00155         return;
00156     }
00157
00158     xEventGroupClearBits(mqtt_event_group,
00159                         MQTT_CONNECTED_BIT | MQTT_PUBLISHED_BIT | MQTT_ERROR_BIT);
00160
00161     if (mqtt_client != NULL)
00162     {
00163         return;
00164     }
00165
00166     esp_mqtt_client_config_t mqtt_cfg = {
00167         .broker.address.uri = MQTT_BROKER_URI,
00168     };
00169
00170     mqtt_client = esp_mqtt_client_init(&mqtt_cfg);
00171     if (mqtt_client == NULL)
00172     {
00173         ESP_LOGE(TAG, "No se pudo crear cliente MQTT");
00174         return;
00175     }
00176
00177     esp_mqtt_client_register_event(
00178         mqtt_client,
00179         ESP_EVENT_ANY_ID,
00180         mqtt_event_handler,
00181         NULL);
00182
00183     ESP_LOGI(TAG, "Conectando a broker MQTT: %s", MQTT_BROKER_URI);
00184     esp_mqtt_client_start(mqtt_client);
00185 }
```

6.47.2.3. mqtt_manager_stop()

```
void mqtt_manager_stop (
    void )
```

Detiene el cliente MQTT.

Detiene el servicio MQTT.

Finaliza la conexión con el broker y libera los recursos utilizados por el módulo.

Detiene el cliente MQTT.

Finaliza la conexión con el broker, destruye el cliente MQTT y limpia los bits de sincronización utilizados por el sistema.

Definición en la línea 276 del archivo `mqtt_manager.c`.

```
00277 {
00278     if (mqtt_client != NULL)
00279     {
00280         esp_mqtt_client_stop(mqtt_client);
00281         esp_mqtt_client_destroy(mqtt_client);
00282         mqtt_client = NULL;
00283     }
00284
00285     if (mqtt_event_group != NULL)
00286     {
00287         xEventGroupClearBits(mqtt_event_group,
00288                             MQTT_CONNECTED_BIT | MQTT_PUBLISHED_BIT | MQTT_ERROR_BIT);
00289     }
00290 }
```

6.47.2.4. `mqtt_publish()`

```
bool mqtt_publish (
    const char * topic,
    const char * payload,
    int qos,
    uint32_t timeout_ms)
```

Publica un mensaje en un tópico MQTT.

Envía una cadena de texto a un tópico específico utilizando el nivel de calidad de servicio (QoS) indicado.

Parámetros

<i>topic</i>	Tópico MQTT de destino.
<i>payload</i>	Mensaje a publicar.
<i>qos</i>	Nivel de QoS utilizado en la publicación.
<i>timeout_ms</i>	Tiempo máximo de espera para completar la operación.

Devuelve

true si la publicación fue exitosa.
false si ocurrió un error o timeout.

Envía un mensaje al broker MQTT utilizando el nivel de QoS especificado.

Para QoS mayores a cero, la función espera la confirmación de publicación mediante Event Groups.

Parámetros

<i>topic</i>	Tópico MQTT de destino.
<i>payload</i>	Mensaje a publicar.
<i>qos</i>	Nivel de calidad de servicio.

<i>timeout_ms</i>	Tiempo máximo de espera para la confirmación.
-------------------	---

Devuelve

true si la publicación fue exitosa.

false si ocurrió un error o timeout.

Definición en la línea 232 del archivo `mqtt_manager.c`.

```

00233 {
00234     if (mqtt_client == NULL || mqtt_event_group == NULL)
00235     {
00236         return false;
00237     }
00238
00239     xEventGroupClearBits(mqtt_event_group, MQTT_PUBLISHED_BIT | MQTT_ERROR_BIT);
00240
00241     int msg_id = esp_mqtt_client_publish(
00242         mqtt_client,
00243         topic,
00244         payload,
00245         0,
00246         qos,
00247         0);
00248
00249     if (msg_id < 0)
00250     {
00251         ESP_LOGE(TAG, "No se pudo publicar en MQTT");
00252         return false;
00253     }
00254
00255     if (qos == 0)
00256     {
00257         return true;
00258     }
00259
00260     EventBits_t bits = xEventGroupWaitBits(
00261         mqtt_event_group,
00262         MQTT_PUBLISHED_BIT | MQTT_ERROR_BIT,
00263         pdFALSE,
00264         pdFALSE,
00265         pdMS_TO_TICKS(timeout_ms));
00266
00267     return (bits & MQTT_PUBLISHED_BIT) != 0;
00268 }

```

6.47.2.5. mqtt_wait_connected()

```

bool mqtt_wait_connected (
    uint32_t timeout_ms)

```

Espera hasta que se establezca la conexión MQTT.

Espera la conexión con el broker MQTT.

Bloquea la ejecución hasta que el cliente MQTT se conecte al broker o hasta que expire el tiempo de espera.

Parámetros

<i>timeout_ms</i>	Tiempo máximo de espera en milisegundos.
-------------------	--

Devuelve

true si la conexión se estableció correctamente.
false si ocurrió un timeout o error de conexión.

Espera hasta que se establezca la conexión MQTT.

La función bloquea la ejecución hasta que se reciba el evento de conexión o se alcance el tiempo máximo especificado.

Parámetros

<i>timeout_ms</i>	Tiempo máximo de espera en milisegundos.
-------------------	--

Devuelve

true si se estableció la conexión.
false si ocurrió timeout o error.

Definición en la línea 198 del archivo `mqtt_manager.c`.

```

00199 {
00200     if (mqtt_event_group == NULL)
00201     {
00202         return false;
00203     }
00204
00205     EventBits_t bits = xEventGroupWaitBits(
00206         mqtt_event_group,
00207         MQTT_CONNECTED_BIT | MQTT_ERROR_BIT,
00208         pdFALSE,
00209         pdFALSE,
00210         pdMS_TO_TICKS(timeout_ms));
00211
00212     return (bits & MQTT_CONNECTED_BIT) != 0;
00213 }
```

6.48. mqtt_manager.h

[Ir a la documentación de este archivo.](#)

```

00001
00023
00024 #ifndef MQTT_MANAGER_H
00025 #define MQTT_MANAGER_H
00026
00027 #include <stdbool.h>
00028 #include <stdint.h>
00029
00036 void mqtt_manager_start(void);
00037
00049 bool mqtt_wait_connected(uint32_t timeout_ms);
00050
00065 bool mqtt_publish(const char *topic,
00066                  const char *payload,
00067                  int qos,
00068                  uint32_t timeout_ms);
00069
00076 void mqtt_manager_stop(void);
00077
00084 bool mqtt_is_connected(void);
00085
00086 #endif /* MQTT_MANAGER_H */
```


6.49. Referencia del archivo main/tasks/task_actuadores.c

Implementación de la tarea de ejecución de actuadores.

```
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "driver/gpio.h"
#include "esp_log.h"
#include "utils/data_types.h"
#include "queues.h"
#include "drivers/servo.h"
```

defines

- #define `PIN_BOMBA` `GPIO_NUM_25`
GPIO de control de la bomba.
- #define `PIN_MOTOR` `GPIO_NUM_26`
GPIO de control del motor de mezcla.
- #define `PIN_SERVO` `GPIO_NUM_27`
GPIO utilizado por el servomotor.
- #define `MIX_DURATION_MS` `20000`
Duración del ciclo de mezcla.

Funciones

- static void `actuadores_init` (void)
Inicializa los actuadores del sistema.
- void `task_actuadores` (void *pvParameters)
Tarea encargada de ejecutar acciones físicas.

Variables

- TaskHandle_t `task_energia_handle`
Handle de la tarea de energía.
- static const char * `TAG` = "ACTUADORES"
Etiqueta utilizada para mensajes de depuración.

6.49.1. Descripción detallada

Implementación de la tarea de ejecución de actuadores.

Esta tarea recibe comandos generados por la lógica de control mediante una cola FreeRTOS y ejecuta las acciones físicas correspondientes sobre los actuadores del sistema.

Actuadores gestionados:

- Bomba de circulación.
- Motor de mezcla.

- Servomotor.

Características:

- Consumo de comandos mediante colas FreeRTOS.
- Control temporizado del motor de mezcla.
- Notificación a la tarea de energía al finalizar una mezcla.
- Separación entre lógica de control y acceso a hardware.

Mecanismos FreeRTOS utilizados:

- Colas.
- Notificaciones entre tareas.

Autor

Fernando Plazas Trujillo Isabella Ordoñez Juan Daniel Constain

Definición en el archivo [task_actuadores.c](#).

6.49.2. Documentación de «define»

6.49.2.1. MIX_DURATION_MS

```
#define MIX_DURATION_MS 20000
```

Duración del ciclo de mezcla.

Tiempo durante el cual el motor permanece activo.

Definición en la línea 74 del archivo [task_actuadores.c](#).

6.49.2.2. PIN_BOMBA

```
#define PIN_BOMBA GPIO_NUM_25
```

GPIO de control de la bomba.

Definición en la línea 57 del archivo [task_actuadores.c](#).

6.49.2.3. PIN_MOTOR

```
#define PIN_MOTOR GPIO_NUM_26
```

GPIO de control del motor de mezcla.

Definición en la línea 62 del archivo [task_actuadores.c](#).

6.49.2.4. PIN_SERVO

```
#define PIN_SERVO GPIO_NUM_27
```

GPIO utilizado por el servomotor.

Definición en la línea 67 del archivo [task_actuadores.c](#).

6.49.3. Documentación de funciones

6.49.3.1. actuadores_init()

```
void actuadores_init (  
    void ) [static]
```

Inicializa los actuadores del sistema.

Configura los GPIO utilizados por la bomba y el motor en modo salida e inicializa el controlador del servomotor.

Además establece todos los actuadores en estado seguro al arrancar el sistema.

Definición en la línea 89 del archivo [task_actuadores.c](#).

```
00090 {  
00091     gpio_config_t io_conf = {  
00092         .pin_bit_mask = (1ULL << PIN_BOMBA) | (1ULL << PIN_MOTOR),  
00093         .mode = GPIO_MODE_OUTPUT,  
00094         .pull_up_en = GPIO_PULLUP_DISABLE,  
00095         .pull_down_en = GPIO_PULLDOWN_DISABLE,  
00096         .intr_type = GPIO_INTR_DISABLE  
00097     };  
00098  
00099     gpio_config(&io_conf);  
00100  
00101     gpio_set_level(PIN_BOMBA, 0);  
00102     gpio_set_level(PIN_MOTOR, 0);  
00103  
00104     servo_init(PIN_SERVO);  
00105 }
```

6.49.3.2. task_actuadores()

```
void task_actuadores (  
    void * pvParameters)
```

Tarea encargada de ejecutar acciones físicas.

Tarea de ejecución de actuadores.

Esta tarea permanece bloqueada esperando comandos provenientes de `queue_control`.

Funciones principales:

- Activación y desactivación de la bomba.
- Posicionamiento del servomotor.
- Gestión del motor de mezcla.
- Notificación a `task_energia` al finalizar una mezcla.

La tarea no implementa lógica de decisión; únicamente ejecuta los comandos generados por task_control.

Parámetros

<i>pvParameters</i>	Parámetros de la tarea (no utilizados).
---------------------	---

Tarea encargada de ejecutar acciones físicas.

Espera comandos provenientes de la cola de control y ejecuta las acciones correspondientes sobre los actuadores físicos del sistema.

La tarea opera como consumidora de estructuras `control_cmd_t`, permitiendo separar la lógica de decisión de la manipulación directa del hardware.

Parámetros

<i>pvParameters</i>	Parámetros de la tarea (no utilizados).
---------------------	---

Definición en la línea 125 del archivo `task_actuadores.c`.

```

00126 {
00127     control_cmd_t cmd;
00128
00129     actuadores_init();
00130
00131     bool motor_activo = false;
00132     TickType_t motor_start_time = 0;
00133
00134     float last_servo_angle = -1;
00135
00136     ESP_LOGI(TAG, "Task actuadores iniciada");
00137
00138     while (1) {
00139         if (xQueueReceive(queue_control, &cmd, pdMS_TO_TICKS(100))) {
00140             ESP_LOGI(TAG, "Ejecutando acciones...");
00141
00142             gpio_set_level(PIN_BOMBA, cmd.bomba ? 1 : 0);
00143
00144             if (cmd.servo_angle != last_servo_angle) {
00145                 servo_set_angle(cmd.servo_angle);
00146                 last_servo_angle = cmd.servo_angle;
00147             }
00148
00149             if (cmd.mezclar && !motor_activo) {
00150                 ESP_LOGI(TAG, "Iniciando mezcla");
00151
00152                 gpio_set_level(PIN_MOTOR, 1);
00153
00154                 motor_activo = true;
00155                 motor_start_time = xTaskGetTickCount();
00156             }
00157
00158             if (motor_activo) {
00159                 TickType_t now = xTaskGetTickCount();
00160
00161                 if ((now - motor_start_time) >= pdMS_TO_TICKS(MIX_DURATION_MS)) {
00162                     ESP_LOGI(TAG, "Deteniendo mezcla");
00163
00164                     gpio_set_level(PIN_MOTOR, 0);
00165                     motor_activo = false;
00166
00167                     xTaskNotifyGive(task_energia_handle);
00168                 }
00169             }
00170
00171             vTaskDelay(pdMS_TO_TICKS(50));
00172         }
00173     }
00174 }

```

6.49.4. Documentación de variables

6.49.4.1. TAG

```
const char* TAG = "ACTUADORES" [static]
```

Etiqueta utilizada para mensajes de depuración.

Definición en la línea 49 del archivo [task_actuadores.c](#).

6.49.4.2. task_energia_handle

```
TaskHandle_t task_energia_handle [extern]
```

Handle de la tarea de energía.

Utilizado para enviar notificaciones cuando finaliza un ciclo de mezcla.

Utilizado para coordinar el ciclo general del sistema.

Definición en la línea 109 del archivo [main.c](#).

6.50. task_actuadores.c

[Ir a la documentación de este archivo.](#)

```
00001
00029 #include "freertos/FreeRTOS.h"
00030 #include "freertos/task.h"
00031 #include "driver/gpio.h"
00032 #include "esp_log.h"
00033
00034 #include "utils/data_types.h"
00035 #include "queues.h"
00036 #include "drivers/servo.h"
00037
00044 extern TaskHandle_t task_energia_handle;
00045
00049 static const char *TAG = "ACTUADORES";
00050
00051 // =====
00052 // PINES
00053 // =====
00057 #define PIN_BOMBA GPIO_NUM_25
00058
00062 #define PIN_MOTOR GPIO_NUM_26
00063
00067 #define PIN_SERVO GPIO_NUM_27
00068
00074 #define MIX_DURATION_MS 20000
00075
00076 // =====
00077 // INIT GPIO + SERVO
00078 // =====
00079
00089 static void actuadores_init(void)
00090 {
00091     gpio_config_t io_conf = {
00092         .pin_bit_mask = (1ULL << PIN_BOMBA) | (1ULL << PIN_MOTOR),
00093         .mode = GPIO_MODE_OUTPUT,
00094         .pull_up_en = GPIO_PULLUP_DISABLE,
00095         .pull_down_en = GPIO_PULLDOWN_DISABLE,
00096         .intr_type = GPIO_INTR_DISABLE
00097     };
00098
00099     gpio_config(&io_conf);
00100
00101     gpio_set_level(PIN_BOMBA, 0);
```

```

00102     gpio_set_level(PIN_MOTOR, 0);
00103
00104     servo_init(PIN_SERVO);
00105 }
00106
00107
00125 void task_actuadores(void *pvParameters)
00126 {
00127     control_cmd_t cmd;
00128
00129     actuadores_init();
00130
00131     bool motor_activo = false;
00132     TickType_t motor_start_time = 0;
00133
00134     float last_servo_angle = -1;
00135
00136     ESP_LOGI(TAG, "Task actuadores iniciada");
00137
00138     while (1) {
00139
00140         if (xQueueReceive(queue_control, &cmd, pdMS_TO_TICKS(100))) {
00141
00142             ESP_LOGI(TAG, "Ejecutando acciones...");
00143
00144             gpio_set_level(PIN_BOMBA, cmd.bomba ? 1 : 0);
00145
00146             if (cmd.servo_angle != last_servo_angle) {
00147                 servo_set_angle(cmd.servo_angle);
00148                 last_servo_angle = cmd.servo_angle;
00149             }
00150
00151             if (cmd.mezclar && !motor_activo) {
00152
00153                 ESP_LOGI(TAG, "Iniciando mezcla");
00154
00155                 gpio_set_level(PIN_MOTOR, 1);
00156
00157                 motor_activo = true;
00158                 motor_start_time = xTaskGetTickCount();
00159             }
00160
00161
00162             if (motor_activo) {
00163
00164                 TickType_t now = xTaskGetTickCount();
00165
00166                 if ((now - motor_start_time) >= pdMS_TO_TICKS(MIX_DURATION_MS)) {
00167
00168                     ESP_LOGI(TAG, "Deteniendo mezcla");
00169
00170                     gpio_set_level(PIN_MOTOR, 0);
00171                     motor_activo = false;
00172
00173                     xTaskNotifyGive(task_energia_handle);
00174                 }
00175             }
00176
00177             vTaskDelay(pdMS_TO_TICKS(50));
00178         }
00179     }

```

6.51. Referencia del archivo main/tasks/task_actuadores.h

Declaración de la tarea encargada del control de actuadores.

Funciones

- void `task_actuadores` (void *pvParameters)

Tarea de ejecución de actuadores.

6.51.1. Descripción detallada

Declaración de la tarea encargada del control de actuadores.

Este módulo define la tarea responsable de ejecutar las acciones físicas del sistema a partir de los comandos generados por la lógica de control.

La tarea recibe estructuras [control_cmd_t](#) mediante una cola FreeRTOS y actúa sobre los dispositivos de salida del sistema.

Actuadores gestionados:

- Sistema de enfriamiento.
- Bomba de circulación.
- Motor de mezcla.
- Servomotor.

Esta arquitectura mantiene desacopladas las decisiones de control del acceso directo al hardware.

Autor

Fernando Plazas Trujillo Isabella Ordoñez Juan Daniel Constain

Definición en el archivo [task_actuadores.h](#).

6.51.2. Documentación de funciones

6.51.2.1. task_actuadores()

```
void task_actuadores (  
    void * pvParameters)
```

Tarea de ejecución de actuadores.

Tarea encargada de ejecutar acciones físicas.

Espera comandos provenientes de la cola de control y ejecuta las acciones correspondientes sobre los actuadores físicos del sistema.

La tarea opera como consumidora de estructuras [control_cmd_t](#), permitiendo separar la lógica de decisión de la manipulación directa del hardware.

Parámetros

<i>pvParameters</i>	Parámetros de la tarea (no utilizados).
---------------------	---

Tarea de ejecución de actuadores.

Esta tarea permanece bloqueada esperando comandos provenientes de `queue_control`.

Funciones principales:

- Activación y desactivación de la bomba.
- Posicionamiento del servomotor.
- Gestión del motor de mezcla.
- Notificación a task_energia al finalizar una mezcla.

La tarea no implementa lógica de decisión; únicamente ejecuta los comandos generados por task_control.

Parámetros

<i>pvParameters</i>	Parámetros de la tarea (no utilizados).
---------------------	---

Definición en la línea 125 del archivo `task_actuadores.c`.

```

00126 {
00127     control_cmd_t cmd;
00128
00129     actuadores_init();
00130
00131     bool motor_activo = false;
00132     TickType_t motor_start_time = 0;
00133
00134     float last_servo_angle = -1;
00135
00136     ESP_LOGI(TAG, "Task actuadores iniciada");
00137
00138     while (1) {
00139         if (xQueueReceive(queue_control, &cmd, pdMS_TO_TICKS(100))) {
00140             ESP_LOGI(TAG, "Ejecutando acciones...");
00141
00142             gpio_set_level(PIN_BOMBA, cmd.bomba ? 1 : 0);
00143
00144             if (cmd.servo_angle != last_servo_angle) {
00145                 servo_set_angle(cmd.servo_angle);
00146                 last_servo_angle = cmd.servo_angle;
00147             }
00148
00149             if (cmd.mezclar && !motor_activo) {
00150                 ESP_LOGI(TAG, "Iniciando mezcla");
00151
00152                 gpio_set_level(PIN_MOTOR, 1);
00153
00154                 motor_activo = true;
00155                 motor_start_time = xTaskGetTickCount();
00156             }
00157
00158             if (motor_activo) {
00159                 TickType_t now = xTaskGetTickCount();
00160
00161                 if ((now - motor_start_time) >= pdMS_TO_TICKS(MIX_DURATION_MS)) {
00162                     ESP_LOGI(TAG, "Deteniendo mezcla");
00163
00164                     gpio_set_level(PIN_MOTOR, 0);
00165                     motor_activo = false;
00166
00167                     xTaskNotifyGive(task_energia_handle);
00168                 }
00169             }
00170
00171             vTaskDelay(pdMS_TO_TICKS(50));
00172         }
00173     }
00174 }

```

6.52. task_actuadores.h

[Ir a la documentación de este archivo.](#)


```
00001
00026
00027 #ifndef TASK_ACTUADORES_H
00028 #define TASK_ACTUADORES_H
00029
00043 void task_actuadores(void *pvParameters);
00044
00045 #endif /* TASK_ACTUADORES_H */
```

6.53. Referencia del archivo main/tasks/task_control.c

Implementación de la lógica de control del sistema.

```
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/queue.h"
#include "esp_log.h"
#include "utils/data_types.h"
#include "queues.h"
```

defines

- #define `CO2_THRESHOLD` 0.02
Umbral de concentración de CO2.
- #define `MIX_INTERVAL_CYCLES` 3
Número de ciclos entre mezclas consecutivas.

Funciones

- void `task_control` (void *pvParameters)
Tarea principal de control.

Variables

- static const char * `TAG` = "CONTROL"
Etiqueta utilizada para mensajes de depuración.

6.53.1. Descripción detallada

Implementación de la lógica de control del sistema.

Esta tarea recibe mediciones provenientes de los sensores, evalúa las condiciones del proceso y genera comandos para los actuadores.

La lógica de control se mantiene desacoplada del hardware, produciendo estructuras `control_cmd_t` que son enviadas mediante una cola FreeRTOS hacia la tarea de actuadores.

Funciones principales:

- Procesamiento de datos de sensores.

- Evaluación de condiciones del proceso.
- Generación de comandos de control.
- Coordinación de ciclos de mezcla.

Arquitectura:

queue_sensores | v task_control | v queue_control

Mecanismos FreeRTOS utilizados:

- Colas de comunicación entre tareas.

Autor

Fernando Plazas Trujillo Isabella Ordoñez Juan Daniel Constain

Definición en el archivo [task_control.c](#).

6.53.2. Documentación de «define»

6.53.2.1. CO2_THRESHOLD

```
#define CO2_THRESHOLD 0.02
```

Umbral de concentración de CO2.

Utilizado para determinar condiciones específicas del proceso de fermentación.

Definición en la línea 58 del archivo [task_control.c](#).

6.53.2.2. MIX_INTERVAL_CYCLES

```
#define MIX_INTERVAL_CYCLES 3
```

Número de ciclos entre mezclas consecutivas.

Cada cierto número de ciclos de adquisición se genera una orden de mezcla para homogeneizar el proceso.

Definición en la línea 66 del archivo [task_control.c](#).

6.53.3. Documentación de funciones

6.53.3.1. task_control()

```
void task_control (
    void * pvParameters)
```

Tarea principal de control.

Espera datos provenientes de queue_sensores, procesa las mediciones recibidas y genera comandos para los actuadores.

La tarea implementa únicamente la lógica de decisión del sistema y no accede directamente a hardware.

Flujo de ejecución:

- Recibe datos de sensores.
- Evalúa condiciones del proceso.
- Genera comandos de actuación.
- Envía comandos a queue_control.

Parámetros

<i>pvParameters</i>	Parámetros de la tarea (no utilizados).
---------------------	---

Recibe datos provenientes de los sensores mediante una cola FreeRTOS, evalúa las condiciones del proceso y genera comandos para los actuadores.

La tarea no accede directamente al hardware, sino que envía estructuras [control_cmd_t](#) hacia la tarea de actuadores mediante queue_control.

Parámetros

<i>pvParameters</i>	Parámetros de la tarea (no utilizados).
---------------------	---

Incrementar contador de ciclos.

Se utiliza para determinar cuándo debe ejecutarse una nueva mezcla.

Evaluación del nivel de CO2.

Variable reservada para futuras estrategias de control.

Programar mezcla periódica.

Lógica de enfriamiento.

Actualmente deshabilitada.

La bomba sigue el estado del sistema de enfriamiento.

Posición por defecto del servomotor.

Definición en la línea 86 del archivo [task_control.c](#).

```

00087 {
00088     sensor_data_t data;
00089     control_cmd_t cmd;
00090
00091     int cycle_count = 0;
00092
00093     while (1)
00094     {
00095         if (xQueueReceive(queue_sensores,
00096                         &data,
00097                         portMAX_DELAY))
00098         {
00105             cycle_count++;
00106
00107             ESP_LOGI(TAG, "Procesando datos...");
00108
00115             bool aire_activo = (data.co2 > CO2_THRESHOLD);
00116
00117             (void)aire_activo;
00118
00122             cmd.mezclar =
00123                 (cycle_count % MIX_INTERVAL_CYCLES == 0);
00124
00130             cmd.enfriar = false;
00131
00136             cmd.bomba = cmd.enfriar;
00137
00141             cmd.servo_angle = 0.0f;
00142
00143             ESP_LOGI(TAG, "Ciclo: %d", cycle_count);
00144             ESP_LOGI(TAG, "Mezclar: %d", cmd.mezclar);
00145
00146             xQueueSend(queue_control,
00147                       &cmd,
00148                       portMAX_DELAY);
00149         }
00150     }
00151 }

```

6.53.4. Documentación de variables

6.53.4.1. TAG

```
const char* TAG = "CONTROL" [static]
```

Etiqueta utilizada para mensajes de depuración.

Definición en la línea 50 del archivo [task_control.c](#).

6.54. task_control.c

[Ir a la documentación de este archivo.](#)

```

00001
00037
00038 #include "freertos/FreeRTOS.h"
00039 #include "freertos/task.h"
00040 #include "freertos/queue.h"
00041
00042 #include "esp_log.h"
00043
00044 #include "utils/data_types.h"
00045 #include "queues.h"
00046
00050 static const char *TAG = "CONTROL";
00051
00058 #define CO2_THRESHOLD 0.02
00059
00066 #define MIX_INTERVAL_CYCLES 3
00067
00086 void task_control(void *pvParameters)
00087 {

```

```

00088     sensor_data_t data;
00089     control_cmd_t cmd;
00090
00091     int cycle_count = 0;
00092
00093     while (1)
00094     {
00095         if (xQueueReceive(queue_sensores,
00096                         &data,
00097                         portMAX_DELAY))
00098         {
00105             cycle_count++;
00106
00107             ESP_LOGI(TAG, "Procesando datos...");
00108
00115             bool aire_activo = (data.co2 > CO2_THRESHOLD);
00116
00117             (void)aire_activo;
00118
00122             cmd.mezclar =
00123                 (cycle_count % MIX_INTERVAL_CYCLES == 0);
00124
00130             cmd.enfriar = false;
00131
00136             cmd.bomba = cmd.enfriar;
00137
00141             cmd.servo_angle = 0.0f;
00142
00143             ESP_LOGI(TAG, "Ciclo: %d", cycle_count);
00144             ESP_LOGI(TAG, "Mezclar: %d", cmd.mezclar);
00145
00146             xQueueSend(queue_control,
00147                       &cmd,
00148                       portMAX_DELAY);
00149         }
00150     }
00151 }

```

6.55. Referencia del archivo main/tasks/task_control.h

Declaración de la tarea de control del sistema.

Funciones

- void [task_control](#) (void *pvParameters)
Tarea principal de control.

6.55.1. Descripción detallada

Declaración de la tarea de control del sistema.

Este módulo define la tarea responsable de procesar las mediciones adquiridas por los sensores y generar los comandos necesarios para los actuadores.

La tarea implementa la lógica de decisión del sistema, manteniendo separada la capa de control de la capa de acceso al hardware.

Flujo de operación:

queue_sensores | v task_control | v queue_control

Esta arquitectura permite desacoplar la adquisición de datos, la toma de decisiones y la ejecución de acciones.

Autor

Fernando Plazas Trujillo Isabella Ordoñez Juan Daniel Constain

Definición en el archivo [task_control.h](#).

6.55.2. Documentación de funciones

6.55.2.1. task_control()

```
void task_control (
    void * pvParameters)
```

Tarea principal de control.

Recibe datos provenientes de los sensores mediante una cola FreeRTOS, evalúa las condiciones del proceso y genera comandos para los actuadores.

La tarea no accede directamente al hardware, sino que envía estructuras `control_cmd_t` hacia la tarea de actuadores mediante `queue_control`.

Parámetros

<i>pvParameters</i>	Parámetros de la tarea (no utilizados).
---------------------	---

Espera datos provenientes de `queue_sensores`, procesa las mediciones recibidas y genera comandos para los actuadores.

La tarea implementa únicamente la lógica de decisión del sistema y no accede directamente a hardware.

Flujo de ejecución:

- Recibe datos de sensores.
- Evalúa condiciones del proceso.
- Genera comandos de actuación.
- Envía comandos a `queue_control`.

Parámetros

<i>pvParameters</i>	Parámetros de la tarea (no utilizados).
---------------------	---

Incrementar contador de ciclos.

Se utiliza para determinar cuándo debe ejecutarse una nueva mezcla.

Evaluación del nivel de CO2.

Variable reservada para futuras estrategias de control.

Programar mezcla periódica.

Lógica de enfriamiento.

Actualmente deshabilitada.

La bomba sigue el estado del sistema de enfriamiento.

Posición por defecto del servomotor.

Definición en la línea 86 del archivo [task_control.c](#).

```

00087 {
00088     sensor_data_t data;
00089     control_cmd_t cmd;
00090
00091     int cycle_count = 0;
00092
00093     while (1)
00094     {
00095         if (xQueueReceive(queue_sensores,
00096                           &data,
00097                           portMAX_DELAY))
00098         {
00105             cycle_count++;
00106
00107             ESP_LOGI(TAG, "Procesando datos...");
00108
00115             bool aire_activo = (data.co2 > CO2_THRESHOLD);
00116
00117             (void)aire_activo;
00118
00122             cmd.mezclar =
00123                 (cycle_count % MIX_INTERVAL_CYCLES == 0);
00124
00130             cmd.enfriar = false;
00131
00136             cmd.bomba = cmd.enfriar;
00137
00141             cmd.servo_angle = 0.0f;
00142
00143             ESP_LOGI(TAG, "Ciclo: %d", cycle_count);
00144             ESP_LOGI(TAG, "Mezclar: %d", cmd.mezclar);
00145
00146             xQueueSend(queue_control,
00147                        &cmd,
00148                        portMAX_DELAY);
00149         }
00150     }
00151 }

```

6.56. task_control.h

[Ir a la documentación de este archivo.](#)

```

00001
00031
00032 #ifndef TASK_CONTROL_H
00033 #define TASK_CONTROL_H
00034
00048 void task_control(void *pvParameters);
00049
00050 #endif /* TASK_CONTROL_H */

```

6.57. Referencia del archivo main/tasks/task_display.c

Implementación de la tarea de visualización del sistema.

```

#include "tasks/task_display.h"
#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/queue.h"
#include "drivers/oled_display.h"
#include "queues.h"
#include "utils/data_types.h"

```

Funciones

- void [task_display](#) (void *pvParameters)
Tarea encargada de actualizar la pantalla OLED.

6.57.1. Descripción detallada

Implementación de la tarea de visualización del sistema.

Esta tarea recibe información mediante una cola FreeRTOS y actualiza la pantalla OLED con el estado actual del proceso de fermentación.

Información mostrada:

- Temperatura.
- pH.
- Concentración de CO2.
- Estado de WiFi.
- Estado de MQTT.
- Estado de la tarjeta SD.
- Hora del sistema.

La tarea se encarga exclusivamente de la presentación de datos, manteniendo desacoplada la interfaz de usuario de las tareas de adquisición, control y comunicación.

Arquitectura:

queue_display | v task_display | v OLED

Mecanismos FreeRTOS utilizados:

- Colas para recepción de datos.

Autor

Fernando Plazas Trujillo Isabella Ordoñez Juan Daniel Constain

Definición en el archivo [task_display.c](#).

6.57.2. Documentación de funciones

6.57.2.1. task_display()

```
void task_display (
    void * pvParameters)
```

Tarea encargada de actualizar la pantalla OLED.

Espera datos provenientes de queue_display y muestra información relevante del proceso de fermentación.

La tarea permanece bloqueada hasta recibir nuevos datos, minimizando el consumo de CPU.

Información visualizada:

- Temperatura.
- pH.
- CO2.
- Estado de WiFi.
- Estado de MQTT.
- Estado de la tarjeta SD.
- Hora actual.

Parámetros

<i>pvParameters</i>	Parámetros de la tarea (no utilizados).
---------------------	---

Espera información proveniente de queue_display y actualiza la interfaz de usuario con el estado actual del proceso de fermentación.

La tarea no realiza procesamiento de sensores ni toma decisiones de control; únicamente presenta información al usuario.

Parámetros

<i>pvParameters</i>	Parámetros de la tarea (no utilizados).
---------------------	---

Inicialización de la pantalla OLED.

Limpiar pantalla antes de redibujar.

Actualizar contenido físico de la pantalla.

Definición en la línea 74 del archivo [task_display.c](#).

```
00075 {
00076     display_data_t data;
00077
00078     char line[32];
00079
00083     oled_init();
00084     oled_clear();
00085
00086     while (1)
```

```

00087     {
00088         if (xQueueReceive(
00089             queue_display,
00090             &data,
00091             portMAX_DELAY))
00092         {
00093             oled_clear();
00094
00095             // =====
00096             // TEMPERATURA
00097             // =====
00098
00099             snprintf(
00100                 line,
00101                 sizeof(line),
00102                 "T: %.1f C",
00103                 data.temperatura);
00104
00105             oled_draw_string(
00106                 0,
00107                 0,
00108                 line);
00109
00110             // =====
00111             // PH
00112             // =====
00113
00114             snprintf(
00115                 line,
00116                 sizeof(line),
00117                 "PH: %.2f",
00118                 data.ph);
00119
00120             oled_draw_string(
00121                 0,
00122                 12,
00123                 line);
00124
00125             // =====
00126             // CO2
00127             // =====
00128
00129             snprintf(
00130                 line,
00131                 sizeof(line),
00132                 "CO2: %.2f",
00133                 data.co2);
00134
00135             oled_draw_string(
00136                 0,
00137                 24,
00138                 line);
00139
00140             // =====
00141             // ESTADOS DEL SISTEMA
00142             // =====
00143
00144             snprintf(
00145                 line,
00146                 sizeof(line),
00147                 "W: %s M: %s S: %s",
00148                 data.wifi_ok ? "OK" : "OFF",
00149                 data.mqtt_ok ? "OK" : "OFF",
00150                 data.sd_ok ? "OK" : "OFF");
00151
00152             oled_draw_string(
00153                 0,
00154                 36,
00155                 line);
00156
00157             // =====
00158             // HORA
00159             // =====
00160
00161             printf(
00162                 "DISPLAY -> %02d: %02d: %02d\n",
00163                 data.datetime.hours,
00164                 data.datetime.minutes,
00165                 data.datetime.seconds);
00166
00167             snprintf(
00168                 line,
00169                 sizeof(line),
00170                 " %02d: %02d: %02d",
00171                 data.datetime.hours,
00172                 data.datetime.minutes,
00173                 data.datetime.seconds);

```

```

00177
00178         oled_draw_string(
00179             0,
00180             48,
00181             line);
00182
00186         oled_update();
00187     }
00188 }
00189 }

```

6.58. task_display.c

[Ir a la documentación de este archivo.](#)

```

00001
00040
00041 #include "tasks/task_display.h"
00042
00043 #include <stdio.h>
00044
00045 #include "freertos/FreeRTOS.h"
00046 #include "freertos/task.h"
00047 #include "freertos/queue.h"
00048
00049 #include "drivers/oled_display.h"
00050
00051 #include "queues.h"
00052 #include "utils/data_types.h"
00053
00074 void task_display(void *pvParameters)
00075 {
00076     display_data_t data;
00077
00078     char line[32];
00079
00083     oled_init();
00084     oled_clear();
00085
00086     while (1)
00087     {
00088         if (xQueueReceive(
00089             queue_display,
00090             &data,
00091             portMAX_DELAY)
00092         {
00096             oled_clear();
00097
00098             // =====
00099             //  TEMPERATURA
00100             //  =====
00101
00102             snprintf(
00103                 line,
00104                 sizeof(line),
00105                 "T: %.1f C",
00106                 data.temperatura);
00107
00108             oled_draw_string(
00109                 0,
00110                 0,
00111                 line);
00112
00113             // =====
00114             //  PH
00115             //  =====
00116
00117             snprintf(
00118                 line,
00119                 sizeof(line),
00120                 "PH: %.2f",
00121                 data.ph);
00122
00123             oled_draw_string(
00124                 0,
00125                 12,
00126                 line);
00127
00128             // =====
00129             //  CO2
00130             //  =====
00131

```

```

00132         snprintf(
00133             line,
00134             sizeof(line),
00135             "CO2: %.2f",
00136             data.co2);
00137
00138         oled_draw_string(
00139             0,
00140             24,
00141             line);
00142
00143         // =====
00144         // ESTADOS DEL SISTEMA
00145         // =====
00146
00147         snprintf(
00148             line,
00149             sizeof(line),
00150             "W: %s M: %s S: %s",
00151             data.wifi_ok ? "OK" : "OFF",
00152             data.mqtt_ok ? "OK" : "OFF",
00153             data.sd_ok ? "OK" : "OFF");
00154
00155         oled_draw_string(
00156             0,
00157             36,
00158             line);
00159
00160         // =====
00161         // HORA
00162         // =====
00163
00164         printf(
00165             "DISPLAY -> %02d: %02d: %02d\n",
00166             data.datetime.hours,
00167             data.datetime.minutes,
00168             data.datetime.seconds);
00169
00170         snprintf(
00171             line,
00172             sizeof(line),
00173             "%02d: %02d: %02d",
00174             data.datetime.hours,
00175             data.datetime.minutes,
00176             data.datetime.seconds);
00177
00178         oled_draw_string(
00179             0,
00180             48,
00181             line);
00182
00186         oled_update();
00187     }
00188 }
00189 }

```

6.59. Referencia del archivo main/tasks/task_display.h

Declaración de la tarea de visualización del sistema.

Funciones

- void `task_display` (void *pvParameters)
Tarea encargada de actualizar la pantalla OLED.

6.59.1. Descripción detallada

Declaración de la tarea de visualización del sistema.

Este módulo define la tarea responsable de actualizar la pantalla OLED con la información más relevante del proceso de fermentación.

La tarea recibe datos mediante una cola FreeRTOS y se encarga exclusivamente de la presentación de información, manteniendo desacoplada la interfaz de usuario de las tareas de adquisición, control y comunicación.

Información mostrada:

- Temperatura.
- pH.
- Concentración de CO2.
- Estado de WiFi.
- Estado de MQTT.
- Estado de la tarjeta SD.
- Fecha y hora del sistema.

Arquitectura:

queue_display | v task_display | v OLED

Autor

Fernando Plazas Trujillo Isabella Ordoñez Juan Daniel Constain

Definición en el archivo [task_display.h](#).

6.59.2. Documentación de funciones

6.59.2.1. task_display()

```
void task_display (  
    void * pvParameters)
```

Tarea encargada de actualizar la pantalla OLED.

Espera información proveniente de queue_display y actualiza la interfaz de usuario con el estado actual del proceso de fermentación.

La tarea no realiza procesamiento de sensores ni toma decisiones de control; únicamente presenta información al usuario.

Parámetros

<i>pvParameters</i>	Parámetros de la tarea (no utilizados).
---------------------	---

Espera datos provenientes de queue_display y muestra información relevante del proceso de fermentación.

La tarea permanece bloqueada hasta recibir nuevos datos, minimizando el consumo de CPU.

Información visualizada:

- Temperatura.

- pH.
- CO2.
- Estado de WiFi.
- Estado de MQTT.
- Estado de la tarjeta SD.
- Hora actual.

Parámetros

<i>pvParameters</i>	Parámetros de la tarea (no utilizados).
---------------------	---

Inicialización de la pantalla OLED.

Limpiar pantalla antes de redibujar.

Actualizar contenido físico de la pantalla.

Definición en la línea 74 del archivo `task_display.c`.

```

00075 {
00076     display_data_t data;
00077
00078     char line[32];
00079
00083     oled_init();
00084     oled_clear();
00085
00086     while (1)
00087     {
00088         if (xQueueReceive(
00089             queue_display,
00090             &data,
00091             portMAX_DELAY)
00092         {
00096             oled_clear();
00097
00098             // =====
00099             // TEMPERATURA
00100             // =====
00101
00102             snprintf(
00103                 line,
00104                 sizeof(line),
00105                 "T: %.1f C",
00106                 data.temperatura);
00107
00108             oled_draw_string(
00109                 0,
00110                 0,
00111                 line);
00112
00113             // =====
00114             // PH
00115             // =====
00116
00117             snprintf(
00118                 line,
00119                 sizeof(line),
00120                 "PH: %.2f",
00121                 data.ph);
00122
00123             oled_draw_string(
00124                 0,
00125                 12,
00126                 line);
00127
00128             // =====
00129             // CO2
00130             // =====
00131
00132             snprintf(
00133                 line,
00134                 sizeof(line),

```

```

00135         "CO2: %.2f",
00136         data.co2);
00137
00138     oled_draw_string(
00139         0,
00140         24,
00141         line);
00142
00143     // =====
00144     // ESTADOS DEL SISTEMA
00145     // =====
00146
00147     snprintf(
00148         line,
00149         sizeof(line),
00150         "W: %s M: %s S: %s",
00151         data.wifi_ok ? "OK" : "OFF",
00152         data.mqtt_ok ? "OK" : "OFF",
00153         data.sd_ok ? "OK" : "OFF");
00154
00155     oled_draw_string(
00156         0,
00157         36,
00158         line);
00159
00160     // =====
00161     // HORA
00162     // =====
00163
00164     printf(
00165         "DISPLAY -> %02d: %02d: %02d\n",
00166         data.datetime.hours,
00167         data.datetime.minutes,
00168         data.datetime.seconds);
00169
00170     snprintf(
00171         line,
00172         sizeof(line),
00173         "%02d: %02d: %02d",
00174         data.datetime.hours,
00175         data.datetime.minutes,
00176         data.datetime.seconds);
00177
00178     oled_draw_string(
00179         0,
00180         48,
00181         line);
00182
00186     oled_update();
00187 }
00188 }
00189 }

```

6.60. task_display.h

[Ir a la documentación de este archivo.](#)

```

00001
00038
00039 #ifndef TASK_DISPLAY_H
00040 #define TASK_DISPLAY_H
00041
00055 void task_display(void *pvParameters);
00056
00057 #endif /* TASK_DISPLAY_H */

```

6.61. Referencia del archivo main/tasks/task_energia.c

Implementación de la gestión energética del sistema.

```

#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "esp_log.h"
#include "esp_sleep.h"

```

Funciones

- void [task_energia](#) (void *pvParameters)
Tarea principal de gestión energética.

Variables

- TaskHandle_t [task_sensores_handle](#)
Handle de la tarea de sensores.
- static const char * [TAG](#) = "ENERGIA"
Etiqueta utilizada para mensajes de depuración.

6.61.1. Descripción detallada

Implementación de la gestión energética del sistema.

Esta tarea coordina el ciclo operativo general del sistema de fermentación y gestiona la transición hacia modos de bajo consumo.

El flujo de operación sigue la secuencia:

Medir ↓ Procesar ↓ Actuar ↓ Registrar ↓ Dormir

La sincronización entre tareas se realiza mediante notificaciones de FreeRTOS, permitiendo una ejecución eficiente y evitando ciclos de espera activa.

Funciones principales:

- Inicio de ciclos de adquisición.
- Coordinación entre tareas.
- Espera de finalización de procesos.
- Activación del modo Deep Sleep.

Mecanismos FreeRTOS utilizados:

- Notificaciones entre tareas.

Modos de bajo consumo:

- Deep Sleep.

Autor

Fernando Plazas Trujillo Isabella Ordoñez Juan Daniel Constain

Definición en el archivo [task_energia.c](#).

6.61.2. Documentación de funciones

6.61.2.1. task_energia()

```
void task_energia (
    void * pvParameters)
```

Tarea principal de gestión energética.

La tarea permanece bloqueada esperando una notificación que indique el inicio de un nuevo ciclo operativo.

Flujo de ejecución:

- Esperar activación.
- Iniciar adquisición de sensores.
- Esperar finalización de procesos.
- Entrar en modo Deep Sleep.

La coordinación se realiza mediante notificaciones directas de FreeRTOS para minimizar el uso de memoria y CPU.

Parámetros

<i>pvParameters</i>	Parámetros de la tarea (no utilizados).
---------------------	---

Coordina el ciclo de ejecución del sistema y supervisa la finalización de las tareas necesarias antes de activar mecanismos de ahorro energético.

Dependiendo del estado del sistema, puede preparar la transición hacia modos de bajo consumo como Deep Sleep.

Parámetros

<i>pvParameters</i>	Parámetros de la tarea (no utilizados).
---------------------	---

Esperar inicio de ciclo.

Activar adquisición de sensores.

Esperar finalización de procesamiento y actuación.

Actualmente se utiliza un tiempo máximo de espera como mecanismo de sincronización.

Esperar finalización de tareas pendientes como almacenamiento o comunicaciones.

Transición al modo Deep Sleep.

El sistema permanecerá suspendido hasta que ocurra el evento de despertar configurado.

Definición en la línea 79 del archivo [task_energia.c](#).

```
00080 {
00081     while (1)
00082     {
00086         ulTaskNotifyTake(
00087             pdTRUE,
00088             portMAX_DELAY);
```

```

00089
00090     ESP_LOGI(TAG, "Ciclo iniciado");
00091
00095     xTaskNotifyGive(
00096         task_sensores_handle);
00097
00105     ulTaskNotifyTake(
00106         pdTRUE,
00107         pdMS_TO_TICKS(25000));
00108
00113     ulTaskNotifyTake(
00114         pdTRUE,
00115         pdMS_TO_TICKS(5000));
00116
00117     ESP_LOGI(TAG, "Entrando en deep sleep");
00118
00125     esp_deep_sleep_start();
00126 }
00127 }

```

6.61.3. Documentación de variables

6.61.3.1. TAG

```
const char* TAG = "ENERGIA" [static]
```

Etiqueta utilizada para mensajes de depuración.

Definición en la línea 59 del archivo [task_energia.c](#).

6.61.3.2. task_sensores_handle

```
TaskHandle_t task_sensores_handle [extern]
```

Handle de la tarea de sensores.

Utilizado para iniciar un nuevo ciclo de adquisición.

Utilizado por `task_energia` para iniciar nuevos ciclos de adquisición.

Definición en la línea 101 del archivo [main.c](#).

6.62. task_energia.c

[Ir a la documentación de este archivo.](#)

```

00001
00042
00043 #include "freertos/FreeRTOS.h"
00044 #include "freertos/task.h"
00045
00046 #include "esp_log.h"
00047 #include "esp_sleep.h"
00048
00054 extern TaskHandle_t task_sensores_handle;
00055
00059 static const char *TAG = "ENERGIA";
00060
00079 void task_energia(void *pvParameters)
00080 {
00081     while (1)
00082     {
00086         ulTaskNotifyTake(
00087             pdTRUE,
00088             portMAX_DELAY);

```

```
00089
00090     ESP_LOGI(TAG, "Ciclo iniciado");
00091
00095     xTaskNotifyGive(
00096         task_sensores_handle);
00097
00105     ulTaskNotifyTake(
00106         pdTRUE,
00107         pdMS_TO_TICKS(25000));
00108
00113     ulTaskNotifyTake(
00114         pdTRUE,
00115         pdMS_TO_TICKS(5000));
00116
00117     ESP_LOGI(TAG, "Entrando en deep sleep");
00118
00125     esp_deep_sleep_start();
00126 }
00127 }
```

6.63. Referencia del archivo main/tasks/task_energia.h

Declaración de la tarea de gestión energética del sistema.

Funciones

- void [task_energia](#) (void *pvParameters)
Tarea principal de gestión energética.

6.63.1. Descripción detallada

Declaración de la tarea de gestión energética del sistema.

Este módulo define la tarea responsable de coordinar el ciclo operativo del sistema y gestionar los modos de bajo consumo.

La tarea supervisa la finalización de actividades críticas del proceso de fermentación y determina cuándo el sistema puede entrar en estados de ahorro energético.

Funciones principales:

- Coordinación del ciclo de operación.
- Sincronización con otras tareas.
- Gestión de modos de bajo consumo.
- Preparación para entrada en Deep Sleep.
- Optimización del consumo energético.

Arquitectura:

task_actuadores | v Notificación FreeRTOS | v task_energia | v Deep Sleep

Esta tarea forma parte de la estrategia de eficiencia energética del sistema embebido.

Autor

Fernando Plazas Trujillo Isabella Ordoñez Juan Daniel Constain

Definición en el archivo [task_energia.h](#).

6.63.2. Documentación de funciones

6.63.2.1. task_energia()

```
void task_energia (
    void * pvParameters)
```

Tarea principal de gestión energética.

Coordina el ciclo de ejecución del sistema y supervisa la finalización de las tareas necesarias antes de activar mecanismos de ahorro energético.

Dependiendo del estado del sistema, puede preparar la transición hacia modos de bajo consumo como Deep Sleep.

Parámetros

<i>pvParameters</i>	Parámetros de la tarea (no utilizados).
---------------------	---

La tarea permanece bloqueada esperando una notificación que indique el inicio de un nuevo ciclo operativo.

Flujo de ejecución:

- Esperar activación.
- Iniciar adquisición de sensores.
- Esperar finalización de procesos.
- Entrar en modo Deep Sleep.

La coordinación se realiza mediante notificaciones directas de FreeRTOS para minimizar el uso de memoria y CPU.

Parámetros

<i>pvParameters</i>	Parámetros de la tarea (no utilizados).
---------------------	---

Esperar inicio de ciclo.

Activar adquisición de sensores.

Esperar finalización de procesamiento y actuación.

Actualmente se utiliza un tiempo máximo de espera como mecanismo de sincronización.

Esperar finalización de tareas pendientes como almacenamiento o comunicaciones.

Transición al modo Deep Sleep.

El sistema permanecerá suspendido hasta que ocurra el evento de despertar configurado.

Definición en la línea 79 del archivo [task_energia.c](#).

```
00080 {
00081     while (1)
00082     {
00086         ulTaskNotifyTake(
00087             pdTRUE,
00088             portMAX_DELAY);
```

```

00089
00090     ESP_LOGI(TAG, "Ciclo iniciado");
00091
00095     xTaskNotifyGive(
00096         task_sensores_handle);
00097
00105     ulTaskNotifyTake(
00106         pdTRUE,
00107         pdMS_TO_TICKS(25000));
00108
00113     ulTaskNotifyTake(
00114         pdTRUE,
00115         pdMS_TO_TICKS(5000));
00116
00117     ESP_LOGI(TAG, "Entrando en deep sleep");
00118
00125     esp_deep_sleep_start();
00126 }
00127 }

```

6.64. task_energia.h

[Ir a la documentación de este archivo.](#)

```

00001
00040
00041 #ifndef TASK_ENERGIA_H
00042 #define TASK_ENERGIA_H
00043
00056 void task_energia(void *pvParameters);
00057
00058 #endif /* TASK_ENERGIA_H */

```

6.65. Referencia del archivo main/tasks/task_logger.c

Implementación de la tarea de registro de datos.

```

#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/queue.h"
#include "esp_log.h"
#include "utils/data_types.h"
#include "drivers/sd_card.h"
#include "queues.h"

```

Funciones

- void `task_logger` (void *pvParameters)
Tarea principal de registro de datos.

Variables

- static const char * `TAG` = "LOGGER"
Etiqueta utilizada para mensajes de depuración.

6.65.1. Descripción detallada

Implementación de la tarea de registro de datos.

Esta tarea recibe mediciones provenientes de los sensores, genera registros en formato CSV y los almacena de forma persistente en la tarjeta SD.

Funciones principales:

- Recepción de datos mediante colas FreeRTOS.
- Formateo de registros CSV.
- Escritura en memoria SD.
- Generación de historial de fermentación.

Arquitectura:

queue_logger | v task_logger | v SD Card

La tarea se ejecuta de forma independiente para evitar que las operaciones de almacenamiento bloqueen la adquisición de sensores o la lógica de control.

Mecanismos FreeRTOS utilizados:

- Colas para comunicación entre tareas.

Autor

Fernando Plazas Trujillo Isabella Ordoñez Juan Daniel Constain

Definición en el archivo [task_logger.c](#).

6.65.2. Documentación de funciones

6.65.2.1. task_logger()

```
void task_logger (  
    void * pvParameters)
```

Tarea principal de registro de datos.

Espera mediciones provenientes de queue_logger, genera una línea en formato CSV y la almacena en la tarjeta SD.

Cada registro contiene:

- Fecha.
- Hora.
- Temperatura.

- pH.
- CO2 procesado.
- CO2 crudo.
- Voltaje.
- Corriente.

La tarea permanece bloqueada mientras no existan nuevos datos disponibles, optimizando el uso de CPU.

Parámetros

<i>pvParameters</i>	Parámetros de la tarea (no utilizados).
---------------------	---

Espera información proveniente de `queue_logger`, genera registros en formato CSV y los almacena en la tarjeta SD.

La tarea se ejecuta de forma independiente para evitar que las operaciones de escritura afecten el tiempo de respuesta de las tareas críticas del sistema.

Parámetros

<i>pvParameters</i>	Parámetros de la tarea (no utilizados).
---------------------	---

Buffer utilizado para generar la línea CSV.

Inicializar tarjeta SD.

Esperar nuevos datos para registrar.

Generar registro CSV.

Almacenar registro en tarjeta SD.

Definición en la línea 77 del archivo `task_logger.c`.

```

00078 {
00079     sensor_data_t data;
00080
00084     char line[160];
00085
00089     if (sd_card_init() != ESP_OK)
00090     {
00091         ESP_LOGE(TAG, "SD no inicializada");
00092     }
00093
00094     while (1)
00095     {
00099         if (xQueueReceive(
00100             queue_logger,
00101             &data,
00102             portMAX_DELAY))
00103         {
00107             snprintf(
00108                 line,
00109                 sizeof(line),
00110                 "%02d-%02d-%04d, %02d: %02d: %02d, %.2f, %.2f, %.2f, %.2f, %.2f, %.2f",
00111                 data.datetime.day,
00112                 data.datetime.month,
00113                 data.datetime.year,
00114                 data.datetime.hours,
00115                 data.datetime.minutes,
00116                 data.datetime.seconds,
00117                 data.temperatura,
00118                 data.ph,
00119                 data.co2,

```

```

00120         data.co2_raw,
00121         data.voltaje,
00122         data.corriente);
00123
00124     ESP_LOGI(TAG, "LOG -> %s", line);
00125
00129     if (sd_card_write_line(line) != ESP_OK)
00130     {
00131         ESP_LOGE(TAG, "Error escribiendo en SD");
00132     }
00133     else
00134     {
00135         ESP_LOGI(TAG, "Dato guardado en SD");
00136     }
00137 }
00138 }
00139 }

```

6.65.3. Documentación de variables

6.65.3.1. TAG

```
const char* TAG = "LOGGER" [static]
```

Etiqueta utilizada para mensajes de depuración.

Definición en la línea 53 del archivo `task_logger.c`.

6.66. task_logger.c

[Ir a la documentación de este archivo.](#)

```

00001
00037
00038 #include <stdio.h>
00039
00040 #include "freertos/FreeRTOS.h"
00041 #include "freertos/task.h"
00042 #include "freertos/queue.h"
00043
00044 #include "esp_log.h"
00045
00046 #include "utils/data_types.h"
00047 #include "drivers/sd_card.h"
00048 #include "queues.h"
00049
00053 static const char *TAG = "LOGGER";
00054
00077 void task_logger(void *pvParameters)
00078 {
00079     sensor_data_t data;
00080
00084     char line[160];
00085
00089     if (sd_card_init() != ESP_OK)
00090     {
00091         ESP_LOGE(TAG, "SD no inicializada");
00092     }
00093
00094     while (1)
00095     {
00099         if (xQueueReceive(
00100             queue_logger,
00101             &data,
00102             portMAX_DELAY))
00103         {
00107             snprintf(
00108                 line,
00109                 sizeof(line),
00110                 "%02d-%02d-%04d, %02d: %02d: %02d, %.2f, %.2f, %.2f, %.2f, %.2f",
00111                 data.datetime.day,
00112                 data.datetime.month,
00113                 data.datetime.year,

```



```

00114         data.datetime.hours,
00115         data.datetime.minutes,
00116         data.datetime.seconds,
00117         data.temperatura,
00118         data.ph,
00119         data.co2,
00120         data.co2_raw,
00121         data.voltaje,
00122         data.corriente);
00123
00124     ESP_LOGI(TAG, "LOG -> %s", line);
00125
00129     if (sd_card_write_line(line) != ESP_OK)
00130     {
00131         ESP_LOGE(TAG, "Error escribiendo en SD");
00132     }
00133     else
00134     {
00135         ESP_LOGI(TAG, "Dato guardado en SD");
00136     }
00137 }
00138 }
00139 }

```

6.67. Referencia del archivo main/tasks/task_logger.h

Declaración de la tarea de registro de datos.

Funciones

- void `task_logger` (void *pvParameters)
Tarea principal de registro de datos.

6.67.1. Descripción detallada

Declaración de la tarea de registro de datos.

Este módulo define la tarea responsable de almacenar las mediciones generadas durante el proceso de fermentación.

La tarea recibe datos provenientes de los sensores mediante una cola FreeRTOS, los formatea adecuadamente y los registra en la tarjeta SD para su posterior análisis.

Funciones principales:

- Recepción de mediciones del sistema.
- Formateo de registros en formato CSV.
- Almacenamiento persistente en tarjeta SD.
- Conservación del historial de fermentación.

Arquitectura:

queue_logger | v task_logger | v SD Card

Esta tarea permite desacoplar el almacenamiento de datos de las tareas de adquisición y control.

Autor

Fernando Plazas Trujillo Isabella Ordoñez Juan Daniel Constain

Definición en el archivo `task_logger.h`.

6.67.2. Documentación de funciones

6.67.2.1. task_logger()

```
void task_logger (  
    void * pvParameters)
```

Tarea principal de registro de datos.

Espera información proveniente de queue_logger, genera registros en formato CSV y los almacena en la tarjeta SD.

La tarea se ejecuta de forma independiente para evitar que las operaciones de escritura afecten el tiempo de respuesta de las tareas críticas del sistema.

Parámetros

<i>pvParameters</i>	Parámetros de la tarea (no utilizados).
---------------------	---

Espera mediciones provenientes de queue_logger, genera una línea en formato CSV y la almacena en la tarjeta SD.

Cada registro contiene:

- Fecha.
- Hora.
- Temperatura.
- pH.
- CO2 procesado.
- CO2 crudo.
- Voltaje.
- Corriente.

La tarea permanece bloqueada mientras no existan nuevos datos disponibles, optimizando el uso de CPU.

Parámetros

<i>pvParameters</i>	Parámetros de la tarea (no utilizados).
---------------------	---

Buffer utilizado para generar la línea CSV.

Inicializar tarjeta SD.

Esperar nuevos datos para registrar.

Generar registro CSV.

Almacenar registro en tarjeta SD.

Definición en la línea 77 del archivo `task_logger.c`.

```

00078 {
00079     sensor_data_t data;
00080
00084     char line[160];
00085
00089     if (sd_card_init() != ESP_OK)
00090     {
00091         ESP_LOGE(TAG, "SD no inicializada");
00092     }
00093
00094     while (1)
00095     {
00099         if (xQueueReceive(
00100             queue_logger,
00101             &data,
00102             portMAX_DELAY))
00103         {
00107             snprintf(
00108                 line,
00109                 sizeof(line),
00110                 "%02d-%02d-%04d, %02d: %02d: %02d, %.2f, %.2f, %.2f, %.2f, %.2f",
00111                 data.datetime.day,
00112                 data.datetime.month,
00113                 data.datetime.year,
00114                 data.datetime.hours,
00115                 data.datetime.minutes,
00116                 data.datetime.seconds,
00117                 data.temperatura,
00118                 data.ph,
00119                 data.co2,
00120                 data.co2_raw,
00121                 data.voltaje,
00122                 data.corriente);
00123
00124             ESP_LOGI(TAG, "LOG -> %s", line);
00125
00129             if (sd_card_write_line(line) != ESP_OK)
00130             {
00131                 ESP_LOGE(TAG, "Error escribiendo en SD");
00132             }
00133             else
00134             {
00135                 ESP_LOGI(TAG, "Dato guardado en SD");
00136             }
00137         }
00138     }
00139 }

```

6.68. task_logger.h

[Ir a la documentación de este archivo.](#)

```

00001
00036
00037 #ifndef TASK_LOGGER_H
00038 #define TASK_LOGGER_H
00039
00053 void task_logger(void *pvParameters);
00054
00055 #endif /* TASK_LOGGER_H */

```

6.69. Referencia del archivo main/tasks/task_mqtt.c

Implementación de la tarea de comunicación MQTT.

```

#include <stdbool.h>
#include <stdio.h>
#include <string.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "drivers/sd_card.h"

```

```
#include "mqtt/mqtt_manager.h"
#include "queues.h"
#include "utils/data_types.h"
```

defines

- #define `MQTT_TOPIC` "fermentador/espInterior/datos"
Tópico MQTT utilizado para publicar mediciones.
- #define `MQTT_CONNECT_TIMEOUT_MS` 8000
Tiempo máximo de espera para conexión MQTT.
- #define `MQTT_PUBLISH_TIMEOUT_MS` 5000
Tiempo máximo de espera para publicación MQTT.
- #define `MQTT_PENDING_PAYLOAD_SIZE` 320
Tamaño máximo permitido para payloads MQTT.

Funciones

- static void `save_pending_payload` (const char *payload)
Guarda un payload MQTT pendiente en la SD.
- static bool `resend_pending_mqtt_messages` (void)
Reenvía mensajes MQTT almacenados en la SD.
- static void `build_payload` (const `sensor_data_t` *data, char *payload, size_t payload_size)
Construye un payload JSON a partir de una medición.
- void `task_mqtt` (void *pvParameters)
Tarea principal de comunicación MQTT.

Variables

- TaskHandle_t `task_energia_handle`
Handle de la tarea de energía.

6.69.1. Descripción detallada

Implementación de la tarea de comunicación MQTT.

Esta tarea recibe mediciones provenientes del sistema, genera mensajes JSON y los publica en un broker MQTT.

Funcionalidades principales:

- Publicación de datos mediante MQTT.
- Generación de payloads JSON.
- Reintento automático de mensajes pendientes.
- Almacenamiento local cuando no existe conectividad.
- Recuperación de mensajes almacenados en SD.

Arquitectura:

queue_mqtt | v task_mqtt | +-----> MQTT Broker | +-----> Archivo de pendientes (SD)

Estrategia de tolerancia a fallos:

Si el broker MQTT no está disponible:

- El dato no se pierde.
- Se almacena en la tarjeta SD.
- Se reenvía automáticamente cuando la conexión se recupera.

Mecanismos FreeRTOS utilizados:

- Colas.
- Notificaciones entre tareas.

Autor

Fernando Plazas Trujillo Isabella Ordoñez Juan Daniel Constain

Definición en el archivo [task_mqtt.c](#).

6.69.2. Documentación de «define»

6.69.2.1. MQTT_CONNECT_TIMEOUT_MS

```
#define MQTT_CONNECT_TIMEOUT_MS 8000
```

Tiempo máximo de espera para conexión MQTT.

Definición en la línea 63 del archivo [task_mqtt.c](#).

6.69.2.2. MQTT_PENDING_PAYLOAD_SIZE

```
#define MQTT_PENDING_PAYLOAD_SIZE 320
```

Tamaño máximo permitido para payloads MQTT.

Definición en la línea 73 del archivo [task_mqtt.c](#).

6.69.2.3. MQTT_PUBLISH_TIMEOUT_MS

```
#define MQTT_PUBLISH_TIMEOUT_MS 5000
```

Tiempo máximo de espera para publicación MQTT.

Definición en la línea 68 del archivo [task_mqtt.c](#).

6.69.2.4. MQTT_TOPIC

```
#define MQTT_TOPIC "fermentador/espInterior/datos"
```

Tópico MQTT utilizado para publicar mediciones.

Definición en la línea 58 del archivo [task_mqtt.c](#).

6.69.3. Documentación de funciones

6.69.3.1. build_payload()

```
void build_payload (
    const sensor_data_t * data,
    char * payload,
    size_t payload_size) [static]
```

Construye un payload JSON a partir de una medición.

Convierte una estructura [sensor_data_t](#) en un mensaje JSON compatible con el broker MQTT y los servicios de monitoreo externos.

Parámetros

<i>data</i>	Datos del sensor.
<i>payload</i>	Buffer de salida.
<i>payload_size</i>	Tamaño del buffer.

Definición en la línea 190 del archivo [task_mqtt.c](#).

```
00194 {
00195     snprintf(
00196         payload,
00197         payload_size,
00198         "{ \"timestamp\": \"%04d-%02d-%02d %02d: %02d\", \"
00199         \"temperatura\": %.2f, \"ph\": %.2f, \"co2\": %.4f, \"
00200         \"co2_raw\": %.4f, \"voltaje\": %.2f, \"corriente\": %.2f }\",
00201         data->datetime.year,
00202         data->datetime.month,
00203         data->datetime.day,
00204         data->datetime.hours,
00205         data->datetime.minutes,
00206         data->datetime.seconds,
00207         data->temperatura,
00208         data->ph,
00209         data->co2,
00210         data->co2_raw,
00211         data->voltaje,
00212         data->corriente);
00213 }
```

6.69.3.2. resend_pending_mqtt_messages()

```
bool resend_pending_mqtt_messages (
    void ) [static]
```

Reenvía mensajes MQTT almacenados en la SD.

Lee todos los mensajes pendientes almacenados localmente y los publica nuevamente en el broker.

Si todos los mensajes son enviados correctamente, el archivo de pendientes es eliminado.

Devuelve

true si todos los mensajes fueron reenviados.

false si ocurrió algún error.

Definición en la línea 115 del archivo `task_mqtt.c`.

```

00116 {
00117     char pending_payload[MQTT_PENDING_PAYLOAD_SIZE];
00118     FILE *pending_file;
00119
00120     if (!sd_card_pending_mqtt_exists())
00121     {
00122         return true;
00123     }
00124
00125     printf("Reenviando mensajes MQTT pendientes desde SD\n");
00126
00127     pending_file = sd_card_open_pending_mqtt_read();
00128
00129     if (pending_file == NULL)
00130     {
00131         return false;
00132     }
00133
00134     while (sd_card_read_pending_mqtt_line(
00135         pending_file,
00136         pending_payload,
00137         sizeof(pending_payload)) != NULL)
00138     {
00139         pending_payload[strcspn(
00140             pending_payload,
00141             "\r\n")] = '\0';
00142
00143         if (pending_payload[0] == '\0')
00144         {
00145             continue;
00146         }
00147
00148         printf("MQTT pendiente -> %s\n", pending_payload);
00149
00150         if (!mqtt_publish(
00151             MQTT_TOPIC,
00152             pending_payload,
00153             1,
00154             MQTT_PUBLISH_TIMEOUT_MS))
00155         {
00156             printf("Error reenviando payload MQTT pendiente\n");
00157
00158             sd_card_close_pending_mqtt(
00159                 pending_file);
00160
00161             return false;
00162         }
00163     }
00164
00165     sd_card_close_pending_mqtt(
00166         pending_file);
00167
00168     if (sd_card_delete_pending_mqtt() != ESP_OK)
00169     {
00170         printf("Mensajes pendientes reenviados, pero no se pudo eliminar el archivo\n");
00171         return false;
00172     }
00173
00174     printf("Mensajes MQTT pendientes reenviados correctamente\n");
00175
00176     return true;
00177 }

```

6.69.3.3. save_pending_payload()

```

void save_pending_payload (
    const char * payload) [static]

```

Guarda un payload MQTT pendiente en la SD.

Se utiliza cuando el broker MQTT no está disponible o cuando ocurre un error durante la publicación.

Parámetros

<i>payload</i>	Cadena JSON a almacenar.
----------------	--------------------------

Definición en la línea 91 del archivo [task_mqtt.c](#).

```
00092 {
00093     if (sd_card_append_pending_mqtt(payload) == ESP_OK)
00094     {
00095         printf("Payload MQTT guardado en pendientes\n");
00096     }
00097     else
00098     {
00099         printf("Error guardando payload MQTT pendiente\n");
00100     }
00101 }
```

6.69.3.4. task_mqtt()

```
void task_mqtt (
    void * pvParameters)
```

Tarea principal de comunicación MQTT.

Tarea principal de publicación MQTT.

Espera nuevas mediciones desde `queue_mqtt`, genera el payload JSON correspondiente y realiza la publicación en el broker MQTT.

En caso de fallo:

- El dato se almacena en SD.
- Se reintentará posteriormente.

Una vez finalizado el proceso, se notifica a `task_energia` para continuar el ciclo operativo.

Parámetros

<i>pvParameters</i>	Parámetros de la tarea (no utilizados).
---------------------	---

Tarea principal de comunicación MQTT.

Espera datos provenientes de `queue_mqtt`, genera los mensajes correspondientes y los publica en el broker MQTT.

La tarea opera de forma independiente para evitar que las latencias de red afecten la adquisición de sensores o la ejecución de la lógica de control.

Parámetros

<i>pvParameters</i>	Parámetros de la tarea (no utilizados).
---------------------	---

Definición en la línea 231 del archivo [task_mqtt.c](#).

```
00232 {
00233     sensor_data_t data;
00234     char payload[MQTT_PENDING_PAYLOAD_SIZE];
00235
00236     while (1)
```



```

00237     {
00238         if (xQueueReceive(
00239             queue_mqtt,
00240             &data,
00241             portMAX_DELAY))
00242         {
00243             build_payload(
00244                 &data,
00245                 payload,
00246                 sizeof(payload));
00247
00248             printf(
00249                 "MQTT payload -> %s\n",
00250                 payload);
00251
00252             mqtt_manager_start();
00253
00254             if (!mqtt_wait_connected(
00255                 MQTT_CONNECT_TIMEOUT_MS))
00256             {
00257                 printf("MQTT no conectado\n");
00258
00259                 save_pending_payload(payload);
00260
00261                 mqtt_manager_stop();
00262
00263                 xTaskNotifyGive(
00264                     task_energia_handle);
00265
00266                 continue;
00267             }
00268
00269             if (!resend_pending_mqtt_messages())
00270             {
00271                 printf("No se pudieron reenviar todos los pendientes MQTT\n");
00272
00273                 save_pending_payload(payload);
00274
00275                 mqtt_manager_stop();
00276
00277                 xTaskNotifyGive(
00278                     task_energia_handle);
00279
00280                 continue;
00281             }
00282
00283             if (mqtt_publish(
00284                 MQTT_TOPIC,
00285                 payload,
00286                 1,
00287                 MQTT_PUBLISH_TIMEOUT_MS))
00288             {
00289                 printf("Dato publicado por MQTT\n");
00290             }
00291             else
00292             {
00293                 printf("Error publicando por MQTT");
00294
00295                 save_pending_payload(payload);
00296             }
00297
00298             mqtt_manager_stop();
00299
00300             xTaskNotifyGive(
00301                 task_energia_handle);
00302         }
00303     }
00304 }

```

6.69.4. Documentación de variables

6.69.4.1. task_energia_handle

TaskHandle_t task_energia_handle [extern]

Handle de la tarea de energía.

Se utiliza para notificar la finalización del proceso de publicación.

Utilizado para coordinar el ciclo general del sistema.

Utilizado para enviar notificaciones cuando finaliza un ciclo de mezcla.

Definición en la línea 109 del archivo `main.c`.

6.70. `task_mqtt.c`

[Ir a la documentación de este archivo.](#)

```

00001
00042
00043 #include <stdbool.h>
00044 #include <stdio.h>
00045 #include <string.h>
00046
00047 #include "freertos/FreeRTOS.h"
00048 #include "freertos/task.h"
00049
00050 #include "drivers/sd_card.h"
00051 #include "mqtt/mqtt_manager.h"
00052 #include "queues.h"
00053 #include "utils/data_types.h"
00054
00058 #define MQTT_TOPIC "fermentador/espInterior/datos"
00059
00063 #define MQTT_CONNECT_TIMEOUT_MS 8000
00064
00068 #define MQTT_PUBLISH_TIMEOUT_MS 5000
00069
00073 #define MQTT_PENDING_PAYLOAD_SIZE 320
00074
00081 extern TaskHandle_t task_energia_handle;
00082
00091 static void save_pending_payload(const char *payload)
00092 {
00093     if (sd_card_append_pending_mqtt(payload) == ESP_OK)
00094     {
00095         printf("Payload MQTT guardado en pendientes\n");
00096     }
00097     else
00098     {
00099         printf("Error guardando payload MQTT pendiente\n");
00100     }
00101 }
00102
00115 static bool resend_pending_mqtt_messages(void)
00116 {
00117     char pending_payload[MQTT_PENDING_PAYLOAD_SIZE];
00118     FILE *pending_file;
00119
00120     if (!sd_card_pending_mqtt_exists())
00121     {
00122         return true;
00123     }
00124
00125     printf("Reenviando mensajes MQTT pendientes desde SD\n");
00126
00127     pending_file = sd_card_open_pending_mqtt_read();
00128
00129     if (pending_file == NULL)
00130     {
00131         return false;
00132     }
00133
00134     while (sd_card_read_pending_mqtt_line(
00135         pending_file,
00136         pending_payload,
00137         sizeof(pending_payload)) != NULL)
00138     {
00139         pending_payload[strcspn(
00140             pending_payload,
00141             "\r\n")] = '\0';
00142
00143         if (pending_payload[0] == '\0')
00144         {
00145             continue;
00146         }
00147
00148         printf("MQTT pendiente -> %s\n", pending_payload);

```

```

00149
00150     if (!mqtt_publish(
00151         MQTT_TOPIC,
00152         pending_payload,
00153         1,
00154         MQTT_PUBLISH_TIMEOUT_MS))
00155     {
00156         printf("Error reenviando payload MQTT pendiente\n");
00157
00158         sd_card_close_pending_mqtt(
00159             pending_file);
00160
00161         return false;
00162     }
00163 }
00164
00165 sd_card_close_pending_mqtt(
00166     pending_file);
00167
00168 if (sd_card_delete_pending_mqtt() != ESP_OK)
00169 {
00170     printf("Mensajes pendientes reenviados, pero no se pudo eliminar el archivo\n");
00171     return false;
00172 }
00173
00174 printf("Mensajes MQTT pendientes reenviados correctamente\n");
00175
00176 return true;
00177 }
00178
00190 static void build_payload(
00191     const sensor_data_t *data,
00192     char *payload,
00193     size_t payload_size)
00194 {
00195     snprintf(
00196         payload,
00197         payload_size,
00198         "{ \"timestamp\": \"%04d-%02d-%02d %02d: %02d\", \"
00199         \"temperatura\": %.2f, \"ph\": %.2f, \"co2\": %.4f, \"
00200         \"co2_raw\": %.4f, \"voltaje\": %.2f, \"corriente\": %.2f}\",
00201         data->datetime.year,
00202         data->datetime.month,
00203         data->datetime.day,
00204         data->datetime.hours,
00205         data->datetime.minutes,
00206         data->datetime.seconds,
00207         data->temperatura,
00208         data->ph,
00209         data->co2,
00210         data->co2_raw,
00211         data->voltaje,
00212         data->corriente);
00213 }
00214
00231 void task_mqtt(void *pvParameters)
00232 {
00233     sensor_data_t data;
00234     char payload[MQTT_PENDING_PAYLOAD_SIZE];
00235
00236     while (1)
00237     {
00238         if (xQueueReceive(
00239             queue_mqtt,
00240             &data,
00241             portMAX_DELAY))
00242         {
00243             build_payload(
00244                 &data,
00245                 payload,
00246                 sizeof(payload));
00247
00248             printf(
00249                 "MQTT payload -> %s\n",
00250                 payload);
00251
00252             mqtt_manager_start();
00253
00254             if (!mqtt_wait_connected(
00255                 MQTT_CONNECT_TIMEOUT_MS))
00256             {
00257                 printf("MQTT no conectado\n");
00258
00259                 save_pending_payload(payload);
00260
00261                 mqtt_manager_stop();
00262

```

```

00263         xTaskNotifyGive (
00264             task_energia_handle);
00265     }
00266     continue;
00267 }
00268
00269 if (!resend_pending_mqtt_messages())
00270 {
00271     printf("No se pudieron reenviar todos los pendientes MQTT\n");
00272     save_pending_payload(payload);
00273     mqtt_manager_stop();
00274     xTaskNotifyGive (
00275         task_energia_handle);
00276     continue;
00277 }
00278
00279 if (mqtt_publish(
00280     MQTT_TOPIC,
00281     payload,
00282     1,
00283     MQTT_PUBLISH_TIMEOUT_MS))
00284 {
00285     printf("Dato publicado por MQTT\n");
00286 }
00287 else
00288 {
00289     printf("Error publicando por MQTT");
00290     save_pending_payload(payload);
00291 }
00292 mqtt_manager_stop();
00293 xTaskNotifyGive (
00294     task_energia_handle);
00295 }
00296 }
00297 }
00298 }
00299 }
00300 }
00301 }
00302 }
00303 }
00304 }

```

6.71. Referencia del archivo main/tasks/task_mqtt.h

Declaración de la tarea de comunicación MQTT.

Funciones

- void `task_mqtt` (void *pvParameters)
Tarea principal de publicación MQTT.

6.71.1. Descripción detallada

Declaración de la tarea de comunicación MQTT.

Este módulo define la tarea responsable de publicar los datos del sistema en un broker MQTT para monitoreo remoto.

La tarea recibe mediciones provenientes de los sensores mediante una cola FreeRTOS y realiza su envío utilizando el módulo `mqtt_manager`.

Funciones principales:

- Recepción de datos de sensores.

- Formateo de mensajes MQTT.
- Publicación en tópicos MQTT.
- Monitoreo del estado de conexión.

Arquitectura:

queue_mqtt | v task_mqtt | v Broker MQTT

Esta tarea desacopla las comunicaciones de red de las tareas de adquisición y control del sistema.

Autor

Fernando Plazas Trujillo Isabella Ordoñez Juan Daniel Constain

Definición en el archivo [task_mqtt.h](#).

6.71.2. Documentación de funciones

6.71.2.1. task_mqtt()

```
void task_mqtt (  
    void * pvParameters)
```

Tarea principal de publicación MQTT.

Tarea principal de comunicación MQTT.

Espera datos provenientes de queue_mqtt, genera los mensajes correspondientes y los publica en el broker MQTT.

La tarea opera de forma independiente para evitar que las latencias de red afecten la adquisición de sensores o la ejecución de la lógica de control.

Parámetros

<i>pvParameters</i>	Parámetros de la tarea (no utilizados).
---------------------	---

Tarea principal de publicación MQTT.

Espera nuevas mediciones desde queue_mqtt, genera el payload JSON correspondiente y realiza la publicación en el broker MQTT.

En caso de fallo:

- El dato se almacena en SD.
- Se reintentará posteriormente.

Una vez finalizado el proceso, se notifica a task_energia para continuar el ciclo operativo.

Parámetros

<i>pvParameters</i>	Parámetros de la tarea (no utilizados).
---------------------	---

Definición en la línea 231 del archivo `task_mqtt.c`.

```

00232 {
00233     sensor_data_t data;
00234     char payload[MQTT_PENDING_PAYLOAD_SIZE];
00235
00236     while (1)
00237     {
00238         if (xQueueReceive(
00239             queue_mqtt,
00240             &data,
00241             portMAX_DELAY))
00242         {
00243             build_payload(
00244                 &data,
00245                 payload,
00246                 sizeof(payload));
00247
00248             printf(
00249                 "MQTT payload -> %s\n",
00250                 payload);
00251
00252             mqtt_manager_start();
00253
00254             if (!mqtt_wait_connected(
00255                 MQTT_CONNECT_TIMEOUT_MS))
00256             {
00257                 printf("MQTT no conectado\n");
00258
00259                 save_pending_payload(payload);
00260
00261                 mqtt_manager_stop();
00262
00263                 xTaskNotifyGive(
00264                     task_energia_handle);
00265
00266                 continue;
00267             }
00268
00269             if (!resend_pending_mqtt_messages())
00270             {
00271                 printf("No se pudieron reenviar todos los pendientes MQTT\n");
00272
00273                 save_pending_payload(payload);
00274
00275                 mqtt_manager_stop();
00276
00277                 xTaskNotifyGive(
00278                     task_energia_handle);
00279
00280                 continue;
00281             }
00282
00283             if (mqtt_publish(
00284                 MQTT_TOPIC,
00285                 payload,
00286                 1,
00287                 MQTT_PUBLISH_TIMEOUT_MS))
00288             {
00289                 printf("Dato publicado por MQTT\n");
00290             }
00291             else
00292             {
00293                 printf("Error publicando por MQTT");
00294
00295                 save_pending_payload(payload);
00296             }
00297
00298             mqtt_manager_stop();
00299
00300             xTaskNotifyGive(
00301                 task_energia_handle);
00302         }
00303     }
00304 }

```

6.72. task_mqtt.h

[Ir a la documentación de este archivo.](#)

```
00001
00036
00037 #ifndef TASK_MQTT_H
00038 #define TASK_MQTT_H
00039
00052 void task_mqtt(void *pvParameters);
00053
00054 #endif /* TASK_MQTT_H */
```

6.73. Referencia del archivo main/tasks/task_sensores.c

Implementación de la tarea de adquisición de sensores.

```
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "esp_log.h"
#include "utils/data_types.h"
#include "drivers/rtc_ds3231.h"
#include "drivers/mq135.h"
#include "drivers/ph_sensor.h"
#include "drivers/ds18b20.h"
#include "queues.h"
#include "wifi/wifi_manager.h"
#include "mqtt/mqtt_manager.h"
#include "drivers/sd_card.h"
```

defines

- #define ALPHA 0.2f
Constante del filtro exponencial.

Funciones

- void task_sensores (void *pvParameters)
Tarea principal de adquisición de sensores.

Variables

- static const char * TAG = "SENSORES"
Etiqueta utilizada para mensajes de depuración.
- static RTC_DATA_ATTR float baseline = 0
Valor de referencia del sensor MQ135.
- static RTC_DATA_ATTR int baseline_initialized = 0
Indica si el baseline ya fue inicializado.
- static float filtered = 0
Valor filtrado actual del sensor MQ135.

6.73.1. Descripción detallada

Implementación de la tarea de adquisición de sensores.

Esta tarea es responsable de realizar la lectura de los sensores del sistema de fermentación, procesar las mediciones obtenidas y distribuir los datos a las diferentes tareas mediante colas FreeRTOS.

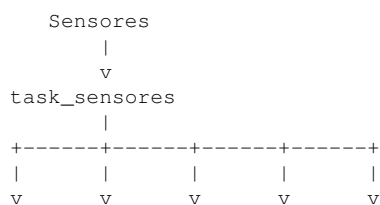
Sensores gestionados:

- DS18B20 (temperatura).
- MQ-135 (CO2).
- Sensor de pH.
- RTC DS3231.

Funciones principales:

- Adquisición de datos.
- Filtrado de señales.
- Cálculo de referencia (baseline).
- Obtención de fecha y hora.
- Distribución de información mediante colas.

Arquitectura:



control logger mqtt display

Mecanismos FreeRTOS utilizados:

- Notificaciones de tareas.
- Colas de comunicación.

Persistencia:

- Uso de RTC_DATA_ATTR para conservar variables durante ciclos de Deep Sleep.

Autor

Fernando Plazas Trujillo Isabella Ordoñez Juan Daniel Constain

Definición en el archivo [task_sensores.c](#).

6.73.2. Documentación de «define»

6.73.2.1. ALPHA

```
#define ALPHA 0.2f
```

Constante del filtro exponencial.

Definición en la línea 94 del archivo [task_sensores.c](#).

6.73.3. Documentación de funciones

6.73.3.1. task_sensores()

```
void task_sensores (  
    void * pvParameters)
```

Tarea principal de adquisición de sensores.

Permanece bloqueada esperando una notificación de activación proveniente de task_energia.

Una vez activada:

- Lee todos los sensores.
- Procesa las mediciones.
- Obtiene fecha y hora.
- Construye estructuras compartidas.
- Distribuye los datos mediante colas FreeRTOS.

Parámetros

<i>pvParameters</i>	Parámetros de la tarea (no utilizados).
---------------------	---

Realiza la lectura de los sensores del sistema, ejecuta el procesamiento inicial de las mediciones y distribuye los datos a través de las colas FreeRTOS correspondientes.

Funciones principales:

- Lectura de sensores.
- Obtención de fecha y hora.
- Construcción de estructuras [sensor_data_t](#).
- Envío de datos a tareas consumidoras.

Parámetros

<i>pvParameters</i>	Parámetros de la tarea (no utilizados).
---------------------	---

Aplicar filtro exponencial para reducir ruido en la señal.

Inicializar referencia base del sensor.

Definición en la línea 116 del archivo `task_sensores.c`.

```

00117 {
00118     sensor_data_t data;
00119
00120     // =====
00121     // INICIALIZACIÓN DE DRIVERS
00122     // =====
00123
00124     mq135_init();
00125
00126     ph_init();
00127
00128     ds18b20_init(GPIO_NUM_4);
00129
00130     while (1)
00131     {
00132         // =====
00133         // ESPERAR ACTIVACIÓN
00134         // =====
00135
00136         ulTaskNotifyTake(
00137             pdTRUE,
00138             portMAX_DELAY);
00139
00140         ESP_LOGI(TAG, "Leyendo sensores...");
00141
00142         // =====
00143         // SENSOR MQ135
00144         // =====
00145
00146         float raw_voltage =
00147             mq135_read();
00148
00149         int raw_adc =
00150             mq135_read_raw();
00151
00152         ESP_LOGI(
00153             TAG,
00154             "MQ135 ADC RAW: %d",
00155             raw_adc);
00156
00161         filtered =
00162             ALPHA * raw_voltage +
00163             (1.0f - ALPHA) * filtered;
00164
00168         if (!baseline_initialized)
00169         {
00170             baseline = filtered;
00171
00172             baseline_initialized = 1;
00173
00174             ESP_LOGI(
00175                 TAG,
00176                 "Baseline inicial: %.4f",
00177                 baseline);
00178         }
00179
00180         data.co2_raw = raw_voltage;
00181
00182         data.co2 =
00183             filtered - baseline;
00184
00185         // =====
00186         // SENSOR DE PH
00187         // =====
00188
00189         data.ph =
00190             ph_read();
00191
00192         // =====
00193         // TEMPERATURA DS18B20
00194         // =====
00195

```

```

00196     float temp =
00197         ds18b20_read_temperature();
00198
00199     if (temp == -127.0f)
00200     {
00201         ESP_LOGW(
00202             TAG,
00203             "Error leyendo DS18B20");
00204
00205         temp = 25.0f;
00206     }
00207
00208     data.temperatura = temp;
00209
00210     // =====
00211     // VARIABLES DE PRUEBA
00212     // =====
00213
00214     data.voltaje = 5.0f;
00215
00216     data.corriente = 0.5f;
00217
00218     // =====
00219     // RTC
00220     // =====
00221
00222     ds3231_get_datetime(
00223         &data.datetime);
00224
00225     printf(
00226         "RTC -> %02d:%02d:%02d\n",
00227         data.datetime.hours,
00228         data.datetime.minutes,
00229         data.datetime.seconds);
00230
00231     // =====
00232     // ENVÍO A TASK CONTROL
00233     // =====
00234
00235     xQueueSend(
00236         queue_sensores,
00237         &data,
00238         portMAX_DELAY);
00239
00240     // =====
00241     // ENVÍO A LOGGER
00242     // =====
00243
00244     xQueueSend(
00245         queue_logger,
00246         &data,
00247         portMAX_DELAY);
00248
00249     // =====
00250     // ENVÍO A MQTT
00251     // =====
00252
00253     xQueueSend(
00254         queue_mqtt,
00255         &data,
00256         portMAX_DELAY);
00257
00258     // =====
00259     // DATOS DE DISPLAY
00260     // =====
00261
00262     display_data_t display;
00263
00264     display.temperatura =
00265         data.temperatura;
00266
00267     display.ph =
00268         data.ph;
00269
00270     display.co2 =
00271         data.co2;
00272
00273     display.wifi_ok =
00274         wifi_is_connected();
00275
00276     display.mqtt_ok =
00277         mqtt_is_connected();
00278
00279     display.sd_ok =
00280         sd_card_is_mounted();
00281
00282     display.datetime =

```

```
00283         data.datetime;
00284
00285         xQueueSend(
00286             queue_display,
00287             &display,
00288             0);
00289
00290         ESP_LOGI(
00291             TAG,
00292             "Datos enviados");
00293     }
00294 }
```

6.73.4. Documentación de variables

6.73.4.1. baseline

```
RTC_DATA_ATTR float baseline = 0 [static]
```

Valor de referencia del sensor MQ135.

Se conserva durante ciclos de Deep Sleep mediante memoria RTC.

Definición en la línea 80 del archivo [task_sensores.c](#).

6.73.4.2. baseline_initialized

```
RTC_DATA_ATTR int baseline_initialized = 0 [static]
```

Indica si el baseline ya fue inicializado.

Definición en la línea 85 del archivo [task_sensores.c](#).

6.73.4.3. filtered

```
float filtered = 0 [static]
```

Valor filtrado actual del sensor MQ135.

Definición en la línea 99 del archivo [task_sensores.c](#).

6.73.4.4. TAG

```
const char* TAG = "SENSORES" [static]
```

Etiqueta utilizada para mensajes de depuración.

Definición en la línea 68 del archivo [task_sensores.c](#).

6.74. task_sensores.c

[Ir a la documentación de este archivo.](#)

```

00001
00048
00049 #include "freertos/FreeRTOS.h"
00050 #include "freertos/task.h"
00051
00052 #include "esp_log.h"
00053
00054 #include "utils/data_types.h"
00055 #include "drivers/rtc_ds3231.h"
00056 #include "drivers/mq135.h"
00057 #include "drivers/ph_sensor.h"
00058 #include "drivers/ds18b20.h"
00059 #include "queues.h"
00060
00061 #include "wifi/wifi_manager.h"
00062 #include "mqtt/mqtt_manager.h"
00063 #include "drivers/sd_card.h"
00064
00068 static const char *TAG = "SENSORES";
00069
00070 // =====
00071 // PERSISTENCIA EN DEEP SLEEP
00072 // =====
00073
00080 RTC_DATA_ATTR static float baseline = 0;
00081
00085 RTC_DATA_ATTR static int baseline_initialized = 0;
00086
00087 // =====
00088 // FILTRO EXPONENCIAL
00089 // =====
00090
00094 #define ALPHA 0.2f
00095
00099 static float filtered = 0;
00100
00116 void task_sensores(void *pvParameters)
00117 {
00118     sensor_data_t data;
00119
00120     // =====
00121     // INICIALIZACIÓN DE DRIVERS
00122     // =====
00123
00124     mq135_init();
00125
00126     ph_init();
00127
00128     ds18b20_init(GPIO_NUM_4);
00129
00130     while (1)
00131     {
00132         // =====
00133         // ESPERAR ACTIVACIÓN
00134         // =====
00135
00136         ulTaskNotifyTake(
00137             pdTRUE,
00138             portMAX_DELAY);
00139
00140         ESP_LOGI(TAG, "Leyendo sensores...");
00141
00142         // =====
00143         // SENSOR MQ135
00144         // =====
00145
00146         float raw_voltage =
00147             mq135_read();
00148
00149         int raw_adc =
00150             mq135_read_raw();
00151
00152         ESP_LOGI(
00153             TAG,
00154             "MQ135 ADC RAW: %d",
00155             raw_adc);
00156
00161         filtered =
00162             ALPHA * raw_voltage +
00163             (1.0f - ALPHA) * filtered;
00164
00168         if (!baseline_initialized)

```

```

00169     {
00170         baseline = filtered;
00171
00172         baseline_initialized = 1;
00173
00174         ESP_LOGI(
00175             TAG,
00176             "Baseline inicial: %.4f",
00177             baseline);
00178     }
00179
00180     data.co2_raw = raw_voltage;
00181
00182     data.co2 =
00183         filtered - baseline;
00184
00185     // =====
00186     // SENSOR DE PH
00187     // =====
00188
00189     data.ph =
00190         ph_read();
00191
00192     // =====
00193     // TEMPERATURA DS18B20
00194     // =====
00195
00196     float temp =
00197         ds18b20_read_temperature();
00198
00199     if (temp == -127.0f)
00200     {
00201         ESP_LOGW(
00202             TAG,
00203             "Error leyendo DS18B20");
00204
00205         temp = 25.0f;
00206     }
00207
00208     data.temperatura = temp;
00209
00210     // =====
00211     // VARIABLES DE PRUEBA
00212     // =====
00213
00214     data.voltaje = 5.0f;
00215
00216     data.corriente = 0.5f;
00217
00218     // =====
00219     // RTC
00220     // =====
00221
00222     ds3231_get_datetime(
00223         &data.datetime);
00224
00225     printf(
00226         "RTC -> %02d: %02d: %02d\n",
00227         data.datetime.hours,
00228         data.datetime.minutes,
00229         data.datetime.seconds);
00230
00231     // =====
00232     // ENVÍO A TASK CONTROL
00233     // =====
00234
00235     xQueueSend(
00236         queue_sensores,
00237         &data,
00238         portMAX_DELAY);
00239
00240     // =====
00241     // ENVÍO A LOGGER
00242     // =====
00243
00244     xQueueSend(
00245         queue_logger,
00246         &data,
00247         portMAX_DELAY);
00248
00249     // =====
00250     // ENVÍO A MQTT
00251     // =====
00252
00253     xQueueSend(
00254         queue_mqtt,
00255         &data,

```

```

00256         portMAX_DELAY);
00257
00258     // =====
00259     // DATOS DE DISPLAY
00260     // =====
00261
00262     display_data_t display;
00263
00264     display.temperatura =
00265         data.temperatura;
00266
00267     display.ph =
00268         data.ph;
00269
00270     display.co2 =
00271         data.co2;
00272
00273     display.wifi_ok =
00274         wifi_is_connected();
00275
00276     display.mqtt_ok =
00277         mqtt_is_connected();
00278
00279     display.sd_ok =
00280         sd_card_is_mounted();
00281
00282     display.datetime =
00283         data.datetime;
00284
00285     xQueueSend(
00286         queue_display,
00287         &display,
00288         0);
00289
00290     ESP_LOGI(
00291         TAG,
00292         "Datos enviados");
00293 }
00294 }

```

6.75. Referencia del archivo main/tasks/task_sensores.h

Declaración de la tarea de adquisición de sensores.

Funciones

- void `task_sensores` (void *pvParameters)
Tarea principal de adquisición de sensores.

6.75.1. Descripción detallada

Declaración de la tarea de adquisición de sensores.

Este módulo define la tarea responsable de realizar la adquisición periódica de las variables físicas del sistema de fermentación.

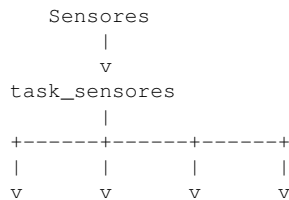
La tarea obtiene información desde los sensores conectados al ESP32, realiza el procesamiento inicial de los datos y distribuye los resultados hacia las diferentes tareas del sistema mediante colas FreeRTOS.

Sensores gestionados:

- DS18B20 (temperatura).
- MQ-135 (CO2).

- Sensor de pH.
- Monitor de voltaje.
- Monitor de corriente.
- RTC DS3231.

Arquitectura:



logger mqtt display control

Esta tarea actúa como productor principal de datos dentro de la arquitectura del sistema.

Autor

Fernando Plazas Trujillo Isabella Ordoñez Juan Daniel Constain

Definición en el archivo [task_sensores.h](#).

6.75.2. Documentación de funciones

6.75.2.1. task_sensores()

```
void task_sensores (
    void * pvParameters)
```

Tarea principal de adquisición de sensores.

Realiza la lectura de los sensores del sistema, ejecuta el procesamiento inicial de las mediciones y distribuye los datos a través de las colas FreeRTOS correspondientes.

Funciones principales:

- Lectura de sensores.
- Obtención de fecha y hora.
- Construcción de estructuras [sensor_data_t](#).
- Envío de datos a tareas consumidoras.

Parámetros

<i>pvParameters</i>	Parámetros de la tarea (no utilizados).
---------------------	---

Permanece bloqueada esperando una notificación de activación proveniente de task_energia.

Una vez activada:

- Lee todos los sensores.
- Procesa las mediciones.
- Obtiene fecha y hora.
- Construye estructuras compartidas.
- Distribuye los datos mediante colas FreeRTOS.

Parámetros

<i>pvParameters</i>	Parámetros de la tarea (no utilizados).
---------------------	---

Aplicar filtro exponencial para reducir ruido en la señal.

Inicializar referencia base del sensor.

Definición en la línea 116 del archivo [task_sensores.c](#).

```

00117 {
00118     sensor_data_t data;
00119
00120     // =====
00121     // INICIALIZACIÓN DE DRIVERS
00122     // =====
00123
00124     mq135_init();
00125
00126     ph_init();
00127
00128     ds18b20_init(GPIO_NUM_4);
00129
00130     while (1)
00131     {
00132         // =====
00133         // ESPERAR ACTIVACIÓN
00134         // =====
00135
00136         ulTaskNotifyTake(
00137             pdTRUE,
00138             portMAX_DELAY);
00139
00140         ESP_LOGI(TAG, "Leyendo sensores...");
00141
00142         // =====
00143         // SENSOR MQ135
00144         // =====
00145
00146         float raw_voltage =
00147             mq135_read();
00148
00149         int raw_adc =
00150             mq135_read_raw();
00151
00152         ESP_LOGI(
00153             TAG,
00154             "MQ135 ADC RAW: %d",
00155             raw_adc);
00156
00161         filtered =
00162             ALPHA * raw_voltage +
00163             (1.0f - ALPHA) * filtered;
00164

```

```

00168     if (!baseline_initialized)
00169     {
00170         baseline = filtered;
00171
00172         baseline_initialized = 1;
00173
00174         ESP_LOGI(
00175             TAG,
00176             "Baseline inicial: %.4f",
00177             baseline);
00178     }
00179
00180     data.co2_raw = raw_voltage;
00181
00182     data.co2 =
00183         filtered - baseline;
00184
00185     // =====
00186     // SENSOR DE PH
00187     // =====
00188
00189     data.ph =
00190         ph_read();
00191
00192     // =====
00193     // TEMPERATURA DS18B20
00194     // =====
00195
00196     float temp =
00197         ds18b20_read_temperature();
00198
00199     if (temp == -127.0f)
00200     {
00201         ESP_LOGW(
00202             TAG,
00203             "Error leyendo DS18B20");
00204
00205         temp = 25.0f;
00206     }
00207
00208     data.temperatura = temp;
00209
00210     // =====
00211     // VARIABLES DE PRUEBA
00212     // =====
00213
00214     data.voltaje = 5.0f;
00215
00216     data.corriente = 0.5f;
00217
00218     // =====
00219     // RTC
00220     // =====
00221
00222     ds3231_get_datetime(
00223         &data.datetime);
00224
00225     printf(
00226         "RTC -> %02d:%02d:%02d\n",
00227         data.datetime.hours,
00228         data.datetime.minutes,
00229         data.datetime.seconds);
00230
00231     // =====
00232     // ENVÍO A TASK CONTROL
00233     // =====
00234
00235     xQueueSend(
00236         queue_sensores,
00237         &data,
00238         portMAX_DELAY);
00239
00240     // =====
00241     // ENVÍO A LOGGER
00242     // =====
00243
00244     xQueueSend(
00245         queue_logger,
00246         &data,
00247         portMAX_DELAY);
00248
00249     // =====
00250     // ENVÍO A MQTT
00251     // =====
00252
00253     xQueueSend(
00254         queue_mqtt,

```

```

00255         &data,
00256         portMAX_DELAY);
00257
00258         // =====
00259         // DATOS DE DISPLAY
00260         // =====
00261
00262         display_data_t display;
00263
00264         display.temperatura =
00265             data.temperatura;
00266
00267         display.ph =
00268             data.ph;
00269
00270         display.co2 =
00271             data.co2;
00272
00273         display.wifi_ok =
00274             wifi_is_connected();
00275
00276         display.mqtt_ok =
00277             mqtt_is_connected();
00278
00279         display.sd_ok =
00280             sd_card_is_mounted();
00281
00282         display.datetime =
00283             data.datetime;
00284
00285         xQueueSend(
00286             queue_display,
00287             &display,
00288             0);
00289
00290         ESP_LOGI(
00291             TAG,
00292             "Datos enviados");
00293     }
00294 }

```

6.76. task_sensores.h

[Ir a la documentación de este archivo.](#)

```

00001
00042
00043 #ifndef TASK_SENSORES_H
00044 #define TASK_SENSORES_H
00045
00062 void task_sensores(void *pvParameters);
00063
00064 #endif /* TASK_SENSORES_H */

```

6.77. Referencia del archivo main/utils/data_types.h

Definición de estructuras de datos compartidas del sistema.

```

#include <stdint.h>
#include <stdbool.h>

```

Clases

- struct [datetime_t](#)
Estructura para almacenar fecha y hora.
- struct [sensor_data_t](#)
Datos adquiridos por los sensores del sistema.
- struct [control_cmd_t](#)
Comandos generados por la lógica de control.
- struct [display_data_t](#)
Información destinada a la interfaz de usuario.

6.77.1. Descripción detallada

Definición de estructuras de datos compartidas del sistema.

Este archivo contiene las estructuras utilizadas para el intercambio de información entre las tareas de FreeRTOS mediante colas de comunicación.

Define los tipos de datos empleados para:

- Gestión de fecha y hora.
- Adquisición de sensores.
- Comandos de control.
- Información para visualización.

Estas estructuras permiten desacoplar las tareas del sistema, facilitando una arquitectura modular y escalable.

Autor

Fernando Plazas Trujillo Isabella Ordoñez Juan Daniel Constain

Definición en el archivo [data_types.h](#).

6.78. data_types.h

[Ir a la documentación de este archivo.](#)

```

00001
00023
00024 #ifndef DATA_TYPES_H
00025 #define DATA_TYPES_H
00026
00027 #include <stdint.h>
00028 #include <stdbool.h>
00029
00030 // =====
00031 // ESTRUCTURAS COMPARTIDAS ENTRE TAREAS FREERTOS
00032 // =====
00033
00040 typedef struct
00041 {
00042     uint8_t seconds;
00043     uint8_t minutes;
00044     uint8_t hours;
00045     uint8_t day;
00046     uint8_t month;
00047     uint16_t year;
00048 } datetime_t;
00049
00050
00063 typedef struct
00064 {
00065     float temperatura;
00066     float ph;
00067     float co2;
00068     float co2_raw;
00069
00070     float voltaje;
00071     float corriente;
00072
00073     datetime_t datetime;
00074
00075 } sensor_data_t;
00076
00086 typedef struct
00087 {

```

```

00088     bool  enfriar;
00089
00090     bool  bomba;
00091     bool  mezclar;
00092
00093     float servo_angle;
00094
00095 } control_cmd_t;
00096
00104 typedef struct
00105 {
00106     float temperatura;
00107     float ph;
00108     float co2;
00109
00110     bool  wifi_ok;
00111     bool  mqtt_ok;
00112     bool  sd_ok;
00113
00114     datetime_t datetime;
00115
00116 } display_data_t;
00117
00118 #endif /* DATA_TYPES_H */

```

6.79. Referencia del archivo main/utils/queues.c

Implementación de las colas de comunicación del sistema.

```
#include "queues.h"
```

Funciones

- void [queues_init](#) (void)
Inicializa todas las colas del sistema.

Variables

- QueueHandle_t [queue_sensores](#)
Cola de datos provenientes de sensores.
- QueueHandle_t [queue_control](#)
Cola de comandos de control.
- QueueHandle_t [queue_logger](#)
Cola para almacenamiento de datos.
- QueueHandle_t [queue_mqtt](#)
Cola para publicación MQTT.
- QueueHandle_t [queue_display](#)
Cola de actualización de pantalla.

6.79.1. Descripción detallada

Implementación de las colas de comunicación del sistema.

Este módulo crea e inicializa las colas FreeRTOS utilizadas para el intercambio de información entre las distintas tareas del sistema.

Las colas permiten desacoplar la adquisición de datos, la lógica de control, la visualización, el almacenamiento y la comunicación MQTT.

Arquitectura de comunicación:

task_sensores | +----> queue_sensores +----> queue_logger +----> queue_mqtt +----> queue_display

task_control | +----> queue_control

Las colas garantizan una comunicación segura entre tareas sin necesidad de compartir variables globales.

Autor

Fernando Plazas Trujillo Isabella Ordoñez Juan Daniel Constain

Definición en el archivo [queues.c](#).

6.79.2. Documentación de funciones

6.79.2.1. queues_init()

```
void queues_init (
    void )
```

Inicializa todas las colas del sistema.

Crea las colas FreeRTOS utilizadas para la comunicación entre tareas y reserva la memoria necesaria para cada una.

Tamaños configurados:

- queue_sensores : 5 elementos.
- queue_control : 5 elementos.
- queue_logger : 5 elementos.
- queue_mqtt : 10 elementos.
- queue_display : 5 elementos.

Crea las colas FreeRTOS necesarias para la comunicación entre tareas y verifica la correcta asignación de memoria.

Definición en la línea 90 del archivo [queues.c](#).

```
00091 {
00092     queue_sensores = xQueueCreate(5, sizeof(sensor_data_t));
00093
00094     queue_control  = xQueueCreate(5, sizeof(control_cmd_t));
00095
00096     queue_logger   = xQueueCreate(5, sizeof(sensor_data_t));
00097
00098     queue_mqtt     = xQueueCreate(10, sizeof(sensor_data_t));
00099
00100     queue_display  = xQueueCreate(5, sizeof(display_data_t));
00101 }
```

6.79.3. Documentación de variables

6.79.3.1. queue_control

`QueueHandle_t queue_control`

Cola de comandos de control.

Transporta estructuras `control_cmd_t` desde la tarea de control hacia la tarea de actuadores.

Utilizada para enviar estructuras `control_cmd_t` desde la tarea de control hacia la tarea de actuadores.

Definición en la línea 51 del archivo `queues.c`.

6.79.3.2. queue_display

`QueueHandle_t queue_display`

Cola de actualización de pantalla.

Transporta estructuras `display_data_t` hacia la tarea encargada de la interfaz de usuario.

Definición en la línea 75 del archivo `queues.c`.

6.79.3.3. queue_logger

`QueueHandle_t queue_logger`

Cola para almacenamiento de datos.

Cola para registro de datos.

Utilizada para transferir mediciones hacia la tarea encargada del registro en tarjeta SD.

Cola para almacenamiento de datos.

Permite transferir mediciones hacia la tarea encargada del almacenamiento en tarjeta SD.

Definición en la línea 59 del archivo `queues.c`.

6.79.3.4. queue_mqtt

`QueueHandle_t queue_mqtt`

Cola para publicación MQTT.

Permite enviar datos hacia la tarea responsable de la comunicación con el broker MQTT.

Permite enviar información desde las tareas de aplicación hacia la tarea responsable de la comunicación MQTT.

Definición en la línea 67 del archivo `queues.c`.

6.79.3.5. queue_sensores

QueueHandle_t queue_sensores

Cola de datos provenientes de sensores.

Cola de datos provenientes de los sensores.

Transporta estructuras [sensor_data_t](#) entre las tareas de adquisición y procesamiento.

Cola de datos provenientes de sensores.

Transporta estructuras [sensor_data_t](#) desde la tarea de adquisición hacia las demás tareas consumidoras.

Definición en la línea [43](#) del archivo [queues.c](#).

6.80. queues.c

[Ir a la documentación de este archivo.](#)

```
00001
00034
00035 #include "queues.h"
00036
00043 QueueHandle_t queue_sensores;
00044
00051 QueueHandle_t queue_control;
00052
00059 QueueHandle_t queue_logger;
00060
00067 QueueHandle_t queue_mqtt;
00068
00075 QueueHandle_t queue_display;
00076
00090 void queues_init(void)
00091 {
00092     queue_sensores = xQueueCreate(5, sizeof(sensor_data_t));
00093
00094     queue_control = xQueueCreate(5, sizeof(control_cmd_t));
00095
00096     queue_logger = xQueueCreate(5, sizeof(sensor_data_t));
00097
00098     queue_mqtt = xQueueCreate(10, sizeof(sensor_data_t));
00099
00100     queue_display = xQueueCreate(5, sizeof(display_data_t));
00101 }
```

6.81. Referencia del archivo main/utils/queues.h

Declaración de las colas de comunicación del sistema.

```
#include "freertos/FreeRTOS.h"
#include "freertos/queue.h"
#include "utils/data_types.h"
```

Funciones

- void [queues_init](#) (void)

Inicializa todas las colas del sistema.

Variables

- QueueHandle_t [queue_sensores](#)
Cola de datos provenientes de los sensores.
- QueueHandle_t [queue_control](#)
Cola de comandos de control.
- QueueHandle_t [queue_logger](#)
Cola para registro de datos.
- QueueHandle_t [queue_mqtt](#)
Cola para publicación MQTT.
- QueueHandle_t [queue_display](#)
Cola de actualización de pantalla.

6.81.1. Descripción detallada

Declaración de las colas de comunicación del sistema.

Este módulo define las colas FreeRTOS utilizadas para el intercambio de información entre las diferentes tareas del sistema embebido.

Las colas permiten una comunicación segura y desacoplada entre productores y consumidores, evitando el uso de variables globales compartidas.

Flujo principal:

task_sensores | v queue_sensores | v task_control | v queue_control | v task_actuadores

Además, los datos pueden ser distribuidos hacia:

- queue_logger
- queue_display
- queue_mqtt

Autor

Fernando Plazas Trujillo Isabella Ordoñez Juan Daniel Constain

Definición en el archivo [queues.h](#).

6.81.2. Documentación de funciones

6.81.2.1. queues_init()

```
void queues_init (
    void )
```

Inicializa todas las colas del sistema.

Crea las colas FreeRTOS necesarias para la comunicación entre tareas y verifica la correcta asignación de memoria.

Crea las colas FreeRTOS utilizadas para la comunicación entre tareas y reserva la memoria necesaria para cada una.

Tamaños configurados:

- queue_sensores : 5 elementos.
- queue_control : 5 elementos.
- queue_logger : 5 elementos.
- queue_mqtt : 10 elementos.
- queue_display : 5 elementos.

Definición en la línea 90 del archivo [queues.c](#).

```
00091 {
00092     queue_sensores = xQueueCreate(5, sizeof(sensor_data_t));
00093     queue_control  = xQueueCreate(5, sizeof(control_cmd_t));
00094     queue_logger   = xQueueCreate(5, sizeof(sensor_data_t));
00095     queue_mqtt     = xQueueCreate(10, sizeof(sensor_data_t));
00096     queue_display  = xQueueCreate(5, sizeof(display_data_t));
00097 }
00101 }
```

6.81.3. Documentación de variables

6.81.3.1. queue_control

```
QueueHandle_t queue_control [extern]
```

Cola de comandos de control.

Utilizada para enviar estructuras [control_cmd_t](#) desde la tarea de control hacia la tarea de actuadores.

Transporta estructuras [control_cmd_t](#) desde la tarea de control hacia la tarea de actuadores.

Definición en la línea 51 del archivo [queues.c](#).

6.81.3.2. queue_display

```
QueueHandle_t queue_display [extern]
```

Cola de actualización de pantalla.

Transporta estructuras [display_data_t](#) hacia la tarea encargada de la interfaz de usuario.

Definición en la línea 75 del archivo [queues.c](#).

6.81.3.3. queue_logger

```
QueueHandle_t queue_logger [extern]
```

Cola para registro de datos.

Cola para almacenamiento de datos.

Permite transferir mediciones hacia la tarea encargada del almacenamiento en tarjeta SD.

Cola para registro de datos.

Utilizada para transferir mediciones hacia la tarea encargada del registro en tarjeta SD.

Definición en la línea 59 del archivo [queues.c](#).

6.81.3.4. queue_mqtt

```
QueueHandle_t queue_mqtt [extern]
```

Cola para publicación MQTT.

Permite enviar información desde las tareas de aplicación hacia la tarea responsable de la comunicación MQTT.

Permite enviar datos hacia la tarea responsable de la comunicación con el broker MQTT.

Definición en la línea 67 del archivo [queues.c](#).

6.81.3.5. queue_sensores

```
QueueHandle_t queue_sensores [extern]
```

Cola de datos provenientes de los sensores.

Cola de datos provenientes de sensores.

Transporta estructuras [sensor_data_t](#) desde la tarea de adquisición hacia las demás tareas consumidoras.

Cola de datos provenientes de los sensores.

Transporta estructuras [sensor_data_t](#) entre las tareas de adquisición y procesamiento.

Definición en la línea 43 del archivo [queues.c](#).

6.82. queues.h

[Ir a la documentación de este archivo.](#)

```

00001
00039
00040 #ifndef QUEUES_H
00041 #define QUEUES_H
00042
00043 #include "freertos/FreeRTOS.h"
00044 #include "freertos/queue.h"
00045
00046 #include "utils/data_types.h"
00047
00054 extern QueueHandle_t queue_sensores;
00055
00062 extern QueueHandle_t queue_control;
00063
00070 extern QueueHandle_t queue_logger;
00071
00078 extern QueueHandle_t queue_mqtt;
00079
00086 extern QueueHandle_t queue_display;
00087
00094 void queues_init(void);
00095
00096 #endif /* QUEUES_H */

```

6.83. Referencia del archivo main/wifi/wifi_manager.c

Implementación del gestor de conectividad WiFi.

```

#include "wifi_manager.h"
#include <string.h>
#include "freertos/FreeRTOS.h"
#include "freertos/event_groups.h"
#include "freertos/task.h"
#include "esp_event.h"
#include "esp_log.h"
#include "esp_netif.h"
#include "esp_wifi.h"
#include "nvs_flash.h"

```

defines

- #define [WIFI_SSID](#) "Sala331"
- #define [WIFI_PASS](#) "Sala331tm"
- #define [WIFI_MAX_RETRIES](#) 4
Número máximo de reintentos de conexión WiFi.
- #define [WIFI_SCAN_MAX_AP](#) 20
Número máximo de puntos de acceso almacenados durante un escaneo.
- #define [WIFI_CONNECTED_BIT](#) BIT0
Bit que indica conexión WiFi exitosa.
- #define [WIFI_FAIL_BIT](#) BIT1

Funciones

- static void `wifi_event_handler` (void *arg, esp_event_base_t event_base, int32_t event_id, void *event_data)
Manejador de eventos WiFi e IP.
- void `wifi_init` (void)
Inicializa y conecta el ESP32 a una red WiFi.
- bool `wifi_wait_connected` (uint32_t timeout_ms)
Espera hasta que el ESP32 se conecte a la red WiFi.
- static const char * `auth_mode_name` (wifi_auth_mode_t authmode)
Convierte un modo de autenticación WiFi a texto.
- void `wifi_scan_print` (void)
Realiza un escaneo de redes WiFi disponibles.
- bool `wifi_is_connected` (void)
Verifica si existe una conexión WiFi activa.

Variables

- static const char * `TAG` = "wifi_manager"
Etiqueta utilizada para mensajes de depuración.
- static int `retry_count` = 0
Contador de reintentos de conexión WiFi.
- static EventGroupHandle_t `wifi_event_group`
Event group para manejar estados del WiFi.

6.83.1. Descripción detallada

Implementación del gestor de conectividad WiFi.

Este módulo implementa la inicialización, conexión y monitoreo de la interfaz WiFi del ESP32 utilizando el framework ESP-IDF.

Funcionalidades principales:

- Inicialización del subsistema WiFi.
- Gestión automática de conexión y reconexión.
- Obtención de dirección IP mediante DHCP.
- Escaneo de redes inalámbricas disponibles.
- Consulta del estado de conexión.
- Sincronización mediante Event Groups de FreeRTOS.

El módulo abstrae la complejidad de la pila de red y proporciona una interfaz sencilla para las tareas de aplicación.

Mecanismos FreeRTOS utilizados:

- Event Groups para sincronización de estados de conexión.

Eventos gestionados:

- Inicio del modo estación.
- Conexión y desconexión WiFi.
- Obtención de dirección IP.
- Reintentos automáticos de conexión.

Autor

Fernando Plazas Trujillo Isabella Ordoñez Juan Daniel Constain

Definición en el archivo [wifi_manager.c](#).

6.83.2. Documentación de «define»**6.83.2.1. WIFI_CONNECTED_BIT**

```
#define WIFI_CONNECTED_BIT BIT0
```

Bit que indica conexión WiFi exitosa.

Definición en la línea [83](#) del archivo [wifi_manager.c](#).

6.83.2.2. WIFI_FAIL_BIT

```
#define WIFI_FAIL_BIT BIT1
```

Definición en la línea [84](#) del archivo [wifi_manager.c](#).

6.83.2.3. WIFI_MAX_RETRIES

```
#define WIFI_MAX_RETRIES 4
```

Número máximo de reintentos de conexión WiFi.

Definición en la línea [58](#) del archivo [wifi_manager.c](#).

6.83.2.4. WIFI_PASS

```
#define WIFI_PASS "Sala331tm"
```

Definición en la línea [49](#) del archivo [wifi_manager.c](#).

6.83.2.5. WIFI_SCAN_MAX_AP

```
#define WIFI_SCAN_MAX_AP 20
```

Número máximo de puntos de acceso almacenados durante un escaneo.

Definición en la línea [63](#) del archivo [wifi_manager.c](#).

6.83.2.6. WIFI_SSID

```
#define WIFI_SSID "Sala331"
```

Definición en la línea [48](#) del archivo [wifi_manager.c](#).

6.83.3. Documentación de funciones

6.83.3.1. auth_mode_name()

```
const char * auth_mode_name (
    wifi_auth_mode_t authmode) [static]
```

Convierte un modo de autenticación WiFi a texto.

Se utiliza para mostrar información legible durante el escaneo de redes disponibles.

Parámetros

<i>authmode</i>	Modo de autenticación reportado por ESP-IDF.
-----------------	--

Devuelve

Cadena descriptiva del tipo de autenticación.

Definición en la línea 334 del archivo [wifi_manager.c](#).

```
00335 {
00336     switch (authmode)
00337     {
00338         case WIFI_AUTH_OPEN:
00339             return "OPEN";
00340         case WIFI_AUTH_WEP:
00341             return "WEP";
00342         case WIFI_AUTH_WPA_PSK:
00343             return "WPA";
00344         case WIFI_AUTH_WPA2_PSK:
00345             return "WPA2";
00346         case WIFI_AUTH_WPA_WPA2_PSK:
00347             return "WPA/WPA2";
00348         case WIFI_AUTH_WPA2_ENTERPRISE:
00349             return "WPA2_ENT";
00350         case WIFI_AUTH_WPA3_PSK:
00351             return "WPA3";
00352         case WIFI_AUTH_WPA2_WPA3_PSK:
00353             return "WPA2/WPA3";
00354         default:
00355             return "UNKNOWN";
00356     }
00357 }
```

6.83.3.2. wifi_event_handler()

```
void wifi_event_handler (
    void * arg,
    esp_event_base_t event_base,
    int32_t event_id,
    void * event_data) [static]
```

Manejador de eventos WiFi e IP.

Esta función es registrada en el sistema de eventos del ESP-IDF y es ejecutada automáticamente cuando ocurre un evento relacionado con la conectividad de red.

Eventos gestionados:

- Inicio de la interfaz WiFi.

- Desconexión de la red.
- Obtención de dirección IP.

Además actualiza los Event Groups utilizados por las tareas para sincronizar el estado de conexión.

Parámetros

<i>arg</i>	Argumento de usuario.
<i>event_base</i>	Base del evento recibido.
<i>event_id</i>	Identificador del evento.
<i>event_data</i>	Datos asociados al evento.

WiFi iniciado.

WiFi desconectado.

IP obtenida correctamente.

Definición en la línea 106 del archivo [wifi_manager.c](#).

```

00111 {
00115     if (event_base == WIFI_EVENT &&
00116         event_id == WIFI_EVENT_STA_START)
00117     {
00118         ESP_LOGI(TAG, "Conectando al WiFi...");
00119         retry_count = 0;
00120         xEventGroupClearBits(
00121             wifi_event_group,
00122             WIFI_CONNECTED_BIT | WIFI_FAIL_BIT);
00123         esp_wifi_connect();
00124     }
00125
00129     else if (event_base == WIFI_EVENT &&
00130              event_id == WIFI_EVENT_STA_DISCONNECTED)
00131     {
00132         wifi_event_sta_disconnected_t *event =
00133             (wifi_event_sta_disconnected_t *) event_data;
00134
00135         xEventGroupClearBits(
00136             wifi_event_group,
00137             WIFI_CONNECTED_BIT);
00138
00139         if (retry_count < WIFI_MAX_RETRIES)
00140         {
00141             retry_count++;
00142             ESP_LOGW(TAG,
00143                 "WiFi desconectado. Razon=%d. Reintento %d/ %d...",
00144                 event->reason,
00145                 retry_count,
00146                 WIFI_MAX_RETRIES);
00147             esp_wifi_connect();
00148         }
00149         else
00150         {
00151             ESP_LOGW(TAG,
00152                 "WiFi desconectado. Razon=%d. Sin mas reintentos.",
00153                 event->reason);
00154             xEventGroupSetBits(
00155                 wifi_event_group,
00156                 WIFI_FAIL_BIT);
00157         }
00158     }
00159
00163     else if (event_base == IP_EVENT &&
00164              event_id == IP_EVENT_STA_GOT_IP)
00165     {
00166         ip_event_got_ip_t *event =
00167             (ip_event_got_ip_t *) event_data;
00168
00169         ESP_LOGI(TAG,
00170             "WiFi conectado");
00171
00172         ESP_LOGI(TAG,

```



```
00173         "IP obtenida: " IPSTR,
00174         IP2STR(&event->ip_info.ip));
00175
00176     xEventGroupSetBits(
00177         wifi_event_group,
00178         WIFI_CONNECTED_BIT);
00179     xEventGroupClearBits(
00180         wifi_event_group,
00181         WIFI_FAIL_BIT);
00182     retry_count = 0;
00183 }
00184 }
```

6.83.3.3. wifi_init()

```
void wifi_init (
    void )
```

Inicializa y conecta el ESP32 a una red WiFi.

Inicializa la interfaz WiFi y establece la conexión.

Realiza la inicialización de:

- NVS Flash.
- Pila TCP/IP.
- Event Loop del sistema.
- Interfaz WiFi en modo estación.

Posteriormente configura las credenciales de acceso, registra los manejadores de eventos y comienza el proceso de conexión a la red inalámbrica.

Inicializa y conecta el ESP32 a una red WiFi.

Configura el subsistema WiFi del ESP32, registra los eventos necesarios y comienza el proceso de conexión a la red previamente configurada. Crear event group.

Inicializar NVS.

Inicializar TCP/IP stack.

Crear event loop.

Crear interfaz WiFi station.

Configuración WiFi por defecto.

Registrar handlers de eventos.

Configuración de credenciales WiFi.

Configurar modo station.

Aplicar configuración WiFi.

Iniciar WiFi.

Definición en la línea 199 del archivo [wifi_manager.c](#).

```

00200 {
00204     wifi_event_group = xEventGroupCreate();
00205
00209     esp_err_t ret = nvs_flash_init();
00210
00211     if (ret == ESP_ERR_NVS_NO_FREE_PAGES ||
00212         ret == ESP_ERR_NVS_NEW_VERSION_FOUND)
00213     {
00214         ESP_ERROR_CHECK(nvs_flash_erase());
00215         ret = nvs_flash_init();
00216     }
00217
00218     ESP_ERROR_CHECK(ret);
00219
00220
00224     ESP_ERROR_CHECK(esp_netif_init());
00225
00229     ESP_ERROR_CHECK(
00230         esp_event_loop_create_default());
00231
00235     esp_netif_create_default_wifi_sta();
00236
00240     wifi_init_config_t cfg =
00241         WIFI_INIT_CONFIG_DEFAULT();
00242
00243     ESP_ERROR_CHECK(esp_wifi_init(&cfg));
00244
00248     ESP_ERROR_CHECK(
00249         esp_event_handler_instance_register(
00250             WIFI_EVENT,
00251             ESP_EVENT_ANY_ID,
00252             &wifi_event_handler,
00253             NULL,
00254             NULL));
00255
00256     ESP_ERROR_CHECK(
00257         esp_event_handler_instance_register(
00258             IP_EVENT,
00259             IP_EVENT_STA_GOT_IP,
00260             &wifi_event_handler,
00261             NULL,
00262             NULL));
00263
00267     wifi_config_t wifi_config = {
00268         .sta = {
00269             .ssid = WIFI_SSID,
00270             .password = WIFI_PASS,
00271         },
00272     };
00273
00277     ESP_ERROR_CHECK(
00278         esp_wifi_set_mode(WIFI_MODE_STA));
00279
00283     ESP_ERROR_CHECK(
00284         esp_wifi_set_config(
00285             WIFI_IF_STA,
00286             &wifi_config));
00287
00291     ESP_ERROR_CHECK(esp_wifi_start());
00292
00293     ESP_LOGI(TAG,
00294         "Iniciación WiFi completada");
00295 }

```

6.83.3.4. wifi_is_connected()

```

bool wifi_is_connected (
    void )

```

Verifica si existe una conexión WiFi activa.

Consulta el estado actual de la conexión WiFi.

Consulta el estado actual almacenado en el Event Group del módulo WiFi.

Devuelve

true si el ESP32 está conectado a una red.
false si no existe conexión activa.

Verifica si existe una conexión WiFi activa.

Devuelve

true si el ESP32 está conectado a una red WiFi.
false si no existe conexión activa.

Definición en la línea 428 del archivo [wifi_manager.c](#).

```
00429 {  
00430     if (wifi_event_group == NULL)  
00431     {  
00432         return false;  
00433     }  
00434  
00435     EventBits_t bits = xEventGroupGetBits(wifi_event_group);  
00436  
00437     return (bits & WIFI_CONNECTED_BIT) != 0;  
00438 }
```

6.83.3.5. wifi_scan_print()

```
void wifi_scan_print (  
    void )
```

Realiza un escaneo de redes WiFi disponibles.

Escanea las redes WiFi disponibles.

Ejecuta un escaneo activo del entorno inalámbrico, obtiene la lista de puntos de acceso detectados y muestra por consola información relevante:

- SSID
- Intensidad de señal (RSSI)
- Canal
- Tipo de autenticación

Los resultados son utilizados principalmente para diagnóstico y configuración del sistema.

Realiza un escaneo de redes WiFi disponibles.

Realiza un escaneo activo del entorno inalámbrico e imprime los resultados mediante la consola de depuración.

Definición en la línea 374 del archivo [wifi_manager.c](#).

```
00375 {  
00376     wifi_ap_record_t ap_records[WIFI_SCAN_MAX_AP];  
00377     uint16_t ap_count = WIFI_SCAN_MAX_AP;  
00378  
00379     wifi_scan_config_t scan_config = {  
00380         .ssid = NULL,  
00381         .bssid = NULL,  
00382         .channel = 0,  
00383         .show_hidden = true  
00384     };  
00385 }
```

```

00386     ESP_LOGI(TAG, "Escaneando redes WiFi visibles...");
00387
00388     esp_wifi_disconnect();
00389     vTaskDelay(pdMS_TO_TICKS(500));
00390
00391     esp_err_t ret = esp_wifi_scan_start(&scan_config, true);
00392     if (ret != ESP_OK)
00393     {
00394         ESP_LOGE(TAG, "Error iniciando escaneo WiFi:%s", esp_err_to_name(ret));
00395         return;
00396     }
00397
00398     ret = esp_wifi_scan_get_ap_records(&ap_count, ap_records);
00399     if (ret != ESP_OK)
00400     {
00401         ESP_LOGE(TAG, "Error leyendo escaneo WiFi:%s", esp_err_to_name(ret));
00402         return;
00403     }
00404
00405     ESP_LOGI(TAG, "Redes encontradas:%d", ap_count);
00406
00407     for (int i = 0; i < ap_count; i++)
00408     {
00409         ESP_LOGI(TAG,
00410             "%02d SSID='%s' RSSI=%d canal=%d auth=%s",
00411             i + 1,
00412             (char *) ap_records[i].ssid,
00413             ap_records[i].rssi,
00414             ap_records[i].primary,
00415             auth_mode_name(ap_records[i].authmode));
00416     }
00417 }

```

6.83.3.6. wifi_wait_connected()

```

bool wifi_wait_connected (
    uint32_t timeout_ms)

```

Espera hasta que el ESP32 se conecte a la red WiFi.

Espera hasta que se establezca una conexión WiFi.

La función bloquea la ejecución hasta recibir un evento de conexión exitosa o hasta que se detecte un fallo de conexión.

Parámetros

<i>timeout_ms</i>	Tiempo máximo de espera en milisegundos.
-------------------	--

Devuelve

true si la conexión fue exitosa.

false si ocurrió timeout o fallo de conexión.

Espera hasta que el ESP32 se conecte a la red WiFi.

La función bloquea la ejecución hasta que el dispositivo obtenga una dirección IP válida o hasta que expire el tiempo máximo de espera especificado.

Parámetros

<i>timeout_ms</i>	Tiempo máximo de espera en milisegundos.
-------------------	--

Devuelve

true si la conexión fue exitosa.

false si ocurrió timeout o error.

Definición en la línea 307 del archivo [wifi_manager.c](#).

```
00308 {  
00309     if (wifi_event_group == NULL)  
00310     {  
00311         return false;  
00312     }  
00313  
00314     EventBits_t bits = xEventGroupWaitBits(  
00315         wifi_event_group,  
00316         WIFI_CONNECTED_BIT | WIFI_FAIL_BIT,  
00317         pdFALSE,  
00318         pdFALSE,  
00319         pdMS_TO_TICKS(timeout_ms));  
00320  
00321     return (bits & WIFI_CONNECTED_BIT) != 0;  
00322 }
```

6.83.4. Documentación de variables

6.83.4.1. retry_count

```
int retry_count = 0 [static]
```

Contador de reintentos de conexión WiFi.

Definición en la línea 73 del archivo [wifi_manager.c](#).

6.83.4.2. TAG

```
const char* TAG = "wifi_manager" [static]
```

Etiqueta utilizada para mensajes de depuración.

Definición en la línea 68 del archivo [wifi_manager.c](#).

6.83.4.3. wifi_event_group

```
EventGroupHandle_t wifi_event_group [static]
```

Event group para manejar estados del WiFi.

Definición en la línea 78 del archivo [wifi_manager.c](#).

6.84. wifi_manager.c

[Ir a la documentación de este archivo.](#)

```

00001
00033 #include "wifi_manager.h"
00034
00035 #include <string.h>
00036
00037 #include "freertos/FreeRTOS.h"
00038 #include "freertos/event_groups.h"
00039 #include "freertos/task.h"
00040
00041 #include "esp_event.h"
00042 #include "esp_log.h"
00043 #include "esp_netif.h"
00044 #include "esp_wifi.h"
00045
00046 #include "nvs_flash.h"
00047
00048 #define WIFI_SSID      "Sala331"
00049 #define WIFI_PASS      "Sala331tm"
00050 // #define WIFI_SSID    "TP-LINK_DDAE"
00051 // #define WIFI_PASS    "24890717"
00052 // #define WIFI_SSID    "S25 Ultra de fernando"
00053 // #define WIFI_PASS    "fer12345"
00054
00058 #define WIFI_MAX_RETRIES 4
00059
00063 #define WIFI_SCAN_MAX_AP 20
00064
00068 static const char *TAG = "wifi_manager";
00069
00073 static int retry_count = 0;
00074
00078 static EventGroupHandle_t wifi_event_group;
00079
00083 #define WIFI_CONNECTED_BIT BIT0
00084 #define WIFI_FAIL_BIT      BIT1
00085
00106 static void wifi_event_handler(
00107     void *arg,
00108     esp_event_base_t event_base,
00109     int32_t event_id,
00110     void *event_data)
00111 {
00115     if (event_base == WIFI_EVENT &&
00116         event_id == WIFI_EVENT_STA_START)
00117     {
00118         ESP_LOGI(TAG, "Conectando al WiFi...");
00119         retry_count = 0;
00120         xEventGroupClearBits(
00121             wifi_event_group,
00122             WIFI_CONNECTED_BIT | WIFI_FAIL_BIT);
00123         esp_wifi_connect();
00124     }
00125
00129     else if (event_base == WIFI_EVENT &&
00130         event_id == WIFI_EVENT_STA_DISCONNECTED)
00131     {
00132         wifi_event_sta_disconnected_t *event =
00133             (wifi_event_sta_disconnected_t *) event_data;
00134
00135         xEventGroupClearBits(
00136             wifi_event_group,
00137             WIFI_CONNECTED_BIT);
00138
00139         if (retry_count < WIFI_MAX_RETRIES)
00140         {
00141             retry_count++;
00142             ESP_LOGW(TAG,
00143                 "WiFi desconectado. Razon=%d. Reintento %d/ %d...",
00144                 event->reason,
00145                 retry_count,
00146                 WIFI_MAX_RETRIES);
00147             esp_wifi_connect();
00148         }
00149         else
00150         {
00151             ESP_LOGW(TAG,
00152                 "WiFi desconectado. Razon=%d. Sin mas reintentos.",
00153                 event->reason);
00154             xEventGroupSetBits(
00155                 wifi_event_group,
00156                 WIFI_FAIL_BIT);
00157         }

```

```

00158     }
00159
00163     else if (event_base == IP_EVENT &&
00164             event_id == IP_EVENT_STA_GOT_IP)
00165     {
00166         ip_event_got_ip_t *event =
00167             (ip_event_got_ip_t *) event_data;
00168
00169         ESP_LOGI(TAG,
00170                 "WiFi conectado");
00171
00172         ESP_LOGI(TAG,
00173                 "IP obtenida: " IPSTR,
00174                 IP2STR(&event->ip_info.ip));
00175
00176         xEventGroupSetBits(
00177             wifi_event_group,
00178             WIFI_CONNECTED_BIT);
00179         xEventGroupClearBits(
00180             wifi_event_group,
00181             WIFI_FAIL_BIT);
00182         retry_count = 0;
00183     }
00184 }
00185
00199 void wifi_init(void)
00200 {
00204     wifi_event_group = xEventGroupCreate();
00205
00209     esp_err_t ret = nvs_flash_init();
00210
00211     if (ret == ESP_ERR_NVS_NO_FREE_PAGES ||
00212         ret == ESP_ERR_NVS_NEW_VERSION_FOUND)
00213     {
00214         ESP_ERROR_CHECK(nvs_flash_erase());
00215
00216         ret = nvs_flash_init();
00217     }
00218
00219     ESP_ERROR_CHECK(ret);
00220
00224     ESP_ERROR_CHECK(esp_netif_init());
00225
00229     ESP_ERROR_CHECK(
00230         esp_event_loop_create_default());
00231
00235     esp_netif_create_default_wifi_sta();
00236
00240     wifi_init_config_t cfg =
00241         WIFI_INIT_CONFIG_DEFAULT();
00242
00243     ESP_ERROR_CHECK(esp_wifi_init(&cfg));
00244
00248     ESP_ERROR_CHECK(
00249         esp_event_handler_instance_register(
00250             WIFI_EVENT,
00251             ESP_EVENT_ANY_ID,
00252             &wifi_event_handler,
00253             NULL,
00254             NULL));
00255
00256     ESP_ERROR_CHECK(
00257         esp_event_handler_instance_register(
00258             IP_EVENT,
00259             IP_EVENT_STA_GOT_IP,
00260             &wifi_event_handler,
00261             NULL,
00262             NULL));
00263
00267     wifi_config_t wifi_config = {
00268         .sta = {
00269             .ssid = WIFI_SSID,
00270             .password = WIFI_PASS,
00271         },
00272     };
00273
00277     ESP_ERROR_CHECK(
00278         esp_wifi_set_mode(WIFI_MODE_STA));
00279
00283     ESP_ERROR_CHECK(
00284         esp_wifi_set_config(
00285             WIFI_IF_STA,
00286             &wifi_config));
00287
00291     ESP_ERROR_CHECK(esp_wifi_start());
00292
00293     ESP_LOGI(TAG,

```

```

00294         "Inicialización WiFi completada");
00295     }
00307 bool wifi_wait_connected(uint32_t timeout_ms)
00308 {
00309     if (wifi_event_group == NULL)
00310     {
00311         return false;
00312     }
00313
00314     EventBits_t bits = xEventGroupWaitBits(
00315         wifi_event_group,
00316         WIFI_CONNECTED_BIT | WIFI_FAIL_BIT,
00317         pdFALSE,
00318         pdFALSE,
00319         pdMS_TO_TICKS(timeout_ms));
00320
00321     return (bits & WIFI_CONNECTED_BIT) != 0;
00322 }
00323
00334 static const char *auth_mode_name(wifi_auth_mode_t authmode)
00335 {
00336     switch (authmode)
00337     {
00338         case WIFI_AUTH_OPEN:
00339             return "OPEN";
00340         case WIFI_AUTH_WEP:
00341             return "WEP";
00342         case WIFI_AUTH_WPA_PSK:
00343             return "WPA";
00344         case WIFI_AUTH_WPA2_PSK:
00345             return "WPA2";
00346         case WIFI_AUTH_WPA_WPA2_PSK:
00347             return "WPA/WPA2";
00348         case WIFI_AUTH_WPA2_ENTERPRISE:
00349             return "WPA2_ENT";
00350         case WIFI_AUTH_WPA3_PSK:
00351             return "WPA3";
00352         case WIFI_AUTH_WPA2_WPA3_PSK:
00353             return "WPA2/WPA3";
00354         default:
00355             return "UNKNOWN";
00356     }
00357 }
00358
00374 void wifi_scan_print(void)
00375 {
00376     wifi_ap_record_t ap_records[WIFI_SCAN_MAX_AP];
00377     uint16_t ap_count = WIFI_SCAN_MAX_AP;
00378
00379     wifi_scan_config_t scan_config = {
00380         .ssid = NULL,
00381         .bssid = NULL,
00382         .channel = 0,
00383         .show_hidden = true
00384     };
00385
00386     ESP_LOGI(TAG, "Escaneando redes WiFi visibles...");
00387
00388     esp_wifi_disconnect();
00389     vTaskDelay(pdMS_TO_TICKS(500));
00390
00391     esp_err_t ret = esp_wifi_scan_start(&scan_config, true);
00392     if (ret != ESP_OK)
00393     {
00394         ESP_LOGE(TAG, "Error iniciando escaneo WiFi:%s", esp_err_to_name(ret));
00395         return;
00396     }
00397
00398     ret = esp_wifi_scan_get_ap_records(&ap_count, ap_records);
00399     if (ret != ESP_OK)
00400     {
00401         ESP_LOGE(TAG, "Error leyendo escaneo WiFi:%s", esp_err_to_name(ret));
00402         return;
00403     }
00404
00405     ESP_LOGI(TAG, "Redes encontradas:%d", ap_count);
00406
00407     for (int i = 0; i < ap_count; i++)
00408     {
00409         ESP_LOGI(TAG,
00410             "%02d SSID='%s' RSSI=%d canal=%d auth=%s",
00411             i + 1,
00412             (char *) ap_records[i].ssid,
00413             ap_records[i].rssi,
00414             ap_records[i].primary,
00415             auth_mode_name(ap_records[i].authmode));
00416     }

```



```
00417 }
00418
00428 bool wifi_is_connected(void)
00429 {
00430     if (wifi_event_group == NULL)
00431     {
00432         return false;
00433     }
00434
00435     EventBits_t bits = xEventGroupGetBits(wifi_event_group);
00436
00437     return (bits & WIFI_CONNECTED_BIT) != 0;
00438 }
```

6.85. Referencia del archivo main/wifi/wifi_manager.h

Interfaz para la gestión de conectividad WiFi.

```
#include <stdbool.h>
#include <stdint.h>
```

Funciones

- void `wifi_init` (void)
Inicializa la interfaz WiFi y establece la conexión.
- bool `wifi_wait_connected` (uint32_t timeout_ms)
Espera hasta que se establezca una conexión WiFi.
- void `wifi_scan_print` (void)
Escanea las redes WiFi disponibles.
- bool `wifi_is_connected` (void)
Consulta el estado actual de la conexión WiFi.

6.85.1. Descripción detallada

Interfaz para la gestión de conectividad WiFi.

Este módulo proporciona las funciones necesarias para inicializar la interfaz WiFi del ESP32, establecer conexión con una red inalámbrica y consultar el estado de conectividad.

Funcionalidades:

- Inicialización del subsistema WiFi.
- Conexión automática a una red configurada.
- Espera sincronizada de conexión.
- Escaneo de redes disponibles.
- Consulta del estado de conexión.

El módulo abstrae la complejidad de la configuración WiFi del ESP-IDF y ofrece una interfaz simplificada para las tareas de aplicación.

Autor

Fernando Plazas Trujillo Isabella Ordoñez Juan Daniel Constain

Definición en el archivo `wifi_manager.h`.

6.85.2. Documentación de funciones

6.85.2.1. `wifi_init()`

```
void wifi_init (
    void )
```

Inicializa la interfaz WiFi y establece la conexión.

Inicializa y conecta el ESP32 a una red WiFi.

Configura el subsistema WiFi del ESP32, registra los eventos necesarios y comienza el proceso de conexión a la red previamente configurada.

Inicializa la interfaz WiFi y establece la conexión.

Realiza la inicialización de:

- NVS Flash.
- Pila TCP/IP.
- Event Loop del sistema.
- Interfaz WiFi en modo estación.

Posteriormente configura las credenciales de acceso, registra los manejadores de eventos y comienza el proceso de conexión a la red inalámbrica. Crear event group.

Inicializar NVS.

Inicializar TCP/IP stack.

Crear event loop.

Crear interfaz WiFi station.

Configuración WiFi por defecto.

Registrar handlers de eventos.

Configuración de credenciales WiFi.

Configurar modo station.

Aplicar configuración WiFi.

Iniciar WiFi.

Definición en la línea [199](#) del archivo [wifi_manager.c](#).

```
00200 {
00204     wifi_event_group = xEventGroupCreate();
00205
00209     esp_err_t ret = nvs_flash_init();
00210
00211     if (ret == ESP_ERR_NVS_NO_FREE_PAGES ||
00212         ret == ESP_ERR_NVS_NEW_VERSION_FOUND)
00213     {
00214         ESP_ERROR_CHECK(nvs_flash_erase());
00215
00216         ret = nvs_flash_init();
00217     }
```

```

00218
00219     ESP_ERROR_CHECK(ret);
00220
00224     ESP_ERROR_CHECK(esp_netif_init());
00225
00229     ESP_ERROR_CHECK(
00230         esp_event_loop_create_default());
00231
00235     esp_netif_create_default_wifi_sta();
00236
00240     wifi_init_config_t cfg =
00241         WIFI_INIT_CONFIG_DEFAULT();
00242
00243     ESP_ERROR_CHECK(esp_wifi_init(&cfg));
00244
00248     ESP_ERROR_CHECK(
00249         esp_event_handler_instance_register(
00250             WIFI_EVENT,
00251             ESP_EVENT_ANY_ID,
00252             &wifi_event_handler,
00253             NULL,
00254             NULL));
00255
00256     ESP_ERROR_CHECK(
00257         esp_event_handler_instance_register(
00258             IP_EVENT,
00259             IP_EVENT_STA_GOT_IP,
00260             &wifi_event_handler,
00261             NULL,
00262             NULL));
00263
00267     wifi_config_t wifi_config = {
00268         .sta = {
00269             .ssid = WIFI_SSID,
00270             .password = WIFI_PASS,
00271         },
00272     };
00273
00277     ESP_ERROR_CHECK(
00278         esp_wifi_set_mode(WIFI_MODE_STA));
00279
00283     ESP_ERROR_CHECK(
00284         esp_wifi_set_config(
00285             WIFI_IF_STA,
00286             &wifi_config));
00287
00291     ESP_ERROR_CHECK(esp_wifi_start());
00292
00293     ESP_LOGI(TAG,
00294         "Iniciación WiFi completada");
00295 }

```

6.85.2.2. wifi_is_connected()

```

bool wifi_is_connected (
    void )

```

Consulta el estado actual de la conexión WiFi.

Verifica si existe una conexión WiFi activa.

Devuelve

true si el ESP32 está conectado a una red WiFi.

false si no existe conexión activa.

Consulta el estado actual de la conexión WiFi.

Consulta el estado actual almacenado en el Event Group del módulo WiFi.

Devuelve

true si el ESP32 está conectado a una red.

false si no existe conexión activa.

Definición en la línea 428 del archivo [wifi_manager.c](#).

```
00429 {
00430     if (wifi_event_group == NULL)
00431     {
00432         return false;
00433     }
00434     EventBits_t bits = xEventGroupGetBits(wifi_event_group);
00435     return (bits & WIFI_CONNECTED_BIT) != 0;
00436 }
00437
00438 }
```

6.85.2.3. wifi_scan_print()

```
void wifi_scan_print (
    void )
```

Escanea las redes WiFi disponibles.

Realiza un escaneo de redes WiFi disponibles.

Realiza un escaneo activo del entorno inalámbrico e imprime los resultados mediante la consola de depuración.

Escanea las redes WiFi disponibles.

Ejecuta un escaneo activo del entorno inalámbrico, obtiene la lista de puntos de acceso detectados y muestra por consola información relevante:

- SSID
- Intensidad de señal (RSSI)
- Canal
- Tipo de autenticación

Los resultados son utilizados principalmente para diagnóstico y configuración del sistema.

Definición en la línea 374 del archivo [wifi_manager.c](#).

```
00375 {
00376     wifi_ap_record_t ap_records[WIFI_SCAN_MAX_AP];
00377     uint16_t ap_count = WIFI_SCAN_MAX_AP;
00378
00379     wifi_scan_config_t scan_config = {
00380         .ssid = NULL,
00381         .bssid = NULL,
00382         .channel = 0,
00383         .show_hidden = true
00384     };
00385     ESP_LOGI(TAG, "Escaneando redes WiFi visibles...");
00386     esp_wifi_disconnect();
00387     vTaskDelay(pdMS_TO_TICKS(500));
00388     esp_err_t ret = esp_wifi_scan_start(&scan_config, true);
00389     if (ret != ESP_OK)
00390     {
00391         ESP_LOGE(TAG, "Error iniciando escaneo WiFi:%s", esp_err_to_name(ret));
00392         return;
00393     }
00394 }
00395
00396
00397 }
```

```

00398     ret = esp_wifi_scan_get_ap_records(&ap_count, ap_records);
00399     if (ret != ESP_OK)
00400     {
00401         ESP_LOGE(TAG, "Error leyendo escaneo WiFi: %s", esp_err_to_name(ret));
00402         return;
00403     }
00404
00405     ESP_LOGI(TAG, "Redes encontradas: %d", ap_count);
00406
00407     for (int i = 0; i < ap_count; i++)
00408     {
00409         ESP_LOGI(TAG,
00410             "%02d SSID='%s' RSSI=%d canal=%d auth=%s",
00411             i + 1,
00412             (char *) ap_records[i].ssid,
00413             ap_records[i].rssi,
00414             ap_records[i].primary,
00415             auth_mode_name(ap_records[i].authmode));
00416     }
00417 }

```

6.85.2.4. wifi_wait_connected()

```

bool wifi_wait_connected (
    uint32_t timeout_ms)

```

Espera hasta que se establezca una conexión WiFi.

Espera hasta que el ESP32 se conecte a la red WiFi.

La función bloquea la ejecución hasta que el dispositivo obtenga una dirección IP válida o hasta que expire el tiempo máximo de espera especificado.

Parámetros

<i>timeout_ms</i>	Tiempo máximo de espera en milisegundos.
-------------------	--

Devuelve

true si la conexión fue exitosa.
false si ocurrió timeout o error.

Espera hasta que se establezca una conexión WiFi.

La función bloquea la ejecución hasta recibir un evento de conexión exitosa o hasta que se detecte un fallo de conexión.

Parámetros

<i>timeout_ms</i>	Tiempo máximo de espera en milisegundos.
-------------------	--

Devuelve

true si la conexión fue exitosa.

false si ocurrió timeout o fallo de conexión.

Definición en la línea 307 del archivo [wifi_manager.c](#).

```
00308 {
00309     if (wifi_event_group == NULL)
00310     {
00311         return false;
00312     }
00313
00314     EventBits_t bits = xEventGroupWaitBits(
00315         wifi_event_group,
00316         WIFI_CONNECTED_BIT | WIFI_FAIL_BIT,
00317         pdFALSE,
00318         pdFALSE,
00319         pdMS_TO_TICKS(timeout_ms));
00320
00321     return (bits & WIFI_CONNECTED_BIT) != 0;
00322 }
```

6.86. wifi_manager.h

[Ir a la documentación de este archivo.](#)

```
00001
00025
00026 #ifndef WIFI_MANAGER_H
00027 #define WIFI_MANAGER_H
00028
00029 #include <stdbool.h>
00030 #include <stdint.h>
00031
00039 void wifi_init(void);
00040
00053 bool wifi_wait_connected(uint32_t timeout_ms);
00054
00061 void wifi_scan_print(void);
00062
00069 bool wifi_is_connected(void);
00070
00071 #endif /* WIFI_MANAGER_H */
```

Índice alfabético

- actuadores_init
 - task_actuadores.c, [173](#)
- adc1_cali_handle
 - adc_manager.c, [124](#)
- adc1_handle
 - adc_manager.c, [124](#)
- adc_manager.c
 - adc1_cali_handle, [124](#)
 - adc1_handle, [124](#)
 - adc_manager_init, [122](#)
 - adc_manager_read_voltage, [123](#)
 - TAG, [124](#)
- adc_manager.h
 - adc_manager_init, [127](#)
 - adc_manager_read_voltage, [128](#)
- adc_manager_init
 - adc_manager.c, [122](#)
 - adc_manager.h, [127](#)
- adc_manager_read_voltage
 - adc_manager.c, [123](#)
 - adc_manager.h, [128](#)
- ADC_OFFSET
 - voltage_sensor.c, [115](#)
- ALPHA
 - task_sensores.c, [219](#)
- app_main
 - main.c, [148](#)
- auth_mode_name
 - wifi_manager.c, [241](#)
- baseline
 - task_sensores.c, [222](#)
- baseline_initialized
 - task_sensores.c, [222](#)
- bcd_to_dec
 - rtc_ds3231.c, [71](#)
- bomba
 - control_cmd_t, [13](#)
- build_payload
 - task_mqtt.c, [208](#)
- co2
 - display_data_t, [16](#)
 - sensor_data_t, [18](#)
- co2_raw
 - sensor_data_t, [18](#)
- CO2_THRESHOLD
 - task_control.c, [180](#)
- control_cmd_t, [13](#)
 - bomba, [13](#)
 - enfriar, [13](#)
 - mezclar, [14](#)
 - servo_angle, [14](#)
- corriente
 - sensor_data_t, [18](#)
- datetime
 - display_data_t, [16](#)
 - sensor_data_t, [18](#)
- datetime_t, [14](#)
 - day, [15](#)
 - hours, [15](#)
 - minutes, [15](#)
 - month, [15](#)
 - seconds, [15](#)
 - year, [15](#)
- day
 - datetime_t, [15](#)
- dec_to_bcd
 - rtc_ds3231.c, [71](#)
- display_data_t, [16](#)
 - co2, [16](#)
 - datetime, [16](#)
 - mqtt_ok, [16](#)
 - ph, [16](#)
 - sd_ok, [17](#)
 - temperatura, [17](#)
 - wifi_ok, [17](#)
- DIVIDER_RATIO
 - voltage_sensor.c, [115](#)
- ds18b20.c
 - ds18b20_init, [22](#)
 - ds18b20_read_temperature, [23](#)
 - ds_pin, [27](#)
 - ow_low, [24](#)
 - ow_read, [24](#)
 - ow_read_bit, [24](#)
 - ow_read_byte, [25](#)
 - ow_release, [25](#)
 - ow_reset, [25](#)
 - ow_write_bit, [26](#)
 - ow_write_byte, [26](#)
 - TAG, [27](#)
- ds18b20.h
 - ds18b20_init, [30](#)
 - ds18b20_read_temperature, [31](#)
- ds18b20_init
 - ds18b20.c, [22](#)
 - ds18b20.h, [30](#)

- ds18b20_read_temperature
 - ds18b20.c, [23](#)
 - ds18b20.h, [31](#)
- DS3231_ADDR
 - rtc_ds3231.c, [70](#)
- ds3231_clear_alarm_flag
 - rtc_ds3231.c, [71](#)
 - rtc_ds3231.h, [78](#)
- DS3231_CONTROL_A1_INTERRUPT
 - rtc_ds3231.c, [70](#)
- ds3231_get_datetime
 - rtc_ds3231.c, [72](#)
 - rtc_ds3231.h, [79](#)
- ds3231_init
 - rtc_ds3231.c, [73](#)
 - rtc_ds3231.h, [80](#)
- ds3231_set_alarm_seconds
 - rtc_ds3231.c, [74](#)
 - rtc_ds3231.h, [80](#)
- ds_pin
 - ds18b20.c, [27](#)
- enfriar
 - control_cmd_t, [13](#)
- filtered
 - task_sensores.c, [222](#)
- font5x7
 - font5x7.h, [33](#)
- font5x7.h
 - font5x7, [33](#)
- framebuffer
 - oled_display.c, [51](#)
- hours
 - datetime_t, [15](#)
- i2c_manager.c
 - i2c_manager_init, [132](#)
 - i2c_manager_read, [132](#)
 - i2c_manager_write, [134](#)
 - i2c_manager_write_raw, [135](#)
 - I2C_MASTER_FREQ_HZ, [131](#)
 - I2C_MASTER_NUM, [131](#)
 - I2C_MASTER_SCL_IO, [131](#)
 - I2C_MASTER_SDA_IO, [131](#)
 - mutex_i2c, [136](#)
 - TAG, [131](#)
- i2c_manager.h
 - i2c_manager_init, [140](#)
 - i2c_manager_read, [141](#)
 - i2c_manager_write, [142](#)
 - i2c_manager_write_raw, [144](#)
 - mutex_i2c, [145](#)
- i2c_manager_init
 - i2c_manager.c, [132](#)
 - i2c_manager.h, [140](#)
- i2c_manager_read
 - i2c_manager.c, [132](#)
- i2c_manager.h, [141](#)
- i2c_manager_write
 - i2c_manager.c, [134](#)
- i2c_manager.h, [142](#)
- i2c_manager_write_raw
 - i2c_manager.c, [135](#)
- i2c_manager.h, [144](#)
- I2C_MASTER_FREQ_HZ
 - i2c_manager.c, [131](#)
- I2C_MASTER_NUM
 - i2c_manager.c, [131](#)
- I2C_MASTER_SCL_IO
 - i2c_manager.c, [131](#)
- I2C_MASTER_SDA_IO
 - i2c_manager.c, [131](#)
- main.c
 - app_main, [148](#)
 - TAG, [151](#)
 - task_energia_handle, [151](#)
 - task_sensores_handle, [151](#)
 - WIFI_CONNECT_TIMEOUT_MS, [148](#)
- main/drivers/ds18b20.c, [21](#), [27](#)
- main/drivers/ds18b20.h, [29](#), [32](#)
- main/drivers/font5x7.h, [32](#), [34](#)
- main/drivers/mq135.c, [35](#), [39](#)
- main/drivers/mq135.h, [39](#), [42](#)
- main/drivers/oled_display.c, [43](#), [51](#)
- main/drivers/oled_display.h, [54](#), [59](#)
- main/drivers/ph_sensor.c, [59](#), [64](#)
- main/drivers/ph_sensor.h, [65](#), [68](#)
- main/drivers/rtc_ds3231.c, [69](#), [76](#)
- main/drivers/rtc_ds3231.h, [77](#), [82](#)
- main/drivers/sd_card.c, [82](#), [92](#)
- main/drivers/sd_card.h, [95](#), [102](#)
- main/drivers/servo.c, [103](#), [109](#)
- main/drivers/servo.h, [110](#), [113](#)
- main/drivers/voltage_sensor.c, [114](#), [117](#)
- main/drivers/voltage_sensor.h, [118](#), [120](#)
- main/hal/adc_manager.c, [120](#), [125](#)
- main/hal/adc_manager.h, [126](#), [129](#)
- main/hal/i2c_manager.c, [129](#), [137](#)
- main/hal/i2c_manager.h, [139](#), [146](#)
- main/main.c, [146](#), [152](#)
- main/mqtt/mqtt_manager.c, [154](#), [163](#)
- main/mqtt/mqtt_manager.h, [165](#), [170](#)
- main/tasks/task_actuadores.c, [171](#), [175](#)
- main/tasks/task_actuadores.h, [176](#), [178](#)
- main/tasks/task_control.c, [179](#), [182](#)
- main/tasks/task_control.h, [183](#), [185](#)
- main/tasks/task_display.c, [185](#), [189](#)
- main/tasks/task_display.h, [190](#), [193](#)
- main/tasks/task_energia.c, [193](#), [196](#)
- main/tasks/task_energia.h, [197](#), [199](#)
- main/tasks/task_logger.c, [199](#), [202](#)
- main/tasks/task_logger.h, [203](#), [205](#)
- main/tasks/task_mqtt.c, [205](#), [212](#)
- main/tasks/task_mqtt.h, [214](#), [217](#)
- main/tasks/task_sensores.c, [217](#), [223](#)

- main/tasks/task_sensores.h, 225, 229
- main/utils/data_types.h, 229, 230
- main/utils/queues.c, 231, 234
- main/utils/queues.h, 234, 238
- main/wifi/wifi_manager.c, 238, 248
- main/wifi/wifi_manager.h, 251, 256
- mezclar
 - control_cmd_t, 14
- minutes
 - datetime_t, 15
- MIX_DURATION_MS
 - task_actuadores.c, 172
- MIX_INTERVAL_CYCLES
 - task_control.c, 180
- month
 - datetime_t, 15
- mount_point
 - sd_card.c, 91
- mq135.c
 - MQ135_ADC_CHANNEL, 36
 - mq135_init, 37
 - mq135_read, 37
 - mq135_read_raw, 37
 - mq135_read_voltage, 38
 - NUM_SAMPLES, 36
 - TAG, 36
- mq135.h
 - mq135_init, 40
 - mq135_read, 40
 - mq135_read_raw, 41
 - mq135_read_voltage, 41
- MQ135_ADC_CHANNEL
 - mq135.c, 36
- mq135_init
 - mq135.c, 37
 - mq135.h, 40
- mq135_read
 - mq135.c, 37
 - mq135.h, 40
- mq135_read_raw
 - mq135.c, 37
 - mq135.h, 41
- mq135_read_voltage
 - mq135.c, 38
 - mq135.h, 41
- MQTT_BROKER_URI
 - mqtt_manager.c, 156
- mqtt_client
 - mqtt_manager.c, 162
- MQTT_CONNECT_TIMEOUT_MS
 - task_mqtt.c, 207
- mqtt_connected
 - mqtt_manager.c, 162
- MQTT_CONNECTED_BIT
 - mqtt_manager.c, 156
- MQTT_ERROR_BIT
 - mqtt_manager.c, 157
- mqtt_event_group
 - mqtt_manager.c, 163
- mqtt_event_handler
 - mqtt_manager.c, 157
- mqtt_is_connected
 - mqtt_manager.c, 158
 - mqtt_manager.h, 166
- mqtt_manager.c
 - MQTT_BROKER_URI, 156
 - mqtt_client, 162
 - mqtt_connected, 162
 - MQTT_CONNECTED_BIT, 156
 - MQTT_ERROR_BIT, 157
 - mqtt_event_group, 163
 - mqtt_event_handler, 157
 - mqtt_is_connected, 158
 - mqtt_manager_start, 159
 - mqtt_manager_stop, 159
 - mqtt_publish, 160
 - MQTT_PUBLISHED_BIT, 157
 - mqtt_wait_connected, 161
 - TAG, 163
- mqtt_manager.h
 - mqtt_is_connected, 166
 - mqtt_manager_start, 166
 - mqtt_manager_stop, 167
 - mqtt_publish, 168
 - mqtt_wait_connected, 169
- mqtt_manager_start
 - mqtt_manager.c, 159
 - mqtt_manager.h, 166
- mqtt_manager_stop
 - mqtt_manager.c, 159
 - mqtt_manager.h, 167
- mqtt_ok
 - display_data_t, 16
- MQTT_PENDING_PAYLOAD_SIZE
 - task_mqtt.c, 207
- mqtt_publish
 - mqtt_manager.c, 160
 - mqtt_manager.h, 168
- MQTT_PUBLISH_TIMEOUT_MS
 - task_mqtt.c, 207
- MQTT_PUBLISHED_BIT
 - mqtt_manager.c, 157
- MQTT_TOPIC
 - task_mqtt.c, 207
- mqtt_wait_connected
 - mqtt_manager.c, 161
 - mqtt_manager.h, 169
- mutex_i2c
 - i2c_manager.c, 136
 - i2c_manager.h, 145
 - oled_display.c, 51
 - rtc_ds3231.c, 75
- NUM_SAMPLES
 - mq135.c, 36
 - ph_sensor.c, 61
 - voltage_sensor.c, 115

- OLED_ADDR
 - oled_display.c, [44](#)
- oled_clear
 - oled_display.c, [45](#)
 - oled_display.h, [55](#)
- oled_display.c
 - framebuffer, [51](#)
 - mutex_i2c, [51](#)
 - OLED_ADDR, [44](#)
 - oled_clear, [45](#)
 - oled_draw_char, [45](#)
 - oled_draw_pixel, [46](#)
 - oled_draw_string, [47](#)
 - OLED_HEIGHT, [44](#)
 - oled_init, [47](#)
 - OLED_PAGES, [44](#)
 - oled_send_command, [48](#)
 - oled_send_data, [49](#)
 - oled_update, [50](#)
 - OLED_WIDTH, [45](#)
- oled_display.h
 - oled_clear, [55](#)
 - oled_draw_char, [55](#)
 - oled_draw_pixel, [56](#)
 - oled_draw_string, [56](#)
 - oled_init, [57](#)
 - oled_update, [58](#)
- oled_draw_char
 - oled_display.c, [45](#)
 - oled_display.h, [55](#)
- oled_draw_pixel
 - oled_display.c, [46](#)
 - oled_display.h, [56](#)
- oled_draw_string
 - oled_display.c, [47](#)
 - oled_display.h, [56](#)
- OLED_HEIGHT
 - oled_display.c, [44](#)
- oled_init
 - oled_display.c, [47](#)
 - oled_display.h, [57](#)
- OLED_PAGES
 - oled_display.c, [44](#)
- oled_send_command
 - oled_display.c, [48](#)
- oled_send_data
 - oled_display.c, [49](#)
- oled_update
 - oled_display.c, [50](#)
 - oled_display.h, [58](#)
- OLED_WIDTH
 - oled_display.c, [45](#)
- ow_low
 - ds18b20.c, [24](#)
- ow_read
 - ds18b20.c, [24](#)
- ow_read_bit
 - ds18b20.c, [24](#)
- ow_read_byte
 - ds18b20.c, [25](#)
- ow_release
 - ds18b20.c, [25](#)
- ow_reset
 - ds18b20.c, [25](#)
- ow_write_bit
 - ds18b20.c, [26](#)
- ow_write_byte
 - ds18b20.c, [26](#)
- pending_mqtt_path
 - sd_card.c, [91](#)
- ph
 - display_data_t, [16](#)
 - sensor_data_t, [18](#)
- PH_ADC_CHANNEL
 - ph_sensor.c, [61](#)
- ph_init
 - ph_sensor.c, [62](#)
 - ph_sensor.h, [66](#)
- PH_MAX_VOLTAGE
 - ph_sensor.c, [61](#)
- PH_MIN_VOLTAGE
 - ph_sensor.c, [61](#)
- PH_OFFSET
 - ph_sensor.c, [61](#)
- ph_read
 - ph_sensor.c, [62](#)
 - ph_sensor.h, [66](#)
- ph_read_voltage
 - ph_sensor.c, [63](#)
 - ph_sensor.h, [67](#)
- ph_sensor.c
 - NUM_SAMPLES, [61](#)
 - PH_ADC_CHANNEL, [61](#)
 - ph_init, [62](#)
 - PH_MAX_VOLTAGE, [61](#)
 - PH_MIN_VOLTAGE, [61](#)
 - PH_OFFSET, [61](#)
 - ph_read, [62](#)
 - ph_read_voltage, [63](#)
 - PH_SLOPE, [61](#)
 - TAG, [64](#)
- ph_sensor.h
 - ph_init, [66](#)
 - ph_read, [66](#)
 - ph_read_voltage, [67](#)
- PH_SLOPE
 - ph_sensor.c, [61](#)
- PIN_BOMBA
 - task_actuadores.c, [172](#)
- PIN_CLK
 - sd_card.c, [84](#)
- PIN_CS
 - sd_card.c, [84](#)
- PIN_MISO
 - sd_card.c, [84](#)
- PIN_MOSI

- sd_card.c, [84](#)
- PIN_MOTOR
 - task_actuadores.c, [172](#)
- PIN_SERVO
 - task_actuadores.c, [172](#)
- queue_control
 - queues.c, [233](#)
 - queues.h, [236](#)
- queue_display
 - queues.c, [233](#)
 - queues.h, [236](#)
- queue_logger
 - queues.c, [233](#)
 - queues.h, [237](#)
- queue_mqtt
 - queues.c, [233](#)
 - queues.h, [237](#)
- queue_sensores
 - queues.c, [233](#)
 - queues.h, [237](#)
- queues.c
 - queue_control, [233](#)
 - queue_display, [233](#)
 - queue_logger, [233](#)
 - queue_mqtt, [233](#)
 - queue_sensores, [233](#)
 - queues_init, [232](#)
- queues.h
 - queue_control, [236](#)
 - queue_display, [236](#)
 - queue_logger, [237](#)
 - queue_mqtt, [237](#)
 - queue_sensores, [237](#)
 - queues_init, [236](#)
- queues_init
 - queues.c, [232](#)
 - queues.h, [236](#)
- Referencia del directorio main, [10](#)
- Referencia del directorio main/drivers, [9](#)
- Referencia del directorio main/hal, [10](#)
- Referencia del directorio main/mqtt, [10](#)
- Referencia del directorio main/tasks, [11](#)
- Referencia del directorio main/utils, [11](#)
- Referencia del directorio main/wifi, [11](#)
- resend_pending_mqtt_messages
 - task_mqtt.c, [208](#)
- retry_count
 - wifi_manager.c, [247](#)
- rtc_ds3231.c
 - bcd_to_dec, [71](#)
 - dec_to_bcd, [71](#)
 - DS3231_ADDR, [70](#)
 - ds3231_clear_alarm_flag, [71](#)
 - DS3231_CONTROL_A1_INTERRUPT, [70](#)
 - ds3231_get_datetime, [72](#)
 - ds3231_init, [73](#)
 - ds3231_set_alarm_seconds, [74](#)
 - mutex_i2c, [75](#)
 - TAG, [75](#)
- rtc_ds3231.h
 - ds3231_clear_alarm_flag, [78](#)
 - ds3231_get_datetime, [79](#)
 - ds3231_init, [80](#)
 - ds3231_set_alarm_seconds, [80](#)
- save_pending_payload
 - task_mqtt.c, [209](#)
- sd_card.c
 - mount_point, [91](#)
 - pending_mqtt_path, [91](#)
 - PIN_CLK, [84](#)
 - PIN_CS, [84](#)
 - PIN_MISO, [84](#)
 - PIN_MOSI, [84](#)
 - sd_card_append_pending_mqtt, [85](#)
 - sd_card_close_pending_mqtt, [85](#)
 - sd_card_delete_pending_mqtt, [86](#)
 - sd_card_init, [86](#)
 - sd_card_is_mounted, [87](#)
 - sd_card_open_pending_mqtt_read, [88](#)
 - sd_card_pending_mqtt_exists, [88](#)
 - sd_card_read_pending_mqtt_line, [89](#)
 - sd_card_write_line, [89](#)
 - sd_mounted, [91](#)
 - spi_mutex, [91](#)
 - TAG, [91](#)
- sd_card.h
 - sd_card_append_pending_mqtt, [96](#)
 - sd_card_close_pending_mqtt, [97](#)
 - sd_card_delete_pending_mqtt, [97](#)
 - sd_card_init, [98](#)
 - sd_card_is_mounted, [99](#)
 - sd_card_open_pending_mqtt_read, [100](#)
 - sd_card_pending_mqtt_exists, [100](#)
 - sd_card_read_pending_mqtt_line, [100](#)
 - sd_card_write_line, [101](#)
- sd_card_append_pending_mqtt
 - sd_card.c, [85](#)
 - sd_card.h, [96](#)
- sd_card_close_pending_mqtt
 - sd_card.c, [85](#)
 - sd_card.h, [97](#)
- sd_card_delete_pending_mqtt
 - sd_card.c, [86](#)
 - sd_card.h, [97](#)
- sd_card_init
 - sd_card.c, [86](#)
 - sd_card.h, [98](#)
- sd_card_is_mounted
 - sd_card.c, [87](#)
 - sd_card.h, [99](#)
- sd_card_open_pending_mqtt_read
 - sd_card.c, [88](#)
 - sd_card.h, [100](#)
- sd_card_pending_mqtt_exists
 - sd_card.c, [88](#)

- sd_card.h, 100
- sd_card_read_pending_mqtt_line
 - sd_card.c, 89
 - sd_card.h, 100
- sd_card_write_line
 - sd_card.c, 89
 - sd_card.h, 101
- sd_mounted
 - sd_card.c, 91
- sd_ok
 - display_data_t, 17
- seconds
 - datetime_t, 15
- sensor_data_t, 17
 - co2, 18
 - co2_raw, 18
 - corriente, 18
 - datetime, 18
 - ph, 18
 - temperatura, 19
 - voltaje, 19
- servo.c
 - SERVO_CHANNEL, 104
 - SERVO_FREQ, 104
 - servo_init, 106
 - SERVO_MAX_DUTY, 105
 - SERVO_MAX_PULSE_US, 105
 - SERVO_MIN_PULSE_US, 105
 - SERVO_MODE, 105
 - SERVO_PERIOD_US, 105
 - servo_pin, 108
 - SERVO_RESOLUTION, 105
 - servo_set_angle, 107
 - SERVO_TIMER, 106
 - TAG, 108
- servo.h
 - servo_init, 111
 - servo_set_angle, 112
- servo_angle
 - control_cmd_t, 14
- SERVO_CHANNEL
 - servo.c, 104
- SERVO_FREQ
 - servo.c, 104
- servo_init
 - servo.c, 106
 - servo.h, 111
- SERVO_MAX_DUTY
 - servo.c, 105
- SERVO_MAX_PULSE_US
 - servo.c, 105
- SERVO_MIN_PULSE_US
 - servo.c, 105
- SERVO_MODE
 - servo.c, 105
- SERVO_PERIOD_US
 - servo.c, 105
- servo_pin
 - servo.c, 108
- SERVO_RESOLUTION
 - servo.c, 105
- servo_set_angle
 - servo.c, 107
 - servo.h, 112
- SERVO_TIMER
 - servo.c, 106
- spi_mutex
 - sd_card.c, 91
- TAG
 - adc_manager.c, 124
 - ds18b20.c, 27
 - i2c_manager.c, 131
 - main.c, 151
 - mq135.c, 36
 - mqtt_manager.c, 163
 - ph_sensor.c, 64
 - rtc_ds3231.c, 75
 - sd_card.c, 91
 - servo.c, 108
 - task_actuadores.c, 175
 - task_control.c, 182
 - task_energia.c, 196
 - task_logger.c, 202
 - task_sensores.c, 222
 - voltage_sensor.c, 115
 - wifi_manager.c, 247
- task_actuadores
 - task_actuadores.c, 173
 - task_actuadores.h, 177
- task_actuadores.c
 - actuadores_init, 173
 - MIX_DURATION_MS, 172
 - PIN_BOMBA, 172
 - PIN_MOTOR, 172
 - PIN_SERVO, 172
 - TAG, 175
 - task_actuadores, 173
 - task_energia_handle, 175
- task_actuadores.h
 - task_actuadores, 177
- task_control
 - task_control.c, 181
 - task_control.h, 184
- task_control.c
 - CO2_THRESHOLD, 180
 - MIX_INTERVAL_CYCLES, 180
 - TAG, 182
 - task_control, 181
- task_control.h
 - task_control, 184
- task_display
 - task_display.c, 187
 - task_display.h, 191
- task_display.c
 - task_display, 187
- task_display.h
 - task_display, 187

- task_display, 191
- task_energia
 - task_energia.c, 195
 - task_energia.h, 198
- task_energia.c
 - TAG, 196
 - task_energia, 195
 - task_sensores_handle, 196
- task_energia.h
 - task_energia, 198
- task_energia_handle
 - main.c, 151
 - task_actuadores.c, 175
 - task_mqtt.c, 211
- task_logger
 - task_logger.c, 200
 - task_logger.h, 204
- task_logger.c
 - TAG, 202
 - task_logger, 200
- task_logger.h
 - task_logger, 204
- task_mqtt
 - task_mqtt.c, 210
 - task_mqtt.h, 215
- task_mqtt.c
 - build_payload, 208
 - MQTT_CONNECT_TIMEOUT_MS, 207
 - MQTT_PENDING_PAYLOAD_SIZE, 207
 - MQTT_PUBLISH_TIMEOUT_MS, 207
 - MQTT_TOPIC, 207
 - resend_pending_mqtt_messages, 208
 - save_pending_payload, 209
 - task_energia_handle, 211
 - task_mqtt, 210
- task_mqtt.h
 - task_mqtt, 215
- task_sensores
 - task_sensores.c, 219
 - task_sensores.h, 226
- task_sensores.c
 - ALPHA, 219
 - baseline, 222
 - baseline_initialized, 222
 - filtered, 222
 - TAG, 222
 - task_sensores, 219
- task_sensores.h
 - task_sensores, 226
- task_sensores_handle
 - main.c, 151
 - task_energia.c, 196
- temperatura
 - display_data_t, 17
 - sensor_data_t, 19
- VOLTAGE_ADC_CHANNEL
 - voltage_sensor.c, 115
- voltage_sensor.c
 - ADC_OFFSET, 115
 - DIVIDER_RATIO, 115
 - NUM_SAMPLES, 115
 - TAG, 115
 - VOLTAGE_ADC_CHANNEL, 115
 - voltage_sensor_init, 116
 - voltage_sensor_read, 116
- voltage_sensor.h
 - voltage_sensor_init, 119
 - voltage_sensor_read, 119
- voltage_sensor_init
 - voltage_sensor.c, 116
 - voltage_sensor.h, 119
- voltage_sensor_read
 - voltage_sensor.c, 116
 - voltage_sensor.h, 119
- voltaje
 - sensor_data_t, 19
- WIFI_CONNECT_TIMEOUT_MS
 - main.c, 148
- WIFI_CONNECTED_BIT
 - wifi_manager.c, 240
- wifi_event_group
 - wifi_manager.c, 247
- wifi_event_handler
 - wifi_manager.c, 241
- WIFI_FAIL_BIT
 - wifi_manager.c, 240
- wifi_init
 - wifi_manager.c, 243
 - wifi_manager.h, 252
- wifi_is_connected
 - wifi_manager.c, 244
 - wifi_manager.h, 253
- wifi_manager.c
 - auth_mode_name, 241
 - retry_count, 247
 - TAG, 247
 - WIFI_CONNECTED_BIT, 240
 - wifi_event_group, 247
 - wifi_event_handler, 241
 - WIFI_FAIL_BIT, 240
 - wifi_init, 243
 - wifi_is_connected, 244
 - WIFI_MAX_RETRIES, 240
 - WIFI_PASS, 240
 - WIFI_SCAN_MAX_AP, 240
 - wifi_scan_print, 245
 - WIFI_SSID, 240
 - wifi_wait_connected, 246
- wifi_manager.h
 - wifi_init, 252
 - wifi_is_connected, 253
 - wifi_scan_print, 254
 - wifi_wait_connected, 255
- WIFI_MAX_RETRIES
 - wifi_manager.c, 240
- wifi_ok

- display_data_t, [17](#)
- WIFI_PASS
 - wifi_manager.c, [240](#)
- WIFI_SCAN_MAX_AP
 - wifi_manager.c, [240](#)
- wifi_scan_print
 - wifi_manager.c, [245](#)
 - wifi_manager.h, [254](#)
- WIFI_SSID
 - wifi_manager.c, [240](#)
- wifi_wait_connected
 - wifi_manager.c, [246](#)
 - wifi_manager.h, [255](#)
- year
 - datetime_t, [15](#)